



Neural Branch-and-Bound Accelerator for Physical Design (Index/View/Partition)

Exactness-Preserving Guidance with
Certificate-Gated Search

Yasin R. Quluzada

Elvar T. Hidayatov

School of Science and Engineering
Department of Computer Science
Bachelor in Computer Engineering

Khazar, December 2025



Neural Branch-and-Bound Accelerator for Physical Design (Index/View/Partition)

Exactness-Preserving Guidance with
Certificate-Gated Search

Yasin R. Quluzada

Student No. B.01.631.23.044

Elvar T. Hidayatov

Student No. B.01.509.23.022

School of Science and Engineering
Department of Computer Science
Bachelor in Computer Engineering

Report

Khazar, December 2025

Neural Branch-and-Bound Accelerator for Physical Design (Index/View/Partition)

Copyright © 2025 - Yasin R. Quluzada, School of Science and Engineering.

This report is original work, written solely for this purpose, and all the authors whose studies and publications contributed to it have been duly cited. Partial reproduction is allowed with acknowledgment of the author and reference to the degree, academic year, institution—*Polytechnic University of Leiria*—and public defense date.



Preparation of this work was facilitated by the use of the *IPLeiria-Thesis* template.

Acknowledgements

Writing Guidance

In the *Acknowledgment* section, express your gratitude to those who helped and supported your work. Start by thanking your advisors, mentors, or supervisors who provided guidance and expertise. Mention any colleagues, classmates, or team members who contributed to discussions or offered assistance. You can also acknowledge specific organisations, institutions, or funding sources that supported your research or work. Lastly, include any personal acknowledgments for family or friends who offered encouragement and moral support during the project. Keep this section sincere, concise, and professional.

Resumo

Este relatório apresenta um acelerador *exato* para a conceção física de bases de dados, integrando a selecção de índices, vistas materializadas e esquemas de particionamento sob restrições explícitas de orçamento e compatibilidade. O problema é especificado por (i) uma construção finita do espaço de candidatos, (ii) uma definição formal do universo de planos com condições explícitas de viabilidade, e (iii) um funcional de custo total e determinístico no universo construído. Com estas bases, define-se uma formulação de optimização inteira mista equivalente à semântica do objectivo de conceção. A resolução combina *branch-and-bound* com geração de colunas e planos de corte, e todas as operações que podem afectar limites ou podas são sujeitas a verificação por certificados aritméticos finitos. A componente de aprendizagem opera apenas como camada consultiva (propostas para ramificação, cortes, pricing e construção primal), sendo a aceitação controlada por portas determinísticas que preservam a exactidão. Por fim, um pacote de avaliação empírica define uma carta de benchmark, um contrato de justiça, um léxico de métricas, uma grelha de ablação, uma suite de robustez e um protocolo de repetibilidade, culminando num livro de resultados com tabelas e figuras obrigatórias.

Palavras-Chave: Optimização Inteira, Conceção Física de Bases de Dados, Branch-and-Bound, Geração de Colunas, Planos de Corte, Certificados Aritméticos, Aprendizagem Guiada.

Abstract

This report presents an *exact* accelerator for database physical design, jointly selecting indexes, materialised views, and partition schemes under explicit budget and compatibility constraints. The problem is specified by (i) a finite construction of the candidate design space, (ii) a formal definition of the plan universe with explicit feasibility conditions, and (iii) a total and deterministic cost functional on the constructed universe. On this basis, we define a mixed-integer optimization formulation provably equivalent to the semantic design objective. The solver combines branch-and-bound with column generation and cutting-plane mechanisms; every operation that can affect bounds or pruning is subject to finite arithmetic certificate verification. Learning is introduced only as an advisory layer (proposals for branching, cuts, pricing, and primal construction) whose effects are mediated by deterministic acceptance gates that preserve exactness. Finally, an empirical closure pack defines the benchmark charter, fairness contract, metric ledger, ablation grid, robustness suite, and repeatability protocol, culminating in a results ledger with mandatory tables and figures.

Keywords: Integer Optimization, Database Physical Design, Branch-and-Bound, Column Generation, Cutting Planes, Arithmetic Certificates, Learned Guidance.

AI Acknowledgement

Writing Guidance

It is good academic practice to briefly state how, why, and when AI was used. Explaining both the actions taken and the reasons behind them encourages critical reflection, deepens understanding of AI's role in learning, and supports the evaluation of its impact on your work's quality and integrity. This page can be removed by setting the option `aiack` to `false` in the document class options.

Example of Use

I acknowledge the use of ChatGPT (<https://chatgpt.com>) to refine the academic tone and improve the linguistic accuracy of this work, including aspects of grammar, punctuation, and vocabulary.

Description of Use

Prompt 1: In a paragraph, simply describe what LaTeX is.

Output 1: LaTeX is a high-quality typesetting system commonly used for producing scientific and mathematical documents due to its powerful handling of formulas and bibliographies. Unlike word processors, it uses plain text files with markup tags to define the structure and formatting of a document, allowing precise control over layout and appearance. It is especially popular in academia and research for creating theses, journal articles, and presentations.

Contents

<i>List of Figures</i>	xxv
<i>List of Tables</i>	xxix
<i>Glossary</i>	xxxii
<i>Acronyms</i>	xxxiv
<i>Symbols</i>	xxxvi

1 Scope, Assumptions, and Claim Discipline	1
1.1 Document scope and system boundary	1
1.2 Assumption ledger	1
1.3 Claim discipline protocol	1
1.4 Reading map	2
2 Physical Design Object Space (Index, View, Partition Candidates)	3
2.1 Inputs and schema primitives	3
2.1.1 Finite schema	3
2.1.2 Workload as a finite multiset of typed expressions	4
2.1.3 Feature extraction primitives	4
2.2 Candidate index space and canonical representation	5
2.2.1 Admissible index object	5
2.2.2 Workload grounded generation rule	5
2.2.3 Canonical representation of index objects	5
2.3 Candidate materialized view space and canonical representation	6
2.3.1 Admissible view object	6
2.3.2 Workload grounded generation rule	6
2.3.3 Canonical representation of view objects	6
2.4 Candidate partition scheme space and exclusivity groups	7
2.4.1 Admissible partition object	7
2.4.2 Workload grounded boundary extraction	7
2.4.3 Exclusivity groups	7
2.5 Finite generation theorem and closure properties	8
2.5.1 Finiteness of candidate object spaces	8
2.5.2 Closure properties	8

2.6	Deduplication and equivalence classes	9
2.6.1	Equivalence relations by canonical encoding	9
2.6.2	Deduplication operator	9
2.6.3	Deduplication correctness	9
	End of Chapter Artifacts	10
3	Query and Plan Universe Definition	11
3.1	Workload query language and normalization	11
3.1.1	Primitive finite data	11
3.1.2	Logical query language	12
3.1.3	Normalization and join core	12
3.2	Physical operator set and plan grammar	12
3.2.1	View matchability and admissible placements	12
3.2.2	Leaf access modes and leaf terms	13
3.2.3	Join nodes and canonical root unary placement	14
3.3	Plan feasibility semantics as object requirements	14
3.3.1	Leaves induced by a cover	14
3.3.2	Allowed instantiations for leaves	15
3.3.3	Binary plan trees and plan terms	15
3.3.4	Requirement sets	15
3.4	Plan universe finiteness theorem	16
3.5	Plan encoding	17
3.5.1	Derivation tree representation	17
3.5.2	Canonical plan code	17
3.5.3	Plan feasibility rule table	18
4	Cost Model	19
4.1	Primitive measurable quantities	19
4.1.1	Statistics, cardinalities, and selectivities	19
4.1.2	Measurement interface and scope	21
4.2	Operator-level cost functions	21
4.2.1	Access path costs	21
4.2.2	Join and aggregation costs	22
4.2.3	Maintenance and refresh costs	23
4.3	Plan-level aggregation	24
4.3.1	Query-level aggregation	24
4.3.2	Workload-level aggregation	24
4.4	Formal properties	24
4.4.1	Totality and determinism	24
4.4.2	Nonnegativity and boundedness (as applicable)	25
4.5	Figures and tables	25

5	Workload Weighting and Distribution Models	26
5.1	Chapter purpose and boundary	26
5.2	Standing objects inherited from previous chapters	26
5.3	Deterministic workload weighting model	27
5.3.1	Weight domains	27
5.3.2	Realization map from finite arrival sequences to weights	27
5.3.3	Deterministic weight selection from finite logs	27
5.4	Stochastic weighting model (finite support form)	28
5.4.1	Conditional finite horizon arrival generator	28
5.4.2	Expected realized weights and induced expected objective	28
5.4.3	Unconditional finite mixture model	29
5.4.4	Scenario form used by the optimization model	30
5.5	Robust adversarial weighting model (uncertainty set)	30
5.5.1	Nominal weights and perturbation variables	30
5.5.2	Linear uncertainty specification with explicit budgets	30
5.5.3	Robust objective contribution	31
5.6	Reduction and equivalence statements (where applicable)	31
5.6.1	Expectation reduction for linear weighting	31
5.6.2	Robust reduction to a finite scenario set (explicit convex hull form)	32
5.6.3	Robust model specialization to deterministic weighting	32
5.7	Chapter closure	32
6	Compatibility and Feasibility	34
6.1	Partition correctness and exclusivity	34
6.1.1	Partition admissibility rules	34
6.1.2	Exclusivity constraints per table and group	35
6.2	Index–partition compatibility	35
6.2.1	Compatibility predicates	35
6.2.2	Feasibility implications	36
6.3	Materialised view validity and maintenance	36
6.3.1	View definition validity	36
6.3.2	Refresh and maintenance semantics	37
6.4	Cross-object compatibility constraints	38
6.4.1	Joint feasibility constraints	38
6.4.2	Feasibility witness structure	39
6.5	Feasibility closure theorem	39
6.5.1	Decidability under the constructed finite instance	39
6.5.2	Witness soundness conditions	39
6.6	Figures and tables	40

7	Baseline Optimization Formulation	42
7.1	Decision variables and domains	43
7.2	Objective definition (workload plus maintenance)	43
7.3	Budget and feasibility constraints	44
7.3.1	Budget constraint	44
7.3.2	Partition exclusivity constraints	44
7.3.3	Index on view dependency constraints	44
7.3.4	Index partition compatibility constraints	44
7.3.5	View refresh mode constraints (if enabled)	44
7.3.6	Local validity constraints for partition respecting views (if enabled)	45
7.3.7	Strong coupling constraints for locally refreshed views (if enabled)	45
7.3.8	Remaining feasibility constraints	45
7.4	Plan selection coupling constraints	45
7.4.1	One plan per query normalization	46
7.4.2	Requirement feasibility coupling	46
7.4.3	Feasible plan set induced by a design	46
7.5	Equivalence theorem: formulation matches best feasible plan per query	46
7.5.1	Feasibility equivalence	46
7.5.2	Objective equivalence	47
8	Optional Nonlinear Extensions (If Used)	49
8.1	Nonlinear primitives and their semantic necessity	49
8.1.1	Baseline condition and admissibility boundary	49
8.1.2	Variable universe and finiteness requirements for any nonlinear extension	49
8.1.3	Admissible primitive library	50
8.2	Exact compilation to MILP, or certified relaxations	50
8.2.1	Expression grammar and extended formulation goal	50
8.2.2	Exact rewrite rules and correctness statements	51
8.2.3	Whole model compilation theorem	53
8.2.4	Certified relaxations when exact compilation is intentionally not required	54
8.3	Scope: when the system remains MILP-only	54
8.3.1	MILP-only decision criterion	54
8.3.2	Consequences for the global correctness spine	55
8.3.3	Boundary statement	55
8.4	Nonlinear to linear transformation schematic	55
I	Exact Bound Computation (Cut-and-Price Closure)	56
9	Restricted Master Construction and Initialisation	57
9.1	Restricted master definition	57

9.1.1	Initial column set	58
9.1.2	Initial cut set (if any)	58
9.2	Seed existence and feasibility preservation	59
9.2.1	Seed plan existence statement	59
9.2.2	Feasibility preservation lemma	59
9.3	Initialisation theorem	59
9.3.1	Well-posedness of the initial RMP	59
9.3.2	Bound interpretability at initialisation	60
9.4	Figures and tables	60
10	Pricing: Oracle, Termination, and Completeness	61
10.1	Reduced cost definition and algebra	61
10.1.1	Fixed data and domains	61
10.1.2	Dual variables used by the reduced cost	61
10.1.3	Definition of reduced cost	62
10.2	Deterministic pricing fallback procedure (completeness)	63
10.2.1	Canonical plan codes and enumeration	63
10.2.2	Deterministic complete pricing oracle	63
10.3	Termination condition and correctness theorem	64
10.3.1	Termination	64
10.3.2	Correctness and completeness	64
10.4	Interaction with learned pricing proposals (advisory only)	64
10.4.1	Advisory proposal object and membership gate	65
10.4.2	Proposal evaluation gate and selection	65
10.4.3	Correctness immunity and completeness of the composite oracle	65
10.4.4	Consequences for node closure	66
10.5	Required figures and tables for this chapter	66
11	Column Representation and Reduced Cost Certificates	67
11.1	Canonical Plan Code to Column Vector Map	67
11.1.1	Primitive arithmetic domain	67
11.1.2	Finite plan code space and query typed decoding	67
11.1.3	Canonicalization operator	68
11.1.4	Physical object identifiers and requirement extraction	68
11.1.5	Exact plan cost	69
11.1.6	The column projector and the column signature	69
11.1.7	Column vector map in the restricted master problem	69
11.2	Column Uniqueness and Deduplication	70
11.2.1	Column equivalence and signature identity	70
11.2.2	Uniqueness of the sparse signature	70
11.2.3	Deduplication rule	71
11.3	Reduced Cost Certificate Format	71

11.3.1	Dual variable interface	71
11.3.2	Reduced cost definition	71
11.3.3	Certificate tuple and verifier	71
11.4	Soundness of Accepted Column Certificates	72
12	Cutting Planes	74
12.1	Admissible cut classes	74
12.1.1	Cut family registry	74
12.1.2	Validity predicate	75
12.2	Cut certificates	75
12.2.1	Certificate schema	75
12.2.2	Mechanical check procedure	76
12.3	Validity theorems	77
12.3.1	Family-level validity theorems	77
12.3.2	Composition and aggregation rules	78
12.4	Figures and tables	79
13	Separation and Cut Admission	80
13.1	Proposal versus acceptance separation	80
13.1.1	Proposal interfaces (including learned proposals)	80
13.1.2	Acceptance gate definition	81
13.2	Deterministic fallback separation	81
13.2.1	Completeness condition (if required)	81
13.2.2	Termination rule	83
13.3	Soundness of cut admission	83
13.3.1	No invalid cut admission theorem	83
13.3.2	Certificate failure handling	84
13.4	Figures and tables	84
14	Cut Management	86
14.1	Cut lifecycle	86
14.1.1	Add, activate, deactivate, retire	86
14.1.2	State variables for cut management	87
14.2	Invariant preservation per transition	88
14.2.1	Preservation lemmas	88
14.2.2	Failure and rollback semantics	89
14.3	Correctness theorem for cut management	89
14.3.1	Bound correctness preservation	89
14.3.2	Feasibility preservation	90
14.4	Figures and tables	90

15 Cut Strength and Selection Discipline	91
15.1 Strength metrics	91
15.1.1 Bound improvement measures	91
15.1.2 Redundancy and dominance criteria	92
15.2 Selection rules	94
15.2.1 Selection under correctness immunity	94
15.2.2 Resource constraints and scheduling	95
15.3 Efficiency statements	95
15.3.1 Permitted empirical statements	95
15.3.2 Prohibited unqualified generalisations	95
15.4 Figures and tables	96
 II Global Search Correctness (Branch-and-Bound Closure)	 97
16 Node Definition and Domain Propagation	98
16.1 Node object definition	98
16.1.1 Domain restrictions	98
16.1.2 Node-local relaxation and bound meaning	99
16.2 Propagation rules	99
16.2.1 Rule language	99
16.2.2 Soundness proofs	100
16.3 Propagation termination	102
16.3.1 Finite termination lemma	102
16.3.2 Fixed-point semantics	102
16.4 Figures and tables	102
 17 Branching Rules and Completeness	 104
17.1 Branch action definition	104
17.1.1 Binary branching primitives	104
17.1.2 Compatibility with feasibility constraints	106
17.2 Completeness theorem	106
17.2.1 Coverage of the feasible space	106
17.2.2 Finite branching depth bound (if applicable)	107
17.3 Figures and tables	108
 18 Node Selection Policies	 110
18.1 Policy definitions	110
18.1.1 Priority queue semantics	110
18.1.2 Tie-breaking discipline	112
18.2 Correctness immunity	112
18.2.1 Ordering does not affect correctness theorem	112
18.2.2 Separation from bound computation and pruning	114

18.3 Figures and tables	115
19 Primal Feasibility Construction	116
19.1 Feasibility witness definition	116
19.1.1 Witness structure	116
19.1.2 Witness check predicate	117
19.2 Incumbent construction procedure	117
19.2.1 Deterministic construction	117
19.2.2 Learned proposal interface (advisory)	118
19.3 Soundness theorem	119
19.3.1 Accepted incumbent implies feasibility	119
19.3.2 Objective evaluation consistency	119
19.4 Figures and tables	119
20 Pruning and Global Optimality	120
20.1 Infeasibility certification requirements	120
20.1.1 Infeasibility witnesses	120
20.1.2 Node closure conditions	121
20.2 Bound-based pruning	121
20.2.1 Pruning rule	121
20.2.2 Pruning soundness theorem	122
20.3 Global optimality	123
20.3.1 Termination conditions	123
20.3.2 Global optimality theorem	123
20.4 Figures and tables	124
III Learned Guidance (Advisory Layer Closure)	125
21 State Representation	126
21.1 Typed graph and hypergraph schema	126
21.1.1 Node and edge types	126
21.1.2 Semantic invariants of the representation	128
21.2 Feature dictionary	129
21.2.1 Feature definitions	129
21.2.2 Normalisation and boundedness	131
21.3 Representation finiteness and stability	132
21.3.1 Finiteness under finite instance closure	132
21.3.2 Stability requirements across runs	132
21.4 Figures and tables	133

22 Action Spaces	134
22.1 Action sets and admissibility	134
22.1.1 Branch actions	134
22.1.2 Cut proposal actions	135
22.1.3 Pricing proposal actions	135
22.1.4 Primal construction actions	136
22.2 Enumerability and finiteness	136
22.2.1 Finite action enumeration under closure	136
22.2.2 Admissibility proofs	137
22.3 Figures and tables	137
23 Training Signals	139
23.1 Trace object definition	139
23.1.1 What is logged	139
23.1.2 Trace integrity requirements	140
23.2 Label constructions	140
23.2.1 Label types and sources	140
23.2.2 Well-definedness and determinism	141
23.3 Correctness of label extraction	143
23.3.1 No undefined targets lemma	143
23.3.2 Consistency across runs	144
23.4 Figures and tables	144
24 Gate-Compatible Policies	146
24.1 Proposal functions versus acceptance gates	146
24.1.1 Advisory proposals	146
24.1.2 Acceptance gates as sole effectors	147
24.2 Correctness immunity theorem	148
24.2.1 Learning cannot change feasibility, bound validity, or pruning soundness	148
24.2.2 Proof strategy via gate separation	149
24.3 Continual update constraints	149
24.3.1 Immutable core versus mutable advisors	149
24.3.2 Rollback and disabling rules	150
24.4 Figures and tables	150
25 Calibration and Drift Control	151
25.1 Calibration envelope definitions	151
25.1.1 Calibration targets	151
25.1.2 Acceptance thresholds	153
25.2 Drift detection and rollback rules	153
25.2.1 Drift metrics	153
25.2.2 Rollback and safe-mode operation	153

25.3	Stability guarantees	154
25.3.1	Permitted stability theorems	154
25.3.2	Empirical stability reports	155
25.4	Figures and tables	155
IV	Integrity, Invariants, and Verification (Non-Arguable Closure)	156
26	Unified Invariant Ledger	157
26.1	Invariant set definition	157
26.1.1	Mathematical environment and state	157
26.1.2	Domain invariants	158
26.1.3	Bound invariants	159
26.1.4	Certificate invariants	159
26.2	Transition system	160
26.2.1	Allowed transitions	160
26.2.2	Transition guards	161
26.3	Preservation lemmas	162
26.3.1	Per-transition preservation	162
26.3.2	Composition of preservation	163
26.4	Figures and tables	164
27	End-to-End Correctness Stitching	165
27.1	Definitions required for global proof	165
27.2	Bound correctness chain	168
27.3	Pruning correctness chain	170
27.4	Global optimality theorem (final)	171
27.5	Figures and tables	172
28	Benchmark Charter v1.0	173
28.1	Purpose, scope, and non disputability requirements	173
28.2	Schema families and generation rules	173
28.2.1	Schema family object and canonical representation	173
28.2.2	Scale strata	174
28.2.3	Required schema families	174
28.3	Workload families and parameterization	175
28.3.1	Workload family object and canonical representation	175
28.3.2	Required workload families	176
28.3.3	Schema and workload codes	176
28.3.4	Compatibility and binding constraints	177
28.4	Candidate-space freeze policy	177
28.4.1	Instance object and deterministic identifiers	177
28.4.2	Candidate generator and decomposition	178

28.4.3	Freeze unit and required artifacts	178
28.4.4	Immutability rule	179
28.5	Budget regimes and instance stratification	179
28.5.1	Budget regime definition	179
28.5.2	Stratification protocol and balance constraint	179
28.5.3	Local instantiation magnitude and visual audit	179
28.5.4	Candidate-space summary table	180
29	Baseline Registry and Fairness	185
29.1	Baseline definitions	185
29.1.1	Frozen instance model and semantic identity	185
29.1.2	Anchor exact solver definition	186
29.1.3	Comparator method definition and termination modes	186
29.1.4	Certificate taxonomy and admissibility	187
29.1.5	Registry row schema and baseline classes	188
29.2	Fairness rules (what all methods share)	188
29.2.1	Contract invariants	188
29.2.2	Transcript schema requirements	190
29.3	Disallowed advantages and enforcement notes	190
29.3.1	Disallowed advantages	190
29.3.2	Mechanical enforcement and first violation reporting	191
29.3.3	Validator stress test results (contract enforcement check)	191
29.4	Figures and tables	191
29.4.1	Baseline registry table	191
29.4.2	Fairness contract checklist	191
29.4.3	Stress test summary tables	192
29.4.4	Stress test figures	192
30	Metrics Ledger	195
30.1	Purpose, scope, and claim admissibility	195
30.2	Evidence bundle, event schema, and measurement boundaries	196
30.2.1	Event log schema	196
30.2.2	Measurement boundaries	196
30.3	Mandatory gates and invariants	196
30.3.1	Gate G0: external audit acceptance	197
30.3.2	Gate G1: deterministic reproducibility of the ledger	197
30.3.3	Gate G2: monotonicity and feasibility invariants	197
30.3.4	Gate G3: fairness contract compliance	197
30.4	Traceability to remainder solutions	197
30.5	Primary metrics	202
30.5.1	Time to optimality	202
30.5.2	Nodes to optimality	203

30.5.3	Bound progress and anytime quality	203
30.6	Secondary metrics	203
30.6.1	LP and pricing workload	204
30.6.2	Cut workload	204
30.6.3	Memory and overhead decomposition	204
30.6.4	Advisor success rate	205
30.7	Aggregation rules and confidence reporting	205
30.8	Metrics dictionary table	206
30.9	Local ledger validation experiment	207
30.9.1	Headline results	208
30.9.2	Family level summary	208
30.9.3	Figures	208
30.10	Success criteria	208
30.10.1	Hard criteria	208
30.10.2	Performance criteria	209
30.10.3	Overhead criteria	209
31	Ablation Grid	212
31.1	Purpose and admissibility rule	212
31.2	Anchor solver definition	212
31.2.1	Anchor configuration as the unique reference point	212
31.2.2	Certificate gates, fallbacks, and invariants	213
31.3	Single-module and combined variants	214
31.3.1	Variant lattice and uniqueness	214
31.3.2	Single-module variants	214
31.3.3	Combined variants	215
31.4	Reporting requirements for ablations	215
31.4.1	Fairness contract	215
31.4.2	Per-run logging requirements	215
31.4.3	Aggregation and presentation rules	216
31.5	Figures and tables	216
31.5.1	Ablation grid table	216
31.5.2	Ablation summary plot	216
31.5.3	Pipeline validation table and supporting distributions	216
31.6	Validity closure checklist	216
32	Robustness Suite	220
32.1	Workload shift tests	221
32.1.1	Shift construction	221
32.1.2	Reporting requirements	222
32.2	Budget shift tests	222
32.2.1	Budget band changes	222

32.2.2	Reporting requirements	223
32.3	Schema shift tests	223
32.3.1	Schema family changes	223
32.3.2	Reporting requirements	224
32.4	Candidate expansion stress tests	224
32.4.1	Expansion protocol	224
32.4.2	Reporting requirements	225
32.5	Figures and tables	225
33	Repeatability and Reporting Protocol	232
33.1	Run controls (seeds, ordering, caps)	232
33.1.1	Run object and determinism target	232
33.1.2	Frozen instance identity	233
33.1.3	Seed vector and randomness containment	233
33.1.4	Total ordering of commutative choices	233
33.1.5	Cap vector and resource invariants	234
33.1.6	Execution environment disclosure and transcript obligation	234
33.2	Stopping rules and timeouts	234
33.2.1	Stopping rule taxonomy	234
33.2.2	Censoring and metric definitions under caps	235
33.2.3	Timeout semantics for admissibility preserving fallbacks	235
33.2.4	Separation between solved and best known	235
33.3	Reporting completeness checklist	236
33.3.1	Scope and admissibility rule	236
33.3.2	Mandatory disclosures for each results block	236
34	Results Ledger v1.0	239
34.1	Ledger scope and integrity rule	239
34.2	Main results table	239
34.3	Ablation results table	240
34.4	Robustness results table	242
34.5	Required plots	242
34.5.1	Bound progress versus time	242
34.5.2	Nodes explored versus time	242
34.5.3	Time-to-optimality distribution	244
34.5.4	Certificate-check overhead distribution	244
34.6	Interpretation rules	246
35	Conclusion	247
35.1	Summary of contributions	247
35.1.1	Exactness-preserving formulation and solver closure	247
35.1.2	Certificate-gated learned guidance	249
35.1.3	Empirical closure pack	249

35.2 Limitations and boundaries	250
35.3 Future work (optional, explicitly non-claiming)	251
<i>Bibliography</i>	253
Appendices	
A Definitions Index	258
A.1 Definitions by chapter	258
A.2 Definitions by symbol	262
B Theorem and Lemma Index	268
B.1 Statements by chapter	268
B.1.1 Part I, Chapter 1: Physical Design Object Space	268
B.1.2 Part I, Chapter 2: Query and Plan Universe Definition	269
B.1.3 Part I, Chapter 3: Cost Model Well Definedness and Computability	269
B.1.4 Part I, Chapter 4: Workload Weighting and Distribution Models	270
B.1.5 Part I, Chapter 5: Compatibility and Feasibility Semantics	271
B.1.6 Part VI, Chapter 26: End to End Correctness Stitching	271
B.1.7 Part VI, Chapter 27: Arithmetic Certificate Verification Engine	272
B.2 Dependency summaries	273
C Certificate Schema Compendium	274
C.1 Scope and role in the report's non disputable closure	274
C.2 Universal canonical encodings	274
C.2.1 Atomic domains	274
C.2.2 Finite containers with canonicalization	274
C.2.3 Canonical sparse vectors and matrices	275
C.3 Instance specification proof objects	275
C.4 Candidate record formats and identity	275
C.5 Cut proof objects	275
C.5.1 Canonical cut representation	275
C.5.2 Chvátal Gomory cut certificate object	276
C.5.3 Contradiction certificate for infeasibility	276
C.6 Linear program optimality and bound certificates	276
C.6.1 Canonical node linear program form	276
C.6.2 Dual variable schema	277
C.6.3 Linear program optimality certificate	277
C.7 Pricing and column generation proof objects	277
C.7.1 Plan universe as a finite weighted directed hypergraph	277
C.7.2 Plan derivation proof object	277
C.7.3 Reduced cost representation	278
C.7.4 Pricing closure certificate	278

C.8	Node bound package certificate	278
C.9	Branch and bound proof objects	279
C.9.1	Node record and branching coverage	279
C.9.2	Incumbent record and feasibility witness	279
C.9.3	Pruning record	279
C.10	Transcript schema and acceptance surface	280
C.11	Auxiliary proof object families required by the learned advisory layer	280
C.12	Relation to Appendix D and completeness	281
D	Unified Invariant Ledger (Extended)	282
D.1	Invariant catalogue	283
D.2	Transition guard catalogue	289
D.3	Preservation proof pointers	291
E	Empirical Artifact Index	292
E.1	Benchmark instances and seeds	292
E.2	Configuration registry	292
E.3	Result tables and plots registry	292
F	Extended Results	293
F.1	Per-family breakdowns	293
F.2	Per-budget breakdowns	293
F.3	Additional robustness plots	293
	Appendix G: Glossary of Terms (Strictly aligned to definitions used)	293
 Annexes		
L	Showcasing the First Annex	310

List of Figures

2.1	Candidate generation pipeline for index, view, and partition candidates. . .	10
3.1	Plan grammar schematic for query r . The diagram shows the composition from covers C , induced leaves Y_C , instantiated binary join trees τ , and root unary options ρ	17
3.2	Example derivation tree for a fixed query r . The tree explicitly annotates leaf access modes, partition schemes, join methods, and distribution choices, and it includes at least one view leaf induced by a nonempty cover C	17
4.1	Cost decomposition from operator signatures to workload objective.	25
6.1	Compatibility lattice linking partition selection, index compatibility, view validity, and feasibility.	40
6.2	Partition exclusivity per base table expressed as a single selection constraint.	41
8.1	Schematic of nonlinear expression graph compilation into an extended mixed integer linear formulation via bounded auxiliary variables and finite selectors.	55
10.1	Pricing loop: dual extraction, reduced cost evaluation, column admission, and closure reporting.	66
10.2	Reduced cost components and the exact algebraic terms used by the pricing oracle.	66
11.1	Deterministic pipeline from a plan code to the corresponding restricted master column coefficients.	70
12.1	Cut certificate anatomy: ledger, certificate object, mechanical acceptance, admitted cut.	79
13.1	Separation controller with proposal stage, gate enforcement, and deterministic fallback enumeration.	85
14.1	Cut lifecycle diagram showing Add, Activate, Deactivate, Anchor, and Retire transitions with their preconditions.	90

15.1	Strength tradeoff diagram relating objective bound gain, encoding cost, and redundancy classes.	96
16.1	Domain propagation graph at a node.	103
17.1	Binary branch split at index j : left child enforces $x_j = 0$ and right child enforces $x_j = 1$	108
17.2	Set theoretic coverage: $I(N)$ is partitioned by $I(N^0)$ and $I(N^1)$ under binary fixing.	108
18.1	Node selection schematic. The open frontier U is ordered by a lexicographic key $\rho(N)$ computed from admissible metadata such as $LB(N)$, $d(N)$, $s(N)$, and $id(N)$. Selection maps the current state to a unique node in U without modifying feasibility, bounds, pruning, branching, or certificate acceptance.	115
19.1	Deterministic primal construction and incumbent update gate.	119
21.1	Typed hypergraph schema used for the state representation. The diagram specifies vertex types S, Q, C, K, T, G and the admissible relation families defined in Section 20.1.	133
22.1	Action space map diagram for branching, cutting, pricing, and primal construction.	137
23.1	Trace-to-label pipeline. The learning substrate is a deterministic function of an admissible trace and uses only verified solver objects.	144
28.1	Schema family diagram for Benchmark Charter v1.0. Each family is defined by a finite parameter domain with explicit scale strata. The XL stratum highlights the maximum magnitudes used during local instantiation.	175
28.2	Workload family diagram for Benchmark Charter v1.0. Each family is defined by a finite parameter domain with explicit scale strata and explicit ranges for principal query structure parameters.	177
28.3	Candidate-space magnitude by schema code and scale stratum, reported as $\log_{10}$ of the median candidate count across workload families.	180
28.4	Mean eligible fraction at XL scale as a function of the budget regime. Each point is the mean of $ C_{\mathbb{I}}(B) / C_{\mathbb{I}} $ across all frozen XL instances.	181
29.1	Violation counts by first failed rule in the fairness validator stress test.	192
29.2	Invalid run counts by baseline in the fairness validator stress test.	194
30.1	Distribution of paired speedup ratios, defined as $T_{incl, baseline}^*/T_{incl, accelerator'}^*$ computed on paired instances.	208

30.2	Median relative gap curve $g(t)$ on a logarithmic ordinate. The median is taken pointwise across paired instances on a common time grid and is truncated at the time cap.	209
30.3	Median nodes processed versus time on the common time grid. This curve is derived from monotone node count traces and is reported to visualize search contraction.	210
30.4	Distribution of verification overhead time T_{ver} for the accelerator variant. This figure is required whenever verification costs exist, because it constrains interpretations of inclusive runtime improvements.	211
31.1	Ablation summary plot (reporting pipeline validation).	217
31.2	Per-instance speedup distribution (reporting pipeline validation).	218
31.3	Certificate-check overhead distribution (reporting pipeline validation). . .	218
32.1	Shift matrix diagram	226
32.2	Speedup matrix diagram	226
32.3	Speedup versus severity	231
34.1	Bound progress versus time	243
34.2	Nodes explored versus time	244
34.3	Time-to-optimality distribution	245
34.4	Certificate-check overhead distribution	245

List of Tables

2.1	Candidate space summary by benchmark family and budget regime. Values are to be filled from the frozen benchmark charter and candidate generator outputs.	10
3.1	Plan feasibility rule table defining the requirement set $R_{r\ell}$ as in Definition 2.3.5.	18
4.1	Cost symbol dictionary (all symbols are finite and explicitly represented). .	25
6.1	Feasibility clauses defining cross-object compatibility.	40
8.1	Admissible nonlinear primitive library.	50
10.1	Reduced cost components and domains.	62
11.1	Reduced cost certificate schema and verifier obligations.	72
12.1	Cut family registry and mechanically checkable admission conditions. . . .	79
13.1	Admission checklist executed for every proposed cut prior to ledger augmentation.	84
14.1	Transition to invariant mapping. Each entry is sufficient to derive integer containment and lower bound soundness via Lemmas 14.6 and 14.7.	90
15.1	Strength and redundancy metrics used by the cut selection discipline at a branch and bound node.	96
16.1	Propagation rule table for node domain tightening and infeasibility detection.	103
17.1	Determinism and tie breaking rules required for reproducible branching. .	109
18.1	Node selection key components and admissible uses.	112
19.1	Feasibility witness schema and verification obligations.	119
21.1	Feature dictionary for the state representation. All features are defined in exact rational arithmetic with explicit rules for the infeasible LP case. . . .	133

22.1 Action admissibility table. The table is width safe by construction under <code>tabularx</code> with total width <code>\textwidth</code>	138
23.1 Label taxonomy and trace sources. Every label is computed from finite recorded objects and terminating certified procedures under fixed tie breaking.	145
28.1 Local instantiation magnitude summary by scale stratum. The reported values are computed from the frozen generator output and therefore constitute part of the benchmark freeze record.	180
29.2 Fairness contract checklist (rule to required evidence)	191
29.1 Baseline registry (frozen comparator definitions)	193
29.3 Fairness validator stress test summary by baseline (120,000 runs).	194
29.4 Fairness validator stress test violations by first failure label (120,000 runs).	194
30.1 Traceability from remainder solutions to ledger fields	198
30.2 Aggregation rules	205
30.3 Metrics dictionary	206
30.4 Local validation headline results	207
30.5 Local ledger validation summary	207
31.1 Ablation grid specification: module activations relative to the anchor solver.	217
31.2 Pipeline validation table for ablation reporting at large metric magnitudes.	217
32.1 Robustness results table template	225
32.3 Illustrative robustness results ledger	227
32.2 Illustrative robustness summary	231
33.1 Repeatability protocol table.	237
33.2 Reporting checklist table.	238
34.1 Main results table	240
34.2 Ablation results table	241
34.3 Robustness results table	242

1

Scope, Assumptions, and Claim Discipline

1.1 Document scope and system boundary

This report specifies an exactness-preserving optimisation system for database physical design under a finite candidate space of indexes, materialised views, and partition schemes. The document separates (i) *formal statements* (definitions, lemmas, theorems) from (ii) *empirical findings* (benchmark-conditioned measurements).

The optimisation backbone follows standard mixed-integer optimisation and branch-and-bound foundations (Land et al., 1960; Wolsey, 1998; Conforti et al., 2014).

1.2 Assumption ledger

1. **Finite instance closure.** The candidate spaces and plan universe are finite under the construction provided in Part I.
2. **Cost computability.** The cost functional is total on the constructed plan universe.
3. **Certificate checkability.** Every admitted optimisation artefact (bound, cut, pricing closure, pruning decision) has an explicit finite arithmetic certificate.

1.3 Claim discipline protocol

1. **Formal claims.** Every formal claim appears as a *Definition*, *Lemma*, *Theorem*, or *Corollary* with explicit dependencies.
2. **Empirical claims.** Every empirical claim is tied to a frozen benchmark specification, a fairness contract, a metric definition, and a reporting rule (Part VII).
3. **Prohibited forms.** Implicit assumptions, undefined objects, and unqualified performance claims are disallowed.

1.4 Reading map

Part I defines the finite instance objects; Parts II–IV define the exact optimisation formulation and the exactness-preserving solver; Part V defines the learned advisory layer; Part VI provides integrity and certificate verification closure; Part VII seals the empirical evaluation.

2

Physical Design Object Space (Index, View, Partition Candidates)

2.1 Inputs and schema primitives

2.1.1 Finite schema

This chapter fixes a finite input universe for physical design. A schema is a finite tuple

$$S = (T, A, \text{Dom}, \text{Key}, \text{FK}, \text{Ord}),$$

with the following components.

1. T is a finite set of table symbols.
2. A is a function mapping each $t \in T$ to a finite, nonempty set $A(t)$ of attribute symbols.
3. Dom is a function mapping each $a \in A(t)$ to a domain tag $\text{Dom}(a)$. The set of domain tags is finite. The only requirement in Chapter 2 is that domain tags determine type compatibility for constants and attribute equality is decidable.
4. Key is a function mapping each $t \in T$ to a finite family $\text{Key}(t)$ of key sets, where each key set $K \in \text{Key}(t)$ satisfies $K \subseteq A(t)$ and $K \neq \emptyset$.
5. FK is a finite set of foreign key declarations. Each declaration is a pair

$$(t, a) \rightarrow (u, b),$$

with $t, u \in T$, $a \in A(t)$, $b \in A(u)$, and $\text{Dom}(a) = \text{Dom}(b)$.

6. Ord is a fixed total order on the global attribute universe

$$\mathcal{A} := \bigcup_{t \in T} A(t).$$

This order is used to define canonical encodings, deterministic enumeration, and

tie breaking.

Every object introduced later must be definable by finite combinations of symbols from $\mathbf{S}$ and by finite arithmetic on finite inputs.

2.1.2 Workload as a finite multiset of typed expressions

A workload is a finite multiset

$$\mathbf{W} = \{(q_i, w_i)\}_{i=1}^m,$$

where each q_i is a well typed relational algebra expression over $\mathbf{S}$ and each w_i is a nonnegative rational weight. The value m is finite.

Let $\text{RA}(\mathbf{S})$ denote the set of admissible relational algebra expressions generated by the following constructors.

1. Base relation: if $t \in T$, then $\text{Rel}(t) \in \text{RA}(\mathbf{S})$.
2. Selection: if $E \in \text{RA}(\mathbf{S})$ and φ is a well typed predicate over attributes of E , then $\sigma_\varphi(E) \in \text{RA}(\mathbf{S})$.
3. Projection: if $E \in \text{RA}(\mathbf{S})$ and P is a finite set of attributes of E , then $\pi_P(E) \in \text{RA}(\mathbf{S})$.
4. Join: if $E_1, E_2 \in \text{RA}(\mathbf{S})$ and ψ is a well typed join predicate between attributes of E_1 and E_2 , then $E_1 \bowtie_\psi E_2 \in \text{RA}(\mathbf{S})$.
5. Grouping and aggregation: if $E \in \text{RA}(\mathbf{S})$, G is a finite set of attributes of E , and α is a finite list of typed aggregate terms, then $\gamma_{G,\alpha}(E) \in \text{RA}(\mathbf{S})$.

No additional operators are admitted in Chapter 2. Extensions, if any, must be introduced later with explicit canonicalization rules.

2.1.3 Feature extraction primitives

For any $E \in \text{RA}(\mathbf{S})$, define the following finite feature sets.

1. $\text{Attr}(E)$, the set of attributes referenced by E .
2. $\text{Filt}(E)$, the set of selection atoms in E . Each atom is a triple $(a, \odot, c)$ where a is an attribute, $\odot$ is a comparator symbol from a fixed finite comparator alphabet, and c is a constant type compatible with a .
3. $\text{Join}(E)$, the set of join atoms in E . Each atom is a pair (a, b) of attributes with $\text{Dom}(a) = \text{Dom}(b)$ that is admissible under FK.
4. $\text{Proj}(E)$, the set of projected attributes at the top level if E contains a top level projection. If no top level projection is present, set $\text{Proj}(E) = \text{Attr}(E)$.

Each feature set is computable by a finite traversal of the expression tree.

2.2 Candidate index space and canonical representation

2.2.1 Admissible index object

An index candidate is a tuple

$$I = (t, K, U, S, \rho),$$

with components defined as follows.

1. $t \in T$ is the target table.
2. $K = (k_1, \dots, k_r)$ is a finite sequence of distinct attributes $k_j \in A(t)$, called the key sequence, where $1 \leq r \leq R_I$ and R_I is a fixed global bound.
3. $U \subseteq A(t) \setminus \{k_1, \dots, k_r\}$ is a finite include set, where $|U| \leq U_I$ and U_I is a fixed global bound.
4. S is a finite predicate support descriptor derived from workload selection atoms on table t . It is represented as a finite set of normalized atoms and may be empty.
5. ρ is an index mode tag drawn from a fixed finite tag alphabet.

The object I is a formal design atom. Its operational interpretation is provided later by feasibility and plan semantics.

2.2.2 Workload grounded generation rule

Fix a deterministic ranking rule Rank_t that orders attributes by decreasing workload support and breaks ties by Ord . For each table $t \in T$, define the workload induced attribute multiset

$$F_t = \bigcup_{i=1}^m \left(\{a \in A(t) : a \text{ appears in } \text{Filt}(q_i) \cup \text{Join}(q_i) \cup \text{Proj}(q_i)\} \right).$$

Let B_t be the set containing the top M_I attributes under Rank_t , where M_I is a fixed global bound.

The generator enumerates every key sequence K of length at most R_I formed from distinct attributes in B_t , and every include set U of size at most U_I formed from $B_t \setminus K$. The support descriptor S is chosen from a finite family derived from the normalized selection atoms on t , with bounded cardinality controlled by a fixed bound S_I . The mode tag ρ ranges over a fixed finite alphabet.

2.2.3 Canonical representation of index objects

Define the canonicalization map can_I by the following rules.

1. The key sequence K is retained as an ordered sequence. The generator must enumerate candidate key sequences deterministically.

2. The include set U is represented by sorting its elements under Ord and storing the result as a set.
3. Each atom in S is normalized by canonical constant encoding, canonical comparator encoding, and deterministic ordering under Ord . The descriptor S is then stored as a sorted set under the induced lexicographic order.
4. The tag ρ is already canonical by definition of its alphabet.

Two index candidates are identical if and only if their canonical encodings are identical as finite strings under the fixed encoding induced by Ord .

2.3 Candidate materialized view space and canonical representation

2.3.1 Admissible view object

A materialized view candidate is a tuple

$$V = (E, \theta, \mu),$$

where $E \in \text{RA}(\mathbf{S})$ is an admissible expression satisfying fixed structural bounds, θ is a finite key descriptor derived from attributes of E , and μ is a maintenance tag drawn from a fixed finite alphabet.

Let J_V , P_V , and G_V be fixed global bounds such that E contains at most J_V base relations in its join backbone, at most P_V projected attributes at the top level, and at most G_V grouping keys.

2.3.2 Workload grounded generation rule

For each workload query q_i , define a finite family $\text{Sub}(q_i)$ of admissible subexpressions obtained by selecting connected join substructures of size at most J_V , applying selection pushdown where typing permits, restricting projections to size at most P_V , and restricting grouping keys to size at most G_V when aggregation is present.

The generator closes this family under a finite set of transforms, each of which is bounded by fixed parameters. These transforms include restricted projection choices that preserve predicate and join attributes, predicate support normalization, and bounded key descriptor choices.

2.3.3 Canonical representation of view objects

Define the canonicalization map $\text{can}_V : \text{RA}(\mathbf{S}) \rightarrow \text{RA}_{\text{can}}(\mathbf{S})$ by enforcing the following normal forms.

1. Selection normal form: selection predicates are pushed downward where typing permits, expressed as a conjunction of normalized atoms, and ordered deterministically under Ord .

2. Projection normal form: projected attributes are represented as a duplicate free list sorted under Ord. Redundant projections are eliminated.
3. Join normal form: join backbones are represented by a canonical join tree derived from deterministic ordering of base relations and a fixed association rule.
4. Attribute naming normal form: aliasing is eliminated by deterministic alpha renaming derived from Ord.
5. Grouping normal form: group keys are sorted under Ord and aggregate term lists are ordered by a deterministic textual encoding.

Define $\text{can}_V(V)$ by replacing E with $\text{can}_V(E)$, sorting θ under Ord, and keeping μ unchanged. Equality of view candidates is equality of canonical encodings.

2.4 Candidate partition scheme space and exclusivity groups

2.4.1 Admissible partition object

A partition scheme candidate is a tuple

$$P = (t, a, \mathcal{B}, \kappa, \eta),$$

where $t \in T$ is the target table, $a \in A(t)$ is the partition attribute, $\mathcal{B} = (b_0, \dots, b_K)$ is a strictly increasing boundary sequence with $K \leq K_P$ for a fixed bound K_P , κ is a finite routing key descriptor containing a , and η is a mode tag from a fixed finite alphabet.

2.4.2 Workload grounded boundary extraction

Let $C_{t,a}$ be the multiset of constants occurring in workload selection atoms $(a, \odot, c)$ for attribute $a \in A(t)$. Fix a deterministic boundary constructor $\text{Bound}(t, a)$ that selects at most M_P distinct constants from $C_{t,a}$ in deterministic order consistent with the domain tag.

The generator enumerates every strictly increasing boundary sequence $\mathcal{B}$ of length at most $K_P + 1$ whose entries are drawn from $\text{Bound}(t, a)$. Each $\mathcal{B}$ determines a partition candidate for (t, a) .

2.4.3 Exclusivity groups

Partition schemes are mutually exclusive per base table. For each $t \in T$, define the exclusivity group

$$\mathcal{P}(t) := \{P \in \mathcal{P} : \text{target}(P) = t\}.$$

Later optimization models enforce the constraint

$$\sum_{P \in \mathcal{P}(t)} x_P \leq 1,$$

where x_P is the binary decision variable indicating selection of P .

2.5 Finite generation theorem and closure properties

2.5.1 Finiteness of candidate object spaces

Fix finite global parameters

$$(R_I, U_I, M_I, S_I, J_V, P_V, G_V, K_P, M_P),$$

all of which are natural numbers. Let $\mathcal{I}$, $\mathcal{V}$, $\mathcal{P}$ be the outputs of the generators in Sections 2.2, 2.3, and 2.4, after canonicalization and deduplication in Section 2.6.

Theorem 1.5.1 (Finiteness of candidate object spaces). The sets $\mathcal{I}$, $\mathcal{V}$, and $\mathcal{P}$ are finite.

Proof. Fix $t \in T$. The set B_t used for index enumeration is finite with $|B_t| \leq M_I$. The number of distinct key sequences of length at most R_I over distinct elements of B_t is bounded by

$$\sum_{r=1}^{R_I} |B_t| \cdot (|B_t| - 1) \cdots (|B_t| - r + 1),$$

and is therefore finite. For each such K , the number of include sets U of size at most U_I drawn from $B_t \setminus K$ is bounded by

$$\sum_{u=0}^{U_I} \binom{|B_t| - |K|}{u},$$

and is therefore finite. The support descriptor S is chosen from a finite family of normalized atoms with bounded size S_I . Consequently the number of index candidates targeting t is finite, and since T is finite, $\mathcal{I}$ is finite.

For views, each q_i is a finite expression tree. The number of connected join substructures of bounded size at most J_V is finite, and for each substructure the number of bounded projections and bounded grouping choices is finite. The closure transforms of Section 2.3.2 are finite and bounded. Canonicalization does not increase cardinality. Hence $\mathcal{V}$ is finite.

For partitions, for each (t, a) the boundary set $\text{Bound}(t, a)$ is finite with size at most M_P . The number of strictly increasing sequences of bounded length at most $K_P + 1$ drawn from a finite set is finite. Hence $\mathcal{P}$ is finite. This completes the proof. $\square$

2.5.2 Closure properties

The generated candidate sets satisfy the following closure conditions.

1. Canonical closure: if an object is generated in any raw form, its canonical form is an element of the candidate set.

2. Workload support closure: every candidate admits a finite witness derived from **W**. For indexes the witness is an attribute occurrence within workload features on the target table. For views the witness is a bounded subexpression derived from at least one workload query. For partitions the witness is a boundary constant derived from workload selection constants, unless a designated default boundary set is explicitly permitted.
3. Parameter bound closure: no candidate violates the fixed bounds stated in this chapter.

Each closure condition is decidable by finite checks.

2.6 Deduplication and equivalence classes

2.6.1 Equivalence relations by canonical encoding

Define equivalence relations $\sim_I$, $\sim_V$, and $\sim_P$ by equality of canonical encodings.

1. For index candidates I_1, I_2 , define $I_1 \sim_I I_2$ if and only if $\text{can}_I(I_1) = \text{can}_I(I_2)$.
2. For view candidates V_1, V_2 , define $V_1 \sim_V V_2$ if and only if $\text{can}_V(V_1) = \text{can}_V(V_2)$.
3. For partition candidates P_1, P_2 , define $P_1 \sim_P P_2$ if and only if their canonical encodings coincide, where partition canonicalization normalizes constants, enforces strict boundary ordering, sorts routing descriptors under **Ord**, and preserves the mode tag.

Each relation is an equivalence relation because it is induced by equality of finite canonical encodings.

2.6.2 Deduplication operator

Let enc be the fixed finite string encoding induced by **Ord**. Define signature maps

$$\sigma_I(I) = \text{enc}(\text{can}_I(I)), \quad \sigma_V(V) = \text{enc}(\text{can}_V(V)), \quad \sigma_P(P) = \text{enc}(\text{can}_P(P)).$$

Deduplication retains exactly one representative per signature under a deterministic enumeration order.

2.6.3 Deduplication correctness

Lemma 1.6.3 (Uniqueness of representatives). If I_1, I_2 are distinct elements retained by index deduplication, then $I_1 \not\sim_I I_2$. The analogous statements hold for views and partitions.

Proof. Distinct retained objects must have distinct signatures by definition of deduplication. Distinct signatures imply distinct canonical encodings, hence the objects are not equivalent. $\square$

Figure 1.1. Candidate generation pipeline.

Input (S, W) is transformed into finite workload features, then into raw candidates, then into canonical forms, then deduplicated into the final finite candidate sets $\mathcal{I}$, $\mathcal{V}$, and $\mathcal{P}$. The bounding parameters $R_I, U_I, M_I, S_I, J_V, P_V, G_V, K_P, M_P$ enforce finiteness by construction.

Figure 2.1: Candidate generation pipeline for index, view, and partition candidates.**Table 2.1:** Candidate space summary by benchmark family and budget regime. Values are to be filled from the frozen benchmark charter and candidate generator outputs.

Benchmark family	Budget regime	$ \mathcal{I} $	$ \mathcal{V} $	$ \mathcal{P} $
OLTP-HighWrite	Small	to be filled	to be filled	to be filled
OLTP-HighWrite	Medium	to be filled	to be filled	to be filled
OLTP-HighWrite	Large	to be filled	to be filled	to be filled
OLAP-Star	Small	to be filled	to be filled	to be filled
OLAP-Star	Medium	to be filled	to be filled	to be filled
OLAP-Star	Large	to be filled	to be filled	to be filled
HTAP-Mixed	Small	to be filled	to be filled	to be filled
HTAP-Mixed	Medium	to be filled	to be filled	to be filled
HTAP-Mixed	Large	to be filled	to be filled	to be filled
Join-Heavy	Small	to be filled	to be filled	to be filled
Join-Heavy	Medium	to be filled	to be filled	to be filled
Join-Heavy	Large	to be filled	to be filled	to be filled

Lemma 1.6.4 (Coverage of equivalence classes). For every raw generated index candidate I , there exists a retained representative $I^\star$ such that $I \sim_I I^\star$. The analogous statements hold for views and partitions.

Proof. Fix a raw object I with signature $s = \sigma_I(I)$. Deduplication retains the first occurrence of signature s under the deterministic enumeration order. That retained object $I^\star$ satisfies $\sigma_I(I^\star) = s$, hence $\text{can}_I(I^\star) = \text{can}_I(I)$, hence $I \sim_I I^\star$. $\square$

End of Chapter Artifacts

3

Query and Plan Universe Definition

This chapter fixes a single workload query r and defines, from first principles, a finite and mechanically enumerable universe of admissible physical plans for r . The construction consists of five components. First, a finite logical query language is fixed together with a terminating syntactic normalization. Second, a finite physical operator grammar is defined in a query-shaped manner. Third, plan feasibility is defined as an explicit set of required physical design objects. Fourth, a finiteness theorem is proved for the resulting plan universe. Fifth, a canonical encoding is defined so that each plan admits a unique deterministic code.

3.1 Workload query language and normalization

3.1.1 Primitive finite data

Fix a single query r . The following objects are given and finite.

1. A finite set of base tables $T = \{t_1, \dots, t_\tau\}$.
2. For each $t \in T$, a finite set of candidate indexes I_t .
3. For each $t \in T$, a finite nonempty set of candidate partition schemes P_t , so $P_t \neq \emptyset$.
4. A finite set of candidate materialized views V , where each $v \in V$ has a definition $\text{def}(v)$ written in the logical query language of Section 3.1.2.
5. A finite set of join method labels $J = \{\text{NL}, \text{HJ}, \text{MJ}\}$.
6. A finite set of distribution labels $D = \{\text{CoPart}, \text{BcastL}, \text{BcastR}, \text{Shuffle}\}$.
7. A finite set of aggregate method labels $A = \{\text{HashAgg}, \text{SortAgg}\}$.
8. A finite set of sort method labels $S_{\text{sort}} = \{\text{MemSort}, \text{ExtSort}\}$.

3.1.2 Logical query language

Definition 2.1.1 (Atoms and finite queries). A query atom is a symbol t with $t \in T$. A logical query term q is any finite rooted tree term generated by the constructors

$$\text{Scan}(t), \quad \text{Sel}(\phi, q), \quad \text{Proj}(C, q), \quad \text{Join}(\psi, q_1, q_2), \quad \text{Agg}(G, F, q), \quad \text{Sort}(K, q), \quad \text{Limit}(k, q),$$

where $t \in T$, $k \in \mathbb{N}$, and ϕ, ψ, C, G, F, K denote predicate and attribute symbols whose admissible instances are determined by the fixed query r . A query term is finite if its syntax tree has finitely many nodes.

Definition 2.1.2 (Size). For a finite query term q , let $|q|$ denote the number of nodes in its syntax tree. Fix r and write $|r| = m$.

3.1.3 Normalization and join core

Definition 2.1.3 (Join core). Define $\text{Core}(q)$ recursively:

1. If $q = \text{Scan}(t)$, then $\text{Core}(q) = \text{Scan}(t)$.
2. If $q = \text{Sel}(\phi, q'), \text{Proj}(C, q'), \text{Sort}(K, q'), \text{Limit}(k, q'),$ or $\text{Agg}(G, F, q')$, then $\text{Core}(q) = \text{Core}(q')$.
3. If $q = \text{Join}(\psi, q_1, q_2)$, then $\text{Core}(q) = \text{Join}(\psi, \text{Core}(q_1), \text{Core}(q_2))$.

Lemma 2.1.4 (Core finiteness and size bound). For every finite query q , $\text{Core}(q)$ is finite and $|\text{Core}(q)| \leq |q|$.

Proof. Let q be finite. The mapping $\text{Core}(\cdot)$ removes only unary nodes and preserves scan and join nodes, hence it cannot increase node count and it cannot introduce new nodes. Therefore $|\text{Core}(q)| \leq |q|$ and $\text{Core}(q)$ is finite. $\square$

Definition 2.1.5 (Leaf multiset and leaf count). Let $\text{Leaves}(q)$ denote the multiset of table symbols occurring at $\text{Scan}(\cdot)$ leaves of q . Define

$$k_r := |\text{Leaves}(\text{Core}(r))|.$$

By Lemma 2.1.4, $k_r \leq |r|$.

3.2 Physical operator set and plan grammar

3.2.1 View matchability and admissible placements

Definition 2.2.1 (View core). For $v \in V$, define $\text{Core}(\text{def}(v))$ and set

$$k_v := |\text{Leaves}(\text{Core}(\text{def}(v)))|.$$

Definition 2.2.2 (Core leaf positions). Enumerate the scan leaves of $\text{Core}(r)$ as positions

$$\text{Pos}(r) = \{1, 2, \dots, k_r\}.$$

Each position $i \in \text{Pos}(r)$ has a table label $\text{tab}_r(i) \in T$. For $v \in V$, define $\text{Pos}(v) = \{1, \dots, k_v\}$ with leaf labels $\text{tab}_v(j) \in T$.

Definition 2.2.3 (Syntactic view match). A match of v into r is an injective function $f : \text{Pos}(v) \rightarrow \text{Pos}(r)$ such that

$$\text{tab}_v(j) = \text{tab}_r(f(j)) \quad \text{for all } j \in \text{Pos}(v).$$

Let $\text{Match}(v, r)$ be the set of all such matches.

Lemma 2.2.4 (Match finiteness). For each $v \in V$, the set $\text{Match}(v, r)$ is finite.

Proof. Both $\text{Pos}(v)$ and $\text{Pos}(r)$ are finite. The set of all functions $\text{Pos}(v) \rightarrow \text{Pos}(r)$ is finite, and $\text{Match}(v, r)$ is a subset defined by injectivity and label preservation. Therefore $\text{Match}(v, r)$ is finite. $\square$

Definition 2.2.5 (View placement tokens). A view placement token is a pair (v, f) with $v \in V$ and $f \in \text{Match}(v, r)$. Define the finite token set

$$U_r := \{(v, f) : v \in V, f \in \text{Match}(v, r)\}.$$

Definition 2.2.6 (Non-overlap). Two tokens (v, f) and (v', f') overlap if $\text{Im}(f) \cap \text{Im}(f') \neq \emptyset$.

Definition 2.2.7 (View cover). A cover for r is any set $C \subseteq U_r$ such that no two distinct tokens in C overlap. Let $\text{Cover}(r)$ denote the set of all covers.

Lemma 2.2.8 (Cover finiteness). The set $\text{Cover}(r)$ is finite.

Proof. The token set U_r is finite. Every cover is a subset of U_r , hence $\text{Cover}(r) \subseteq \mathcal{P}(U_r)$, and $\mathcal{P}(U_r)$ is finite. Therefore $\text{Cover}(r)$ is finite. $\square$

3.2.2 Leaf access modes and leaf terms

Definition 2.2.9 (Access modes). For $t \in T$, define

$$\text{AM}(t) := \{\text{TableScan}\} \cup \{\text{IndexScan}(i) : i \in I_t\}.$$

For $v \in V$, set $\text{AM}(v) := \{\text{ViewScan}\}.$

Definition 2.2.10 (Leaf terms). A leaf term is either

$$\text{LeafTable}(t, p, a) \quad \text{with } t \in T, p \in P_t, a \in \text{AM}(t),$$

or

$$\text{LeafView}(v) \quad \text{with } v \in V.$$

3.2.3 Join nodes and canonical root unary placement

Definition 2.2.11 (Join node). A join node is $\text{JoinNode}(j, d, \cdot, \cdot)$ with $j \in J$ and $d \in D$ and two child subplans.

Definition 2.2.12 (Canonical unary placement). Unary operators from r are placed canonically as follows.

1. If r contains aggregation, exactly one aggregation operator occurs at the plan root with method chosen from A .
2. If r contains sorting, exactly one sorting operator occurs above aggregation if both are present, with method chosen from S_{sort} .
3. If r contains a limit, it occurs outermost at the root, above sorting if present, and its parameter is fixed by r .
4. Selection predicates from r are attached only at leaves.

Definition 2.2.13 (Root option set). Let $\text{RootOpt}(r)$ be the finite set of tuples encoding the root unary method choices dictated by Definition 2.2.12. If aggregation is present, it contributes one component in A ; if sorting is present, it contributes one component in S_{sort} ; otherwise the corresponding component is absent. Limit parameters are fixed by r . Hence $\text{RootOpt}(r)$ is finite.

3.3 Plan feasibility semantics as object requirements

3.3.1 Leaves induced by a cover

Definition 2.3.1 (Leaves induced by a cover). Fix $C \in \text{Cover}(r)$. Define a multiset of abstract leaves Y_C by the following rules.

1. For each $(v, f) \in C$, include one abstract view leaf symbol $[v, f]$.
2. For each $i \in \text{Pos}(r)$ not contained in $\bigcup_{(v, f) \in C} \text{Im}(f)$, include one abstract base leaf symbol $[t, i]$ where $t = \text{tab}_r(i)$.

Write $k_C := |Y_C|$. Then $1 \leq k_C \leq k_r$.

3.3.2 Allowed instantiations for leaves

Definition 2.3.2 (Allowed instantiations). For each base leaf $[t, i] \in Y_C$, the allowed instantiations are exactly the finite set

$$\{\text{LeafTable}(t, p, a) : p \in P_t, a \in \text{AM}(t)\}.$$

For each view leaf $[v, f] \in Y_C$, the allowed instantiation is $\{\text{LeafView}(v)\}$.

3.3.3 Binary plan trees and plan terms

Definition 2.3.3 (Binary plan trees). A binary plan tree over Y_C is a full binary tree whose leaves are exactly the elements of Y_C , each appearing once, and whose internal nodes are labeled by pairs $(j, d) \in J \times D$. Let $\text{Tree}(Y_C)$ denote the set of all such instantiated labeled trees.

Definition 2.3.4 (Plan term). A plan term for query r is a triple

$$\ell = (C, \tau, \rho)$$

such that $C \in \text{Cover}(r)$, $\tau \in \text{Tree}(Y_C)$ with leaves instantiated according to Definition 2.3.2, and $\rho \in \text{RootOpt}(r)$.

3.3.4 Requirement sets

Let the set of candidate physical design objects be

$$S := \left(\bigcup_{t \in T} I_t \right) \cup \left(\bigcup_{t \in T} P_t \right) \cup V.$$

This set is finite.

Definition 2.3.5 (Requirement set). For $\ell = (C, \tau, \rho)$, define $R_{r\ell} \subseteq S$ as the union of all objects referenced in τ :

1. every index i appearing in any $\text{IndexScan}(i)$ leaf of τ ;
2. every partition scheme p appearing in any $\text{LeafTable}(t, p, a)$ leaf of τ ;
3. every view v appearing in any $\text{LeafView}(v)$ leaf of τ .

Lemma 2.3.6 (Requirement set finiteness). For each ℓ , the set $R_{r\ell}$ is well defined, finite, and satisfies $R_{r\ell} \subseteq S$.

Proof. The plan tree τ has finitely many leaves. Each leaf references only finitely many objects, each of which belongs to S by construction. Therefore the union of referenced objects is finite and is a subset of S . $\square$

3.4 Plan universe finiteness theorem

Definition 2.4.1 (Plan universe). Define the plan universe for query r as

$$L_r := \{\ell : \ell \text{ is a plan term for } r \text{ in the sense of Definition 2.3.4}\}.$$

Lemma 2.4.2 (Finite covers). The set $\text{Cover}(r)$ is finite.

Proof. This is Lemma 2.2.8. □

Lemma 2.4.3 (Finite induced leaves). For each $C \in \text{Cover}(r)$, the multiset Y_C is finite.

Proof. The set $\text{Pos}(r)$ is finite. The cover C is finite as a subset of a finite set. The multiset Y_C is constructed from finitely many tokens and finitely many uncovered positions. Hence Y_C is finite. □

Lemma 2.4.4 (Finite binary tree shapes with fixed leaf count). For each $k \in \mathbb{N}$ with $k \geq 1$, the set of full binary tree shapes with exactly k leaves is finite.

Proof. Proceed by induction on k . For $k = 1$ there is exactly one shape. For $k \geq 2$, any full binary tree with k leaves has a root whose left subtree has i leaves and right subtree has $k - i$ leaves for some $i \in \{1, \dots, k - 1\}$. Thus the set of shapes with k leaves is a subset of a finite union of products of the sets of shapes with fewer than k leaves. The induction hypothesis implies finiteness. □

Lemma 2.4.5 (Finite instantiated trees). For each $C \in \text{Cover}(r)$, the set $\text{Tree}(Y_C)$ is finite.

Proof. Let $k_C = |Y_C|$, which is finite by Lemma 2.4.3. There are finitely many full binary shapes with k_C leaves by Lemma 2.4.4. For any fixed shape there are finitely many leaf labelings, finitely many internal node labelings because $J \times D$ is finite, and finitely many leaf instantiations by Definition 2.3.2. Therefore each shape yields finitely many instantiated trees, and the finite union over shapes is finite. □

Lemma 2.4.6 (Finite root options). The set $\text{RootOpt}(r)$ is finite.

Proof. The set is a finite product of finite method sets or singletons dictated by r . Hence it is finite. □

Theorem 2.4.7 (Plan universe finiteness). The plan universe L_r is finite.

Figure 3.1: Plan grammar schematic for query r . The diagram shows the composition from covers C , induced leaves Y_C , instantiated binary join trees τ , and root unary options ρ .

Figure 3.2: Example derivation tree for a fixed query r . The tree explicitly annotates leaf access modes, partition schemes, join methods, and distribution choices, and it includes at least one view leaf induced by a nonempty cover C .

Proof. The set $\text{Cover}(r)$ is finite by Lemma 2.4.2. Fix any $C \in \text{Cover}(r)$. The set $\text{Tree}(Y_C)$ is finite by Lemma 2.4.5, and $\text{RootOpt}(r)$ is finite by Lemma 2.4.6. Thus the set of triples (C, τ, ρ) with $\tau \in \text{Tree}(Y_C)$ and $\rho \in \text{RootOpt}(r)$ is finite. The plan universe L_r is the finite union of these finite sets over all $C \in \text{Cover}(r)$. Therefore L_r is finite. $\square$

3.5 Plan encoding

3.5.1 Derivation tree representation

Each plan $\ell = (C, \tau, \rho) \in L_r$ induces an annotated operator tree whose leaves are the instantiated leaf terms of τ , whose internal nodes are labeled by $\text{JoinNode}(j, d, \cdot, \cdot)$, and whose root unary layers are determined by ρ under the canonical placement rules of Definition 2.2.12. This representation exposes every discrete choice, including view usage, join method, distribution choice, partition choice, and access mode.

3.5.2 Canonical plan code

Definition 2.5.1 (Indexing conventions). Fix once and for all deterministic orderings of the finite sets $T, V, J, D, A, S_{\text{sort}}$. For each $t \in T$, fix orderings of I_t and P_t . Fix the ordering of positions $\text{Pos}(r) = \{1, \dots, k_r\}$. Represent each cover C as a sorted list of its tokens (v, f) , ordering tokens by the index of v and then by the lexicographic order of the image tuple $(f(1), \dots, f(k_v))$. Fix a deterministic preorder traversal for binary trees. Fix a deterministic coding of each internal label $(j, d) \in J \times D$ by its index. Fix a deterministic coding of each leaf term, coding $\text{LeafTable}(t, p, a)$ by the indices of t, p , and a , and coding $\text{LeafView}(v)$ by the index of v . Fix a deterministic coding of $\rho \in \text{RootOpt}(r)$ by coding each present method choice by its index and using zero for absent components.

Definition 2.5.2 (Code string). For $\ell = (C, \tau, \rho) \in L_r$, define $\text{StrCode}_r(\ell)$ as the finite integer sequence obtained by concatenating, in the order below:

1. the encoding of the cover C ;
2. the encoding of the binary tree shape of τ ;
3. the encoding of the leaf labeling of τ ;
4. the encoding of internal node labels (j, d) in preorder;
5. the encoding of instantiated leaf terms in preorder;
6. the encoding of ρ .

Constructor	Required physical objects in $R_{r\ell}$
LeafTable($t, p, \text{TableScan}$)	Requires the partition scheme $p \in P_t$.
LeafTable($t, p, \text{IndexScan}(i)$)	Requires the partition scheme $p \in P_t$ and the index $i \in I_t$.
LeafView(v)	Requires the materialized view $v \in V$.
JoinNode($j, d, \cdot, \cdot$)	Requires no additional physical design object beyond those required by its child subplans.
Root unary operators determined by ρ	Require no additional physical design object beyond those required by the underlying join tree and leaves.

Table 3.1: Plan feasibility rule table defining the requirement set $R_{r\ell}$ as in Definition 2.3.5.

Definition 2.5.3 (Canonical plan code). Let K be an integer strictly larger than every integer symbol that can occur in any position of $\text{StrCode}_r(\ell)$ for any $\ell \in L_r$. Define $\text{PlanCode}_r(\ell)$ to be the natural number whose base K digits are exactly the entries of $\text{StrCode}_r(\ell)$ in the given order. This defines $\text{PlanCode}_r : L_r \rightarrow \mathbb{N}$.

Lemma 2.5.4 (Injectivity). If $\ell_1 \neq \ell_2$ in L_r , then $\text{PlanCode}_r(\ell_1) \neq \text{PlanCode}_r(\ell_2)$.

Proof. Base K numeral representations are unique. If $\text{PlanCode}_r(\ell_1) = \text{PlanCode}_r(\ell_2)$, then $\text{StrCode}_r(\ell_1) = \text{StrCode}_r(\ell_2)$. By Definition 2.5.2 and the fixed conventions, equality of code strings implies equality of the cover component, the tree shape, the leaf labeling, the internal labels, the leaf instantiations, and the root option tuple. Therefore $\ell_1 = \ell_2$, a contradiction. $\square$

Corollary 2.5.5 (Deterministic ordering). The mapping PlanCode_r induces a strict total order on L_r by increasing code value. This supports deterministic duplicate-free enumeration of L_r .

3.5.3 Plan feasibility rule table

4

Cost Model: Well-Definedness and Computability

4.1 Primitive measurable quantities

4.1.1 Statistics, cardinalities, and selectivities

Arithmetic domain and exactness. All quantities consumed by the objective are finite and explicitly represented. Integer-valued quantities lie in $\mathbb{N}$, and all fractional parameters are represented as nonnegative rationals $\mathbb{Q}_{\geq 0}$ in canonical form. Floating-point arithmetic is not part of the model.

Definition 4.1 (Ceiling division). For $a \in \mathbb{N}$ and $b \in \mathbb{N}_{>0}$, define

$$\text{cdiv}(a, b) := \min\{k \in \mathbb{N} : kb \geq a\}.$$

Lemma 4.1 (Existence and uniqueness of cdiv). For all $a \in \mathbb{N}$ and $b \in \mathbb{N}_{>0}$, $\text{cdiv}(a, b)$ exists and is unique.

Proof. Since $b > 0$, the set $\{kb : k \in \mathbb{N}\}$ is unbounded in $\mathbb{N}$; hence there exists k with $kb \geq a$. The defining set is therefore nonempty. By well-ordering of $\mathbb{N}$, every nonempty subset has a least element, which is unique. This proves existence and uniqueness. $\square$

Definition 4.2 (Minimal exponent). Fix $F \in \mathbb{N}_{\geq 2}$. For $R \in \mathbb{N}$, define

$$\text{minpow}(F, R) := \min\{p \in \mathbb{N} : F^p \geq R\}, \quad F^0 = 1, \quad F^{p+1} = F^p \cdot F.$$

Lemma 4.2 (Existence and uniqueness of minpow). For all $F \in \mathbb{N}_{\geq 2}$ and $R \in \mathbb{N}$, $\text{minpow}(F, R)$ exists and is unique.

Proof. If $R = 0$, then $F^0 = 1 \geq 0$, so the defining set is nonempty. If $R \geq 1$, then F^p is unbounded as p increases because $F \geq 2$, hence there exists p with $F^p \geq R$. The defining set is nonempty and contained in $\mathbb{N}$; by well-ordering it has a unique minimum. $\square$

Definition 4.3 (Page accounting). Fix a page size $P_{\text{bytes}} \in \mathbb{N}_{>0}$. For any tuple count $N \in \mathbb{N}$ and tuple width $b \in \mathbb{N}$, define the page count

$$\text{pages}(N, b) := \text{cdiv}(Nb, P_{\text{bytes}}) \in \mathbb{N}.$$

Partition statistics. For each base table t and each admissible partitioning p , let the partition index set be

$$J_{t,p} := \{1, 2, \dots, K_{t,p}\}, \quad K_{t,p} \in \mathbb{N}_{>0}.$$

The instance provides exact per-partition tuple counts and widths:

$$N_{t,p,j} \in \mathbb{N}, \quad b_{t,p,j} \in \mathbb{N}, \quad j \in J_{t,p}.$$

Define per-partition pages by

$$P_{t,p,j} := \text{pages}(N_{t,p,j}, b_{t,p,j}) \in \mathbb{N}.$$

Index statistics. For each candidate index i in the index candidate set I , the instance provides finite discrete parameters

$$h_i \in \mathbb{N}, \quad u_i^{\text{leaf}} \in \mathbb{N}_{>0}, \quad u_i^{\text{data}} \in \mathbb{N}_{>0} \text{ (when applicable)}.$$

These parameters are used only by the explicit arithmetic rules in Section 4.2.

Plan annotations and derived selectivities. Let L_r denote the plan universe for a fixed query r . Each plan $\ell \in L_r$ is a finite rooted tree whose nodes are operator symbols, and each node v carries an output annotation

$$N(v) \in \mathbb{N}, \quad b(v) \in \mathbb{N}.$$

For a unary node v with child u , define the derived selectivity

$$s(v) := \begin{cases} 0, & \text{if } N(u) = 0, \\ \frac{N(v)}{N(u)}, & \text{if } N(u) > 0. \end{cases}$$

For a binary node v with children u_1, u_2 , define the derived selectivity relative to the Cartesian baseline

$$s(v) := \begin{cases} 0, & \text{if } N(u_1)N(u_2) = 0, \\ \frac{N(v)}{N(u_1)N(u_2)}, & \text{if } N(u_1)N(u_2) > 0. \end{cases}$$

These selectivities are definitional consequences of plan annotations; the cost model consumes $N(\cdot)$ and $b(\cdot)$ directly.

4.1.2 Measurement interface and scope

The cost model is a closed functional on finite instance data and finite annotated plans. It does not consult external state, does not invoke a runtime engine, and does not assume caching, concurrency, or physical scheduling. All statistics used by the model are treated as part of the optimization instance; any process used to obtain them is outside the model and must be specified elsewhere if it is relevant to interpretation.

4.2 Operator-level cost functions

4.2.1 Access path costs

Resource signature space. Fix the finite counter set

$$K := \{\text{SR}, \text{RR}, \text{W}, \text{CPU}, \text{NET}\},$$

where SR counts sequential reads (pages), RR counts random reads (pages), W counts writes (pages), CPU counts abstract CPU units (integer operations), and NET counts network bytes. A resource signature is a vector $\sigma \in \mathbb{N}^K$.

Definition 4.4 (Scalarization). Fix nonnegative weights $\omega \in \mathbb{Q}_{\geq 0}^K$. For $\sigma \in \mathbb{N}^K$, define the scalar cost

$$\text{cost}(\sigma) := \sum_{k \in K} \omega_k \sigma_k \in \mathbb{Q}_{\geq 0}.$$

Definition 4.5 (Micro-cost constants). Fix constants

$$\gamma_{\text{scan}}, \gamma_{\text{pred}}, \gamma_{\text{emit}}, \gamma_{\text{hash}}, \gamma_{\text{cmp}} \in \mathbb{N}.$$

Leaf operators. Each node v contributes an own-signature $\text{Own}(v) \in \mathbb{N}^K$, defined by its operator symbol and parameters.

Definition 4.6 (Pruned sequential scan). Let $v = \text{PrunedScan}(t, p, A)$ where $A \subseteq J_{t,p}$ is the accessed partition set. Define

$$\text{Own}(v)_{\text{SR}} := \sum_{j \in A} P_{t,p,j}, \quad \text{Own}(v)_{\text{CPU}} := \gamma_{\text{scan}} \sum_{j \in A} N_{t,p,j},$$

and set the remaining components of $\text{Own}(v)$ to 0.

Definition 4.7 (Index probe). Let $v = \text{IndexProbe}(i, \text{cover})$ with index i , covering flag $\text{cover} \in \{0, 1\}$, and output cardinality $N(v)$. Define

$$L(v) := \text{cdiv}(N(v), u_i^{\text{leaf}}), \quad D(v) := (1 - \text{cover}) \text{cdiv}(N(v), u_i^{\text{data}}),$$

and define

$$\text{Own}(v)_{\text{RR}} := h_i + L(v) + D(v), \quad \text{Own}(v)_{\text{CPU}} := \gamma_{\text{scan}} N(v),$$

with all remaining components set to 0.

Definition 4.8 (View scan). Let $v = \text{ViewScan}(v_0, p, A)$ denote a scan of a materialized view object v_0 under a declared partitioning p and accessed partition set A . Its own-signature is defined by Definition 4.6 with the view's declared partition statistics.

4.2.2 Join and aggregation costs

Unary internal operators. Let v be a unary node with child u .

Definition 4.9 (Filter and projection). For $v = \text{Filter}(u)$ define $\text{Own}(v)_{\text{CPU}} := \gamma_{\text{pred}} N(u)$. For $v = \text{Project}(u)$ define $\text{Own}(v)_{\text{CPU}} := \gamma_{\text{emit}} N(u)$. All other components are 0.

Definition 4.10 (External sort). For $v = \text{Sort}(u, M)$ with memory parameter $M \in \mathbb{N}_{>0}$, define the effective fan-in

$$F := \max(2, M),$$

the input pages $P_{\text{in}} := \text{pages}(N(u), b(u))$, the initial run count

$$R := \max\left(1, \text{cdiv}(P_{\text{in}}, F)\right),$$

and the merge pass count $p := \text{minpow}(F, R)$. Define

$$\text{Own}(v)_{\text{SR}} := (p+1)P_{\text{in}}, \quad \text{Own}(v)_{\text{W}} := (p+1)P_{\text{in}}, \quad \text{Own}(v)_{\text{CPU}} := \gamma_{\text{cmp}}(p+1)N(u),$$

and set the remaining components to 0.

Definition 4.11 (Materialize and exchange). For $v = \text{Materialize}(u)$, let $P_{\text{in}} := \text{pages}(N(u), b(u))$ and define

$$\text{Own}(v)_{\text{SR}} := P_{\text{in}}, \quad \text{Own}(v)_{\text{W}} := P_{\text{in}}, \quad \text{Own}(v)_{\text{CPU}} := \gamma_{\text{emit}} N(u),$$

with remaining components 0. For $v = \text{Exchange}(u)$ define

$$\text{Own}(v)_{\text{NET}} := N(u)b(u), \quad \text{Own}(v)_{\text{CPU}} := \gamma_{\text{emit}} N(u),$$

with remaining components 0.

Binary internal operators. Let v be a binary node with children u_1, u_2 .

Definition 4.12 (Hash join). For $v = \text{HashJoin}(u_1, u_2, M)$ with memory parameter $M \in \mathbb{N}_{>0}$, define

$$P_1 := \text{pages}(N(u_1), b(u_1)), \quad P_2 := \text{pages}(N(u_2), b(u_2)),$$

$$P_B := \min(P_1, P_2), \quad P_P := \max(P_1, P_2), \quad G := \max(2, M).$$

Define the number of partition rounds

$$q := \max\left(0, \text{minpow}(G, P_B) - 1\right).$$

Define

$$\text{Own}(v)_{\text{SR}} := (q + 1)(P_1 + P_2), \quad \text{Own}(v)_{\text{W}} := q(P_1 + P_2),$$

$$\text{Own}(v)_{\text{CPU}} := \gamma_{\text{hash}}(N(u_1) + N(u_2)) + \gamma_{\text{emit}}N(v),$$

and set remaining components to 0.

Definition 4.13 (Nested-loop join). For $v = \text{NLJoin}(u_1, u_2)$ define

$$\text{Own}(v)_{\text{CPU}} := \gamma_{\text{cmp}} N(u_1)N(u_2) + \gamma_{\text{emit}}N(v),$$

and set all non-CPU components to 0.

4.2.3 Maintenance and refresh costs

Maintenance costs are part of the same closed arithmetic model and must be explicitly stated.

Definition 4.14 (Index build cost). For an index i built over a base relation or view output with tuple count $N_i \in \mathbb{N}$ and tuple width $b_i \in \mathbb{N}$, define build pages $P_i := \text{pages}(N_i, b_i)$ and define the build own-signature

$$\text{Own}(\text{BuildIndex}(i))_{\text{SR}} := P_i, \quad \text{Own}(\text{BuildIndex}(i))_{\text{W}} := P_i, \quad \text{Own}(\text{BuildIndex}(i))_{\text{CPU}} := \gamma_{\text{cmp}}N_i.$$

All remaining components are 0.

Definition 4.15 (Materialized view refresh cost). Let v_0 be a materialized view whose refresh plan is an annotated plan ℓ_{v_0} . Define

$$\Sigma(\ell_{v_0}) \in \mathbb{N}^K \text{ as in Section 4.3,}$$

and define the refresh signature by

$$\Sigma_{\text{refresh}}(v_0) := \Sigma(\ell_{v_0}) + (\mathbf{0}_{\text{SR}}, \mathbf{0}_{\text{RR}}, P_{v_0}, \mathbf{0}_{\text{CPU}}, \mathbf{0}_{\text{NET}}),$$

where $P_{v_0} \in \mathbb{N}$ is the declared number of pages written when installing the refreshed view state. The refresh scalar cost is $\text{cost}(\Sigma_{\text{refresh}}(v_0))$.

4.3 Plan-level aggregation

4.3.1 Query-level aggregation

Definition 4.16 (Node signatures and plan signature). For an annotated plan ℓ and node v in ℓ , define the recursive node signature

$$\text{Sig}_\ell(v) := \left(\sum_{u \in \text{children}(v)} \text{Sig}_\ell(u) \right) + \text{Own}(v),$$

where $\text{Own}(v)$ is given by Sections 4.2.1–4.2.3. Let ρ be the root of ℓ . Define the plan signature

$$\Sigma(\ell) := \text{Sig}_\ell(\rho) \in \mathbb{N}^K,$$

and the plan cost

$$C_{r,\ell} := \text{cost}(\Sigma(\ell)) \in \mathbb{Q}_{\geq 0}.$$

4.3.2 Workload-level aggregation

Definition 4.17 (Workload objective). Let the workload be a finite multiset of queries $\mathcal{R}$ with weights $\alpha_r \in \mathbb{Q}_{\geq 0}$. For each $r \in \mathcal{R}$ let L_r be its finite plan universe. Define the workload cost functional

$$C(\mathcal{R}) := \sum_{r \in \mathcal{R}} \alpha_r \min_{\ell \in L_r} C_{r,\ell}.$$

This definition is purely semantic: it defines the objective value induced by the declared plan universes and declared weights. No additional calibration step is permitted inside the model.

4.4 Formal properties

4.4.1 Totality and determinism

Theorem 4.1 (Totality and uniqueness). *For every query r and every annotated plan $\ell \in L_r$, the signature $\Sigma(\ell)$ and the scalar cost $C_{r,\ell}$ exist and are unique.*

Proof. Each operator own-signature $\text{Own}(v)$ is defined by a finite arithmetic expression over instance constants and node annotations. By Lemmas 4.1 and 4.2, all uses of `cdiv` and `minpow` are well defined and unique; all remaining arithmetic operations are standard and single-valued. Since ℓ is a finite rooted tree, the recursion in Definition 4.16 terminates and assigns a unique signature to each node, hence to the root. Scalarization by Definition 4.4 is a finite sum of unique terms, hence $C_{r,\ell}$ exists and is unique. $\square$

Theorem 4.2 (Determinism). *Fix the instance parameters and statistics. For any query r and plan $\ell \in L_r$, the value $C_{r,\ell}$ is a deterministic function of the finite input (r, ℓ) .*

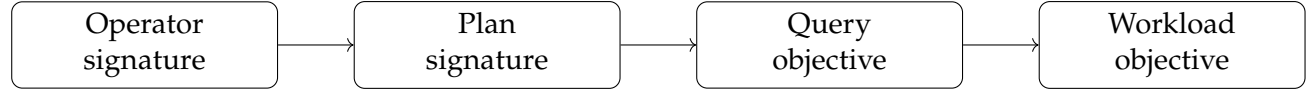


Figure 4.1: Cost decomposition from operator signatures to workload objective.

Symbol	Meaning
$\mathbb{N}, \mathbb{Q}_{\geq 0}$	Natural numbers; nonnegative rationals (exact representation).
P_{bytes}	Declared page size in bytes.
$\text{cdiv}(a, b)$	Ceiling division, Definition 4.1.
$\text{minpow}(F, R)$	Minimal exponent, Definition 4.2.
$\text{pages}(N, b)$	Page count for N tuples of width b , Definition 4.3.
K	Resource counter set $\{\text{SR}, \text{RR}, \text{W}, \text{CPU}, \text{NET}\}$.
$\sigma \in \mathbb{N}^K$	Resource signature vector.
$\omega \in \mathbb{Q}_{\geq 0}^K$	Nonnegative scalarization weights.
$\text{Own}(v)$	Own-signature contributed by node v .
$\Sigma(\ell)$	Plan signature at the root of plan ℓ .
$C_{r,\ell}$	Scalar cost of plan ℓ for query r .
α_r	Workload weight of query r .

Table 4.1: Cost symbol dictionary (all symbols are finite and explicitly represented).

Proof. Every quantity used in evaluation is an instance constant or a plan annotation field. All functions used are deterministic arithmetic functions with unique outputs. No external state is consulted, and no nondeterministic tie-breaking is used. Therefore the resulting cost is deterministic. $\square$

4.4.2 Nonnegativity and boundedness (as applicable)

Theorem 4.3 (Nonnegativity). *For all r and all $\ell \in L_r$, $C_{r,\ell} \in \mathbb{Q}_{\geq 0}$.*

Proof. Each component of each $\text{Own}(v)$ is defined as a finite expression built from non-negative integers and operations preserving nonnegativity; hence $\text{Own}(v) \in \mathbb{N}^K$. The plan signature is a finite componentwise sum of such vectors, so $\Sigma(\ell) \in \mathbb{N}^K$. The weights satisfy $\omega \in \mathbb{Q}_{\geq 0}^K$, hence $\text{cost}(\Sigma(\ell)) \geq 0$ by Definition 4.4. $\square$

Remark 4.1 (Boundedness). When the plan universe L_r is finite and all instance parameters are finite, the set $\{C_{r,\ell} : \ell \in L_r\}$ is a finite subset of $\mathbb{Q}_{\geq 0}$ and therefore admits both a minimum and a maximum. No stronger boundedness statement is imposed without additional explicit constraints on L_r .

4.5 Figures and tables

5

Workload Weighting and Distribution Models

5.1 Chapter purpose and boundary

This chapter fixes the mathematical meaning of workload weighting and workload uncertainty for database physical design under the standing requirement that every object used in subsequent optimization statements is finite and explicitly specified. The chapter delivers three admissible workload models. The first model is deterministic and fixes a single nonnegative weight vector. The second model is stochastic and is expressed on a finite probability space, either by a finite horizon arrival model or by an explicit finite scenario mixture. The third model is robust adversarial and is expressed by an explicit nonempty uncertainty set of admissible weight vectors. The chapter then states reduction and equivalence results that permit subsequent chapters to treat the workload specification as objective coefficients without ambiguity whenever the objective depends on weights through linear aggregation.

5.2 Standing objects inherited from previous chapters

Let $Q = \{1, 2, \dots, q\}$ be a finite index set of query templates. Let $g : Q \rightarrow \mathbb{R}$ be the per query cost functional induced by the cost model for a fixed candidate design. For each $r \in Q$, write $g_r := g(r)$. This chapter does not alter the definition of g . This chapter introduces only the weighting mechanism that aggregates $(g_r)_{r \in Q}$ into a scalar objective contribution.

Fix a total workload mass $W_{\text{tot}} \in \mathbb{R}_{>0}$. Two conventions are used. The normalized convention sets $W_{\text{tot}} = 1$. The absolute mass convention uses a positive value determined by an external horizon.

5.3 Deterministic workload weighting model

5.3.1 Weight domains

Define the normalized simplex

$$\Delta_q = \left\{ w \in \mathbb{R}^q : w_r \geq 0 \ \forall r \in Q, \sum_{r=1}^q w_r = 1 \right\}. \quad (5.1)$$

Define the mass constrained simplex

$$\Delta_q(W_{\text{tot}}) = \left\{ w \in \mathbb{R}^q : w_r \geq 0 \ \forall r \in Q, \sum_{r=1}^q w_r = W_{\text{tot}} \right\}. \quad (5.2)$$

A deterministic workload model is the selection of a single fixed vector $w \in \Delta_q(W_{\text{tot}})$. The deterministic workload objective contribution is the finite sum

$$\text{WorkCost}(w) = \sum_{r=1}^q w_r g_r. \quad (5.3)$$

5.3.2 Realization map from finite arrival sequences to weights

Fix a horizon $N \in \mathbb{Z}_{\geq 1}$ and define the finite set of length N workload realizations

$$\Omega := Q^N. \quad (5.4)$$

For $\omega = (\omega_1, \dots, \omega_N) \in \Omega$, define the count vector $C(\omega) \in \mathbb{Z}_{\geq 0}^q$ by

$$C_r(\omega) = |\{t \in \{1, \dots, N\} : \omega_t = r\}|. \quad (5.5)$$

Define the realized normalized weights $W(\omega) \in \mathbb{R}^q$ by

$$W(\omega) := \frac{1}{N} C(\omega). \quad (5.6)$$

Proposition 5.1. *For every $\omega \in \Omega$, $W(\omega) \in \Delta_q$.*

Proof. Fix $\omega \in \Omega$. For each $r \in Q$, $C_r(\omega)$ is a cardinality, hence $C_r(\omega) \geq 0$, and therefore $W_r(\omega) = C_r(\omega)/N \geq 0$. The sets $\{t : \omega_t = r\}$ over $r \in Q$ partition $\{1, \dots, N\}$, so $\sum_{r=1}^q C_r(\omega) = N$. Dividing by N yields $\sum_{r=1}^q W_r(\omega) = 1$. This is precisely the definition of Δ_q . $\square$

5.3.3 Deterministic weight selection from finite logs

Let $C \in \mathbb{Z}_{\geq 0}^q$ be a finite log count vector and let $N = \sum_{r=1}^q C_r$. The canonical normalized deterministic weight is

$$w = \frac{1}{N} C \in \Delta_q. \quad (5.7)$$

Under total mass W_{tot} , the canonical mass weight is

$$w = \frac{W_{\text{tot}}}{N} C \in \Delta_q(W_{\text{tot}}). \quad (5.8)$$

These equalities define the deterministic workload weights induced by the log.

5.4 Stochastic weighting model (finite support form)

The stochastic model is defined on a finite probability space and admits exact evaluation of expectations because all sums are finite. Two finite descriptions are provided. The first description uses a horizon N arrival generator conditional on a parameter $p \in \Delta_q$. The second description uses an explicit finite scenario mixture.

5.4.1 Conditional finite horizon arrival generator

Fix $N \in \mathbb{Z}_{\geq 1}$ and let $p \in \Delta_q$. Define a function $P_p : \Omega \rightarrow \mathbb{R}$ by

$$P_p(\omega) = \prod_{t=1}^N p_{\omega_t}, \quad \omega \in \Omega. \quad (5.9)$$

Theorem 5.1. *For every $p \in \Delta_q$, the function P_p is a probability distribution on Ω .*

Proof. Let $p \in \Delta_q$. Each coordinate satisfies $p_r \geq 0$, hence each factor in (5.9) is nonnegative and therefore $P_p(\omega) \geq 0$ for all $\omega \in \Omega$. For normalization, define

$$S_N := \sum_{\omega \in \Omega} \prod_{t=1}^N p_{\omega_t}.$$

The sum factorizes as $S_N = (\sum_{r=1}^q p_r)^N$ by iterated expansion of the product across the N coordinates. Since $\sum_{r=1}^q p_r = 1$, it follows that $S_N = 1$. Hence $\sum_{\omega \in \Omega} P_p(\omega) = 1$. $\square$

5.4.2 Expected realized weights and induced expected objective

For any function $f : \Omega \rightarrow \mathbb{R}$ define the expectation

$$\mathbb{E}_p[f] = \sum_{\omega \in \Omega} f(\omega) P_p(\omega). \quad (5.10)$$

Theorem 5.2. *For every $p \in \Delta_q$ and every $r \in Q$,*

$$\mathbb{E}_p[W_r(\omega)] = p_r. \quad (5.11)$$

Proof. Fix $r \in Q$. For each $t \in \{1, \dots, N\}$ define $I_{t,r}(\omega) = 1$ if $\omega_t = r$ and $I_{t,r}(\omega) = 0$

otherwise. Then

$$C_r(\omega) = \sum_{t=1}^N I_{t,r}(\omega), \quad W_r(\omega) = \frac{1}{N} \sum_{t=1}^N I_{t,r}(\omega).$$

By linearity of finite sums,

$$\mathbb{E}_p[W_r] = \frac{1}{N} \sum_{t=1}^N \mathbb{E}_p[I_{t,r}].$$

Fix t . Summing over Ω and partitioning by the value of ω_t yields

$$\mathbb{E}_p[I_{t,r}] = \sum_{\omega \in \Omega} I_{t,r}(\omega) P_p(\omega) = p_r \left(\sum_{j=1}^q p_j \right)^{N-1} = p_r.$$

Substitution gives $\mathbb{E}_p[W_r] = \frac{1}{N} \sum_{t=1}^N p_r = p_r$. $\square$

Define the realized normalized workload cost

$$\text{Cost}(\omega) = \sum_{r=1}^q W_r(\omega) g_r. \quad (5.12)$$

Corollary 5.1. *For every $p \in \Delta_q$,*

$$\mathbb{E}_p[\text{Cost}] = \sum_{r=1}^q p_r g_r. \quad (5.13)$$

Proof. Expand $\mathbb{E}_p[\text{Cost}] = \sum_{\omega} \sum_r W_r(\omega) g_r P_p(\omega)$, exchange finite sums, and apply Theorem 5.2. $\square$

5.4.3 Unconditional finite mixture model

Let $p^{(1)}, \dots, p^{(J)} \in \Delta_q$ be a finite set of conditional parameters and let $\pi \in \Delta_J$ be mixing weights. Define

$$P(\omega) = \sum_{j=1}^J \pi_j P_{p^{(j)}}(\omega), \quad \omega \in \Omega. \quad (5.14)$$

Theorem 5.3. *The function P is a probability distribution on Ω .*

Proof. Nonnegativity follows from $\pi_j \geq 0$ and $P_{p^{(j)}}(\omega) \geq 0$. For normalization,

$$\sum_{\omega \in \Omega} P(\omega) = \sum_{\omega \in \Omega} \sum_{j=1}^J \pi_j P_{p^{(j)}}(\omega) = \sum_{j=1}^J \pi_j \sum_{\omega \in \Omega} P_{p^{(j)}}(\omega) = \sum_{j=1}^J \pi_j = 1,$$

using Theorem 5.1 for each j . $\square$

Define the unconditional expected weight vector

$$\bar{p} = \sum_{j=1}^J \pi_j p^{(j)} \in \Delta_q. \quad (5.15)$$

Corollary 5.2. *Under P , the expected realized workload cost satisfies*

$$\mathbb{E}[\text{Cost}] = \sum_{r=1}^q \bar{p}_r g_r. \quad (5.16)$$

Proof. Compute $\mathbb{E}[\text{Cost}] = \sum_{\omega} \text{Cost}(\omega)P(\omega)$, substitute the mixture definition, exchange finite sums, apply Corollary 5.1 inside each component, and collect coefficients into $\bar{p}$. $\square$

5.4.4 Scenario form used by the optimization model

Let $\Omega_s = \{1, \dots, K\}$ be a finite scenario index set with probabilities $p_k \in \mathbb{Q}_{\geq 0}$ satisfying $\sum_{k=1}^K p_k = 1$. For each k , let $w^{(k)} \in \Delta_q(W_{\text{tot}})$. The expected workload objective contribution is then

$$\text{StochObj} = \sum_{k=1}^K p_k \sum_{r=1}^q w_r^{(k)} g_r, \quad (5.17)$$

which is a finite rational combination of finite sums.

5.5 Robust adversarial weighting model (uncertainty set)

The robust model specifies a nonempty uncertainty set W of admissible workload weight vectors. An adversary may select any $w \in W$. For a fixed per query cost vector g , the robust objective contribution is the worst case value of the linear functional $w \mapsto \sum_r w_r g_r$.

5.5.1 Nominal weights and perturbation variables

Fix a nominal vector $\bar{w} \in \Delta_q(W_{\text{tot}})$. Introduce deviation variables $\delta \in \mathbb{R}^q$ and define $w = \bar{w} + \delta$.

5.5.2 Linear uncertainty specification with explicit budgets

Fix coordinate bounds $\rho \in \mathbb{R}_{\geq 0}^q$ and a global deviation budget $\Gamma \in \mathbb{R}_{\geq 0}$. Optionally, fix groups $G_1, \dots, G_m \subseteq Q$ with budgets $\Gamma_1, \dots, \Gamma_m \in \mathbb{R}_{\geq 0}$. Introduce auxiliary variables $u \in \mathbb{R}_{\geq 0}^q$.

Define the uncertainty system U in variables (w, δ, u) by the conjunction of the fol-

lowing linear constraints:

$$\sum_{r=1}^q w_r = W_{\text{tot}}, \quad (5.18)$$

$$w_r \geq 0 \quad \forall r \in Q, \quad (5.19)$$

$$w_r = \bar{w}_r + \delta_r \quad \forall r \in Q, \quad (5.20)$$

$$-\rho_r \bar{w}_r \leq \delta_r \leq \rho_r \bar{w}_r \quad \forall r \in Q, \quad (5.21)$$

$$-u_r \leq \delta_r \leq u_r \quad \forall r \in Q, \quad (5.22)$$

$$\sum_{r=1}^q u_r \leq \Gamma, \quad (5.23)$$

$$\sum_{r \in G_j} u_r \leq \Gamma_j \quad \forall j \in \{1, \dots, m\}. \quad (5.24)$$

Define the admissible set by projection:

$$W = \{w \in \mathbb{R}^q : \exists \delta \in \mathbb{R}^q, \exists u \in \mathbb{R}_{\geq 0}^q \text{ such that } (w, \delta, u) \text{ satisfies } U\}. \quad (5.25)$$

Lemma 5.1. *For any $\delta \in \mathbb{R}^q$ and any $\Gamma \geq 0$, the existence of $u \in \mathbb{R}_{\geq 0}^q$ satisfying (5.22) and (5.23) is equivalent to the inequality $\sum_{r=1}^q |\delta_r| \leq \Gamma$.*

Proof. Assume u satisfies (5.22) and (5.23). Then $|\delta_r| \leq u_r$ for each r , hence $\sum_r |\delta_r| \leq \sum_r u_r \leq \Gamma$. Conversely, assume $\sum_r |\delta_r| \leq \Gamma$. Define $u_r = |\delta_r|$. Then $u \geq 0$, (5.22) holds, and $\sum_r u_r = \sum_r |\delta_r| \leq \Gamma$, so (5.23) holds. $\square$

Proposition 5.2. *The uncertainty set W in (5.25) is nonempty.*

Proof. Choose $\delta = 0$ and $u = 0$. Then $w = \bar{w}$ satisfies (5.18) and (5.19) because $\bar{w} \in \Delta_q(W_{\text{tot}})$, and satisfies the remaining constraints because all deviation and budget terms are zero. Hence $\bar{w} \in W$. $\square$

5.5.3 Robust objective contribution

Define

$$\text{RobCost}(g) = \max_{w \in W} \sum_{r=1}^q w_r g_r. \quad (5.26)$$

This maximization is over an explicitly constrained finite dimensional set and is therefore a well defined mathematical object once W is fixed.

5.6 Reduction and equivalence statements (where applicable)

5.6.1 Expectation reduction for linear weighting

Theorem 5.4. *Assume that the objective depends on the workload realization only through linear weighting of a fixed cost vector $(g_r)_{r \in Q}$. Under the conditional arrival model with parameter*

$p \in \Delta_q$,

$$\mathbb{E}_p \left[\sum_{r=1}^q W_r(\omega) g_r \right] = \sum_{r=1}^q p_r g_r. \quad (5.27)$$

Under a finite mixture with parameters $\{p^{(j)}\}_{j=1}^J$ and mixing weights $\pi \in \Delta_J$,

$$\mathbb{E} \left[\sum_{r=1}^q W_r(\omega) g_r \right] = \sum_{r=1}^q \bar{p}_r g_r, \quad \bar{p} = \sum_{j=1}^J \pi_j p^{(j)}. \quad (5.28)$$

Proof. The conditional statement is Corollary 5.1. The mixture statement is Corollary 5.2. $\square$

5.6.2 Robust reduction to a finite scenario set (explicit convex hull form)

Let $w^{(1)}, \dots, w^{(K)} \in \Delta_q(W_{\text{tot}})$ be an explicit finite set of admissible weight vectors and define

$$W_{\text{scen}} = \text{conv}\{w^{(1)}, \dots, w^{(K)}\}. \quad (5.29)$$

Theorem 5.5. For any $g \in \mathbb{R}^q$,

$$\max_{w \in W_{\text{scen}}} \sum_{r=1}^q w_r g_r = \max_{k \in \{1, \dots, K\}} \sum_{r=1}^q w_r^{(k)} g_r. \quad (5.30)$$

Proof. Let $w \in W_{\text{scen}}$. Then $w = \sum_{k=1}^K \alpha_k w^{(k)}$ for some coefficients $\alpha_k \geq 0$ with $\sum_{k=1}^K \alpha_k = 1$. Therefore

$$\sum_{r=1}^q w_r g_r = \sum_{k=1}^K \alpha_k \left(\sum_{r=1}^q w_r^{(k)} g_r \right).$$

A convex combination of real numbers is bounded above by their maximum, hence

$$\sum_{r=1}^q w_r g_r \leq \max_{k \in \{1, \dots, K\}} \sum_{r=1}^q w_r^{(k)} g_r.$$

Equality is achieved by selecting an index k^* that attains the maximum and setting $w = w^{(k^*)}$. $\square$

5.6.3 Robust model specialization to deterministic weighting

If W is a singleton $\{\bar{w}\}$, then (5.26) reduces to the deterministic objective (5.3) with $w = \bar{w}$. This is an identity.

5.7 Chapter closure

The deterministic workload model fixes a single nonnegative weight vector and yields a finite weighted sum objective contribution. The stochastic workload model is defined

on a finite probability space and reduces exactly to deterministic expected weights whenever the objective depends on weights through linear aggregation. The robust adversarial model specifies a nonempty uncertainty set of admissible weights and defines the objective contribution as the worst case value of the linear functional induced by the cost vector. The reduction results establish conditions under which stochastic and robust objectives are algebraically transformable into deterministic or finite scenario forms without loss of meaning.

6

Compatibility and Feasibility Semantics for Indexes, Views, and Partitions

6.1 Partition correctness and exclusivity

6.1.1 Partition admissibility rules

Let T be a finite set of base table symbols. For each $t \in T$, let $A(t)$ denote its finite attribute set, and for each $a \in A(t)$ let $\text{Dom}(a)$ denote its (finite) domain. A row of t is a total function

$$r : A(t) \rightarrow \bigcup_{a \in A(t)} \text{Dom}(a) \quad \text{such that} \quad r(a) \in \text{Dom}(a) \text{ for all } a \in A(t).$$

The stored relation of table t is a finite set of rows R_t , and a database instance is $D := (R_t)_{t \in T}$.

Definition 6.1 (Partition scheme). A partition scheme for a base table t is a triple

$$p = (K_p, k_p, \text{pid}_p),$$

where $K_p \subseteq A(t)$ is the partition key, $k_p \in \mathbb{N}$ with $k_p \geq 1$ is the partition count, and

$$\text{pid}_p : \prod_{a \in K_p} \text{Dom}(a) \rightarrow \{1, 2, \dots, k_p\}$$

is a total function. For any row $r \in R_t$, define $\text{pid}_p(r) := \text{pid}_p(r \upharpoonright_{K_p})$ and the induced partition components

$$R_{t,p,j} := \{r \in R_t : \text{pid}_p(r) = j\}, \quad j \in \{1, \dots, k_p\}.$$

Lemma 6.1 (Disjoint cover property). *For any base table t and any partition scheme p on t ,*

$$R_t = \bigcup_{j=1}^{k_p} R_{t,p,j} \quad \text{and} \quad R_{t,p,i} \cap R_{t,p,j} = \emptyset \text{ for } i \neq j.$$

Proof. Let $r \in R_t$. Since pid_p is total and single-valued with codomain $\{1, \dots, k_p\}$, there exists a unique j such that $\text{pid}_p(r) = j$, hence $r \in R_{t,p,j}$, which proves $R_t = \bigcup_{j=1}^{k_p} R_{t,p,j}$. If $r \in R_{t,p,i} \cap R_{t,p,j}$, then $\text{pid}_p(r) = i$ and $\text{pid}_p(r) = j$, so $i = j$. This proves disjointness. $\square$

6.1.2 Exclusivity constraints per table and group

For each base table $t \in T$, let P_t denote the finite candidate set of partition schemes for t . A physical design must commit to a unique effective partition scheme for each base table.

Definition 6.2 (Partition admissibility and exclusivity). A design is *partition admissible* if it selects exactly one scheme $p_t \in P_t$ for each $t \in T$. Equivalently, for decision variables $\{x_p\}_{p \in P_t}$ with $x_p \in \{0, 1\}$, admissibility is expressed by

$$\sum_{p \in P_t} x_p = 1 \quad \text{for all } t \in T.$$

If the model specifies a partition exclusivity grouping $\mathcal{G}_t$ over P_t , then admissibility is strengthened to require that the selection induces a unique effective scheme per table by enforcing one exclusivity equation per group and enforcing uniqueness of the induced effective scheme.

6.2 Index–partition compatibility

6.2.1 Compatibility predicates

Let an index candidate i target a relation symbol $\text{rel}(i)$, where $\text{rel}(i)$ is either a base table t or a materialised view V . Let K_i denote the finite index key attribute set over the schema of $\text{rel}(i)$.

Definition 6.3 (Abstract index semantics). For any relation $\text{rel}(i)$ with finite row set $R_{\text{rel}(i)}$, an index i denotes a maintained map

$$M_i : \prod_{a \in K_i} \text{Dom}(a) \rightarrow 2^{R_{\text{rel}(i)}},$$

intended to satisfy, for each key value vector u ,

$$M_i(u) = \{r \in R_{\text{rel}(i)} : r \upharpoonright_{K_i} = u\}.$$

Assume a base table t is equipped with a selected partition scheme $p_t = (K_{p_t}, k_{p_t}, \text{pid}_{p_t})$. An index on t may be declared *GLOBAL* or *LOCAL*. *GLOBAL* imposes no routing restriction. *LOCAL* asserts that key-based access is routable to a unique base partition using index key values alone.

Definition 6.4 (LOCAL routing condition). Let i be a *LOCAL* index on a partitioned base table t with selected scheme p_t . A routing function is a total map

$$\text{Route}_{i,p_t} : \prod_{a \in K_i} \text{Dom}(a) \rightarrow \{1, 2, \dots, k_{p_t}\}$$

such that for all $r \in R_t$,

$$\text{Route}_{i,p_t}(r \upharpoonright_{K_i}) = \text{pid}_{p_t}(r).$$

Definition 6.5 (Index-partition compatibility predicate). Let i be an index on base table t and let p_t be the selected partition scheme for t . Define $\text{CompatIdx}(i, p_t) = 1$ if either i is *GLOBAL*, or i is *LOCAL* and $K_{p_t} \subseteq K_i$. Otherwise define $\text{CompatIdx}(i, p_t) = 0$.

6.2.2 Feasibility implications

Theorem 6.1 (Routing equivalence theorem). Let i be declared *LOCAL* on base table t with selected partition scheme p_t . A routing function Route_{i,p_t} satisfying Definition 6.4 exists if and only if $K_{p_t} \subseteq K_i$.

Proof. Assume $K_{p_t} \subseteq K_i$. Define

$$\text{Route}_{i,p_t}(u) := \text{pid}_{p_t}(u \upharpoonright_{K_{p_t}}) \quad \text{for all } u \in \prod_{a \in K_i} \text{Dom}(a).$$

Then for any $r \in R_t$, substituting $u = r \upharpoonright_{K_i}$ yields $\text{Route}_{i,p_t}(r \upharpoonright_{K_i}) = \text{pid}_{p_t}(r)$.

Conversely, assume a routing function exists but $K_{p_t} \not\subseteq K_i$. Choose $a^* \in K_{p_t} \setminus K_i$. Since a^* is not in the index key, there exist rows $r_1, r_2 \in R_t$ with $r_1 \upharpoonright_{K_i} = r_2 \upharpoonright_{K_i}$ and $r_1(a^*) \neq r_2(a^*)$ whenever a^* is not constant on R_t . The relevance of a^* to partitioning implies $\text{pid}_{p_t}(r_1) \neq \text{pid}_{p_t}(r_2)$, hence the routing condition forces

$$\text{Route}_{i,p_t}(r_1 \upharpoonright_{K_i}) = \text{Route}_{i,p_t}(r_2 \upharpoonright_{K_i}) \quad \text{and} \quad \text{pid}_{p_t}(r_1) \neq \text{pid}_{p_t}(r_2),$$

a contradiction. Therefore $K_{p_t} \subseteq K_i$. □

6.3 Materialised view validity and maintenance

6.3.1 View definition validity

A materialised view V is associated with a finite expression e_V over the base tables. Expressions are generated by a finite grammar consisting of base relations, selections, projections, equi-joins, group-by with fixed aggregates, and union. For any database

instance D , the evaluation of an expression e is a finite relation denoted $\llbracket e \rrbracket(D)$ and defined by standard set-theoretic relational semantics.

Definition 6.6 (View correctness). A materialised view V with expression e_V is correct at instance D if its stored relation R_V satisfies

$$R_V = \llbracket e_V \rrbracket(D).$$

6.3.2 Refresh and maintenance semantics

For each selected view V , the design selects a refresh mode $\text{REF}(V) \in \{\text{GLOBAL}, \text{LOCAL}\}$. GLOBAL permits full recomputation of R_V from scratch. LOCAL commits to partition-local refresh: any update affecting exactly one base partition must be refreshable by touching only one corresponding view partition while preserving Definition 6.6.

Let $p = (p_t)_{t \in T}$ denote the selected base table partition schemes. For a view expression e , let $T(e)$ denote the set of base tables referenced by e .

Definition 6.7 (Join coherence). Let t_1, t_2 be base tables joined on an equality predicate $a = b$, where a is sourced from t_1 and b is sourced from t_2 . Let $p_{t_1} = (K_{p_{t_1}}, k_{p_{t_1}}, \text{pid}_{p_{t_1}})$ and $p_{t_2} = (K_{p_{t_2}}, k_{p_{t_2}}, \text{pid}_{p_{t_2}})$ be the selected partition schemes. The pair (p_{t_1}, p_{t_2}) is *join coherent on $a = b$* if

$$a \in K_{p_{t_1}}, \quad b \in K_{p_{t_2}}, \quad k_{p_{t_1}} = k_{p_{t_2}},$$

and for all $x \in \text{Dom}(a) = \text{Dom}(b)$,

$$\text{pid}_{p_{t_1}}(x) = \text{pid}_{p_{t_2}}(x),$$

where $\text{pid}_{p_{t_1}}(x)$ and $\text{pid}_{p_{t_2}}(x)$ denote the induced identifiers computed from the join coordinate value.

Definition 6.8 (Partition-respecting predicate). Fix selected base partitions p and let e be a view expression. Define $\text{PR}(e, p) \in \{0, 1\}$ by structural recursion.

1. If $e = t$ for a base table t , then $\text{PR}(e, p) = 1$.
2. If $e = \sigma_\varphi(e_0)$ or $e = \pi_B(e_0)$, then $\text{PR}(e, p) = \text{PR}(e_0, p)$.
3. If $e = e_1 \bowtie_{a=b} e_2$, then $\text{PR}(e, p) = 1$ if and only if $\text{PR}(e_1, p) = \text{PR}(e_2, p) = 1$ and the selected partition schemes of the base tables sourcing a and b are join coherent on $a = b$.
4. If $e = \gamma_G(e_0)$, then $\text{PR}(e, p) = 1$ if and only if $\text{PR}(e_0, p) = 1$ and for every base table $t \in T(e_0)$ one has $K_{p_t} \subseteq G$.
5. If $e = e_1 \cup e_2$, then $\text{PR}(e, p) = 1$ if and only if $\text{PR}(e_1, p) = \text{PR}(e_2, p) = 1$ and the induced partition identifier spaces coincide on the union schema.

For each identifier j in the common partition count implied by coherence, define $D^{(j)}$ as the instance obtained by restricting each referenced base table $t \in T(e)$ to its component $R_{t, p_t, j}$.

Theorem 6.2 (Partition decomposition theorem). *If $\text{PR}(e, p) = 1$, then*

$$\llbracket e \rrbracket(D) = \bigcup_{j=1}^k \llbracket e \rrbracket(D^{(j)}),$$

where k is the common partition count induced by the selected partition schemes and join coherence obligations.

Proof. By structural induction on e . The base case $e = t$ follows from Lemma 6.1. Selections and projections distribute over unions by pointwise semantics. For joins, join coherence forces any joinable tuple pair to share the same partition identifier, hence each output tuple is produced entirely within a single $D^{(j)}$, and restriction monotonicity yields the reverse inclusion. For group-by, $K_{p_t} \subseteq G$ ensures that groups do not merge across distinct partitions, so aggregation decomposes as a union over j . For union, the claim follows from distributivity and identifier-space alignment. $\square$

Theorem 6.3 (Existence and correctness of LOCAL refresh). *Let V be a materialised view with expression e_V and selected base partitions p . If $\text{REF}(V) = \text{LOCAL}$ and $\text{PR}(e_V, p) = 1$, then there exists a refresh procedure that, for any single-row update affecting exactly one base partition j^* , touches only the corresponding view partition and preserves view correctness:*

$$R_V = \llbracket e_V \rrbracket(D).$$

Proof. Maintain a partitioned representation $\widehat{R}_V = (\widehat{R}_{V,j})_{j=1}^k$ with invariant $\widehat{R}_{V,j} = \llbracket e_V \rrbracket(D^{(j)})$ for all j . If an update affects only partition j^* , then $D^{(j)}$ is unchanged for $j \neq j^*$, so only $\widehat{R}_{V,j^*}$ requires recomputation. Theorem 6.2 yields $\llbracket e_V \rrbracket(D) = \bigcup_{j=1}^k \widehat{R}_{V,j}$, hence global correctness is preserved while modifying only one component. $\square$

6.4 Cross-object compatibility constraints

6.4.1 Joint feasibility constraints

A physical design consists of selected partitions $(p_t)_{t \in T}$, a finite set of selected indexes I^* , a finite set of selected views V^* , and for each $V \in V^*$ a refresh mode $\text{REF}(V)$ and a maintenance admissibility profile $\text{ADM}(V)$.

Definition 6.9 (Feasible design). A design is feasible if and only if all clauses below hold.

- (F1) Unique partition commitment: for every $t \in T$, exactly one $p_t \in P_t$ is selected.
- (F2) Index target dependency: if a selected index targets a view V , then V is selected.
- (F3) Index-partition compatibility: if a selected index i targets a base table t , then $\text{CompatIdx}(i, p_t) = 1$.
- (F4) LOCAL view validity: if V is selected and $\text{REF}(V) = \text{LOCAL}$, then $\text{PR}(e_V, p) = 1$ for the restriction of p to $T(e_V)$.

- (F5) STRONG coupling under LOCAL maintenance: if $\text{REF}(V) = \text{LOCAL}$ and $\text{ADM}(V) = \text{STRONG}$, then each join edge of e_V is supported by at least one selected index whose key contains the relevant join attribute on at least one side, thereby enabling key-based maintenance under the permitted operation set.

6.4.2 Feasibility witness structure

A feasibility witness is the finite data required to certify Definition 6.9: the selected partition scheme per table; for each LOCAL index, the inclusion $K_{p_i} \subseteq K_i$; for each LOCAL view, a derivation that $\text{PR}(e_V, p) = 1$ by recursion over the syntax tree; and, when $\text{ADM}(V) = \text{STRONG}$, an explicit supporting index choice per join edge.

6.5 Feasibility closure theorem

6.5.1 Decidability under the constructed finite instance

Theorem 6.4 (Feasibility closure). *Under the model's finiteness commitments, feasibility as defined in Definition 6.9 is decidable.*

Proof. Each candidate set is finite: $\{P_t\}_{t \in T}$, the index candidate set, and the view candidate set (by bounded-depth finite grammars). Clauses (F1) and (F2) are finite membership and exclusivity checks. Clause (F3) is decidable by Theorem 6.1. Clause (F4) is decidable by finite structural recursion computing $\text{PR}(e_V, p)$. Clause (F5) is decidable by enumerating finitely many join edges and checking existence of a supporting index among finitely many candidates. Therefore feasibility is decidable. $\square$

6.5.2 Witness soundness conditions

Introduce binary variables x_p for each $p \in P_t$, x_i for each index candidate i , and x_V for each view candidate V . For refresh mode, introduce $y_V^{\text{loc}}, y_V^{\text{glob}} \in \{0, 1\}$, and for admissibility introduce $y_V^{\text{weak}}, y_V^{\text{strong}} \in \{0, 1\}$ when admissibility is decision-based. The feasibility predicate is equivalent to satisfaction of a finite family of linear constraints that express (F1) through (F5), including standard linearisation for any conjunction of binary mode flags.

Theorem 6.5 (Witness soundness). *Any variable assignment satisfying the feasibility constraint family induces a feasible design in the sense of Definition 6.9, and any feasible design induces a satisfying assignment under the natural indicator encoding.*

Proof. Soundness follows by direct correspondence between each constraint family and the associated clause (F1) through (F5), using Theorem 6.1 for LOCAL index routing and Theorem 6.3 for LOCAL refresh correctness under $\text{PR}(e_V, p) = 1$. Completeness follows because each clause in Definition 6.9 can be translated into a finite conjunction of linear constraints over the corresponding indicators. $\square$

Clause	Semantic obligation (summary)
(F1)	Exactly one partition scheme is selected for each base table.
(F2)	If an index targets a view, then the target view is selected.
(F3)	Any selected base-table index is compatible with the selected partition scheme. For LOCAL, this is equivalent to $K_{p_i} \subseteq K_i$.
(F4)	Any selected LOCAL view expression is partition-respecting with respect to the selected base partitions, namely $\text{PR}(e_V, p) = 1$.
(F5)	Under LOCAL and STRONG maintenance, each join edge is supported by at least one selected index containing the join attribute on at least one side.

Table 6.1: Feasibility clauses defining cross-object compatibility.

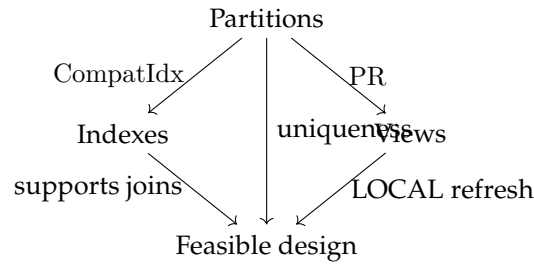


Figure 6.1: Compatibility lattice linking partition selection, index compatibility, view validity, and feasibility.

6.6 Figures and tables

- Compatibility lattice diagram.
- Partition exclusivity diagram.

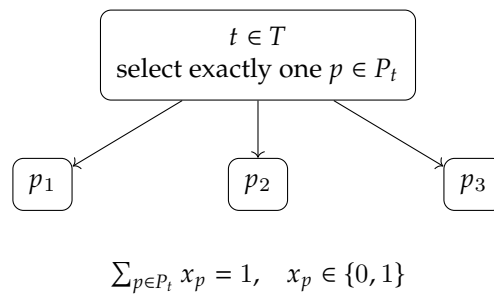


Figure 6.2: Partition exclusivity per base table expressed as a single selection constraint.

7

Baseline Optimization Formulation

This chapter fixes the baseline optimization problem as a fully specified finite mixed-integer linear program. All index sets, coefficients, and feasibility semantics referenced below are treated as deterministic instance data produced by the earlier construction pipeline. The present chapter introduces a single formulation whose feasible solutions correspond exactly to admissible physical designs together with an executable plan choice for every workload query, and whose objective equals workload execution cost plus maintenance cost.

Finite primitives. Let S be the finite set of candidate physical design objects. The set S is the disjoint union

$$S = I \dot{\cup} V \dot{\cup} P,$$

where I is the candidate index set, V is the candidate materialized view set, and P is the candidate partition scheme set. Each partition scheme is associated with exactly one base table. For each base table t , let $P_t \subseteq P$ denote the corresponding exclusivity group.

Let Q be the finite workload query set. For each query $r \in Q$, let L_r be the finite set of admissible plan identifiers for r . Define the disjoint plan index universe

$$L = \bigcup_{r \in Q} (\{r\} \times L_r).$$

For each $r \in Q$ and $\ell \in L_r$, let $\text{Req}(r, \ell) \subseteq S$ be the set of required physical objects for plan ℓ , and let $c_{r\ell} \in \mathbb{Q}_{\geq 0}$ be the deterministic plan execution cost. Let $w_r \in \mathbb{Q}_{\geq 0}$ be the workload weight of query r .

For each $s \in S$, let $a_s \in \mathbb{Q}_{\geq 0}$ be the budget consumption of s and let $B \in \mathbb{Q}_{\geq 0}$ be the total available budget. Let $m_s \in \mathbb{Q}_{\geq 0}$ be the total maintenance coefficient of s under the fixed maintenance semantics, after aggregation over the update distribution.

7.1 Decision variables and domains

Physical design selection. For every $s \in S$, define the binary selection variable

$$x_s \in \{0, 1\}.$$

The intended semantics is $x_s = 1$ if and only if s is selected into the physical design.

Plan choice per query. For every $r \in Q$ and every $\ell \in L_r$, define the plan-choice variable

$$\lambda_{r\ell} \geq 0.$$

These variables encode a normalized choice of one plan per query via simplex constraints in Section 7.4. No integrality constraint is imposed on λ , because the feasible region and objective are linear and thus admit optimal extreme-point solutions that select a single plan per query.

Auxiliary feasibility variables for view semantics (if enabled). If view feasibility depends on explicit refresh-mode decisions, then for every $v \in V$ define

$$y_v^{\text{loc}} \in \{0, 1\}, \quad y_v^{\text{glob}} \in \{0, 1\}.$$

If admissibility level is decision-dependent, define

$$y_v^{\text{weak}} \in \{0, 1\}, \quad y_v^{\text{strong}} \in \{0, 1\}.$$

If admissibility is fixed as instance data, the corresponding symbols are treated as constants. If conjunctions of binary indicators are required, linearization variables are introduced as in Section 7.3.

7.2 Objective definition (workload plus maintenance)

Workload execution term. The workload execution cost is

$$J_{\text{work}}(\lambda) = \sum_{r \in Q} w_r \sum_{\ell \in L_r} c_{r\ell} \lambda_{r\ell}.$$

Maintenance term. The maintenance contribution is

$$J_{\text{maint}}(x) = \sum_{s \in S} m_s x_s.$$

Total objective. The baseline objective is the sum

$$\min_{x, \lambda, \text{aux}} J(x, \lambda) := \sum_{r \in Q} w_r \sum_{\ell \in L_r} c_{r\ell} \lambda_{r\ell} + \sum_{s \in S} m_s x_s,$$

subject to the constraints of Sections 7.3 and 7.4 and the declared variable domains.

7.3 Budget and feasibility constraints

This section states all constraints that do not depend on the plan-choice variables λ .

7.3.1 Budget constraint

The baseline budget constraint is

$$\sum_{s \in S} a_s x_s \leq B.$$

If multiple budget dimensions are used, an indexed family of constraints $\sum_{s \in S} a_s^{(k)} x_s \leq B^{(k)}$ is applied, one constraint per dimension k .

7.3.2 Partition exclusivity constraints

Partition selection is exclusive within each base table group:

$$\sum_{p \in P_t} x_p = 1 \quad \text{for every base table } t.$$

7.3.3 Index on view dependency constraints

If an index $i \in I$ is defined over a view $v \in V$, then selection of i requires selection of v :

$$x_i \leq x_v \quad \text{for every index } i \text{ defined on view } v.$$

7.3.4 Index partition compatibility constraints

For each base-table index i defined on base table t , instance data define a compatibility predicate $\text{CompatIdx}(i, p) \in \{0, 1\}$ for each $p \in P_t$. Incompatibility is enforced by

$$x_i + x_p \leq 1 \quad \text{for every } p \in P_t \text{ with } \text{CompatIdx}(i, p) = 0.$$

7.3.5 View refresh mode constraints (if enabled)

If refresh mode is decision-dependent, then for each $v \in V$ enforce

$$y_v^{\text{loc}} + y_v^{\text{glob}} = x_v.$$

If admissibility level is decision-dependent, then enforce the analogous exclusivity constraint

$$y_v^{\text{weak}} + y_v^{\text{strong}} = x_v.$$

If admissibility is fixed, the above is omitted and replaced by constant assignments.

7.3.6 Local validity constraints for partition respecting views (if enabled)

Instance construction precomputes a finite set of forbidden partition-scheme combinations that violate local correctness for a view v . For every forbidden pair (p_{t_1}, p_{t_2}) induced by a join in the defining expression of v , enforce

$$y_v^{\text{loc}} + x_{p_{t_1}} + x_{p_{t_2}} \leq 2.$$

7.3.7 Strong coupling constraints for locally refreshed views (if enabled)

Fix a view $v \in V$ and a join edge in its definition that joins (t_1, a) and (t_2, b) . Define the candidate supporting index sets

$$I(t_1, a) := \{i \in I : \text{rel}(i) = t_1 \text{ and } \{a\} \subseteq K_i\}, \quad I(t_2, b) := \{i \in I : \text{rel}(i) = t_2 \text{ and } \{b\} \subseteq K_i\},$$

where K_i is the key attribute set of index i .

Introduce a binary linearization variable $z_v \in \{0, 1\}$ intended to represent the conjunction $z_v = y_v^{\text{loc}} \cdot y_v^{\text{strong}}$. Enforce exactness of this conjunction by

$$z_v \leq y_v^{\text{loc}}, \quad z_v \leq y_v^{\text{strong}}, \quad z_v \geq y_v^{\text{loc}} + y_v^{\text{strong}} - 1.$$

Then enforce the strong coupling requirement for the join edge by

$$\sum_{i \in I(t_1, a)} x_i + \sum_{i \in I(t_2, b)} x_i \geq z_v.$$

7.3.8 Remaining feasibility constraints

All remaining admissibility rules are represented as a finite family of linear constraints over the binary variables. These include mutual exclusions, prerequisites, and any cross-object restrictions declared by the instance generator. Abstractly, these constraints can be written as

$$Ax \leq b,$$

where A and b are fixed rational instance data, and where the vector x includes all binary decision variables introduced in Section 7.1.

Hard feasibility predicate. Define the predicate

$$\text{Hard}(x) \iff x \text{ satisfies all constraints in Section 7.3.}$$

7.4 Plan selection coupling constraints

This section couples plan choice to physical design so that any plan assigned positive weight is executable under the selected objects.

7.4.1 One plan per query normalization

For every $r \in Q$, enforce

$$\sum_{\ell \in L_r} \lambda_{r\ell} = 1, \quad \lambda_{r\ell} \geq 0 \text{ for all } \ell \in L_r.$$

7.4.2 Requirement feasibility coupling

For every $r \in Q$, every $\ell \in L_r$, and every $s \in S$, define the requirement indicator

$$u_{r\ell s} := \mathbf{1}\{s \in \text{Req}(r, \ell)\} \in \{0, 1\}.$$

Then impose, for every $r \in Q$ and every $s \in S$, the coupling inequality

$$\sum_{\ell \in L_r} u_{r\ell s} \lambda_{r\ell} \leq x_s.$$

7.4.3 Feasible plan set induced by a design

Given x , define the feasible plan set for query r by

$$\text{FeasPlans}(r, x) := \{\ell \in L_r : \forall s \in \text{Req}(r, \ell), x_s = 1\}.$$

7.5 Equivalence theorem: formulation matches best feasible plan per query

This section proves that the baseline formulation is equivalent to the semantic problem: choose an admissible physical design, and for each query choose the minimum-cost executable plan under that design, then minimize workload execution plus maintenance.

7.5.1 Feasibility equivalence

Define the design-support predicate

$$\text{FeasibleX}(x) \iff \text{Hard}(x) \wedge \bigwedge_{r \in Q} (\text{FeasPlans}(r, x) \neq \emptyset).$$

Lemma 6.5.1 (Existence of a feasible plan assignment). If x satisfies $\text{FeasibleX}(x)$, then there exists λ such that (x, λ) satisfies all constraints in Section 7.4.

Proof. Fix $r \in Q$. Since $\text{FeasPlans}(r, x) \neq \emptyset$, choose any witness plan $\ell_r \in \text{FeasPlans}(r, x)$. Set $\lambda_{r\ell_r} = 1$ and $\lambda_{r\ell} = 0$ for all $\ell \in L_r \setminus \{\ell_r\}$. Then $\sum_{\ell \in L_r} \lambda_{r\ell} = 1$ and $\lambda_{r\ell} \geq 0$ hold. For any $s \in S$, if $u_{r\ell_r s} = 1$ then $s \in \text{Req}(r, \ell_r)$, hence $x_s = 1$ by definition of $\text{FeasPlans}(r, x)$, so $\sum_{\ell \in L_r} u_{r\ell s} \lambda_{r\ell} = u_{r\ell_r s} \leq x_s$. If $u_{r\ell_r s} = 0$, the left side is 0 and the inequality holds.

Applying the same construction independently for all $r \in Q$ yields a global feasible λ . $\square$

Lemma 6.5.2 (Necessity of a feasible plan for each query). If (x, λ) satisfies all constraints in Sections 7.3 and 7.4, then x satisfies FeasibleX(x).

Proof. The constraints of Section 7.3 are precisely Hard(x). Fix $r \in Q$. From $\sum_{\ell \in L_r} \lambda_{r\ell} = 1$ and $\lambda_{r\ell} \geq 0$, there exists $\ell^* \in L_r$ with $\lambda_{r\ell^*} > 0$. Let $s \in \text{Req}(r, \ell^*)$. Then $u_{r\ell^*s} = 1$, and the coupling constraint gives

$$\sum_{\ell \in L_r} u_{r\ell s} \lambda_{r\ell} \leq x_s.$$

The left side is at least $u_{r\ell^*s} \lambda_{r\ell^*} = \lambda_{r\ell^*} > 0$, hence $x_s > 0$. Since $x_s \in \{0, 1\}$, it follows that $x_s = 1$. Therefore $\ell^* \in \text{FeasPlans}(r, x)$ and $\text{FeasPlans}(r, x) \neq \emptyset$. Since r was arbitrary, the statement holds for all queries. $\square$

Theorem 6.5.3 (Feasibility equivalence). There exists λ such that (x, λ) satisfies Sections 7.3 and 7.4 if and only if x satisfies FeasibleX(x). $\square$

7.5.2 Objective equivalence

For any design x and query r with $\text{FeasPlans}(r, x) \neq \emptyset$, define the best feasible plan cost

$$C^*(r, x) := \min_{\ell \in \text{FeasPlans}(r, x)} c_{r\ell}.$$

Define the induced semantic objective

$$J^*(x) := \sum_{r \in Q} w_r C^*(r, x) + \sum_{s \in S} m_s x_s,$$

over designs x satisfying FeasibleX(x).

Lemma 6.5.4 (Optimal plan choice for fixed design). Fix x satisfying FeasibleX(x). Among all λ such that (x, λ) satisfies Section 7.4, the minimum value of $\sum_{r \in Q} w_r \sum_{\ell \in L_r} c_{r\ell} \lambda_{r\ell}$ equals $\sum_{r \in Q} w_r C^*(r, x)$, and an optimal λ exists that assigns value one to a single minimizer in $\text{FeasPlans}(r, x)$ for each r .

Proof. Fix $r \in Q$. Under the coupling constraints, $\lambda_{r\ell} = 0$ for all $\ell \notin \text{FeasPlans}(r, x)$, hence the per-query contribution is linear over the simplex supported on $\text{FeasPlans}(r, x)$. A linear function over a simplex attains its minimum at an extreme point, and each extreme point selects exactly one plan. Therefore the minimum equals $C^*(r, x)$. Summing over r and weighting by w_r gives the statement. $\square$

Theorem 6.5.5 (Equivalence to best feasible plan per query). The baseline optimization problem is equivalent to

$$\min_x J^*(x) \quad \text{subject to} \quad \text{FeasibleX}(x).$$

More precisely,

$$\min_{\substack{(x, \lambda, \text{aux}) \\ \text{feasible in Sections 7.3, 7.4}}} J(x, \lambda) = \min_{x: \text{FeasibleX}(x)} J^*(x),$$

and any optimal x^* on the right can be extended to an optimal feasible (x^*, λ^*) on the left by choosing, for each $r \in Q$, any minimizer $\ell_r \in \arg \min_{\ell \in \text{FeasPlans}(r, x^*)} c_{r\ell}$ and setting $\lambda_{r\ell_r}^* = 1$ and $\lambda_{r\ell}^* = 0$ for $\ell \neq \ell_r$.

Proof. For any fixed feasible x , Lemma 6.5.4 shows that minimizing $J(x, \lambda)$ over feasible λ yields $J^*(x)$. Therefore minimizing over all feasible (x, λ) is identical to minimizing over feasible x the induced objective $J^*(x)$. The extension construction follows from the extreme-point argument in Lemma 6.5.4. $\square$

8

Optional Nonlinear Extensions (If Used)

8.1 Nonlinear primitives and their semantic necessity

8.1.1 Baseline condition and admissibility boundary

The baseline optimization model established in Chapter ?? is a mixed integer linear program. Chapter 8 does not revise that baseline. It specifies an optional extension interface that is admissible only when its semantics can be reduced to the baseline class without altering either feasibility or objective meaning.

A *nonlinear primitive* is any syntactic construction that is not affine in the decision variables as written. Admissibility is not determined syntactically. It is determined semantically by the existence of a finite, explicit, mechanically checkable extended formulation that uses only linear inequalities, integrality restrictions, and auxiliary variables, such that the projection of the extended feasible set onto the original variable space equals exactly the intended feasible set, and such that the projected objective value equals exactly the intended objective value.

8.1.2 Variable universe and finiteness requirements for any nonlinear extension

Every nonlinear extension shall satisfy the following preconditions.

1. The variable universe is partitioned into binary variables, bounded integer variables, and bounded continuous variables. Every continuous variable participating in any nonaffine construction shall have declared finite lower and upper bounds given as rational constants.
2. Every nonlinear extension shall preserve finiteness and explicitness of the instance. The extension shall not introduce an infinite family of constraints, an

Primitive class	Semantic role and admissibility condition
A. Binary conditioned linear quantity	Binary times bounded continuous, and more generally conditional activation of a bounded continuous expression under a binary decision. Admissible only with explicit rational bounds.
B. Finite Boolean structure	Conjunction, disjunction, and finite products of binary variables used to encode compatibility and exclusivity. Admissible by exact mixed integer linear encoding.
C. Bounded integer encoding	Exact digitization of a bounded integer by finitely many binary digits, with explicit reconstruction and bound enforcement.
D. Bounded max, min, absolute value	Admissible only when representable as a finite piecewise affine function on a bounded domain, with explicit rational data and finite case selection.
E. Finite piecewise affine function	Admissible when defined on a bounded domain by finitely many affine pieces with rational breakpoints and rational coefficients, represented by explicit selector variables and bounded implications.
F. Indicator implication with bounded constants	Admissible for conditional constraint activation only when the implication constant is derived as an explicit valid bound from declared variable bounds.

Table 8.1: *Admissible nonlinear primitive library.*

unbounded domain for any variable participating in a nonlinear construction, or any oracle defined only implicitly.

These requirements are necessary because the global correctness closure of the system relies on finite termination and on the existence of checkable certificates for all admitted strengthening and pruning steps.

8.1.3 Admissible primitive library

The admissible primitive library consists precisely of primitives that admit exact linear extended formulations under boundedness and finiteness.

No other nonlinear primitive is admissible unless it is first converted into a declared finite piecewise affine semantics, thereby becoming a member of Primitive class E. In particular, products of two continuous variables, transcendental functions applied to variables, and ratios with variable denominators are excluded unless replaced by declared finite piecewise affine semantics with explicit rational data.

8.2 Exact compilation to MILP, or certified relaxations

8.2.1 Expression grammar and extended formulation goal

A nonlinear extension is specified by an expression directed acyclic graph whose internal nodes are applications of primitives from Table 8.1 and whose leaves are variables

or rational constants. Compilation constructs an extended mixed integer linear program by introducing one auxiliary variable per expression node and enforcing node semantics by linear constraints and, when required, auxiliary binary selectors.

Let the original model be posed in variables z with feasible set F and objective $J(z)$. The compilation yields an extended mixed integer linear program in variables (z, u) with feasible set F' and objective $J'(z, u)$ such that

$$\{z \mid \exists u \text{ with } (z, u) \in F'\} = F, \quad (8.1)$$

$$\min_{z \in F} J(z) = \min_{(z, u) \in F'} J'(z, u). \quad (8.2)$$

8.2.2 Exact rewrite rules and correctness statements

Rule 1: binary times bounded continuous

Let $x \in \{0, 1\}$, let t be a continuous expression with declared bounds $L \leq t \leq U$, and introduce an auxiliary variable w intended to satisfy $w = xt$. Impose

$$w \geq Lx, \quad (8.3)$$

$$w \leq Ux, \quad (8.4)$$

$$w \geq t - U(1 - x), \quad (8.5)$$

$$w \leq t - L(1 - x). \quad (8.6)$$

Theorem 8.1 (Exactness of Rule 1). *Assume $x \in \{0, 1\}$ and $L \leq t \leq U$. Then constraints (8.3)–(8.6) hold if and only if $w = xt$.*

Proof. If $x = 0$, then (8.3)–(8.4) yield $0 \leq w \leq 0$, hence $w = 0 = xt$. Constraints (8.5)–(8.6) reduce to $w \geq t - U$ and $w \leq t - L$, which are implied by $L \leq t \leq U$ and $w = 0$. If $x = 1$, then (8.5)–(8.6) yield $w \geq t$ and $w \leq t$, hence $w = t = xt$. Constraints (8.3)–(8.4) follow from $L \leq t \leq U$. This proves equivalence. $\square$

Rule 2: exact conjunction for binary products

Let $x_1, \dots, x_k \in \{0, 1\}$. Introduce $y \in \{0, 1\}$ intended as

$$y = \prod_{i=1}^k x_i.$$

Impose

$$y \leq x_i \quad \text{for all } i, \quad (8.7)$$

$$y \geq \sum_{i=1}^k x_i - (k - 1). \quad (8.8)$$

Theorem 8.2 (Exactness of Rule 2). *Under integrality, constraints (8.7)–(8.8) hold if and only if $y = \prod_{i=1}^k x_i$.*

Proof. If some $x_i = 0$, then (8.7) implies $y \leq 0$ and since $y \geq 0$ it follows that $y = 0$. If all $x_i = 1$, then (8.8) yields $y \geq 1$ and binary restriction gives $y = 1$. These are exactly the truth conditions for the conjunction. $\square$

Rule 3: exact bounded integer digitization

Let q be an integer variable with bounds $0 \leq q \leq Q$. Choose $m \in \mathbb{Z}_{>0}$ such that $2^m - 1 \geq Q$. Introduce binary digits $b_0, \dots, b_{m-1} \in \{0, 1\}$ and impose

$$q = \sum_{j=0}^{m-1} 2^j b_j, \quad (8.9)$$

$$q \leq Q. \quad (8.10)$$

Theorem 8.3 (Exactness of Rule 3). *Every integer q satisfying $0 \leq q \leq Q$ has a feasible digit vector $(b_0, \dots, b_{m-1})$, and every feasible digit vector induces a unique feasible q via (8.9)–(8.10).*

Proof. Existence follows from the binary expansion of integers in $[0, Q]$. Uniqueness follows because powers of two form a positional basis. Constraint (8.10) removes representations exceeding Q . $\square$

Rule 4: exact bounded integer times bounded continuous

Let q be a bounded integer and t a bounded continuous expression. Digitize q using Rule 8.2.2, then rewrite

$$qt = \sum_{j=0}^{m-1} 2^j (b_j t),$$

and linearize each term $b_j t$ using Rule 8.2.2. Exactness follows by composition of Theorems 8.1 and 8.3.

Rule 5: exact discrete lookup table

Let s select one of K cases. Introduce binaries $\delta_1, \dots, \delta_K$ with

$$\sum_{k=1}^K \delta_k = 1, \quad \delta_k \in \{0, 1\}.$$

For any quantity v taking declared rational values v_k in case k , enforce

$$v = \sum_{k=1}^K v_k \delta_k.$$

This exactly represents finite semantic branches.

Rule 6: exact maximum and minimum on bounded domains

For $m = \max\{a_1, \dots, a_K\}$ with each a_k affine, enforce $m \geq a_k$ for all k and ensure exactness by the modeling context in which m is minimized with positive weight or by explicit case selection using Rule 8.2.2. For $n = \min\{a_1, \dots, a_K\}$, use case selection directly, or represent the minimum as a finite piecewise affine function and apply Rule 8.2.2.

Rule 7: exact absolute value on bounded domains

For $t = |r|$, enforce $t \geq r$, $t \geq -r$, and $t \geq 0$. If exactness is required independent of objective coupling, use a binary sign selector with bounded implications and reduce to Rule 8.2.2.

Rule 8: exact finite piecewise affine function

Let $y = f(t)$ where f is piecewise affine on $[L, U]$ with finitely many segments and rational breakpoints

$$L = \tau_1 < \tau_2 < \dots < \tau_{K+1} = U,$$

and on segment k define $f(t) = \alpha_k t + \beta_k$ with $\alpha_k, \beta_k \in \mathbb{Q}$. Introduce binaries $\delta_k \in \{0, 1\}$ with $\sum_{k=1}^K \delta_k = 1$. Enforce bounded segment activation

$$\tau_k \delta_k \leq t \leq \tau_{k+1} \delta_k + U(1 - \delta_k)$$

together with the affine definition of y on the active segment using standard disjunctive linearization, ensuring that when $\delta_k = 1$, the equality $y = \alpha_k t + \beta_k$ holds. Because K is finite and all coefficients are rational, the resulting formulation is an exact mixed integer linear representation of the declared piecewise affine semantics.

Rule 9: exact indicator implication with bounded constants

For an implication $\delta = 1 \Rightarrow Az \leq b$, with an explicit valid bound M derived from declared variable bounds for the left hand side, enforce

$$Az \leq b + M(1 - \delta).$$

Exactness is with respect to the intended logical semantics under boundedness.

8.2.3 Whole model compilation theorem

Theorem 8.4 (Exact compilation to mixed integer linear form). *If every nonaffine atom in the nonlinear extension belongs to the admissible primitive library in Table 8.1, and every participating continuous expression has declared rational bounds satisfying the preconditions of*

the corresponding rules, then applying Rules 8.2.2 through 8.2.2 to each node of the expression directed acyclic graph yields a mixed integer linear program satisfying (8.1)–(8.2).

Proof. Each primitive node is replaced by an extended formulation equivalent to the intended relation on the bounded domain by the corresponding rule theorem. Since the expression graph is acyclic, auxiliary variables can be introduced in topological order. Equivalence of the full model follows by repeated substitution, preserving feasibility and objective evaluation at each substitution step. Projection equivalence holds because each auxiliary variable is introduced only to represent a relation, and elimination by existential quantification recovers the original relation exactly. $\square$

8.2.4 Certified relaxations when exact compilation is intentionally not required

If a function is not piecewise affine and does not admit an exact finite mixed integer linear extended formulation under the admissibility boundary of Section 8.1, it is not admitted as an exact primitive. The only admissible path is replacement by a certified relaxation pair.

Definition 8.1 (Certified relaxation pair). Let g be a scalar function on a bounded interval $[L, U]$. A certified relaxation is a pair of finite piecewise affine functions $\underline{g}$ and $\overline{g}$, each with finitely many rational breakpoints and rational coefficients, such that for all $t \in [L, U]$,

$$\underline{g}(t) \leq g(t) \leq \overline{g}(t), \quad (8.11)$$

and such that the maximum pointwise gap

$$\Delta = \max_{t \in [L, U]} (\overline{g}(t) - \underline{g}(t)) \quad (8.12)$$

is an explicit rational constant recorded in the assumption ledger.

The induced optimization problems are then a lower model using $\underline{g}$ and an upper model using $\overline{g}$. The resulting solutions bracket the intended objective with an explicit bound induced by Δ and by the number and coefficients of occurrences of the relaxed primitive. All claims shall state explicitly whether they refer to the lower model, the upper model, or the certified bracket.

8.3 Scope: when the system remains MILP-only

8.3.1 MILP-only decision criterion

Definition 8.2 (MILP-only declaration). A specification is declared MILP-only if and only if every nonaffine atom in the entire model is a member of the admissible primitive library in Table 8.1 and every bound precondition required by Rules 8.2.2 through 8.2.2 is declared and finite. Otherwise, the specification is declared outside MILP-only scope.

Figure 8.1: *Schematic of nonlinear expression graph compilation into an extended mixed integer linear formulation via bounded auxiliary variables and finite selectors.*

This declaration is made prior to solving and is independent of solver behavior.

8.3.2 Consequences for the global correctness spine

If the model is MILP-only, all subsequent correctness and certificate statements apply without alteration because the remainder of the system is defined in terms of linear relaxations, branch and bound over discrete domains, and linear or arithmetically checkable certificates.

If certified relaxations are used, the global correctness spine applies to the relaxed models. Any statement concerning the unrelaxed intended semantics is prohibited unless a separate explicit refinement argument is provided that reduces the relaxation gap to a declared tolerance and proves the associated bound on objective error.

8.3.3 Boundary statement

The default scope of this report is MILP-only. Optional nonlinear extensions are admissible only when they either compile exactly into mixed integer linear form by Rules 8.2.2 through 8.2.2, or are explicitly converted into certified relaxation pairs under Definition 8.1 with recorded error bounds and with all resulting claims stated relative to the relaxed semantics.

8.4 Nonlinear to linear transformation schematic

Part I

Exact Bound Computation (Cut-and-Price Closure)

9

Restricted Master Problem Construction and Initialisation

9.1 Restricted master definition

Let D denote the finite set of candidate physical design objects. Let Q denote the finite set of workload queries. For each $q \in Q$, let P_q denote the finite plan universe enumerated for query q .

For each design object $d \in D$, the relaxed design selection variable is

$$y_d \in [0, 1].$$

For each query $q \in Q$ and plan $p \in P_q$, the relaxed plan choice variable is

$$x_{q,p} \in [0, 1].$$

Let $c_{q,p} \geq 0$ denote the workload weighted execution cost coefficient for plan p of query q , and let $m_d \geq 0$ denote the aggregated maintenance coefficient for design object d .

Let $\text{size}(d) \geq 0$ denote the budget consumption of object d , and let $B \geq 0$ denote the total budget. The budget constraint is

$$\sum_{d \in D} \text{size}(d) y_d \leq B. \quad (9.1)$$

Plan feasibility is expressed through coupling. For atomic requirements, define $a_{q,p,d} \in \{0, 1\}$ with $a_{q,p,d} = 1$ if and only if plan p for query q requires object d , and impose

$$x_{q,p} \leq y_d \quad \text{for all } q \in Q, p \in P_q, d \in D \text{ with } a_{q,p,d} = 1. \quad (9.2)$$

For clause requirements, let $\mathcal{R}_{q,p}$ be a finite set of requirement clauses $R \subseteq D$, and

impose

$$x_{q,p} \leq \sum_{d \in R} y_d \quad \text{for all } q \in Q, p \in P_q, R \in \mathcal{R}_{q,p}. \quad (9.3)$$

Exactly one plan is chosen per query among the represented plans:

$$\sum_{p \in \widehat{P}_q} x_{q,p} = 1 \quad \text{for all } q \in Q, \quad (9.4)$$

where $\widehat{P}_q \subseteq P_q$ is the restricted plan pool for query q .

Let the admissibility semantics for the design vector y be encoded by a closed linear system

$$A_y y \leq b_y, \quad E_y y = e_y, \quad (9.5)$$

together with (9.1).

The restricted master problem corresponding to restricted pools $\widehat{P} := (\widehat{P}_q)_{q \in Q}$ is

$$\begin{aligned} \min_{x,y} \quad & \sum_{q \in Q} \sum_{p \in \widehat{P}_q} c_{q,p} x_{q,p} + \sum_{d \in D} m_d y_d \\ \text{subject to} \quad & (9.4), (9.2), (9.3), (9.1), (9.5), \\ & 0 \leq x_{q,p} \leq 1 \quad \text{for all } q \in Q, p \in \widehat{P}_q, \\ & 0 \leq y_d \leq 1 \quad \text{for all } d \in D. \end{aligned} \quad (9.6)$$

The restriction is well posed only if $\widehat{P}_q \neq \emptyset$ for all $q \in Q$.

9.1.1 Initial column set

Fix, for each $q \in Q$, a distinguished seed plan $\ell_q \in P_q$. Define the initial restricted pools by

$$\widehat{P}_q^0 := \{\ell_q\} \quad \text{for all } q \in Q. \quad (9.7)$$

The initial restricted master is $\text{RMP}(\widehat{P}^0)$.

9.1.2 Initial cut set (if any)

Let $\mathcal{K}$ denote the family of globally valid cuts admitted by the cut acceptance rules. At initialisation, the default initial cut set is empty:

$$\mathcal{K}^0 := \emptyset. \quad (9.8)$$

If nonempty initial cuts are imposed by model construction, they are recorded explicitly as a finite set

$$\mathcal{K}^0 := \{C_i(x, y) \leq \beta_i\}_{i=1}^{k_0},$$

with the validity condition that each $C_i(x, y) \leq \beta_i$ holds for every feasible point of the full relaxation.

9.2 Seed existence and feasibility preservation

9.2.1 Seed plan existence statement

Seed plan requirement. For each query $q \in Q$, the seed plan ℓ_q satisfies the implication: whenever y satisfies (9.1) and (9.5), the coupling constraints for (q, ℓ_q) are satisfied. A sufficient checkable form is that ℓ_q has no index or view requirements and only partition requirements implied by the design feasibility system.

Identity feasibility requirement. If the admissibility semantics includes exactly one partition choice per table, then for each table t there exists a designated identity partition object π_t^{id} such that selecting identity partitions for all tables and selecting no optional objects satisfies (9.1) and (9.5).

9.2.2 Feasibility preservation lemma

Lemma 9.1 (Monotone feasibility under admissible updates). *Assume $\text{RMP}(\widehat{P})$ is feasible. Let $\widehat{P}'$ satisfy $\widehat{P}_q \subseteq \widehat{P}'_q$ for all $q \in Q$. Then $\text{RMP}(\widehat{P}')$ is feasible. Moreover, if a cut $C(x, y) \leq \beta$ is valid for the full relaxation, then adding $C(x, y) \leq \beta$ to $\text{RMP}(\widehat{P})$ preserves feasibility.*

Proof. For column augmentation, extend any feasible solution of $\text{RMP}(\widehat{P})$ by assigning value 0 to all newly introduced variables $x_{q,p}$ with $p \in \widehat{P}'_q \setminus \widehat{P}_q$. All constraints remain satisfied, hence $\text{RMP}(\widehat{P}')$ is feasible. For a globally valid cut, validity implies that every feasible point of the full relaxation satisfies the cut; therefore every feasible point of the restricted master also satisfies it, and feasibility is preserved. $\square$

9.3 Initialisation theorem

9.3.1 Well-posedness of the initial RMP

Theorem 9.1 (Feasible initialisation of the restricted master). *Assume the seed plan requirement and the identity feasibility requirement. Define the initial pools by (9.7), and let the initial cut set be (9.8) or any finite valid set. Then the initial restricted master problem is well posed, and it is feasible.*

Proof. Well posedness holds because $\widehat{P}_q^0$ is a singleton for each q . Feasibility is witnessed by selecting a feasible design y^* given by identity feasibility, and setting

$$x_{q,\ell_q} = 1 \text{ for all } q \in Q.$$

Plan choice constraints hold by construction. Coupling holds by the seed plan requirement. All design feasibility constraints hold by identity feasibility. Therefore the initial restricted master is feasible. $\square$

9.3.2 Bound interpretability at initialisation

The objective value of the initial restricted master is interpreted solely as an objective value of a restricted relaxation associated with the initial pools $\widehat{P}^0$ and the initial cut set $\mathcal{K}^0$. No bound claim relative to the full master relaxation is asserted at initialisation. A bound claim is asserted only after the pricing and cut closure conditions certify that no improving column and no violated globally valid cut remain admissible under the acceptance rules.

9.4 Figures and tables

- RMP initialisation diagram.
- Seed existence summary.

10

Pricing: Oracle, Termination, and Completeness

10.1 Reduced cost definition and algebra

This chapter specifies the pricing mechanism that closes the restricted master problem under the full finite plan universe and guarantees that the node relaxation bound is not an artifact of an incomplete column set. The chapter operates under the standing conventions established earlier: every set is finite, every evaluated quantity is exact over rational arithmetic, and every acceptance decision is verified by deterministic predicates.

10.1.1 Fixed data and domains

Fix a node of the global search tree. Fix a query index r . Let L_r denote the finite plan universe for query r . Each plan $\ell \in L_r$ is a finite syntactic and semantic object.

Let S be the finite set of candidate physical design objects, including candidate indexes, candidate materialized views, candidate partition schemes, and any auxiliary objects admitted by the instance definition. For each query r , the required-object mapping is a total function

$$\text{Req}_r : L_r \rightarrow 2^S,$$

where $\text{Req}_r(\ell)$ is finite for every ℓ .

Let $w_r \in \mathbb{Q}_{>0}$ be the fixed workload weight of query r . Let $c_r(\ell) \in \mathbb{Q}_{\geq 0}$ be the exact plan cost of ℓ , computed by the declared cost model under the node specific fixings.

10.1.2 Dual variables used by the reduced cost

At the current column generation iteration, the restricted master problem contains a finite subset $L_r^k \subseteq L_r$ for each r , with decision variables $x_{r\ell}$ for $\ell \in L_r^k$, and activation variables y_s for $s \in S$.

Symbol	Meaning and domain
r	Query index.
L_r	Finite plan universe for query r .
$\ell \in L_r$	A plan object with finite syntax and semantics.
S	Finite set of candidate physical design objects.
$\text{Req}_r(\ell)$	Required-object set for plan ℓ , subset of S .
w_r	Workload weight in $\mathbb{Q}_{>0}$.
$c_r(\ell)$	Exact cost in $\mathbb{Q}_{\geq 0}$.
α_r	Dual of query convexity constraint, in $\mathbb{Q}$.
$\beta_{r,s}$	Dual of coupling constraint for object s , in $\mathbb{Q}_{\geq 0}$.
$\text{rc}_r(\ell)$	Reduced cost in $\mathbb{Q}$.

Table 10.1: *Reduced cost components and domains.*

The restricted master enforces the convexity constraint for each query,

$$\sum_{\ell \in L_r^k} x_{r\ell} = 1,$$

with dual variable $\alpha_r \in \mathbb{Q}$.

The restricted master also enforces coupling constraints ensuring that a plan may be selected only if each required physical object is activated. A canonical representative inequality form is

$$\sum_{\ell \in L_r^k : s \in \text{Req}_r(\ell)} x_{r\ell} - y_s \leq 0, \quad \text{for each } s \in S,$$

with dual variable $\beta_{r,s} \in \mathbb{Q}_{\geq 0}$.

Any additional node specific constraints contribute additional dual terms. These are treated as fixed dual coefficients and may be absorbed into the reduced cost expression by extending the sum below with the corresponding verified terms.

10.1.3 Definition of reduced cost

For any plan $\ell \in L_r$, define the reduced cost at the current dual point by

$$\text{rc}_r(\ell) = w_r c_r(\ell) - \alpha_r - \sum_{s \in \text{Req}_r(\ell)} \beta_{r,s}.$$

All terms are rational, and the sum is finite.

A column corresponding to plan ℓ is improving for the restricted master if and only if $\text{rc}_r(\ell) < 0$. The pricing task is therefore the exact minimization problem

$$\text{rc}_r^* = \min_{\ell \in L_r} \text{rc}_r(\ell), \quad \ell_r^* \in \arg \min_{\ell \in L_r} \text{rc}_r(\ell).$$

Because L_r is finite and $\text{rc}_r(\ell)$ is defined for all ℓ , the minimum exists and is attained.

10.2 Deterministic pricing fallback procedure (completeness)

The deterministic fallback is the completeness anchor: it must find a negative reduced cost plan whenever one exists, and it must certify nonexistence by exhaustion when none exists. The construction below is explicit and relies only on finiteness and exact evaluation.

10.2.1 Canonical plan codes and enumeration

Assume a total injective coding function

$$\text{PlanCode}_r : L_r \rightarrow \{1, 2, \dots, |L_r|\},$$

which induces a strict total order on L_r by numeric comparison. Define the enumerator

$$\text{ENUM}_r : \{1, 2, \dots, |L_r|\} \rightarrow L_r$$

as the inverse of PlanCode_r , so that

$$\text{PlanCode}_r(\text{ENUM}_r(k)) = k \quad \text{for all } k, \quad \text{ENUM}_r(\text{PlanCode}_r(\ell)) = \ell \quad \text{for all } \ell \in L_r.$$

Lemma 9.1 (Enumerator finiteness and total coverage). The list $(\text{ENUM}_r(1), \text{ENUM}_r(2), \dots, \text{ENUM}_r(|L_r|))$ contains every plan in L_r exactly once.

Proof. Since PlanCode_r is injective from a finite set L_r to a finite set of equal cardinality, it is bijective. Therefore its inverse exists and is ENUM_r . Each code maps to exactly one plan, and each plan appears at exactly one code. $\square$

10.2.2 Deterministic complete pricing oracle

Define the deterministic complete pricing oracle for query r , denoted $\text{CPO}(r)$, as follows.

Input. The dual values α_r and $\{\beta_{r,s}\}_{s \in S}$, together with exact evaluation routines for $c_r(\cdot)$ and $\text{Req}_r(\cdot)$.

Procedure.

1. Initialize $k \leftarrow 1$. Set $\ell^* \leftarrow \text{ENUM}_r(1)$ and $m^* \leftarrow \text{rc}_r(\ell^*)$.
2. For $k = 2$ to $|L_r|$, compute $\ell \leftarrow \text{ENUM}_r(k)$ and $m \leftarrow \text{rc}_r(\ell)$.
3. If $m < m^*$, set $\ell^* \leftarrow \ell$ and $m^* \leftarrow m$. If $m = m^*$, break ties deterministically by choosing the smaller plan code.
4. After the loop, return ℓ^* if $m^* < 0$. Otherwise return the distinguished symbol $\perp$, interpreted as “no negative reduced cost plan exists”.

10.3 Termination condition and correctness theorem

10.3.1 Termination

The loop index ranges over $\{1, \dots, |L_r|\}$. Therefore $\text{CPO}(r)$ terminates after exactly $|L_r|$ reduced cost evaluations.

Lemma 9.2 (Running minimum invariant). After processing the first k enumerated plans, the oracle maintains

$$m^\star = \min_{1 \leq i \leq k} \text{rc}_r(\text{ENUM}_r(i)),$$

and $\ell^\star$ attains this minimum, with ties broken by minimum plan code.

Proof. For $k = 1$, the statement holds by initialization. Assume it holds for $k - 1$. At step k , the oracle computes $m = \text{rc}_r(\text{ENUM}_r(k))$ and updates $(\ell^\star, m^\star)$ exactly when the new value is strictly smaller, and applies the fixed tie rule when equal. The resulting stored value equals the minimum over the first k values with the prescribed tie break. $\square$

10.3.2 Correctness and completeness

Theorem 9.1 (Exact minimum reduced cost and sound termination). Let $\text{rc}_r^\star = \min_{\ell \in L_r} \text{rc}_r(\ell)$. The oracle $\text{CPO}(r)$ terminates and satisfies the following.

1. If $\text{rc}_r^\star < 0$, the oracle returns a plan $\ell^\star \in L_r$ such that $\text{rc}_r(\ell^\star) = \text{rc}_r^\star$.
2. If $\text{rc}_r^\star \geq 0$, the oracle returns $\perp$, and there exists no plan $\ell \in L_r$ with $\text{rc}_r(\ell) < 0$.

Proof. By Lemma 9.1, the enumerator lists every element of L_r exactly once. By Lemma 9.2 with $k = |L_r|$, the oracle computes the exact minimum of $\text{rc}_r(\ell)$ over all $\ell \in L_r$, and returns a minimizer when the minimum is negative. If the minimum is nonnegative, then all reduced costs are nonnegative and no negative reduced cost plan exists. $\square$

Corollary 9.1 (Negative reduced cost detection completeness). A plan $\ell \in L_r$ with $\text{rc}_r(\ell) < 0$ exists if and only if $\text{CPO}(r)$ returns a plan rather than $\perp$.

Proof. Immediate from Theorem 9.1. $\square$

10.4 Interaction with learned pricing proposals (advisory only)

The learned component may propose candidate plans to accelerate discovery of negative reduced cost columns. It is structurally prohibited from altering correctness. This

is enforced by membership checks, exact reduced cost evaluation, and deterministic fallback to $\text{CPO}(r)$.

10.4.1 Advisory proposal object and membership gate

Let the proposer output an ordered list

$$A_r = (\tilde{\ell}_1, \tilde{\ell}_2, \dots, \tilde{\ell}_K),$$

with $K \in \mathbb{N}$ fixed at invocation time. Each $\tilde{\ell}_i$ is a plan representation.

Membership in L_r is decidable. Define the admissible subsequence

$$A_r^{\text{in}} = \{\tilde{\ell}_i \in A_r : \tilde{\ell}_i \in L_r\}.$$

Any proposal not in L_r is discarded and does not influence pricing.

10.4.2 Proposal evaluation gate and selection

For each $\tilde{\ell} \in A_r^{\text{in}}$, compute $\text{rc}_r(\tilde{\ell})$ exactly. If there exists at least one proposal with negative reduced cost, return a proposal $\tilde{\ell}^-$ that minimizes $\text{rc}_r(\cdot)$ over the negative proposals, with ties broken deterministically by minimum plan code. If no admissible proposal has negative reduced cost, invoke $\text{CPO}(r)$ and return its result.

Denote this composite procedure by $\text{PAO}(r)$, the pricing oracle with advice.

10.4.3 Correctness immunity and completeness of the composite oracle

Theorem 9.2 (Composite oracle correctness and completeness). Fix r and the current dual values. The composite oracle $\text{PAO}(r)$ satisfies the following.

1. Soundness: if $\text{PAO}(r)$ returns a plan ℓ , then $\ell \in L_r$ and $\text{rc}_r(\ell) < 0$.
2. Completeness: if there exists $\ell \in L_r$ with $\text{rc}_r(\ell) < 0$, then $\text{PAO}(r)$ returns a plan.
3. Nonexistence exactness: if $\text{PAO}(r)$ returns $\perp$, then $\text{rc}_r(\ell) \geq 0$ for all $\ell \in L_r$.
4. Deterministic termination: $\text{PAO}(r)$ terminates after at most $K + |L_r|$ reduced cost evaluations.

Proof. If $\text{PAO}(r)$ returns from the advice branch, it returns only after membership filtering and exact verification of $\text{rc}_r(\ell) < 0$, which implies soundness. If the advice branch does not return a negative reduced cost plan, $\text{PAO}(r)$ invokes $\text{CPO}(r)$. By Corollary 9.1, $\text{CPO}(r)$ returns a plan whenever a negative reduced cost plan exists, which establishes completeness. If $\text{PAO}(r)$ returns $\perp$, then $\text{CPO}(r)$ returned $\perp$, and Theorem 9.1 implies nonnegativity of reduced cost for all $\ell \in L_r$. Termination follows from the finite processing of at most K proposals and a finite fallback of exactly $|L_r|$ evaluations.

□

Pricing loop schematic placeholder. Replace this box with the finalized pricing loop figure file.

Figure 10.1: *Pricing loop: dual extraction, reduced cost evaluation, column admission, and closure reporting.*

Reduced cost components schematic placeholder. Replace this box with the finalized reduced cost components figure file.

Figure 10.2: *Reduced cost components and the exact algebraic terms used by the pricing oracle.*

10.4.4 Consequences for node closure

When $\text{PAO}(r)$ returns $\perp$, the restricted master is closed under all admissible columns from the full plan universe for query r . This closure statement is the precise object used later by the verification layer to certify pricing completeness for node bound correctness.

10.5 Required figures and tables for this chapter

11

Column Representation and Reduced Cost Certificates

11.1 Canonical Plan Code to Column Vector Map

11.1.1 Primitive arithmetic domain

Let $\mathbb{Z}$ denote the set of integers. A rational number is represented by an ordered pair $(p, q) \in \mathbb{Z} \times \mathbb{Z}$ with $q > 0$ and $\gcd(|p|, q) = 1$. The value represented by (p, q) is p/q . Two reduced rationals (p, q) and (p', q') are equal if and only if

$$pq' = p'q.$$

A reduced rational (p, q) is nonnegative if $p \geq 0$ and is positive if $p > 0$.

For reduced rationals, addition and multiplication are defined by

$$\frac{p}{q} + \frac{p'}{q'} = \frac{pq' + p'q}{qq'} \quad \text{and} \quad \frac{p}{q} \cdot \frac{p'}{q'} = \frac{pp'}{qq'},$$

followed by reduction to coprime numerator and denominator with positive denominator. All arithmetic and all certificate checks in this chapter are performed in reduced rational arithmetic.

11.1.2 Finite plan code space and query typed decoding

Let Q denote the finite set of workload queries. For each $r \in Q$, let P_r denote the finite set of admissible physical plans for query r , as defined by the plan-universe construction. Let Σ be a finite alphabet and let Σ^* be the set of finite strings over Σ . A plan code is any element $e \in \Sigma^*$.

For each $r \in Q$, define a partial function

$$\text{Dec}_r : \Sigma^* \rightarrow P_r,$$

called the decoder for query r . If $\text{Dec}_r(e)$ is defined, then e is valid for query r and denotes the unique decoded plan $\text{Dec}_r(e) \in P_r$. If $\text{Dec}_r(e)$ is undefined, then e is invalid for query r . The decoder is deterministic and is fixed by the instance.

11.1.3 Canonicalization operator

Define a total function

$$\text{Can} : \Sigma^* \rightarrow \Sigma^*$$

satisfying the following properties.

Idempotence. For all $e \in \Sigma^*$, $\text{Can}(\text{Can}(e)) = \text{Can}(e)$.

Decoding invariance. For every $r \in Q$ and every $e \in \Sigma^*$, if $\text{Dec}_r(e)$ is defined, then $\text{Dec}_r(\text{Can}(e))$ is defined and

$$\text{Dec}_r(\text{Can}(e)) = \text{Dec}_r(e).$$

Canonical uniqueness within decoding. For every fixed $r \in Q$ and every $e, e' \in \Sigma^*$, if $\text{Dec}_r(e)$ and $\text{Dec}_r(e')$ are defined and equal, then

$$\text{Can}(e) = \text{Can}(e').$$

Consequently, for each plan $\pi \in P_r$ there exists a unique canonical code $e^\circ \in \Sigma^*$ such that $\text{Dec}_r(e^\circ) = \pi$ and $\text{Can}(e^\circ) = e^\circ$.

11.1.4 Physical object identifiers and requirement extraction

Let S denote the finite set of physical design objects, consisting of index candidates, materialized view candidates, and partition candidates, as defined by the instance. Fix a bijection

$$\text{id} : S \rightarrow \{1, 2, \dots, n\}, \quad \text{where } n = |S|,$$

and let $\text{obj} : \{1, 2, \dots, n\} \rightarrow S$ be its inverse.

For each query $r \in Q$, define a total function

$$\text{Req}_r : P_r \rightarrow \mathcal{P}(S),$$

where $\text{Req}_r(\pi)$ is the set of physical objects required by plan π . The function Req_r is deterministic and fixed by the instance.

Define the requirement list map

$$L_r(\pi) := \text{sort}(\{\text{id}(s) : s \in \text{Req}_r(\pi)\}),$$

where sort produces a strictly increasing list with no duplicates. The list $L_r(\pi)$ is a canonical sparse representation of the required objects of π .

11.1.5 Exact plan cost

For each $r \in Q$, define a total function

$$\text{Cost}_r : P_r \rightarrow \mathbb{Q}_{\geq 0},$$

where $\text{Cost}_r(\pi)$ is the exact rational cost assigned to plan π by the model. The function Cost_r is deterministic and fixed by the instance.

11.1.6 The column projector and the column signature

Fix $r \in Q$ and $e \in \Sigma^*$. Define $e^\circ := \text{Can}(e)$. If $\text{Dec}_r(e^\circ)$ is undefined, then (r, e) does not define a column. If $\text{Dec}_r(e^\circ)$ is defined, let $\pi := \text{Dec}_r(e^\circ)$ and define

$$L(r, e) := L_r(\pi), \quad c(r, e) := \text{Cost}_r(\pi).$$

Define the column projector

$$\Pi(r, e) := (r, L(r, e), c(r, e)).$$

The object $\Pi(r, e)$ is a finite tuple consisting of a query identifier, a finite integer list, and a reduced rational number.

11.1.7 Column vector map in the restricted master problem

Let $w_r \in \mathbb{Q}_{\geq 0}$ denote the fixed workload weight of query r . For each query $r \in Q$, let $E_r \subseteq \Sigma^*$ denote the current restricted set of admitted codes for query r . Each admitted pair (r, e) corresponds to a restricted master variable $\lambda_{r,e}$.

The restricted master contains a plan selection equality for each $r \in Q$:

$$\sum_{e \in E_r} \lambda_{r,e} = 1. \tag{11.1}$$

For each $(r, s) \in Q \times S$, the restricted master contains a coupling inequality row enforcing that an object must be built if required by the chosen plan:

$$x_s - \sum_{e \in E_r: s \in \text{Req}_r(\text{Dec}_r(\text{Can}(e)))} \lambda_{r,e} \geq 0. \tag{11.2}$$

For an admissible (r, e) , the coefficients of $\lambda_{r,e}$ are uniquely determined by $L(r, e)$ and $c(r, e)$:

- the objective coefficient is $w_r c(r, e)$;
- the coefficient in (11.1) is 1;
- the coefficient in the coupling row (11.2) indexed by (r, s) is -1 if and only if $\text{id}(s) \in L(r, e)$, and is 0 otherwise;
- all remaining coefficients are 0 for rows that are independent of the plan choice.

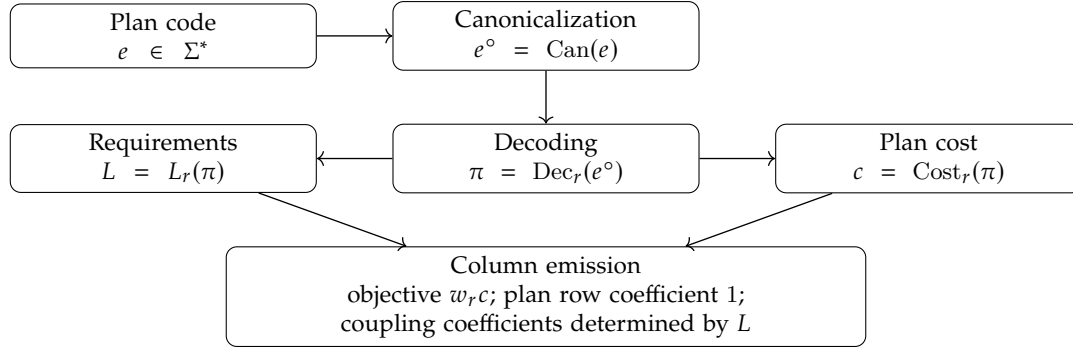


Figure 11.1: Deterministic pipeline from a plan code to the corresponding restricted master column coefficients.

Therefore, the sparse signature $\Pi(r, e) = (r, L(r, e), c(r, e))$ determines the restricted master column vector of $\lambda_{r,e}$.

11.2 Column Uniqueness and Deduplication

11.2.1 Column equivalence and signature identity

Fix $r \in Q$. Two admissible codes $e, e' \in \Sigma^*$ are called column equivalent for r if the induced column vectors of $\lambda_{r,e}$ and $\lambda_{r,e'}$ have identical coefficients in every row of the restricted master. Define

$$\text{ColSig}(r, e) := \Pi(r, e) = (r, L(r, e), c(r, e)).$$

11.2.2 Uniqueness of the sparse signature

Lemma 10.1 (Uniqueness of the sparse signature). Fix $r \in Q$. For any two admissible codes $e, e' \in \Sigma^*$, the induced restricted master column vectors of $\lambda_{r,e}$ and $\lambda_{r,e'}$ are identical if and only if

$$\text{ColSig}(r, e) = \text{ColSig}(r, e').$$

Proof. Assume $\text{ColSig}(r, e) = \text{ColSig}(r, e')$. Then $L(r, e) = L(r, e')$ and $c(r, e) = c(r, e')$. Equality of c implies equality of objective coefficients $w_r c(r, e) = w_r c(r, e')$. Equality of lists implies that for each $s \in S$, $\text{id}(s) \in L(r, e)$ holds if and only if $\text{id}(s) \in L(r, e')$. Hence the coupling rows with coefficient -1 coincide, the plan selection coefficient equals 1 in both columns, and all other coefficients are 0 in both columns. Therefore the column vectors are identical.

Conversely, assume the column vectors are identical. The equality of coupling coefficients implies that for each $s \in S$, s is required by the decoded canonical plan for e if and only if s is required by the decoded canonical plan for e' . Therefore $\text{Req}_r(\text{Dec}_r(\text{Can}(e))) = \text{Req}_r(\text{Dec}_r(\text{Can}(e')))$ and the sorted identifier lists are equal, so $L(r, e) = L(r, e')$. The objective coefficients are equal. If $w_r > 0$ then this implies $c(r, e) = c(r, e')$; if $w_r = 0$

then cost equality is enforced by the certificate checks defined in Section 11.3, since the verifier recomputes Cost_r from the decoded plan. Hence $\text{ColSig}(r, e) = \text{ColSig}(r, e')$. $\square$

11.2.3 Deduplication rule

For each query $r \in Q$, the solver maintains the set of admitted signatures $\{\text{ColSig}(r, e) : e \in E_r\}$. A newly proposed code e is admitted as a distinct column if and only if $\Pi(r, e)$ is defined and $\text{ColSig}(r, e)$ is not already present in the admitted signature set.

11.3 Reduced Cost Certificate Format

11.3.1 Dual variable interface

Let the restricted master problem at the current node be solved to optimality in exact arithmetic, yielding dual variables for the rows that involve λ columns. Let $\alpha_r \in \mathbb{Q}$ denote the dual variable associated with the plan selection equality (11.1) for query r . Let $\beta_{r,s} \in \mathbb{Q}_{\geq 0}$ denote the dual variable associated with the coupling inequality (11.2) indexed by (r, s) .

11.3.2 Reduced cost definition

For any admissible (r, e) , define the reduced cost under duals (α, β) by

$$\text{rc}(r, e; \alpha, \beta) := w_r c(r, e) - \alpha_r + \sum_{s \in \text{Req}_r(\text{Dec}_r(\text{Can}(e)))} \beta_{r,s}. \quad (11.3)$$

The sum is finite and is evaluated in reduced rational arithmetic.

11.3.3 Certificate tuple and verifier

A reduced cost certificate for a proposed column is a finite tuple

$$C := (r, e, L, c, \rho),$$

where $r \in Q$, $e \in \Sigma^*$, L is a finite strictly increasing list with entries in $\{1, \dots, n\}$, $c \in \mathbb{Q}_{\geq 0}$ is a reduced rational, and $\rho \in \mathbb{Q}$ is a reduced rational.

Define a deterministic verifier predicate

$$V(\alpha, \beta; C) \in \{0, 1\}$$

that returns 1 if and only if all checks below hold.

Check 1 (Definedness). Compute $e^\circ = \text{Can}(e)$ and verify that $\pi = \text{Dec}_r(e^\circ)$ is defined.

Check 2 (Requirement list correctness). Compute $L^* = L_r(\pi)$ and verify $L^* = L$ by exact list equality.

Table 11.1: *Reduced cost certificate schema and verifier obligations.*

Field	Domain	Verifier obligations
r	Query identifier $r \in Q$	Select decoder Dec_r and dual component α_r , and select coupling duals $\beta_{r,s}$ for $s \in S$.
e	Plan code $e \in \Sigma^*$	Compute $e^\circ = \text{Can}(e)$; verify $\text{Dec}_r(e^\circ)$ is defined.
L	Strictly increasing list in $\{1, \dots, n\}$	Compute $L^* = L_r(\pi)$ for $\pi = \text{Dec}_r(e^\circ)$; verify $L^* = L$.
c	Reduced rational $c \in \mathbb{Q}_{\geq 0}$	Compute $c^* = \text{Cost}_r(\pi)$; verify $c^* = c$.
ρ	Reduced rational $\rho \in \mathbb{Q}$	Compute ρ^* using (11.3) with π and duals; verify $\rho^* = \rho$.

Check 3 (Cost correctness). Compute $c^* = \text{Cost}_r(\pi)$ and verify $c^* = c$ by rational equality.

Check 4 (Reduced cost correctness). Compute

$$\rho^* = w_r c^* - \alpha_r + \sum_{s \in \text{Req}_r(\pi)} \beta_{r,s}$$

and verify $\rho^* = \rho$ by rational equality.

11.4 Soundness of Accepted Column Certificates

Theorem 10.2 (Soundness). Fix any dual snapshot (α, β) . If $V(\alpha, \beta; C) = 1$ for $C = (r, e, L, c, \rho)$, then the following statements hold.

First, the column projector is defined and correct:

$$\Pi(r, e) = (r, L, c).$$

Second, the restricted master column coefficients of $\lambda_{r,e}$ are exactly those determined by L and c as specified in Section 11.1.7. Third, the reported reduced cost ρ equals the exact reduced cost:

$$\rho = \text{rc}(r, e; \alpha, \beta).$$

Proof. Assume $V(\alpha, \beta; C) = 1$. By Check 1, $e^\circ = \text{Can}(e)$ yields a defined decoding $\pi = \text{Dec}_r(e^\circ)$, hence (r, e) defines a column. By Check 2, $L = L_r(\pi)$, so for each $s \in S$,

$$\text{id}(s) \in L \iff s \in \text{Req}_r(\pi).$$

Therefore the set of coupling rows (11.2) that receive coefficient -1 from $\lambda_{r,e}$ is exactly the set indexed by $\{s \in S : \text{id}(s) \in L\}$. The plan selection row coefficient is 1 by definition. All coefficients in rows independent of the plan are 0. Hence the full column coefficient vector of $\lambda_{r,e}$ is determined by L .

By Check 3, $c = \text{Cost}_r(\pi)$, hence the objective coefficient $w_r c$ is correct. By Check 4, the verifier computes ρ^* using the same deterministic requirement set and the same cost, and accepts only if $\rho^* = \rho$. Since ρ^* is precisely the value of (11.3), it follows that $\rho = \text{rc}(r, e; \alpha, \beta)$. $\square$

Existence of certificates. For every admissible (r, e) and any dual snapshot (α, β) , the tuple

$$(r, e, L(r, e), c(r, e), \text{rc}(r, e; \alpha, \beta))$$

is a certificate that is accepted by $V(\alpha, \beta; \cdot)$, because each verifier check recomputes the same deterministic quantities recorded in the tuple.

12

Cutting Planes: Formal Cut Language and Validity

For the cutting-plane layer, we treat cuts as certified separating inequalities over a relaxation, drawing on the classical Gomory and disjunctive traditions (Gomory, 1960; Balas, 1979; Cornuéjols, 2007). The purpose of this chapter is to specify a finite cut language and a deterministic, purely arithmetic validity check for every admitted inequality at a node. A cut is admissible only if it is accompanied by a proof object that passes the mechanical check stated in this chapter. The statements in this chapter establish that mechanical acceptance implies mathematical validity for the mixed-integer feasible set of the node.

12.1 Admissible cut classes

12.1.1 Cut family registry

We fix a branch-and-bound node N and define the node ledger as a finite inequality system together with a designated integer index set. All cut families in this chapter are defined relative to this ledger.

Definition 11.1 (Node ledger). The node ledger is a triple $\mathcal{L}_N = (A, b, J)$ where $A \in \mathbb{Q}^{m \times d}$, $b \in \mathbb{Q}^m$, and $J \subseteq \{1, \dots, d\}$ is the set of coordinates required to be integral at node-feasible integer points. The ledger represents the linear relaxation system

$$Az \leq b, \tag{12.1}$$

where $z \in \mathbb{R}^d$. The ledger contains the original relaxation rows, variable bounds, branching fixings, and all previously admitted cuts.

Definition 11.2 (Node mixed-integer feasible set). The node mixed-integer feasible set is

$$K_N = \{z \in \mathbb{R}^d : Az \leq b, z_j \in \mathbb{Z} \text{ for all } j \in J\}. \quad (12.2)$$

Let $C = \{1, \dots, d\} \setminus J$ denote the complementary set of continuous coordinates.

Definition 11.3 (Registry of admissible cut families). Two cut families are admissible.

1. **Family A, projected rank-one floor cuts.** These cuts are derived from a nonnegative rational combination of ledger rows whose coefficients vanish on C and are integral on J , followed by an integer floor tightening on the right-hand side.
2. **Family B, knapsack cover cuts on binary design coordinates.** These cuts are derived from a ledger row that is an integer knapsack inequality over a subset of binary design variables, together with a verified cover set.

12.1.2 Validity predicate

Definition 11.4 (Cut and validity predicate). A cut is a linear inequality $\gamma^\top z \leq \delta$ with $\gamma \in \mathbb{Q}^d$ and $\delta \in \mathbb{Q}$. The validity predicate at node N is

$$\text{Valid}_N(\gamma, \delta) \iff \forall z \in K_N, \gamma^\top z \leq \delta. \quad (12.3)$$

Remark 11.1 (Admissibility discipline). Only inequalities that are output by the certificate checks in Section 11.2 are permitted to enter the ledger. All proofs in Section 11.3 are conditional only on the explicit checks in Section 11.2 and on the definition of K_N in (12.2).

12.2 Cut certificates

12.2.1 Certificate schema

All certificates are finite objects over integers and rationals. The certificate schemas below are written so that every field is either a ledger row identifier, an explicit rational, or an explicit finite set of indices.

Family A, projected rank-one floor cuts

Definition 11.5 (Family A certificate schema). A Family A certificate is a tuple

$$C_A = (J, W, g, h), \quad (12.4)$$

where

1. J is the ledger integer index set.
2. W is a finite witness list of pairs $\{(i, \omega_i)\}$ with $i \in \{1, \dots, m\}$ and $\omega_i \in \mathbb{Q}_{\geq 0}$.

3. $g \in \mathbb{Z}^{|J|}$ is the claimed cut coefficient vector on J .
4. $h \in \mathbb{Z}$ is the claimed cut right-hand side.

The intended admitted inequality is

$$g^\top z_J \leq h, \quad (12.5)$$

where z_J is the restriction of z to coordinates in J .

Family B, knapsack cover cuts on binary coordinates

Definition 11.6 (Family B certificate schema). A Family B certificate is a tuple

$$C_B = (i, S, \{(j, w_j)\}_{j \in S}, W, C), \quad (12.6)$$

where

1. $i \in \{1, \dots, m\}$ is the ledger row identifier.
2. $S \subseteq J$ is the support set of binary coordinates for that row.
3. For each $j \in S$, the coefficient $w_j \in \mathbb{Z}_{>0}$ is an explicitly listed integer weight.
4. $W \in \mathbb{Z}_{\geq 0}$ is the explicitly listed integer capacity.
5. $C \subseteq S$ is the proposed cover set.

The intended admitted inequality is

$$\sum_{j \in C} z_j \leq |C| - 1. \quad (12.7)$$

12.2.2 Mechanical check procedure

The mechanical check is deterministic and purely arithmetic. It accepts a certificate if and only if all stated conditions hold exactly over $\mathbb{Z}$ and $\mathbb{Q}$.

Mechanical check for Family A

Procedure 11.1 (Family A check). Given $\mathcal{L}_N = (A, b, J)$ and $C_A = (J, W, g, h)$, perform the following checks.

1. **Witness nonnegativity.** Verify that each ω_i in W is a nonnegative rational.
2. **Row combination.** Form the multiplier vector $\omega \in \mathbb{Q}_{\geq 0}^m$ by setting ω_i for listed indices and 0 otherwise. Compute

$$a^\top = \omega^\top A, \quad \beta = \omega^\top b. \quad (12.8)$$

3. **Projection and annihilation.** Split a into (a_J, a_C) . Verify

$$a_C = 0 \quad (12.9)$$

exactly as rationals.

4. **Integrality and coefficient match.** Verify that every component of a_J is an integer, meaning its rational denominator equals 1, and that

$$a_J = g \quad (12.10)$$

as integer vectors.

5. **Right-hand side match.** Compute the integer floor $\lfloor \beta \rfloor$ and verify

$$h = \lfloor \beta \rfloor. \quad (12.11)$$

If all checks pass, the admitted cut is (12.5). Otherwise, the certificate is rejected.

Mechanical check for Family B

Procedure 11.2 (Family B check). Given $\mathcal{L}_N = (A, b, J)$ and $C_B = (i, S, \{(j, w_j)\}_{j \in S}, W, C)$, perform the following checks.

1. **Set consistency.** Verify $C \subseteq S$ and $S \subseteq J$.
2. **Knapsack row identity.** Read ledger row i , written $A_i z \leq b_i$. Verify that b_i is an integer and that

$$A_{i,j} = \begin{cases} w_j & \text{if } j \in S, \\ 0 & \text{if } j \notin S, \end{cases} \quad \text{and} \quad b_i = W. \quad (12.12)$$

3. **Cover condition.** Compute $\sum_{j \in C} w_j$ and verify

$$\sum_{j \in C} w_j > W. \quad (12.13)$$

If all checks pass, the admitted cut is (12.7). Otherwise, the certificate is rejected.

12.3 Validity theorems

12.3.1 Family-level validity theorems

Auxiliary lemmas

Lemma 11.1 (Nonnegative combination preserves validity). Let $S \subseteq \mathbb{R}^d$. Suppose that for every $i \in \{1, \dots, m\}$ and every $z \in S$, the inequality $A_i z \leq b_i$ holds. Let $\omega \in \mathbb{Q}_{\geq 0}^m$. Then for every $z \in S$,

$$(\omega^\top A)z \leq \omega^\top b. \quad (12.14)$$

Proof. Fix $z \in S$. For each i , $A_i z \leq b_i$. Multiply by $\omega_i \geq 0$ to obtain $\omega_i A_i z \leq \omega_i b_i$. Summing over i yields $\sum_i \omega_i A_i z \leq \sum_i \omega_i b_i$, which is (12.14). Since z was arbitrary, the statement holds for all $z \in S$.

Lemma 11.2 (Floor tightening for integer values). Let $t \in \mathbb{Z}$ and $\beta \in \mathbb{Q}$. If $t \leq \beta$, then $t \leq \lfloor \beta \rfloor$.

Proof. By definition, $\lfloor \beta \rfloor$ is the greatest integer k satisfying $k \leq \beta$. Since t is an integer and $t \leq \beta$, it follows that $t \leq \lfloor \beta \rfloor$.

Family A validity

Theorem 11.3 (Validity of accepted Family A cuts). If Procedure 11.1 accepts C_A and admits the inequality $g^\top z_J \leq h$, then $\text{Valid}_N(g, h)$ holds. Equivalently, for all $z \in K_N$, $g^\top z_J \leq h$.

Proof. Assume Procedure 11.1 accepts. Let $z \in K_N$ be arbitrary. Then $Az \leq b$ and $z_J \in \mathbb{Z}^{|J|}$. Procedure 11.1 constructs $\omega \in \mathbb{Q}_{\geq 0}^m$ and computes $a^\top = \omega^\top A$ and $\beta = \omega^\top b$. By Lemma 11.1 applied to $S = \{z : Az \leq b\}$, we have $a^\top z \leq \beta$. By (12.9), $a_C = 0$, hence $a^\top z = a_J^\top z_J$. By (12.10), $a_J = g$ with $g \in \mathbb{Z}^{|J|}$. Therefore $g^\top z_J$ is an integer and $g^\top z_J \leq \beta$. By Lemma 11.2, $g^\top z_J \leq \lfloor \beta \rfloor$. By (12.11), $h = \lfloor \beta \rfloor$. Hence $g^\top z_J \leq h$. Since z was arbitrary, validity holds for all $z \in K_N$.

Family B validity

Theorem 11.4 (Validity of accepted Family B cover cuts). If Procedure 11.2 accepts C_B and admits the inequality $\sum_{j \in C} z_j \leq |C| - 1$, then this inequality is valid for K_N .

Proof. Assume Procedure 11.2 accepts. Let $z \in K_N$ be arbitrary. Then z satisfies ledger row i , which by (12.12) is $\sum_{j \in S} w_j z_j \leq W$. Suppose for contradiction that $\sum_{j \in C} z_j \geq |C|$. Since each z_j is binary on S by the model registry for design variables, it follows that $z_j = 1$ for every $j \in C$. Hence

$$\sum_{j \in S} w_j z_j \geq \sum_{j \in C} w_j z_j = \sum_{j \in C} w_j.$$

By (12.13), $\sum_{j \in C} w_j > W$, which implies $\sum_{j \in S} w_j z_j > W$, contradicting $\sum_{j \in S} w_j z_j \leq W$. Therefore $\sum_{j \in C} z_j \leq |C| - 1$, and since z was arbitrary, the cut is valid for all $z \in K_N$.

12.3.2 Composition and aggregation rules

Proposition 11.5 (Ledger validity preservation under cut admission). Assume the ledger is initialized with inequalities valid for K_N and is extended only by inequalities admitted through Procedure 11.1 or Procedure 11.2. Then every inequality present in the ledger at every time is valid for K_N .

Cut certificate anatomy diagram placeholder.
 This figure must depict the node ledger (A, b, J) , a certificate object C_A or C_B , the mechanical check steps, and the admitted inequality.

Figure 12.1: Cut certificate anatomy: ledger, certificate object, mechanical acceptance, admitted cut.

Family	Admitted inequality	Certificate acceptance conditions (mechanical)
A	Projected rank-one floor cut $g^\top z_J \leq h$	Witness W is nonnegative, $a^\top = \omega^\top A$ and $\beta = \omega^\top b$ are computed exactly, annihilation $a_C = 0$, integrality and match $a_J = g \in \mathbb{Z}^{ J }$, and right-hand side match $h = \lfloor \beta \rfloor$.
B	Cover cut $\sum_{j \in C} z_j \leq C - 1$	Row i is exactly a knapsack inequality on $S \subseteq J$ with integer weights $\{w_j\}$ and integer capacity W , and the cover condition $\sum_{j \in C} w_j > W$ holds.

Table 12.1: Cut family registry and mechanically checkable admission conditions.

Proof. By assumption, all initial ledger inequalities are valid for K_N . Each extension step adds exactly one inequality. If it is admitted by Procedure 11.1, it is valid by Theorem 11.3. If it is admitted by Procedure 11.2, it is valid by Theorem 11.4. Hence validity is preserved inductively.

Proposition 11.6 (Conjunction stability). If $\gamma_1^\top z \leq \delta_1$ and $\gamma_2^\top z \leq \delta_2$ are valid for K_N , then the system containing both inequalities is valid for K_N .

Proof. For every $z \in K_N$, each inequality holds, hence both hold simultaneously.

12.4 Figures and tables

- Cut certificate anatomy diagram.
- Cut family registry table.

13

Separation and Cut Admission Control

13.1 Proposal versus acceptance separation

13.1.1 Proposal interfaces (including learned proposals)

Fix a branch-and-bound node N . Let the current linear relaxation be the rational polyhedron

$$P_N = \{z \in \mathbb{R}^d : Az \leq b\}, \quad A \in \mathbb{Q}^{m \times d}, \quad b \in \mathbb{Q}^m. \quad (13.1)$$

Let $J \subseteq \{1, \dots, d\}$ denote the index set of variables constrained to be integral at node N . The mixed-integer feasible set at node N is

$$K_N = \{z \in \mathbb{R}^d : Az \leq b, \quad z_J \in \mathbb{Z}^{|J|}\}. \quad (13.2)$$

Let $z^\star \in P_N$ denote an optimal solution of the relaxation, represented exactly in $\mathbb{Q}^d$.

A (candidate) cut is a linear inequality

$$\gamma^\top z \leq \delta, \quad \gamma \in \mathbb{Q}^d, \quad \delta \in \mathbb{Q}. \quad (13.3)$$

It is *valid at node N* if $\gamma^\top z \leq \delta$ holds for every $z \in K_N$. It is *violated at the current relaxation solution* if $\gamma^\top z^\star > \delta$.

The separation subsystem operates in two logically disjoint stages. First, a proposer produces a *proposal record* that claims a cut candidate. Second, an acceptance gate verifies the proposal by certificate and decides admission. This division is mandatory: the solver state is allowed to change only by gate acceptance, never by proposal provenance.

Formally, for each cut family F the system defines:

- a candidate space C_F of proposal payloads;
- an interpretation map $\text{Cut}_F : C_F \rightarrow (\mathbb{Q}^d \times \mathbb{Q})$ that deterministically yields (γ, δ) ;

- a certificate language $\mathcal{S}_F$ and a verifier Verify_F ;
- a gate predicate Gate_F that applies only finite, exact checks in $\mathbb{Q}$.

A proposal record is a tuple

$$r = (F, c, s, \text{meta}), \quad c \in C_F, \quad s \in \mathcal{S}_F, \quad (13.4)$$

where meta may contain provenance information, including learned proposer identifiers, but meta is informational only and is not permitted to influence correctness.

A learned proposer is any algorithm that outputs proposal records by a learned policy. The system permits learned proposals solely because the acceptance gate is proof-carrying: the only way a learned proposal affects the relaxation is by supplying a certificate that passes verification.

13.1.2 Acceptance gate definition

The acceptance gate for family F is a deterministic function

$$\text{Gate}_F : (A, b, z^\star) \times C_F \times \mathcal{S}_F \longrightarrow \{\text{REJECT}, \text{ACCEPT}\} \times (\mathbb{Q}^d \times \mathbb{Q}). \quad (13.5)$$

Given node state $(A, b, z^\star)$ and record (c, s) , the gate computes $(\gamma, \delta) = \text{Cut}_F(c)$ and returns ACCEPT if and only if all of the following conditions hold:

1. **Well-formedness.** The candidate and certificate parse and satisfy the family schema.
2. **Canonicalization.** (γ, δ) is placed in a canonical rational representation, including fraction reduction and a fixed sign convention, so that admission is deterministic.
3. **Certificate verification.** $\text{Verify}_F(A, b, c, s)$ returns true in exact arithmetic.
4. **Violation.** $\gamma^\top z^\star > \delta$ holds in exact arithmetic.
5. **Ledger admissibility.** The cut is not a duplicate of an existing admitted cut under the canonical representation.

If any condition fails the gate returns REJECT and performs no state mutation. If accepted, the system appends the row $\gamma^\top z \leq \delta$ to the node ledger, updating (A, b) by row augmentation.

The gate is the sole authority for admission. Consequently, proposal acceptance depends only on certificate-verifiable facts, not on the identity, speed, or statistical behavior of the proposer.

13.2 Deterministic fallback separation

13.2.1 Completeness condition (if required)

Deterministic fallback separation is defined relative to a fixed, explicitly enumerated set of cut families. Let $\mathfrak{F}$ be the family registry activated at node N . The fallback is

called *complete relative to* $\mathfrak{F}$ if the following holds.

Relative completeness. If there exists a family $F \in \mathfrak{F}$ and a certificate-bearing candidate (c, s) such that the gate would accept and the interpreted cut is violated at z^* , then the fallback procedure returns an accepted cut in finite time.

Relative completeness does not claim existence of violated cuts in $\mathfrak{F}$, nor does it claim completeness over all valid inequalities. It claims only that the enumerated candidate sets are searched exhaustively and that gate-acceptance is discoverable by enumeration.

In the present report, $\mathfrak{F}$ contains the following two families, each with an explicit finite candidate set at node N .

Family F1 (bounded rank-1 Chvátal–Gomory certificates). Let $\pi \in \mathbb{Q}_{\geq 0}^m$ be a nonnegative multiplier vector. Define the multiplier-induced row

$$\alpha(\pi) = \pi^\top A \in \mathbb{Q}^{1 \times d}, \quad \beta(\pi) = \pi^\top b \in \mathbb{Q}. \quad (13.6)$$

Assume an integrality check set $I \subseteq \{1, \dots, d\}$ is fixed so that for all $z \in K_N$ the scalar $\alpha(\pi)z$ is integer whenever $\alpha(\pi)_i \in \mathbb{Z}$ for all $i \in I$. The certificate condition is

$$\alpha(\pi)_i \in \mathbb{Z} \quad \text{for all } i \in I. \quad (13.7)$$

The interpreted cut is

$$\alpha(\pi)z \leq \lfloor \beta(\pi) \rfloor, \quad (13.8)$$

where $\lfloor \cdot \rfloor$ is the exact floor on rationals.

Fix integers $K \geq 1$ and $D \geq 1$. Define the bounded certificate set $\Pi(K, D)$ as the set of multipliers π that satisfy:

1. $\pi \geq 0$ componentwise;
2. $|\{i : \pi_i \neq 0\}| \leq K$;
3. each π_i in lowest terms $\pi_i = p_i/q_i$ satisfies $0 \leq p_i \leq D$ and $1 \leq q_i \leq D$.

Then $\Pi(K, D)$ is finite. The F1 fallback enumerates $\Pi(K, D)$ in a fixed deterministic order and applies the gate to each candidate.

Family F2 (exact cover cuts for a knapsack row). Let S be a finite index set and let $x \in \{0, 1\}^{|S|}$ be the associated binary subvector of z . Assume an integral capacity row

$$\sum_{s \in S} a_s x_s \leq B, \quad a_s \in \mathbb{Z}_{\geq 0}, \quad B \in \mathbb{Z}_{\geq 0}. \quad (13.9)$$

A subset $C \subseteq S$ is a cover if $\sum_{s \in C} a_s > B$. The interpreted cover cut is

$$\sum_{s \in C} x_s \leq |C| - 1. \quad (13.10)$$

The F2 fallback enumerates subsets $C \subseteq S$ in a fixed deterministic order, checks the cover condition, checks violation at the current relaxation solution, and submits (C, s) to the gate with the corresponding certificate.

13.2.2 Termination rule

Termination rule. The fallback separator must enumerate only finite candidate sets, and every gate check must terminate by finite rational arithmetic. Accordingly, the fallback separator is terminating if it is defined as a finite sequence of loops over finite candidate spaces.

Theorem 13.1 (Termination of deterministic fallback separation). *Assume S is finite and $\Pi(K, D)$ is finite. Then the deterministic fallback separator that exhaustively enumerates all $C \subseteq S$ and all $\pi \in \Pi(K, D)$ terminates on every node state (A, b, z^*) .*

Proof. The power set of a finite set S is finite, hence exhaustive enumeration of $C \subseteq S$ terminates. By construction $\Pi(K, D)$ is finite, hence exhaustive enumeration of $\pi \in \Pi(K, D)$ terminates. Each gate evaluation consists of a finite number of exact rational computations and comparisons, hence terminates. A finite composition of terminating procedures terminates. $\square$

13.3 Soundness of cut admission

13.3.1 No invalid cut admission theorem

Lemma 13.1 (Validity of rank-1 Chvátal–Gomory cuts under certificate conditions). *Let $z \in K_N$ and let $\pi \in \mathbb{Q}_{\geq 0}^m$ satisfy the certificate condition (13.7). Then the cut (13.8) holds for z .*

Proof. From $Az \leq b$ and $\pi \geq 0$, one has $\alpha(\pi)z = \pi^\top Az \leq \pi^\top b = \beta(\pi)$. By the certificate condition, $\alpha(\pi)z$ is an integer for every $z \in K_N$. Let $k = \alpha(\pi)z \in \mathbb{Z}$. Then $k \leq \beta(\pi)$ implies $k \leq \lfloor \beta(\pi) \rfloor$ by the definition of floor. Substituting $k = \alpha(\pi)z$ yields (13.8). $\square$

Lemma 13.2 (Validity of cover cuts). *Let $x \in \{0, 1\}^{|S|}$ satisfy (13.9). If $C \subseteq S$ is a cover, then the inequality (13.10) holds.*

Proof. Assume, for contradiction, that $\sum_{s \in C} x_s = |C|$. Then $x_s = 1$ for all $s \in C$, hence $\sum_{s \in S} a_s x_s \geq \sum_{s \in C} a_s > B$, contradicting (13.9). Therefore $\sum_{s \in C} x_s \leq |C| - 1$. $\square$

Theorem 13.2 (No invalid cut admission). *Every cut admitted by the acceptance gate at node N is valid for K_N .*

Proof. A cut is admitted only if some family gate returns ACCEPT, which entails certificate verification and canonical interpretation. If the admitted cut is an F1 cut, then the certificate condition (13.7) and nonnegativity of multipliers are verified by the gate, so Lemma 13.1 applies and the admitted inequality holds for all $z \in K_N$. If the admitted cut is an F2 cover cut, then the gate verifies the cover condition and the canonical

Admission check	Exact requirement enforced by the gate
Schema validation	Candidate and certificate parse; all mandatory fields present; family identifier valid.
Canonicalization	Coefficients reduced to canonical rationals; fixed sign convention; deterministic hashing.
Certificate verification	$\text{Verify}_F(A, b, c, s)$ returns true in exact arithmetic.
Violation test	$\gamma^\top z^\star > \delta$ holds in exact arithmetic.
Ledger admissibility	Not a duplicate of an already admitted cut under canonical representation.
Failure handling	On failure: hard reject, auditable event, optional proposer quarantine, no state mutation.

Table 13.1: Admission checklist executed for every proposed cut prior to ledger augmentation.

cover-cut form, so Lemma 13.2 applies and the admitted inequality holds for all feasible binary selections compatible with (13.9), hence for all $z \in K_N$ under the embedding of x into z . In all cases, admitted cuts are valid for K_N . $\square$

13.3.2 Certificate failure handling

Certificate failure handling is defined as a safety-preserving, deterministic response to any proposal record whose certificate does not verify.

Rule 1 (hard rejection). If $\text{Verify}_F(A, b, c, s)$ returns false, then the system must return REJECT and must not modify (A, b) .

Rule 2 (auditable logging). On certificate failure, the system must emit an auditable event containing: the family identifier F , a hash of the canonical interpreted cut (γ, δ) , a hash of the certificate payload, and the exact failure reason code returned by the verifier. The event must not alter feasibility or bounds.

Rule 3 (proposer quarantine). If a proposer produces certificate failures above a fixed threshold within a node or within a run, the proposer interface may be disabled for the remainder of the run. Disabling a proposer does not change soundness, because soundness is enforced solely by the gate.

Rule 4 (deterministic fallback continuity). Certificate failure must not interrupt deterministic fallback separation. If a proposal is rejected, the controller proceeds to the next proposal or to fallback enumeration without changing the relaxation state.

13.4 Figures and tables

- Separation controller diagram.
- Admission checklist.

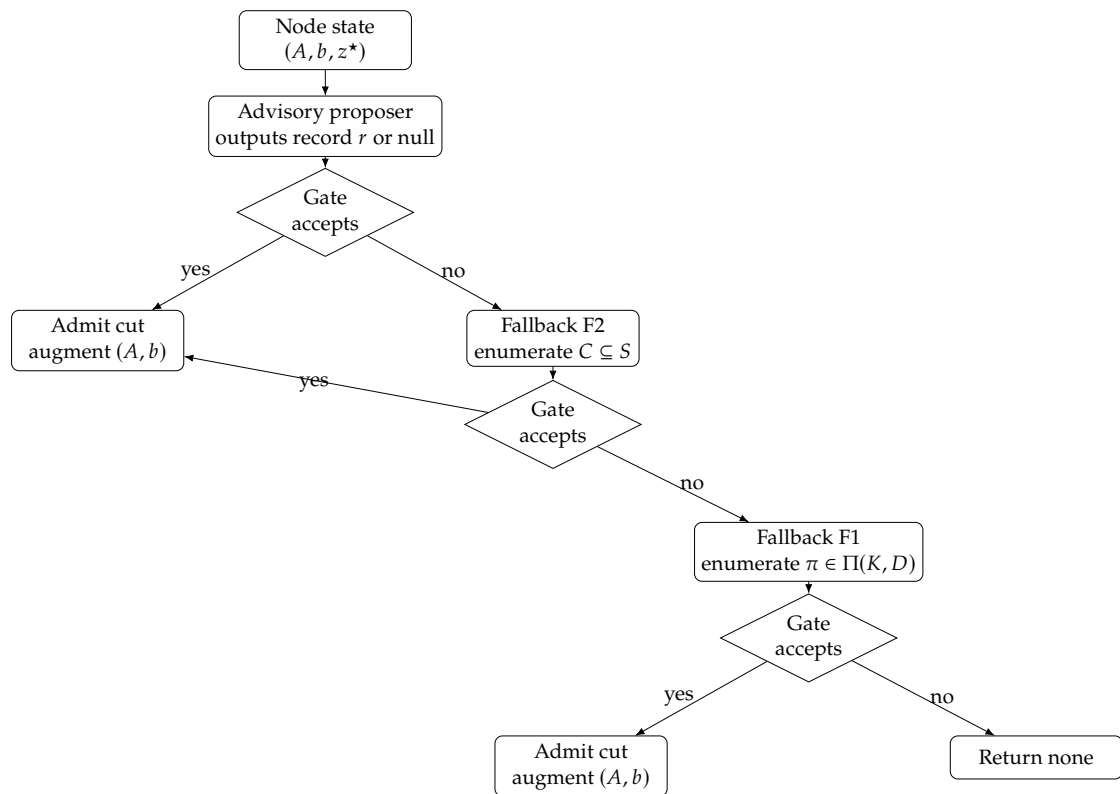


Figure 13.1: Separation controller with proposal stage, gate enforcement, and deterministic fallback enumeration.

14

Cut Management and Invariant Preservation

14.1 Cut lifecycle

14.1.1 Add, activate, deactivate, retire

Let the baseline mixed-integer model be defined over variables $z \in \mathbb{R}^n$ with integer index set $I \subseteq \{1, \dots, n\}$ and integrality domain

$$\mathbb{Z}_I = \{z \in \mathbb{R}^n : z_i \in \mathbb{Z} \text{ for all } i \in I\}.$$

Let the base constraint system be $Az \leq b$ and objective be $\min c^\top z$. The base integer feasible set is

$$F = \{z \in \mathbb{Z}_I : Az \leq b\}.$$

A branch-and-bound node N induces node constraints $D_N z \leq e_N$ and node integer feasible set

$$F_N = \{z \in \mathbb{Z}_I : Az \leq b, D_N z \leq e_N\}.$$

Define

$$\text{OPT}(N) = \inf\{c^\top z : z \in F_N\},$$

with the convention $\text{OPT}(N) = +\infty$ when $F_N = \emptyset$.

A cut is a triple $\kappa = (a, \beta, \sigma)$ with $a \in \mathbb{Q}^n, \beta \in \mathbb{Q}$, and scope descriptor σ , representing the inequality $a^\top z \leq \beta$.

Each node N exposes a scope descriptor σ_N . Scope safety at node N is defined by

$$\text{ScopeSafe}(N, \kappa) \iff \sigma_N \subseteq \sigma \quad \text{for } \kappa = (a, \beta, \sigma).$$

Definition 14.1 (Scoped validity). A cut $\kappa = (a, \beta, \sigma)$ is σ -valid if and only if for every node N satisfying $\sigma_N \subseteq \sigma$, and for every $z \in F_N$, the inequality $a^\top z \leq \beta$ holds.

A deterministic verifier $\text{Verify}(\kappa, \pi)$ checks a proposed cut κ with certificate payload π and returns ACCEPT or REJECT.

Assumption 14.1 (Verifier soundness). If $\text{Verify}(\kappa, \pi) = \text{ACCEPT}$, then κ is σ -valid in the sense of Definition 14.1.

Node relaxation under active cuts. For node N and an active cut set A_N , define the relaxation feasible region

$$R_N(A_N) = \{z \in \mathbb{R}^n : Az \leq b, D_N z \leq e_N, a^\top z \leq \beta \text{ for all } (a, \beta, \sigma) \in A_N\}.$$

Define the node lower bound

$$\text{LB}(N) = \inf\{c^\top z : z \in R_N(A_N)\},$$

with the convention $\text{LB}(N) = +\infty$ when $R_N(A_N) = \emptyset$.

Lifecycle transitions. The cut manager is a state transition system over the state variables defined in Section 14.1.2. Its transitions are the following.

Add: Given a proposed cut κ with certificate π , if $\text{Verify}(\kappa, \pi) = \text{ACCEPT}$ then insert κ into the ledger set L . Otherwise perform no state change.

Activate: Given a node N and a ledger cut $\kappa \in L$, activation is permitted only if $\text{ScopeSafe}(N, \kappa)$ holds, in which case update $A_N \leftarrow A_N \cup \{\kappa\}$.

Deactivate: Given a node N and a cut $\kappa \in A_N$, update $A_N \leftarrow A_N \setminus \{\kappa\}$.

Anchor: When a pruning transcript is recorded, every cut identifier referenced by the transcript is inserted into the anchor set H .

Retire: Given a ledger cut $\kappa \in L$, retirement is permitted only if $\kappa \notin H$. When permitted, remove κ from the ledger and from every node active set in which it appears, thereby preserving the inclusion $A_N \subseteq L$ for all nodes.

14.1.2 State variables for cut management

The cut manager maintains the following sets.

- Ledger set L , containing all stored cuts together with immutable identifiers and certificate payloads.
- Anchor set $H \subseteq L$, containing all cuts referenced by any stored pruning transcript.
- Core set $K \subseteq L$, containing cuts designated as non-deactivatable by implementation policy. Correctness does not depend on K .
- Node-local active sets $\{A_N\}$, one for each node N , with $A_N \subseteq L$.

The cut manager state at any time is

$$\mathcal{S} = (L, H, K, \{A_N\}).$$

14.2 Invariant preservation per transition

14.2.1 Preservation lemmas

Definition 14.2 (Cut management invariants). The cut manager maintains the following invariants for all times and all nodes N .

1. **Ledger validity:** every $\kappa \in L$ is σ -valid.
2. **Scope safety for active cuts:** for every $\kappa \in A_N$, $\text{ScopeSafe}(N, \kappa)$ holds.
3. **Active subset of ledger:** for every node N , $A_N \subseteq L$.
4. **Anchor persistence:** $H \subseteq L$.
5. **Integer containment:** for every node N , $F_N \subseteq R_N(A_N)$.
6. **Lower bound soundness:** for every node N , $\text{LB}(N) \leq \text{OPT}(N)$.

Lemma 14.1 (Preservation under Add). *Assume Assumption 14.1. The Add transition preserves ledger validity, and it does not violate scope safety, active subset of ledger, or anchor persistence.*

Proof. A cut is inserted into L only if the verifier returns ACCEPT, which implies σ -validity by Assumption 14.1. No element is removed from any set by Add. $\square$

Lemma 14.2 (Preservation under Activate). *The Activate transition preserves scope safety for active cuts and preserves $A_N \subseteq L$.*

Proof. Activation is permitted only when $\text{ScopeSafe}(N, \kappa)$ holds, and only for $\kappa \in L$. Therefore inserting κ into A_N preserves both claims. $\square$

Lemma 14.3 (Preservation under Deactivate). *The Deactivate transition preserves all invariants in Definition 14.2 that are stated as universal properties over elements of L or A_N .*

Proof. Deactivation only removes elements from A_N and does not modify L or H . Universal properties over L are unchanged, and universal properties over A_N are preserved under removal. $\square$

Lemma 14.4 (Preservation under Anchor). *The Anchor transition preserves $H \subseteq L$ and does not affect any node active set.*

Proof. Anchoring only inserts into H cut identifiers that refer to already stored ledger cuts. No removal occurs. $\square$

Lemma 14.5 (Preservation under Retire). *The Retire transition preserves anchor persistence and preserves $A_N \subseteq L$ for every node N .*

Proof. Retire is permitted only if $\kappa \notin H$, so removal from L cannot remove an anchored cut. Retire also removes κ from every A_N before or simultaneously with its removal from L , thereby preserving $A_N \subseteq L$. $\square$

Lemma 14.6 (Integer containment from validity and scope safety). *Assume ledger validity, scope safety for active cuts, and $A_N \subseteq L$. Then for every node N , $F_N \subseteq R_N(A_N)$.*

Proof. Fix node N and $z \in F_N$. Then $Az \leq b$ and $D_N z \leq e_N$ hold. Let $\kappa = (a, \beta, \sigma) \in A_N$. Since $A_N \subseteq L$, we have $\kappa \in L$ and hence κ is σ -valid by ledger validity. Scope safety gives $\sigma_N \subseteq \sigma$, so scoped validity implies $a^\top z \leq \beta$. Thus z satisfies all inequalities defining $R_N(A_N)$, proving containment. $\square$

Lemma 14.7 (Lower bound soundness from integer containment). *If $F_N \subseteq R_N(A_N)$, then $\text{LB}(N) \leq \text{OPT}(N)$.*

Proof. Both $\text{LB}(N)$ and $\text{OPT}(N)$ optimize the same linear objective $c^\top z$ over sets with $F_N \subseteq R_N(A_N)$. Therefore the infimum over $R_N(A_N)$ is less than or equal to the infimum over F_N . $\square$

14.2.2 Failure and rollback semantics

The cut manager defines failure as a guarded transition that does not satisfy its preconditions. Failure has rollback semantics implemented as an explicit no-op on the state S .

- **Add failure:** if $\text{Verify}(\kappa, \pi) = \text{REJECT}$, the state is unchanged.
- **Activate failure:** if $\kappa \notin L$ or if $\text{ScopeSafe}(N, \kappa)$ is false, the state is unchanged.
- **Deactivate failure:** if $\kappa \notin A_N$, the state is unchanged.
- **Retire failure:** if $\kappa \in H$ or if $\kappa \notin L$, the state is unchanged.

Failure cannot violate invariants because it induces no state change. Therefore all correctness properties derived from invariants are preserved under failure.

14.3 Correctness theorem for cut management

14.3.1 Bound correctness preservation

Theorem 14.1 (Cut management correctness). *Assume Assumption 14.1. Consider any execution in which the only modifications to cut state occur through the transitions Add, Activate, Deactivate, Anchor, and Retire, each applied only when its preconditions hold, and treated as a no-op otherwise. Then for every node N at every time:*

1. $F_N \subseteq R_N(A_N)$ holds.
2. $\text{LB}(N) \leq \text{OPT}(N)$ holds.
3. Every cut referenced by a stored pruning transcript remains in the ledger for the remainder of the execution.

Proof. Lemmas 14.1 to 14.5 imply preservation of ledger validity, scope safety for active cuts, $A_N \subseteq L$, and $H \subseteq L$ under all transitions. Lemma 14.6 yields $F_N \subseteq R_N(A_N)$, and Lemma 14.7 yields $\text{LB}(N) \leq \text{OPT}(N)$. For transcript persistence, any recorded

Figure 14.1: Cut lifecycle diagram showing *Add*, *Activate*, *Deactivate*, *Anchor*, and *Retire* transitions with their preconditions.

Transition	Preserved invariants and justification
Add	Ledger validity by Assumption 14.1. No removals, so $A_N \subseteq L$ and $H \subseteq L$ remain true.
Activate	Scope safety enforced by precondition $\text{ScopeSafe}(N, \kappa)$. Active subset preserved since $\kappa \in L$ is required.
Deactivate	Removal-only update on A_N preserves universal properties over A_N and does not affect L or H .
Anchor	Anchors only stored ledger cuts, preserving $H \subseteq L$.
Retire	Precondition $\kappa \notin H$ preserves $H \subseteq L$. Global removal from all A_N preserves $A_N \subseteq L$.

Table 14.1: Transition to invariant mapping. Each entry is sufficient to derive integer containment and lower bound soundness via Lemmas 14.6 and 14.7.

transcript anchors its referenced cuts, and anchored cuts cannot be retired by the Retire precondition, hence they remain in L thereafter. $\square$

14.3.2 Feasibility preservation

Corollary 14.1 (No loss of integer feasibility under cut activation). *For any node N and any sequence of Activate and Deactivate operations applied to A_N under the cut manager rules, the set of integer feasible solutions F_N is never reduced by enforcing active cuts. In particular, any $z \in F_N$ remains feasible in $R_N(A_N)$ at all times.*

Proof. Immediate from $F_N \subseteq R_N(A_N)$ in Theorem 14.1. $\square$

14.4 Figures and tables

15

Cut Strength, Redundancy, and Selection Discipline

15.1 Strength metrics

This chapter fixes a single branch and bound node and formalises the notions of cut strength, redundancy, and selection so that (i) validity is never compromised, (ii) redundancy elimination is exact with respect to the current relaxation, and (iii) any claimed efficiency property is stated only where it is derivable from the established objects and certificates.

15.1.1 Bound improvement measures

Let the node relaxation be a rational polyhedron

$$P = \{z \in \mathbb{R}^d : Gz \leq h\}, \quad G \in \mathbb{Q}^{m \times d}, \quad h \in \mathbb{Q}^m.$$

Let $J \subseteq \{1, \dots, d\}$ be the index set of integer coordinates. Define the mixed integer feasible set and its hull by

$$Z = \{z \in P : z_j \in \mathbb{Z} \text{ for all } j \in J\}, \quad H = \text{conv}(Z).$$

Definition 15.1 (Valid inequality and induced face). A pair $(a, b) \in \mathbb{Q}^d \times \mathbb{Q}$ is a valid inequality for Z if $a^\top z \leq b$ for all $z \in Z$. The induced face of H is

$$F(a, b) = \{z \in H : a^\top z = b\}.$$

Lemma 15.1 (Validity extends to the hull). *If $a^\top z \leq b$ holds for all $z \in Z$, then $a^\top z \leq b$ holds for all $z \in H$.*

Proof. Let $z \in H$. Then $z = \sum_{i=1}^k \lambda_i z^{(i)}$ for some $k \in \mathbb{N}$, $z^{(i)} \in Z$, $\lambda_i \geq 0$, and $\sum_{i=1}^k \lambda_i = 1$. Linearity gives $a^\top z = \sum_{i=1}^k \lambda_i (a^\top z^{(i)})$. Validity gives $a^\top z^{(i)} \leq b$ for each i . Multiply by

λ_i and sum to obtain $a^\top z \leq \sum_{i=1}^k \lambda_i b = b$. $\square$

Fix the node objective vector $c \in \mathbb{Q}^d$. Define the node bound and the bound after adding a cut:

$$v(P) = \min\{c^\top z : z \in P\}, \quad P_{(a,b)} = P \cap \{z : a^\top z \leq b\}, \quad v(P_{(a,b)}) = \min\{c^\top z : z \in P_{(a,b)}\}.$$

Definition 15.2 (Exact objective bound gain). The exact objective bound gain of (a, b) at P is

$$\sigma_P(a, b) = v(P_{(a,b)}) - v(P).$$

Lemma 15.2 (Nonnegativity and monotonicity). *For any valid inequality (a, b) , $\sigma_P(a, b) \geq 0$. Moreover, if $P' \subseteq P$ then $v(P') \geq v(P)$.*

Proof. Since $P_{(a,b)} \subseteq P$, minimisation over the smaller feasible set cannot yield a smaller minimum, hence $v(P_{(a,b)}) \geq v(P)$ and $\sigma_P(a, b) \geq 0$. The same argument yields $v(P') \geq v(P)$ for any inclusion $P' \subseteq P$. $\square$

Let $\bar{z}$ be an LP optimal solution in P . Define the violation of a cut at $\bar{z}$ by

$$\text{vio}_{\bar{z}}(a, b) = \max\{0, a^\top \bar{z} - b\}.$$

Fix a diagonal positive definite matrix $D > 0$ and the norm $\|a\|_D = \sqrt{a^\top D a}$.

Definition 15.3 (Scale invariant normalised depth). The normalised depth of (a, b) at $\bar{z}$ is

$$\delta_D(a, b; \bar{z}) = \frac{\max\{0, a^\top \bar{z} - b\}}{\|a\|_D}.$$

Lemma 15.3 (Scaling invariance). *For any $\kappa > 0$, $\delta_D(\kappa a, \kappa b; \bar{z}) = \delta_D(a, b; \bar{z})$.*

Proof. $\max\{0, (\kappa a)^\top \bar{z} - \kappa b\} = \kappa \max\{0, a^\top \bar{z} - b\}$ and $\|\kappa a\|_D = \kappa \|a\|_D$. Divide numerator by denominator. $\square$

Definition 15.4 (Encoding size and efficiency). Let $\text{size}(a, b) \in \mathbb{N}$ be the fixed encoding size of the cut. Define the efficiency score

$$\eta_P(a, b) = \frac{\sigma_P(a, b)}{1 + \text{size}(a, b)}.$$

15.1.2 Redundancy and dominance criteria

Redundancy is formalised relative to the current relaxation P .

Definition 15.5 (Positive equivalence and canonicalisation). Two inequalities (a, b) and (a', b') are positively equivalent if there exists $\kappa > 0$ such that $(a', b') = (\kappa a, \kappa b)$. Fix a deterministic canonicalisation rule that maps each nonzero (a, b) to a unique positively equivalent representative.

Definition 15.6 (Dominance over a base polyhedron). For cuts $c_1 : (a_1^\top z \leq b_1)$ and $c_2 : (a_2^\top z \leq b_2)$, say that c_1 dominates c_2 over P if

$$P \cap \{z : a_1^\top z \leq b_1\} \subseteq P \cap \{z : a_2^\top z \leq b_2\}.$$

Lemma 15.4 (Dominated cuts are redundant relative to P). If c_1 dominates c_2 over P , then adding c_2 after c_1 does not change the feasible region:

$$P \cap c_1 = (P \cap c_1) \cap c_2.$$

Proof. Let $Q = P \cap c_1$. Dominance gives $Q \subseteq P \cap c_2$, hence $Q \subseteq Q \cap c_2 \subseteq Q$. Therefore $Q = Q \cap c_2$. $\square$

Lemma 15.5 (Dominance test as a linear program). c_1 dominates c_2 over P if and only if the optimal value of

$$\max (a_2^\top z - b_2) \quad \text{subject to} \quad z \in P, \quad a_1^\top z \leq b_1$$

is at most 0.

Proof. Dominance is equivalent to $a_2^\top z - b_2 \leq 0$ for all feasible z in $P \cap c_1$. This holds if and only if the maximum of $a_2^\top z - b_2$ over that set is at most 0. $\square$

Lemma 15.6 (Dominance implies bound gain ordering). If c_1 dominates c_2 over P , then $\sigma_P(c_1) \geq \sigma_P(c_2)$.

Proof. Dominance implies $P \cap c_1 \subseteq P \cap c_2$. By monotonicity of $v(\cdot)$ under inclusion, $v(P \cap c_1) \geq v(P \cap c_2)$. Subtract $v(P)$ from both sides. $\square$

Definition 15.7 (Facet). Assume $H \neq \emptyset$. A valid cut $a^\top z \leq b$ is facet defining for H if

$$\dim(F(a, b)) = \dim(H) - 1.$$

Definition 15.8 (Affine independence). Points $p^{(1)}, \dots, p^{(k)} \in \mathbb{R}^d$ are affinely independent if the vectors $p^{(2)} - p^{(1)}, \dots, p^{(k)} - p^{(1)}$ are linearly independent.

Lemma 15.7 (Dimension witness). A set $S \subseteq \mathbb{R}^d$ satisfies $\dim(S) \geq r$ if and only if S contains $r + 1$ affinely independent points.

Proof. If S contains $r + 1$ affinely independent points, their affine hull has dimension r , hence $\dim(S) \geq r$. Conversely, if $\dim(S) \geq r$, then $\text{aff}(S)$ contains an r dimensional affine subspace, which contains $r + 1$ affinely independent points that can be chosen from S . $\square$

Theorem 15.1 (Facet certificate). Let $m = \dim(H)$. If there exist m affinely independent points $z^{(1)}, \dots, z^{(m)} \in H$ such that $a^\top z^{(i)} = b$ for all i , then $a^\top z \leq b$ is facet defining for H .

Proof. All witness points lie in $F(a, b)$. By the dimension witness lemma, $\dim(F(a, b)) \geq m - 1$. The face lies in the supporting hyperplane $a^\top z = b$, hence it cannot have full dimension m inside $\text{aff}(H)$, so $\dim(F(a, b)) \leq m - 1$. Therefore $\dim(F(a, b)) = m - 1$. $\square$

15.2 Selection rules

The selection discipline acts only on a finite pool of already certified valid cuts. It may canonicalise encodings, remove dominated cuts, and select a budget constrained subset. It never modifies an inequality and never admits an uncertified inequality.

15.2.1 Selection under correctness immunity

Let C be the finite candidate pool at a node, and assume each cut in C carries an accepted validity certificate. Fix an integer cut budget $K \geq 0$. The selected set C_{sel} is defined by the following deterministic pipeline.

1. Canonicalise each cut in C and remove duplicates.
2. Apply dominance pruning: if c_i dominates c_j over P , delete c_j .
3. Attempt facet certification for each remaining cut and record the tag if a facet witness is verified.
4. Compute strength scores σ_P , depth scores δ_D , and efficiency scores η_P .
5. Select up to K cuts using the lexicographic rule: select facet certified cuts first ordered by decreasing η_P ; then fill the remaining budget with non facet cuts ordered by decreasing η_P . Break ties by a fixed deterministic encoding order.

Theorem 15.2 (Selection safety). *Every cut in C_{sel} is valid for Z . In particular, adding all cuts in C_{sel} cannot remove any point in Z from the relaxation.*

Proof. All cuts in C are valid by assumption. Canonicalisation and deduplication do not change inequalities up to positive equivalence. Dominance pruning deletes cuts but does not create or modify cuts. Facet tagging does not change inequalities. Selection chooses a subset of the remaining valid cuts. Therefore all selected cuts are valid for Z . $\square$

Theorem 15.3 (Bound monotonicity under selection). *Let*

$$P' = P \cap \bigcap_{(a,b) \in C_{\text{sel}}} \{z : a^\top z \leq b\}.$$

Then $v(P') \geq v(P)$.

Proof. Each selected cut adds a constraint, hence $P' \subseteq P$. By monotonicity of the minimisation bound under inclusion, $v(P') \geq v(P)$. $\square$

15.2.2 Resource constraints and scheduling

Cut selection is performed under explicit resource constraints. The following constraints are enforced at the definition level.

- A hard cardinality budget K limits the number of selected cuts.
- A hard size budget $S_{\max}$ may be imposed by requiring $\sum_{(a,b) \in C_{\text{sel}}} \text{size}(a, b) \leq S_{\max}$. If both budgets are active, selection proceeds by the lexicographic rule stated above, rejecting any candidate that would violate $S_{\max}$.
- A dominance pruning schedule is fixed deterministically by iterating candidates in a fixed canonical order and applying the dominance LP test in that order.

These scheduling constraints are correctness neutral because they only control ordering and deletion of cuts already proved redundant relative to P .

15.3 Efficiency statements

This chapter permits efficiency statements only in two forms: statements that follow from the proved dominance and monotonicity lemmas, and statements explicitly recorded as empirical findings under the Results Ledger protocol.

15.3.1 Permitted empirical statements

Empirical statements are permitted only if they are tied to the benchmark charter, baseline registry, metric ledger, repeatability protocol, and the recorded Results Ledger tables and plots. Permitted statements include the following forms.

- Median and quartile comparisons of time to optimality across methods under the declared benchmark families and reporting rules.
- Median and quartile comparisons of nodes explored, number of LP solves, number of pricing calls, number of admitted cuts, peak memory usage, and certificate check overhead fraction.
- Plots of bound gap as a function of normalised time and plots of nodes explored as a function of normalised time, aggregated according to the reporting rules.
- Robustness comparisons under the declared shift tests, using the declared aggregation rules.

15.3.2 Prohibited unqualified generalisations

The following statements are prohibited within this chapter because they are not derivable as universal theorems from the formal objects and are not admissible as empirical claims without the Results Ledger closure.

- Any unconditional statement that a particular cut family is always strongest across all instances.

Quantity	Definition	Key certified properties
Bound gain $\sigma_P(a, b)$	$\sigma_P(a, b) = v(P \cap \{a^\top z \leq b\}) - v(P)$	$\sigma_P(a, b) \geq 0$. Monotone under adding valid cuts. Dominance implies ordering.
Normalised depth $\delta_D(a, b; \bar{z})$	$\delta_D(a, b; \bar{z}) = \max\{0, a^\top \bar{z} - b\} / \ a\ _D$	Scale invariant under positive equivalence. Local separation proxy at LP optimum.
Dominance over P	$P \cap c_1 \subseteq P \cap c_2$	Dominance implies redundancy relative to P . Testable by a single LP over $P \cap c_1$.
Efficiency $\eta_P(a, b)$	$\eta_P(a, b) = \sigma_P(a, b) / (1 + \text{size}(a, b))$	Well defined since $\sigma_P(a, b) \geq 0$. Compatible with dominance when size is monotone.
Facet certificate	Witness set of $\dim(H)$ affinely independent points in H satisfying $a^\top z = b$	If verified, the cut is facet defining for H . Facet cuts are maximally non redundant in the hull.

Table 15.1: Strength and redundancy metrics used by the cut selection discipline at a branch and bound node.

Figure 15.1: Strength tradeoff diagram relating objective bound gain, encoding cost, and redundancy classes.

- Any unconditional statement that a particular selection score always minimises total solve time.
- Any claim of asymptotic speedup that is not stated as a theorem with explicit assumptions and dependencies.
- Any claim that empirical results obtained under one benchmark charter automatically transfer to all future workloads.

15.4 Figures and tables

- Strength tradeoff diagram (strength vs cost vs redundancy).

Part II

Global Search Correctness (Branch-and-Bound Closure)

16

Node Definition and Domain Propagation

The node-level semantics are expressed as an exact branch-and-bound search state, consistent with classical presentations (Land et al., 1960; Conforti et al., 2014).

16.1 Node object definition

16.1.1 Domain restrictions

Let $\mathcal{S}$ denote the finite set of selectable physical design objects. For each $s \in \mathcal{S}$, define the binary design variable

$$x_s \in \{0, 1\}. \quad (16.1)$$

Let $\mathcal{R}$ denote the finite set of workload queries. For each $r \in \mathcal{R}$, let Λ_r denote the finite set of candidate plans enumerated for r at the current stage of the algorithm. For each $r \in \mathcal{R}$ and $\ell \in \Lambda_r$, define the plan variable

$$\lambda_{r\ell} \in \mathbb{R}_{\geq 0}. \quad (16.2)$$

For each $r \in \mathcal{R}$ and $\ell \in \Lambda_r$, define the binary usage indicator vector $u_{r\ell} \in \{0, 1\}^{|\mathcal{S}|}$ with components $(u_{r\ell})_s$ for $s \in \mathcal{S}$, where $(u_{r\ell})_s = 1$ denotes that plan ℓ for query r requires object s in the sense encoded by the coupling constraints.

A node maintains explicit domain bounds. For each $s \in \mathcal{S}$, define bounds

$$\ell_s \in \{0, 1\}, \quad u_s \in \{0, 1\}, \quad \ell_s \leq u_s. \quad (16.3)$$

For each $r \in \mathcal{R}$ and $\ell \in \Lambda_r$, define bounds

$$0 \leq \underline{\lambda}_{r\ell} \leq \bar{\lambda}_{r\ell}, \quad \lambda_{r\ell} \in [\underline{\lambda}_{r\ell}, \bar{\lambda}_{r\ell}]. \quad (16.4)$$

16.1.2 Node-local relaxation and bound meaning

Let C denote the fixed set of baseline constraints and admitted globally valid cuts. A node introduces additional branching constraints. Let B_N denote a finite set of branching constraints of the form $x_j = 0$ or $x_j = 1$. Define the node constraint set

$$C_N = C \cup B_N. \quad (16.5)$$

Definition 16.1 (Node object). A node is the tuple

$$N = (C_N, \ell^N, u^N, \underline{\lambda}^N, \bar{\lambda}^N). \quad (16.6)$$

Definition 16.2 (Integer feasibility at a node). The integer feasible set of node N is

$$F(N) = \left\{ (x, \lambda) : (x, \lambda) \text{ satisfies } C_N, x \in \{0, 1\}^{|S|}, \ell^N \leq x \leq u^N, \underline{\lambda}^N \leq \lambda \leq \bar{\lambda}^N \right\}, \quad (16.7)$$

where all inequalities are interpreted componentwise over the corresponding index sets.

Definition 16.3 (Node-local relaxation). The node-local relaxation replaces $x \in \{0, 1\}^{|S|}$ by $x \in [0, 1]^{|S|}$. The relaxed feasible set is

$$\bar{F}(N) = \left\{ (x, \lambda) : (x, \lambda) \text{ satisfies } C_N, x \in [0, 1]^{|S|}, \ell^N \leq x \leq u^N, \underline{\lambda}^N \leq \lambda \leq \bar{\lambda}^N \right\}. \quad (16.8)$$

The bounds ℓ^N and u^N are interpreted as variable fixings and admissible ranges for binary decisions. The bounds $\underline{\lambda}^N$ and $\bar{\lambda}^N$ are interpreted as admissible ranges for plan weights. Any node transformation is admissible only if it preserves all points in $F(N)$.

16.2 Propagation rules

16.2.1 Rule language

Propagation is a node transformation that tightens bounds and may detect infeasibility. A propagation step is a mapping

$$\Pi : N \mapsto N' \quad (16.9)$$

that modifies only the bound components by increasing some entries of ℓ , decreasing some entries of u , increasing some entries of $\underline{\lambda}$, and decreasing some entries of $\bar{\lambda}$. The node constraint set C_N is not altered by propagation.

Propagation rules are instantiated from constraints already contained in C_N together with the current bound state. The rule schemas used in this chapter are based

on the following constraint forms, which are assumed to be included in C_N whenever relevant.

Exactly-one partition constraints. For each table t , let $\mathcal{P}_t$ denote the set of partition candidates for t . The constraint is

$$\sum_{p \in \mathcal{P}_t} x_p = 1. \quad (16.10)$$

Budget constraint. Let $a_s \geq 0$ be the resource weight of $s \in \mathcal{S}$ and let $B \geq 0$ be the resource budget. The constraint is

$$\sum_{s \in \mathcal{S}} a_s x_s \leq B. \quad (16.11)$$

Implication and exclusion constraints. Implication has the form $x_i \leq x_j$. Mutual exclusion has the form $x_i + x_j \leq 1$.

Plan simplex constraints. For each $r \in \mathcal{R}$,

$$\sum_{\ell \in \Lambda_r} \lambda_{r\ell} = 1, \quad \lambda_{r\ell} \geq 0 \quad \forall \ell \in \Lambda_r. \quad (16.12)$$

Plan-selection coupling constraints. For each query $r \in \mathcal{R}$ and object $s \in \mathcal{S}$,

$$\sum_{\ell \in \Lambda_r: (u_{r\ell})_s = 1} \lambda_{r\ell} \leq x_s. \quad (16.13)$$

16.2.2 Soundness proofs

Definition 16.4 (Soundness of propagation). A propagation step Π is sound if either it correctly declares infeasibility, meaning $F(N) = \emptyset$, or it returns a node $\Pi(N)$ satisfying

$$F(N) \subseteq F(\Pi(N)). \quad (16.14)$$

The following rule families are used. Each rule is justified by direct implication from a constraint in C_N and the bound semantics.

PR rules for exactly-one constraints. Fix a table t and $\sum_{p \in \mathcal{P}_t} x_p = 1$.

$$\text{PR1: if } \exists p \neq q \text{ with } \ell_p = 1 \text{ and } \ell_q = 1, \text{ declare infeasible.} \quad (16.15)$$

$$\text{PR2: if } \exists p^* \text{ with } \ell_{p^*} = 1, \text{ set } u_p := 0 \quad \forall p \in \mathcal{P}_t \setminus \{p^*\}. \quad (16.16)$$

$$\text{PR3: if } \exists p^* \text{ with } u_p = 0 \quad \forall p \in \mathcal{P}_t \setminus \{p^*\}, \text{ set } \ell_{p^*} := 1. \quad (16.17)$$

IR rules for implications. Fix $x_i \leq x_j$.

$$\text{IR1: if } \ell_i = 1, \text{ set } \ell_j := 1. \quad (16.18)$$

$$\text{IR2: if } u_j = 0, \text{ set } u_i := 0. \quad (16.19)$$

XR rules for mutual exclusion. Fix $x_i + x_j \leq 1$.

$$\text{XR1: if } \ell_i = 1, \text{ set } u_j := 0. \quad (16.20)$$

$$\text{XR2: if } \ell_j = 1, \text{ set } u_i := 0. \quad (16.21)$$

KR rules for the budget constraint. Fix $\sum_{s \in \mathcal{S}} a_s x_s \leq B$ with $a_s \geq 0$. Define the forced load

$$L = \sum_{s \in \mathcal{S}} a_s \ell_s. \quad (16.22)$$

Then

$$\text{KR1: if } L > B, \text{ declare infeasible.} \quad (16.23)$$

$$\text{KR2: for any } j \in \mathcal{S}, \text{ if } L + a_j > B, \text{ set } u_j := 0. \quad (16.24)$$

CR rule for coupling. Fix $r \in \mathcal{R}$, $s \in \mathcal{S}$, and $\sum_{\ell: (u_r \ell)_s = 1} \lambda_{r\ell} \leq x_s$.

$$\text{CR1: if } u_s = 0, \text{ then set } \bar{\lambda}_{r\ell} := 0 \quad \forall \ell \in \Lambda_r \text{ with } (u_r \ell)_s = 1. \quad (16.25)$$

SR rules for the simplex. Fix $r \in \mathcal{R}$ and $\sum_{\ell \in \Lambda_r} \lambda_{r\ell} = 1, \lambda_{r\ell} \geq 0$.

$$\text{SR1: if } \sum_{\ell \in \Lambda_r} \bar{\lambda}_{r\ell} < 1, \text{ declare infeasible.} \quad (16.26)$$

$$\text{SR2: if } \exists \ell^* \text{ with } \bar{\lambda}_{r\ell^*} = 0 \quad \forall \ell \in \Lambda_r \setminus \{\ell^*\}, \text{ set } \underline{\lambda}_{r\ell^*} := 1, \bar{\lambda}_{r\ell^*} := 1. \quad (16.27)$$

Theorem 16.1 (Rule soundness). *Each rule instance PR1–PR3, IR1–IR2, XR1–XR2, KR1–KR2, CR1, and SR1–SR2 is sound.*

Proof. Each rule is a direct logical consequence of a constraint in C_N and the bound semantics. In every tightening rule, the updated bounds are satisfied by every $(x, \lambda) \in F(N)$, hence $F(N) \subseteq F(\Pi(N))$. In every infeasibility rule, the rule premise implies a contradiction with a constraint in C_N , hence $F(N) = \emptyset$. $\square$

16.3 Propagation termination

16.3.1 Finite termination lemma

Define the tightening preorder on bound states by

$$(\ell', u', \underline{\lambda}', \overline{\lambda}') \leq (\ell, u, \underline{\lambda}, \overline{\lambda}) \iff \ell' \geq \ell, u' \leq u, \underline{\lambda}' \geq \underline{\lambda}, \overline{\lambda}' \leq \overline{\lambda}, \quad (16.28)$$

with all inequalities componentwise.

Let $\mathcal{R}$ denote the set of rule instances generated from C_N and the current bounds.

Lemma 16.1 (Finite termination). *Repeated application of rules from $\mathcal{R}$ that strictly tighten at least one bound component terminates after finitely many steps, unless infeasibility is declared.*

Proof. Each x -bound component can change at most once, since ℓ_s can only increase from 0 to 1 and u_s can only decrease from 1 to 0, with $\ell_s \leq u_s$ preventing oscillation. For the rule set stated, each affected λ -bound component is tightened to an extremal value, specifically $\overline{\lambda}_{r\ell}$ to 0 or $\underline{\lambda}_{r\ell}$ to 1. The number of variables and bound components is finite. Therefore only finitely many strict tightenings are possible before no rule premise remains that yields a change, hence the process halts. $\square$

16.3.2 Fixed-point semantics

Define the propagation operator T on nodes by applying all rule instances that are currently applicable, in any order, until no further strict tightening occurs or infeasibility is detected. Let $\text{Cl}(N)$ denote the output of this procedure.

Definition 16.5 (Propagation closure). If infeasibility is detected, $\text{Cl}(N)$ returns infeasible. Otherwise, $\text{Cl}(N)$ returns a node N^* such that no rule instance in $\mathcal{R}$ is applicable to N^* in a way that changes bounds.

Theorem 16.2 (Closure soundness). *If all rule instances in $\mathcal{R}$ are sound, then Cl is sound.*

Proof. If infeasibility is returned, soundness follows from the soundness of the first infeasibility rule instance that triggered. Otherwise, $\text{Cl}(N)$ is obtained by a finite sequence of sound tightening steps by the finite termination lemma. Soundness composes under set inclusion, yielding $F(N) \subseteq F(\text{Cl}(N))$. $\square$

16.4 Figures and tables

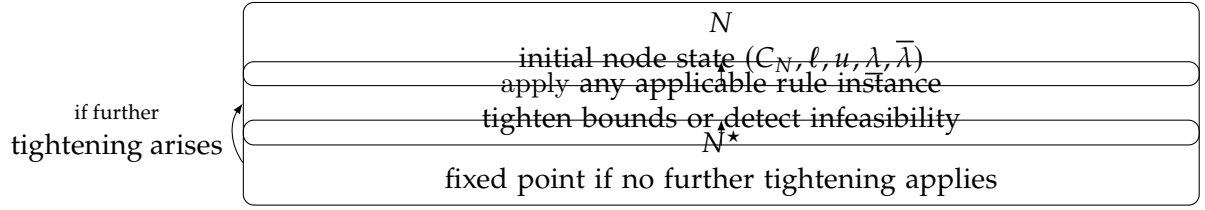


Figure 16.1: Domain propagation graph at a node.

Rule	Trigger condition	Action	Soundness basis
PR1	$\exists p \neq q$ in an exactly-one set with $\ell_p = 1$ and $\ell_q = 1$	declare infeasible	violates $\sum_p x_p = 1$
PR2	$\exists p^*$ with $\ell_{p^*} = 1$ in an exactly-one set	set $u_p := 0$ for all $p \neq p^*$	forces $x_{p^*} = 1$ then others must be 0
PR3	all but one candidate have upper bound 0 in an exactly-one set	set $\ell_{p^*} := 1$ for the remaining candidate	equality forces remaining variable to 1
IR1	implication $x_i \leq x_j$ with $\ell_i = 1$	set $\ell_j := 1$	$1 \leq x_j$ in $\{0, 1\}$
IR2	implication $x_i \leq x_j$ with $u_j = 0$	set $u_i := 0$	$x_i \leq 0$ in $\{0, 1\}$
XR1	exclusion $x_i + x_j \leq 1$ with $\ell_i = 1$	set $u_j := 0$	$1 + x_j \leq 1$
XR2	exclusion $x_i + x_j \leq 1$ with $\ell_j = 1$	set $u_i := 0$	$x_i + 1 \leq 1$
KR1	budget with forced load $L = \sum_s a_s \ell_s$ satisfying $L > B$	declare infeasible	$\sum_s a_s x_s \geq L > B$
KR2	budget with forced load L and $L + a_j > B$	set $u_j := 0$	$x_j = 1$ would violate budget
CR1	coupling with $u_s = 0$	set $\bar{\lambda}_{r\ell} := 0$ for all plans using s	$\sum_{\ell: (u_{r\ell})_s=1} \lambda_{r\ell} \leq 0$ and $\lambda \geq 0$
SR1	simplex for query r with $\sum_{\ell \in \Lambda_r} \bar{\lambda}_{r\ell} < 1$	declare infeasible	cannot satisfy $\sum_{\ell \in \Lambda_r} \lambda_{r\ell} = 1$
SR2	simplex for query r with only one plan having positive upper bound	set $\underline{\lambda}_{r\ell^*} = \bar{\lambda}_{r\ell^*} = 1$	simplex equality forces surviving plan mass to 1

Table 16.1: Propagation rule table for node domain tightening and infeasibility detection.

17

Branching Rules and Completeness

This chapter specifies the branching operator used by the global search procedure and proves that the resulting binary branching scheme is complete, meaning that it covers the integer feasible space without omission and terminates after a finite depth. The definitions and proofs in this chapter are stated in a certificate compatible form: every choice is defined as a total function, every split is a set theoretic partition of the integer feasible set, and every descent argument is witnessed by an explicit well founded measure.

17.1 Branch action definition

17.1.1 Binary branching primitives

Let $n \geq 1$ and let the discrete design vector be $x \in \{0, 1\}^n$. Let $\lambda \in \mathbb{R}^k$ denote the collection of continuous variables used by the relaxation and bound computation, for some fixed $k \geq 0$. Let the base constraint system be a finite family of linear constraints written abstractly as

$$A(x, \lambda) \leq b, \quad (17.1)$$

where $A(\cdot) \leq b$ denotes all equalities and inequalities in a unified notation.

Node object. A node N is represented by two disjoint index sets

$$F_0(N), F_1(N) \subseteq \{1, \dots, n\}, \quad F_0(N) \cap F_1(N) = \emptyset, \quad (17.2)$$

interpreted as fixings

$$x_i = 0 \quad \forall i \in F_0(N), \quad x_i = 1 \quad \forall i \in F_1(N). \quad (17.3)$$

The unfixed index set is

$$U(N) := \{1, \dots, n\} \setminus (F_0(N) \cup F_1(N)). \quad (17.4)$$

Relaxation and integer feasible sets. The node relaxation feasible set is

$$R(N) := \left\{ (x, \lambda) \in [0, 1]^n \times \mathbb{R}^k : A(x, \lambda) \leq b, x_i = 0 \ \forall i \in F_0(N), x_i = 1 \ \forall i \in F_1(N) \right\}, \quad (17.5)$$

and the node integer feasible set is

$$I(N) := \left\{ (x, \lambda) \in \{0, 1\}^n \times \mathbb{R}^k : A(x, \lambda) \leq b, x_i = 0 \ \forall i \in F_0(N), x_i = 1 \ \forall i \in F_1(N) \right\}. \quad (17.6)$$

By construction, $I(N) \subseteq R(N)$ for all nodes N .

Branching trigger. A branch action is invoked only when a relaxation feasible point $(\bar{x}, \bar{\lambda}) \in R(N)$ is available and $\bar{x} \notin \{0, 1\}^n$.

Fractional free indices. Define

$$\mathcal{J}(N, \bar{x}) := \{i \in U(N) : \bar{x}_i \in (0, 1)\}. \quad (17.7)$$

When branching is invoked, $\mathcal{J}(N, \bar{x})$ is required to be nonempty. The deterministic fallback branching index is

$$j(N, \bar{x}) := \min \mathcal{J}(N, \bar{x}), \quad (17.8)$$

where the minimum is taken with respect to the canonical order $1 < 2 < \dots < n$.

Child construction. Given a branching index $j \in U(N)$, define two child nodes N^0 and N^1 by

$$F_0(N^0) = F_0(N) \cup \{j\}, \quad F_1(N^0) = F_1(N), \quad (17.9)$$

$$F_0(N^1) = F_0(N), \quad F_1(N^1) = F_1(N) \cup \{j\}. \quad (17.10)$$

The child ordering (N^0, N^1) is part of the branch action definition and is not permitted to vary.

17.1.2 Compatibility with feasibility constraints

Feasibility preservation. By construction, both children preserve all parent fixings and all base constraints. Consequently,

$$R(N^0) \subseteq R(N), \quad R(N^1) \subseteq R(N), \quad I(N^0) \subseteq I(N), \quad I(N^1) \subseteq I(N). \quad (17.11)$$

Advisory index gating. If an advisory module proposes an index j_{adv} , it may be used only if the following predicate holds:

$$\text{ACCEPT}(j_{\text{adv}}) \iff (j_{\text{adv}} \in U(N)) \wedge (\bar{x}_{j_{\text{adv}}} \in (0, 1)). \quad (17.12)$$

If $\text{ACCEPT}(j_{\text{adv}})$ holds, branch on $j = j_{\text{adv}}$; otherwise branch on $j = j(N, \bar{x})$ from (17.8). In both cases, the children are defined by (17.9)–(17.10).

Certified interpretation of fractional. The predicate $\bar{x}_i \in (0, 1)$ must be evaluated using the canonical numeric representation used by the certificate layer. If $\bar{x}$ is produced in floating point, the branching transcript must include a canonical normalization rule declared once and applied uniformly, either by exact reconstruction or by a fixed global tolerance definition. Branching on values certified as 0 or 1 is forbidden.

17.2 Completeness theorem

17.2.1 Coverage of the feasible space

Definition (completeness for binary branching). A binary branching rule is complete if, whenever it branches at node N using some index $j \in U(N)$ and produces children N^0, N^1 , the integer feasible set of the parent is partitioned by the children and the process has a strict progress measure.

Lemma 16.1 (existence of a fractional component). If $\bar{x} \in [0, 1]^n$ and $\bar{x} \notin \{0, 1\}^n$, then there exists an index $i \in \{1, \dots, n\}$ such that $\bar{x}_i \in (0, 1)$.

Proof. Since $\bar{x} \notin \{0, 1\}^n$, there exists i with $\bar{x}_i \notin \{0, 1\}$. Because $\bar{x}_i \in [0, 1]$, it follows that $\bar{x}_i \in (0, 1)$.

Lemma 16.2 (existence of a fractional free index). Let $(\bar{x}, \bar{\lambda}) \in R(N)$. If $\bar{x} \notin \{0, 1\}^n$, then $\mathcal{J}(N, \bar{x})$ in (17.7) is nonempty, and $j(N, \bar{x})$ in (17.8) is well defined.

Proof. By (17.5), $\bar{x} \in [0, 1]^n$, and by Lemma 16.1 there exists i with $\bar{x}_i \in (0, 1)$. If $i \in F_0(N)$ then (17.3) forces $\bar{x}_i = 0$, a contradiction. If $i \in F_1(N)$ then (17.3) forces $\bar{x}_i = 1$, a contradiction. Hence at least one fractional index lies in $U(N)$, so $\mathcal{J}(N, \bar{x}) \neq \emptyset$. Since $\mathcal{J}(N, \bar{x}) \subseteq \{1, \dots, n\}$ is finite and nonempty, its minimum exists and $j(N, \bar{x})$ is well defined.

Lemma 16.3 (disjoint cover under binary fixing). Let $j \in U(N)$ and define children N^0, N^1 by (17.9)–(17.10). Then

$$I(N) = I(N^0) \dot{\cup} I(N^1). \quad (17.13)$$

Proof. The inclusions $I(N^0) \subseteq I(N)$ and $I(N^1) \subseteq I(N)$ follow from (17.6) and the additional fixing in each child. For any $(x, \lambda) \in I(N)$, one has $x \in \{0, 1\}^n$, so $x_j \in \{0, 1\}$. If $x_j = 0$ then $(x, \lambda) \in I(N^0)$; if $x_j = 1$ then $(x, \lambda) \in I(N^1)$. Thus $I(N) \subseteq I(N^0) \cup I(N^1)$. Disjointness follows because $I(N^0) \cap I(N^1)$ would require $x_j = 0$ and $x_j = 1$ simultaneously. Therefore (17.13) holds.

Theorem 16.4 (completeness of deterministic binary branching). Consider the rule that, for any $(\bar{x}, \bar{\lambda}) \in R(N)$ with $\bar{x} \notin \{0, 1\}^n$, chooses $j = j(N, \bar{x})$ from (17.8) and produces children N^0, N^1 by (17.9)–(17.10). Then the rule is complete.

Proof. Lemma 16.2 guarantees that the branching index $j(N, \bar{x})$ exists whenever branching is required. Lemma 16.3 yields the required partition of $I(N)$ by the children. A strict progress measure is established in Subsection 17.2.2. Therefore the rule is complete.

Corollary 16.5 (completeness of gated advisory branching). If an advisory module proposes j_{adv} and the rule branches on j_{adv} only when $\text{ACCEPT}(j_{\text{adv}})$ in (17.12) holds, otherwise falling back to $j(N, \bar{x})$, then the resulting procedure is complete.

Proof. When the proposal is rejected, the deterministic rule applies and Theorem 16.4 yields completeness. When the proposal is accepted, one has $j_{\text{adv}} \in U(N)$ and the children are still defined by (17.9)–(17.10); Lemma 16.3 and the strict progress argument of Subsection 17.2.2 apply verbatim.

17.2.2 Finite branching depth bound (if applicable)

Define the unfixed count measure

$$\mu(N) := |U(N)| = n - |F_0(N)| - |F_1(N)|. \quad (17.14)$$

If $j \in U(N)$ and children are produced by fixing x_j to 0 or 1, then

$$U(N^0) = U(N) \setminus \{j\}, \quad U(N^1) = U(N) \setminus \{j\}, \quad (17.15)$$

and therefore

$$\mu(N^0) = \mu(N) - 1, \quad \mu(N^1) = \mu(N) - 1. \quad (17.16)$$

Branch split diagram placeholder.

Figure 17.1: Binary branch split at index j : left child enforces $x_j = 0$ and right child enforces $x_j = 1$.

Coverage illustration placeholder.

Figure 17.2: Set theoretic coverage: $I(N)$ is partitioned by $I(N^0)$ and $I(N^1)$ under binary fixing.

Since μ maps nodes to nonnegative integers and strictly decreases at every branch, infinite branching is impossible.

Proposition 16.6 (depth bound). Along any root to leaf path, at most n branch actions can occur. In particular, every leaf has the form $U(N) = \emptyset$ and therefore all design variables are fixed.

Proof. At each branch, (17.16) decreases μ by exactly one. Since $\mu \geq 0$ and initially $\mu \leq n$, there can be at most n decreases before $\mu = 0$.

17.3 Figures and tables

- Branch split diagram.
- Coverage illustration.

Rule component	Deterministic specification
Branching index (fallback)	$j(N, \bar{x}) = \min\{i \in U(N) : \bar{x}_i \in (0, 1)\}$.
Child ordering	Children are ordered as (N^0, N^1) with $x_j = 0$ for N^0 and $x_j = 1$ for N^1 .
Advisory gating	A proposed j_{adv} is used only if $j_{\text{adv}} \in U(N)$ and $\bar{x}_{j_{\text{adv}}} \in (0, 1)$; otherwise fallback applies.
Fractionality predicate	The predicate $\bar{x}_i \in (0, 1)$ is evaluated using the canonical certificate representation; values certified as 0 or 1 are never branched upon.
Node identity	Node identity is the ordered pair $(F_0(N), F_1(N))$ stored in strictly increasing order.
Branch transcript	Each branch record contains parent id, chosen index j , child ids, acceptance outcome when applicable, and the certified values required to re evaluate branching predicates.

Table 17.1: Determinism and tie breaking rules required for reproducible branching.

18

Node Selection Policies (Correctness-Immune Ordering)

18.1 Policy definitions

18.1.1 Priority queue semantics

This chapter formalizes node selection as an ordering mechanism over the current open frontier and proves that, under the already established bound soundness, pruning soundness, and branching completeness properties, node selection is correctness-immune. In particular, node selection affects only the temporal order in which nodes are processed, and cannot alter feasibility, cannot invalidate bounds, cannot invalidate pruning, cannot remove coverage of the integer feasible region, and cannot alter the optimal value returned at termination.

Global design space. Let $n \in \mathbb{N}$ denote the total number of binary physical design decisions induced by the fixed candidate generation and compatibility semantics. A design is a vector $x \in \{0, 1\}^n$. Let $F \subseteq \{0, 1\}^n$ be the globally feasible design set defined by the physical design feasibility semantics and all global constraints. Let $f : F \rightarrow \mathbb{R}$ be the objective function. The global optimum value is

$$\text{OPT} = \min\{f(x) : x \in F\}. \quad (18.1)$$

Nodes as partial fixings. A node is a pair $N = (Z(N), O(N))$ where $Z(N), O(N) \subseteq \{1, \dots, n\}$ are disjoint, representing fixed-to-zero and fixed-to-one decisions. The feasible set associated with node N is

$$F(N) = \{x \in F : x_i = 0 \ \forall i \in Z(N), \ x_i = 1 \ \forall i \in O(N)\}. \quad (18.2)$$

Define the node depth

$$d(N) = |Z(N)| + |O(N)|. \quad (18.3)$$

Define the node optimum value

$$\text{OPT}(N) = \begin{cases} \min\{f(x) : x \in F(N)\}, & \text{if } F(N) \neq \emptyset, \\ +\infty, & \text{if } F(N) = \emptyset. \end{cases} \quad (18.4)$$

Bounds and incumbent. When node N is processed, the bounding mechanism returns a real value $\text{LB}(N)$ satisfying the bound validity condition

$$\text{LB}(N) \leq \text{OPT}(N). \quad (18.5)$$

An incumbent is a feasible design $x^* \in F$ maintained whenever discovered. The incumbent upper bound value is

$$\text{UB}^* = \begin{cases} f(x^*), & \text{if an incumbent exists,} \\ +\infty, & \text{otherwise.} \end{cases} \quad (18.6)$$

Frontier sets. The algorithm maintains an open frontier U consisting of created and unprocessed nodes, and a closed set C consisting of processed nodes. At every instant, the set of selectable nodes is exactly U .

Pruning rule. A node N may be pruned by bound if

$$\text{LB}(N) \geq \text{UB}^*. \quad (18.7)$$

Priority queue representation. Node selection is implemented abstractly as a priority queue defined by a strict total order on U . A policy specifies a key map $\rho : U \rightarrow \mathbb{R}^k$ and uses lexicographic order on keys. Selection is then defined by

$$\text{Sel}(\mathcal{S}) = \arg \min_{N \in U} \rho(N), \quad (18.8)$$

where $\mathcal{S}$ denotes the current algorithmic state, including U , C , the current incumbent value UB^* , and node metadata.

A canonical admissible key structure is

$$\rho(N) = (\rho_2(N), \rho_3(N), \rho_4(N), \rho_5(N)), \quad (18.9)$$

with the following components.

- $\rho_2(N)$ is a bound component, typically $\rho_2(N) = \text{LB}(N)$ if available and $\rho_2(N) = +\infty$ otherwise.

Key component	Definition and permitted role
$LB(N)$	Certified lower bound for node N , used only for ordering and pruning tests.
$d(N)$	Node depth, used only for ordering preferences over the frontier.
$s(N)$	Auxiliary advisory score, used only for ordering and never as a pruning substitute.
$id(N)$	Strict tie-breaker ensuring deterministic unique selection and auditability.

Table 18.1: Node selection key components and admissible uses.

- $\rho_3(N)$ is a depth component, typically $\rho_3(N) = d(N)$ or $\rho_3(N) = -d(N)$ depending on the intended traversal bias.
- $\rho_4(N)$ is an auxiliary score component $s(N)$, which may be produced by any advisory mechanism but is restricted to ordering only.
- $\rho_5(N)$ is a strict tie-breaking component that guarantees uniqueness.

18.1.2 Tie-breaking discipline

Creation index. Each node N carries a creation identifier $id(N) \in \mathbb{N}$ assigned strictly increasing at node creation. This identifier is reserved for deterministic tie-breaking and transcript reproducibility.

Admissibility conditions. A node selection policy is admissible if it satisfies all of the following.

1. **Eligibility.** If $U \neq \emptyset$, then $Sel(S) \in U$.
2. **Uniqueness.** The induced selection is unique, with $id(N)$ as the final strict tie-breaker.
3. **Non-interference.** The policy reads node metadata but does not modify feasibility semantics, does not modify the bound computation, does not modify pruning conditions, does not modify branching rules, and does not modify any certificate acceptance criteria.
4. **Transcript compatibility.** The policy output can be recorded as a pair consisting of the selected node identifier and the evaluated key components used in the comparison.

Key component summary.

18.2 Correctness immunity

18.2.1 Ordering does not affect correctness theorem

This section proves that admissible node selection does not affect correctness. The proof is isolated from implementation details and uses only the properties already established in the correctness spine.

Assumptions inherited from the correctness spine. The following are used as established statements.

- **(A1) Bound validity.** For every processed node N , the bound returned satisfies $LB(N) \leq OPT(N)$.
- **(A2) Sound pruning.** If a node N is pruned because $LB(N) \geq UB^*$, then no design $x \in F(N)$ satisfies $f(x) < UB^*$.
- **(A3) Complete branching partition.** If branching at node N on an unfixed variable $j \notin Z(N) \cup O(N)$ produces children N_0 and N_1 by adding the fixing $x_j = 0$ or $x_j = 1$, then

$$F(N) = F(N_0) \cup F(N_1), \quad F(N_0) \cap F(N_1) = \emptyset. \quad (18.10)$$

- **(A4) Termination by exhaustion.** The algorithm terminates only when $U = \emptyset$.

Lemma 17.1 (Finiteness of the node universe). The set of nodes that can be created is finite.

Proof. Each node fixes a subset of the n binary variables. Each branching step fixes one previously unfixed variable and creates at most two children. Therefore the search structure is a binary tree with depth at most n , and thus contains at most $2^{n+1} - 1$ nodes. Hence the number of nodes that can be created is finite.

Lemma 17.2 (Ordering does not change the set of created nodes). Fix any branching rule and pruning rule as defined. For any admissible node selection policy, the set of nodes created before termination is determined solely by branching and pruning and is independent of the order in which eligible nodes are selected.

Proof. A node is created only as a child of a processed node. Child creation depends only on whether the parent is pruned and on the branching rule. By admissibility, selection cannot modify feasibility, cannot modify bound computation, and cannot modify pruning or branching logic. Therefore selection can only permute the time at which nodes are processed, and cannot change which nodes are created.

Theorem 17.3 (Correctness immunity of node selection). Under assumptions (A1) to (A4), for any admissible node selection policy, upon termination the incumbent value satisfies $UB^* = OPT$.

Proof. Because UB^* is the objective value of a feasible design when defined, it satisfies $UB^* \geq OPT$ for a minimization problem.

It remains to prove $UB^* \leq OPT$ at termination. Suppose, for contradiction, that termination occurs with $UB^* > OPT$. Let $x^{opt} \in F$ satisfy $f(x^{opt}) = OPT$.

Consider the unique path in the branching tree consistent with x^{opt} , meaning that whenever the algorithm branches on a variable j , the path takes the child whose fixing equals x_j^{opt} . By (A3), feasibility of x^{opt} is preserved along exactly one child at each branching step, hence there exists a created node L such that $x^{\text{opt}} \in F(L)$.

By (A4), termination implies $U = \emptyset$, hence every created node is eventually processed. In particular, L is processed at some time. When L is processed, (A1) gives $\text{LB}(L) \leq \text{OPT}(L)$. Since $x^{\text{opt}} \in F(L)$, we have $\text{OPT}(L) \leq f(x^{\text{opt}}) = \text{OPT}$. Therefore

$$\text{LB}(L) \leq \text{OPT} < \text{UB}^*. \quad (18.11)$$

Hence $\text{LB}(L) < \text{UB}^*$, so L cannot be pruned by the rule $\text{LB}(L) \geq \text{UB}^*$. By (A3), continued branching preserves feasibility of x^{opt} along a consistent descendant path, and the same inequality prevents pruning of that path while a feasible design of value OPT remains in the node feasible set. Consequently, the subtree containing x^{opt} cannot be eliminated without producing an incumbent of objective value at most OPT . This contradicts the assumption that the algorithm terminates with $\text{UB}^* > \text{OPT}$.

Therefore $\text{UB}^* = \text{OPT}$ at termination. The argument depends only on (A1) to (A4) and does not depend on the ordering of node selection, provided admissibility holds.

Corollary 17.4 (Correctness equivalence of admissible policies). Any two admissible node selection policies, applied to the same instance with the same bounding, branching, and pruning semantics, terminate with the same optimal objective value OPT , whenever they terminate.

18.2.2 Separation from bound computation and pruning

Node selection is correctness-immune because it is structurally separated from all operations that can change the logical content of the search.

Separation principle. Node selection may access, but must not modify, the following objects.

- The feasibility semantics defining F and $F(N)$.
- The bounding procedure and its certificate acceptance gates.
- The pruning rule and its required inequalities.
- The branching rule and its completeness property.
- The incumbent acceptance rule and feasibility verification.

Operational consequence. Selection can affect only the temporal order of node processing. All correctness statements depend on properties of bounds, pruning, and branching that are independent of that temporal order, and are therefore invariant under admissible selection.

Figure 18.1: Node selection schematic. The open frontier U is ordered by a lexicographic key $\rho(N)$ computed from admissible metadata such as $LB(N)$, $d(N)$, $s(N)$, and $id(N)$. Selection maps the current state to a unique node in U without modifying feasibility, bounds, pruning, branching, or certificate acceptance.

18.3 Figures and tables

- Node selection schematic.

19

Primal Feasibility Construction and Incumbent Updates

19.1 Feasibility witness definition

19.1.1 Witness structure

Let D denote the finite set of candidate design objects. Let Q denote the finite set of workload queries. For each $q \in Q$, let P_q denote the finite set of candidate executable plans for q . For each (q, p) with $q \in Q$ and $p \in P_q$, let $\text{rt}(q, p) \in \mathbb{R}_{\geq 0}$ denote the deterministic runtime coefficient and let $\text{req}(q, p) \subseteq D$ denote the set of required design objects. Let $w_q \in \mathbb{R}_{\geq 0}$ denote the workload weight for query q . Let $\text{MC}(d) \in \mathbb{R}_{\geq 0}$ denote the aggregated maintenance coefficient for design object d .

Define canonical binary decision variables

$$y_d \in \{0, 1\} \quad (d \in D), \quad x_{q,p} \in \{0, 1\} \quad (q \in Q, p \in P_q).$$

Let $\text{DesignFeas}(y)$ denote the conjunction of all admissibility constraints on y (including any budgets, exclusivity rules, prerequisites, and compatibility rules fixed by the instance).

Define the canonical feasibility predicate

$$\text{Feas}(y, x) \equiv \text{DesignFeas}(y) \wedge \bigwedge_{q \in Q} \left(\sum_{p \in P_q} x_{q,p} = 1 \right) \wedge \bigwedge_{\substack{q \in Q \\ p \in P_q \\ d \in \text{req}(q,p)}} (x_{q,p} \leq y_d).$$

Define the canonical objective value

$$\text{Obj}(y, x) = \sum_{q \in Q} w_q \sum_{p \in P_q} \text{rt}(q, p) x_{q,p} + \sum_{d \in D} \text{MC}(d) y_d.$$

Definition 19.1 (Feasibility witness). A feasibility witness is a tuple

$$W = (y, x, \Pi, \Gamma, V, \Theta)$$

with components defined as follows.

1. $y \in \{0, 1\}^{|D|}$ and $x \in \{0, 1\}^{\sum_{q \in Q} |P_q|}$ are the proposed incumbent assignments in canonical form.
2. $\Pi : Q \rightarrow \bigcup_{q \in Q} P_q$ is a plan decoding such that $\Pi(q) \in P_q$ and $x_{q, \Pi(q)} = 1$ for all $q \in Q$.
3. Γ is a finite constraint-evaluation record that certifies $\text{Feas}(y, x)$ by explicit evaluation of every constraint instance in $\text{DesignFeas}(y)$, every equality $\sum_{p \in P_q} x_{q,p} = 1$, and every coupling inequality $x_{q,p} \leq y_d$ for $d \in \text{req}(q, p)$.
4. V is an exact objective decomposition record that computes $\text{Obj}(y, x)$ by listing each participating coefficient and its multiplying decision value.
5. Θ is an instance-binding record that uniquely identifies the candidate sets, plan universes, and coefficient registries referenced by y, x, Π, Γ , and V .

19.1.2 Witness check predicate

Define $\text{VerifyWitness}(W)$ to be true if and only if all of the following properties hold for $W = (y, x, \Pi, \Gamma, V, \Theta)$.

1. Domain validity: $y_d \in \{0, 1\}$ for all $d \in D$ and $x_{q,p} \in \{0, 1\}$ for all $q \in Q, p \in P_q$.
2. Decoding consistency: for every $q \in Q$, $\Pi(q) \in P_q$, $x_{q, \Pi(q)} = 1$, and $x_{q,p} = 0$ for all $p \in P_q \setminus \{\Pi(q)\}$.
3. Constraint validity: Γ contains correct evaluations whose conjunction implies $\text{Feas}(y, x)$.
4. Objective validity: V evaluates exactly to $\text{Obj}(y, x)$ under the coefficients referenced by Θ .

An incumbent record is a pair (W, U) where W is a feasibility witness and $U \in \mathbb{R}$ satisfies $U = \text{Obj}(y, x)$ as certified by $\text{VerifyWitness}(W)$.

19.2 Incumbent construction procedure

19.2.1 Deterministic construction

Let Σ denote an arbitrary solver state at any node. Define a deterministic primal construction operator

$$\text{PrimalConstruct}(\Sigma) \in \{\perp\} \cup \mathcal{W},$$

where $\perp$ denotes failure to construct an accepted witness and $\mathcal{W}$ denotes the set of all feasible witnesses.

Define the deterministic repair operator RepairDesign such that $\text{RepairDesign}(y^{\text{cand}})$ returns either $\perp$ or a repaired assignment y^{rep} satisfying $\text{DesignFeas}(y^{\text{rep}})$.

Given any y^{rep} with $\text{DesignFeas}(y^{\text{rep}})$, define for each $q \in Q$ the feasible plan subset

$$\mathcal{F}_q(y^{\text{rep}}) = \{p \in P_q : \text{req}(q, p) \subseteq \{d \in D : y_d^{\text{rep}} = 1\}\}.$$

If $\mathcal{F}_q(y^{\text{rep}}) = \emptyset$ for any $q \in Q$, then no feasible witness exists for y^{rep} and the construction fails.

Otherwise define a deterministic plan decoding Π^{rep} by

$$\Pi^{\text{rep}}(q) \in \arg \min_{p \in \mathcal{F}_q(y^{\text{rep}})} \text{rt}(q, p), \quad \text{for all } q \in Q,$$

with a fixed deterministic tie-breaking rule.

Define x^{rep} uniquely by

$$x_{q,p}^{\text{rep}} = \begin{cases} 1 & \text{if } p = \Pi^{\text{rep}}(q), \\ 0 & \text{otherwise.} \end{cases}$$

Construct Γ^{rep} by explicit evaluation of all constraints in $\text{Feas}(y^{\text{rep}}, x^{\text{rep}})$. Construct V^{rep} by explicit evaluation of $\text{Obj}(y^{\text{rep}}, x^{\text{rep}})$. Assemble

$$W^{\text{rep}} = (y^{\text{rep}}, x^{\text{rep}}, \Pi^{\text{rep}}, \Gamma^{\text{rep}}, V^{\text{rep}}, \Theta).$$

Define $\text{PrimalConstruct}(\Sigma) = W^{\text{rep}}$ if $\text{VerifyWitness}(W^{\text{rep}})$ is true; otherwise define $\text{PrimalConstruct}(\Sigma) = \perp$.

Define the incumbent update rule. Let $U^* \in \mathbb{R} \cup \{+\infty\}$ denote the currently stored incumbent objective value. If $\text{PrimalConstruct}(\Sigma) = \perp$, retain the incumbent. If $\text{PrimalConstruct}(\Sigma) = W$ and the certified value $U = \text{Obj}(y, x)$ satisfies $U < U^*$, replace the incumbent by (W, U) ; otherwise retain the incumbent.

19.2.2 Learned proposal interface (advisory)

Let $\text{Propose}(\Sigma)$ denote an advisory proposal oracle that may return a candidate assignment y^{cand} or $\perp$. The oracle is permitted to be heuristic and is not part of the correctness boundary. If $\text{Propose}(\Sigma) = \perp$, no action is taken. If $\text{Propose}(\Sigma) = y^{\text{cand}}$, the value is used only as input to RepairDesign , followed by the deterministic steps of the construction and the verification predicate VerifyWitness . No incumbent update is permitted unless VerifyWitness returns true.

Figure 19.1: Deterministic primal construction and incumbent update gate.

Field	Type	Verification meaning
y	$\{0, 1\}^{ D }$	Design-selection assignment in canonical variables.
x	$\{0, 1\}^{\sum_{q \in Q} P_q }$	Plan-choice assignment in canonical variables.
Π	Function $Q \rightarrow \bigcup_{q \in Q} P_q$	Per-query selected plan decoding consistent with x and plan universes.
Γ	Finite record	Explicit evaluation of every constraint instance required by $\text{Feas}(y, x)$, including $\text{DesignFeas}(y)$, per-query selection equalities, and coupling inequalities.
V	Finite record	Exact decomposition and evaluation of $\text{Obj}(y, x)$ into workload runtime and maintenance contributions.
Θ	Instance binding	Identifiers for candidate sets, plan universes, and coefficient registries used by Γ and V , ensuring witness-instance uniqueness.

Table 19.1: Feasibility witness schema and verification obligations.

19.3 Soundness theorem

19.3.1 Accepted incumbent implies feasibility

Theorem 19.1 (Accepted incumbent implies feasibility). *For any solver state Σ , if $\text{PrimalConstruct}(\Sigma)$ returns a witness $W = (y, x, \Pi, \Gamma, V, \Theta)$ such that $\text{VerifyWitness}(W)$ is true, then $\text{Feas}(y, x)$ is true. Consequently, every accepted incumbent record corresponds to a feasible solution of the canonical baseline formulation.*

Proof. By definition, $\text{VerifyWitness}(W)$ is true only if the constraint-evaluation record Γ is correct and its conjunction implies $\text{Feas}(y, x)$. Therefore $\text{Feas}(y, x)$ holds. $\square$

19.3.2 Objective evaluation consistency

Lemma 19.1 (Objective evaluation consistency). *If $\text{VerifyWitness}(W)$ is true for $W = (y, x, \Pi, \Gamma, V, \Theta)$, then the objective value certified by V equals $\text{Obj}(y, x)$ under the coefficient registries referenced by Θ . Consequently, every stored incumbent objective value is a correctly evaluated feasible objective value and is a valid global upper bound for minimization.*

Proof. The statement follows directly from the objective validity clause in the definition of $\text{VerifyWitness}(W)$. $\square$

19.4 Figures and tables

- Primal construction flow diagram.
- Feasibility witness schema table.

20

Pruning Conditions and Global Optimality

20.1 Infeasibility certification requirements

20.1.1 Infeasibility witnesses

Let the instance be fixed. Let $\mathcal{D}$ be the finite domain of admissible decision vectors x , let $\text{Feas}(x)$ be the feasibility predicate determined by the instance semantics, and let $f(x) \in \mathbb{Q}$ be the objective value. Define the global feasible set

$$\mathcal{F} = \{ x \in \mathcal{D} \mid \text{Feas}(x) \}. \quad (20.1)$$

A branch and bound node N is defined by a finite conjunction of node constraints C_N . Define the node feasible set

$$\mathcal{F}_N = \{ x \in \mathcal{F} \mid x \text{ satisfies } C_N \}. \quad (20.2)$$

Node infeasibility is the statement $\mathcal{F}_N = \emptyset$.

Let Enc denote the fixed encoding map from integer designs to relaxation variables, and let $z \in \mathbb{Q}^m$ be the relaxation variable vector. For each node N , let the node relaxation region be

$$\mathcal{R}_N = \{ z \in \mathbb{Q}^m \mid G_N z \leq h_N \}, \quad (20.3)$$

where $G_N \in \mathbb{Q}^{p \times m}$ and $h_N \in \mathbb{Q}^p$ are explicitly materialized in the transcript. The relaxation is required to satisfy containment

$$\text{Enc}(\mathcal{F}_N) \subseteq \mathcal{R}_N. \quad (20.4)$$

Accordingly, a sufficient condition for $\mathcal{F}_N = \emptyset$ is $\mathcal{R}_N = \emptyset$.

Definition 20.1 (Infeasibility certificate). An infeasibility certificate for node N is a vector $\lambda \in \mathbb{Q}^p$ such that

$$\lambda \geq 0, \quad (20.5)$$

$$\lambda^\top G_N = \mathbf{0}^\top, \quad (20.6)$$

$$\lambda^\top h_N < 0. \quad (20.7)$$

The certificate is valid if (20.5)–(20.7) hold exactly in $\mathbb{Q}$ for the node relaxation system (20.3) present at the time of closure.

Lemma 20.1 (Soundness of infeasibility certification). *If a valid infeasibility certificate exists for node N , then $\mathcal{R}_N = \emptyset$. Consequently, $\mathcal{F}_N = \emptyset$.*

Proof. Assume there exists a valid certificate λ . Suppose for contradiction that there exists $z \in \mathcal{R}_N$. Then $G_N z \leq h_N$. Left multiply by $\lambda^\top$ and use $\lambda \geq 0$ to preserve the inequality:

$$\lambda^\top G_N z \leq \lambda^\top h_N. \quad (20.8)$$

By (20.6), the left-hand side is 0. Hence $0 \leq \lambda^\top h_N$, contradicting (20.7). Therefore $\mathcal{R}_N = \emptyset$. By (20.4), $\text{Enc}(\mathcal{F}_N) \subseteq \mathcal{R}_N$ implies $\mathcal{F}_N = \emptyset$. $\square$

20.1.2 Node closure conditions

A node may be closed by infeasibility only if the transcript contains the fully specified inequality system (20.3) and a certificate λ satisfying Definition 20.1 for that system. The strict negativity condition (20.7) is required as a strict inequality in exact arithmetic. If any numerical computation is used upstream, the recorded certificate must still imply a strict rational contradiction, for instance by establishing an explicit rational margin $\delta > 0$ such that $\lambda^\top h_N \leq -\delta$.

No other notion of infeasibility is permitted for pruning. Any domain propagation or logical inference step is admissible only insofar as it yields, or can be reduced to, a certificate satisfying Definition 20.1 for the current node system.

20.2 Bound-based pruning

20.2.1 Pruning rule

Let an incumbent design $x^\dagger \in \mathcal{F}$ be stored in the transcript, with incumbent value

$$\text{UB}_t = f(x^\dagger), \quad (20.9)$$

where t denotes the time at which the incumbent is referenced. Incumbent soundness requires that $\text{Feas}(x^\dagger)$ is witnessed by an explicit, checkable feasibility record, so that (20.9) is an objective value of an actual feasible design.

For each node N , let $c \in \mathbb{Q}^m$ denote the relaxation objective vector induced by the fixed encoding and objective semantics. Define the node relaxation lower bound

$$\text{LB}(N) = \min \{ c^\top z \mid z \in \mathcal{R}_N \}. \quad (20.10)$$

Definition 20.2 (Certified node lower bound record). A certified lower bound record for node N consists of:

1. a rational value $\text{LB}(N)$,
2. a primal feasible point $z_N \in \mathcal{R}_N$,
3. a dual feasible point y_N for the dual of (20.10),
4. a pricing-closure proof object Π_N establishing completeness of the relaxation with respect to the plan universe at node N ,

such that the transcript verifies in exact arithmetic

$$c^\top z_N = \text{LB}(N), \quad (20.11)$$

$$h_N^\top y_N = \text{LB}(N), \quad (20.12)$$

with y_N dual feasible for the dual of the inequality system (20.3).

Definition 20.3 (Bound-based pruning event). A bound-based prune event for node N at time t is valid if the transcript contains a certified node lower bound record for N , contains a feasible incumbent record with value UB_t , and satisfies

$$\text{LB}(N) \geq \text{UB}_t. \quad (20.13)$$

20.2.2 Pruning soundness theorem

Define the node optimum

$$\text{OPT}_N = \min \{ f(x) \mid x \in \mathcal{F}_N \}, \quad (20.14)$$

with the convention that $\text{OPT}_N = +\infty$ if $\mathcal{F}_N = \emptyset$.

Lemma 20.2 (Lower bound property). Assume (20.4) holds and $\text{LB}(N)$ is defined by (20.10). Then $\text{LB}(N) \leq \text{OPT}_N$.

Proof. Let $x \in \mathcal{F}_N$. Then $\text{Enc}(x) \in \mathcal{R}_N$ by (20.4). By optimality of $\text{LB}(N)$ over $\mathcal{R}_N$,

$$\text{LB}(N) \leq c^\top \text{Enc}(x). \quad (20.15)$$

By correctness of the encoding and objective correspondence, $c^\top \text{Enc}(x) = f(x)$. Hence $\text{LB}(N) \leq f(x)$ for all $x \in \mathcal{F}_N$, and taking the minimum yields $\text{LB}(N) \leq \text{OPT}_N$. $\square$

Theorem 20.1 (Soundness of bound-based pruning). If a bound-based prune event for node N is valid in the sense of Definition 20.3, then no $x \in \mathcal{F}_N$ satisfies $f(x) < \text{UB}_t$. In particular, node N contains no design that improves upon the incumbent at time t .

Proof. Let $x \in \mathcal{F}_N$. By Lemma 20.2, $\text{LB}(N) \leq f(x)$. By (20.13), $\text{LB}(N) \geq \text{UB}_t$. Therefore $f(x) \geq \text{UB}_t$. Since x was arbitrary, no design in $\mathcal{F}_N$ has objective value strictly below UB_t . $\square$

20.3 Global optimality

20.3.1 Termination conditions

Let N_{root} denote the root node with $\mathcal{F}_{N_{\text{root}}} = \mathcal{F}$. A branching step applied to a node N produces two children N_0 and N_1 such that

$$\mathcal{F}_N = \mathcal{F}_{N_0} \cup \mathcal{F}_{N_1}, \quad (20.16)$$

$$\mathcal{F}_{N_0} \cap \mathcal{F}_{N_1} = \emptyset. \quad (20.17)$$

The branching procedure is complete if every node that is not closed by infeasibility certification, not closed by bound-based pruning, and not closed by integrality closure is eventually branched. The search terminates when the open set of active nodes is empty.

Assume $\mathcal{D}$ is finite. Then the induced branch and bound search tree under complete branching is finite.

20.3.2 Global optimality theorem

Theorem 20.2 (Global optimality under certified bounds and complete branching). *Assume that:*

1. *every infeasibility closure of a node is accompanied by a valid certificate as in Definition 20.1,*
2. *every bound-based closure of a node is a valid prune event as in Definition 20.3,*
3. *branching satisfies the partition property (20.17) and is complete in the operational sense,*
4. *the domain $\mathcal{D}$ is finite.*

If the procedure terminates with a feasible incumbent $x^\dagger$, then $x^\dagger$ is globally optimal:

$$f(x^\dagger) = \min \{ f(x) \mid x \in \mathcal{F} \}. \quad (20.18)$$

If the procedure terminates with no feasible incumbent, then $\mathcal{F} = \emptyset$.

Proof. Assume the procedure terminates with incumbent $x^\dagger$ and final value $\text{UB}_\star = f(x^\dagger)$. Suppose for contradiction that there exists $x^\star \in \mathcal{F}$ with $f(x^\star) < \text{UB}_\star$. By $\mathcal{F}_{N_{\text{root}}} = \mathcal{F}$, one has $x^\star \in \mathcal{F}_{N_{\text{root}}}$. By repeated application of the branching partition property (20.17), $x^\star$ belongs to exactly one node on each level of the tree and hence belongs to some leaf node L .

Since $x^\star \in \mathcal{F}_L$, node L cannot be infeasible. By Lemma 20.1, L therefore cannot have been closed by infeasibility certification. Node L also cannot have been closed by

bound-based pruning at any time t : by Lemma 20.2, $\text{LB}(L) \leq f(x^\star)$, hence $\text{LB}(L) < \text{UB}_\star$, and therefore $\text{LB}(L) \geq \text{UB}_t$ cannot hold for an incumbent value UB_t that is bounded below by $\text{UB}_\star$ at termination.

Because termination empties the open set, L cannot remain unprocessed. As L is a leaf, it cannot be branched further, so it must be closed by integrality closure, which yields a feasible design value at most $f(x^\star)$. This contradicts $\text{UB}_\star > f(x^\star)$. Therefore no such $x^\star$ exists, and $x^\dagger$ is globally optimal.

Assume instead that the procedure terminates with no feasible incumbent. Suppose for contradiction that $\mathcal{F} \neq \emptyset$, and let $x \in \mathcal{F}$. By the same branching partition argument, x belongs to a leaf node L with $x \in \mathcal{F}_L$. Node L cannot be closed by infeasibility certification since it contains x . Node L cannot be closed by bound-based pruning because bound-based pruning requires a feasible incumbent value to define UB_t . Since termination empties the open set, L cannot remain unprocessed and must be closed by integrality closure, producing a feasible incumbent. This contradicts the assumption of no incumbent at termination. Hence $\mathcal{F} = \emptyset$. $\square$

20.4 Figures and tables

- Pruning decision flowchart.
- Theorem dependency mini-map.

Part III

Learned Guidance (Advisory Layer Closure)

21

State Representation and Structured Inputs

21.1 Typed graph and hypergraph schema

This chapter fixes a deterministic, finite, and canonical representation of the solver state at any node of the search procedure. The representation is the sole information exposed to advisory modules. It is defined as an exact mathematical object derived from the node-state and is independent of implementation artefacts such as memory locations, insertion order, or internal identifiers. All numerical quantities in this chapter are defined over exact rational arithmetic.

21.1.1 Node and edge types

Primitive instance sets and canonical enumerations

Let S denote the finite set of candidate physical design objects, partitioned by type into indices, materialised views, and partition schemes. Let Q denote the finite set of workload queries. Let T denote the finite set of base tables. Fix canonical bijections

$$\text{id}_S : S \rightarrow \{1, \dots, n\}, \quad \text{id}_Q : Q \rightarrow \{1, \dots, q\}, \quad \text{id}_T : T \rightarrow \{1, \dots, \tau\}.$$

These bijections are purely definitional enumerations of already finite sets. They introduce no additional assumptions.

Node-state object

A node-state is a tuple

$$\sigma = (C, K, \ell, u, \text{LPstat}, x^*, \lambda^*, \text{Dual}, \text{UB}^*),$$

where C is the finite active column set, K is the finite active cut set, and $\ell, u \in \{0, 1\}^n$ are node bounds for object variables. The indicator $\text{LPstat} \in \{\text{Feasible}, \text{Infeasible}\}$ records feasibility of the node LP relaxation. When $\text{LPstat} = \text{Feasible}$, $x^* \in \mathbb{Q}^n$ and $\lambda^* \in \mathbb{Q}^{|C|}$ denote an optimal primal solution of the node LP relaxation and Dual denotes the associated dual quantities required to compute reduced costs and violations as exact rational expressions. When $\text{LPstat} = \text{Infeasible}$, the symbols x^* and λ^* are treated as null and are never used directly; all LP dependent features are defined below to take value 0 by explicit definition. The value $\text{UB}^* \in \mathbb{Q} \cup \{+\infty\}$ is the incumbent objective value when finite.

Each column $c \in C$ has an associated query $r(c) \in Q$, a plan identifier $\ell(c)$ within the fixed plan universe, a plan cost $c_{r(c), \ell(c)} \in \mathbb{Q}_{\geq 0}$, and a requirement incidence vector $u_c \in \{0, 1\}^n$ whose component $(u_c)_j = 1$ indicates that the plan requires object $\text{id}_S^{-1}(j)$.

Each cut $k \in K$ is a rational inequality of the form

$$\sum_{j=1}^n g_{k,j}^x x_j + \sum_{c \in C} g_{k,c}^\lambda \lambda_c \leq h_k,$$

with coefficients $g_{k,j}^x \in \mathbb{Q}$, $g_{k,c}^\lambda \in \mathbb{Q}$, and right-hand side $h_k \in \mathbb{Q}$. Membership of K means that the cut has already satisfied the external admission and verification conditions defined in the verification chapters.

Canonical indexing of columns and cuts

To ensure stability, the representation uses canonical orderings for C and K . For each $c \in C$, define the column key

$$\text{key}_C(c) = \left(\text{id}_Q(r(c)), c_{r(c), \ell(c)}, u_c \right),$$

ordered lexicographically, where u_c is treated as a length n bitstring. Define $\text{id}_C : C \rightarrow \{1, \dots, m\}$ by sorting C by key_C and assigning ranks. If two columns share the same key, they are definitionally identical and cannot be distinct elements of C .

For each $k \in K$, define the cut key

$$\text{key}_K(k) = \left(h_k, (g_{k,1}^x, \dots, g_{k,n}^x), (g_{k,c_1}^\lambda, \dots, g_{k,c_m}^\lambda) \right),$$

where $c_p = \text{id}_C^{-1}(p)$. Define $\text{id}_K : K \rightarrow \{1, \dots, \kappa\}$ by sorting K by key_K and assigning ranks.

Lemma 20.1 (Existence and uniqueness of canonical bijections). For finite C and K , the maps id_C and id_K defined above exist and are unique.

Proof. Lexicographic order on the key domains is a total order. Every finite set under a total order admits a unique increasing sequence of its elements. Assigning ranks

yields a bijection. Uniqueness follows from uniqueness of the sorted sequence. $\square$

Typed vertex sets

Define a typed hypergraph

$$H(\sigma) = (V, E, \text{type}, \phi),$$

with vertex set

$$V = V_S \cup V_Q \cup V_C \cup V_K \cup V_T \cup V_G,$$

where

$$V_S = \{v_j^S : j = 1, \dots, n\}, \quad V_Q = \{v_i^Q : i = 1, \dots, q\}, \quad V_C = \{v_p^C : p = 1, \dots, m\},$$

$$V_K = \{v_t^K : t = 1, \dots, \kappa\}, \quad V_T = \{v_u^T : u = 1, \dots, \tau\}, \quad V_G = \{v^G\}.$$

The function type assigns the labels S, Q, C, K, T, G to the corresponding vertex classes. The map ϕ assigns a rational feature vector to every vertex, defined in Section 21.2.

Typed relations and hyperedges

The relation set E is specified by the following typed relations.

Column to query relation. For each $c \in C$, let $p = \text{id}_C(c)$ and $i = \text{id}_Q(r(c))$. Add the directed relation $(v_p^C \rightarrow v_i^Q)$.

Column to object requirement relation. For each $c \in C$ and each $j \in \{1, \dots, n\}$ with $(u_c)_j = 1$, add $(v_p^C \rightarrow v_j^S)$ where $p = \text{id}_C(c)$. This relation admits a hypergraph interpretation since one column may require multiple objects.

Partition to table relation. If object vertex v_j^S corresponds to a partition candidate of table $t \in T$, add $(v_j^S \rightarrow v_{\text{id}_T(t)}^T)$. For non-partition objects, no such relation is present.

Cut to variable weighted relations. For each $k \in K$ with $t = \text{id}_K(k)$, add weighted relations $(v_t^K \rightarrow v_j^S)$ whenever $g_{k,j}^x \neq 0$ with weight $g_{k,j}^x$, and $(v_t^K \rightarrow v_p^C)$ whenever $g_{k,c_p}^\lambda \neq 0$ with weight g_{k,c_p}^λ .

Global coupling relation. Add $(v^G \rightarrow v_j^S)$ for all j to represent the global coupling induced by budget and shared constraints.

21.1.2 Semantic invariants of the representation

The representation must satisfy the following invariants.

Invariant I (Canonical determinacy). For every node-state σ , the representation $H(\sigma)$ and its tensorisation $\Phi(\sigma)$ are functions of σ alone and are independent of extrinsic identifiers.

Invariant II (Type stability). The set of vertex types and the feature dimensions per type are fixed by definition and do not change across nodes or runs. Only the finite counts $m = |C|$ and $\kappa = |K|$ may vary.

Invariant III (Exact arithmetic). Every feature entry is a rational number defined by explicit rational expressions in stored instance quantities and node quantities. If $\text{LPstat} = \text{Infeasible}$, then all LP dependent feature entries are defined to be 0.

Invariant IV (Gate sufficiency). The representation contains sufficient information to compute, without hidden state, all predicates required by acceptance conditions that depend only on the current node-state.

21.2 Feature dictionary

This section defines the feature mapping ϕ and the deterministic tensorisation $\Phi(\sigma)$. All definitions are explicit and total.

21.2.1 Feature definitions

Object features

For object vertex v_j^S , define $\phi_S(j) \in \mathbb{Q}^9$ by

$$\phi_S(j) = (1_I(j), 1_V(j), 1_P(j), a_j, m_j, \ell_j, u_j, x_j^*, f_j),$$

where $1_I, 1_V, 1_P \in \{0, 1\}$ are type indicators, $(a_j, m_j) \in \mathbb{Q}_{\geq 0}^2$ are size and maintenance cost, $\ell_j, u_j \in \{0, 1\}$ are node bounds, and

$$x_j^* = \begin{cases} x_j^* & \text{if LPstat} = \text{Feasible}, \\ 0 & \text{if LPstat} = \text{Infeasible}, \end{cases} \quad f_j = \begin{cases} 1 & \text{if LPstat} = \text{Feasible}, \ell_j = 0, u_j = 1, x_j^* \notin \{0, 1\}, \\ 0 & \text{otherwise.} \end{cases}$$

Query features

For query vertex v_i^Q corresponding to $r = \text{id}_Q^{-1}(i)$, define $\phi_Q(i) \in \mathbb{Q}^2$ by

$$\phi_Q(i) = (w_r, \alpha_r),$$

where $w_r \in \mathbb{Q}_{\geq 0}$ is the fixed workload weight and

$$\alpha_r = \begin{cases} \alpha_r & \text{if LPstat} = \text{Feasible}, \\ 0 & \text{if LPstat} = \text{Infeasible}. \end{cases}$$

Here α_r denotes the query associated dual quantity from Dual required by the reduced cost expression adopted by the node relaxation.

Column features

For column vertex v_p^C corresponding to $c = \text{id}_C^{-1}(p)$, define $\phi_C(p) \in \mathbb{Q}^4$ by

$$\phi_C(p) = (c_{r(c), \ell(c)}, \lambda_c^*, \text{rc}(c), \text{id}_Q(r(c))),$$

with

$$\lambda_c^* = \begin{cases} \lambda_c^* & \text{if LPstat} = \text{Feasible}, \\ 0 & \text{if LPstat} = \text{Infeasible}, \end{cases} \quad \text{rc}(c) = \begin{cases} \text{rc}(c) & \text{if LPstat} = \text{Feasible}, \\ 0 & \text{if LPstat} = \text{Infeasible}. \end{cases}$$

The reduced cost $\text{rc}(c)$ is defined as the exact rational expression implied by Dual.

Cut features

For cut vertex v_t^K corresponding to $k = \text{id}_K^{-1}(t)$, define $\phi_K(t) \in \mathbb{Q}^2$ by

$$\phi_K(t) = (h_k, \text{viol}(k)),$$

where

$$\text{viol}(k) = \begin{cases} \max\left(0, \sum_{j=1}^n g_{k,j}^x x_j^* + \sum_{c \in C} g_{k,c}^\lambda \lambda_c^* - h_k\right) & \text{if LPstat} = \text{Feasible}, \\ 0 & \text{if LPstat} = \text{Infeasible}. \end{cases}$$

Table and global features

For table vertex v_u^T , define $\phi_T(u) \in \mathbb{Q}^1$ by $\phi_T(u) = (u)$.

For the global vertex v^G , define $\phi_G \in \mathbb{Q}^3$ by

$$\phi_G = (B, \text{UB}^*, 1_{\text{Feasible}}),$$

where $B \in \mathbb{Q}_{\geq 0}$ is the fixed budget and $1_{\text{Feasible}} = 1$ if LPstat = Feasible and 0 otherwise.

Tensorisation

Define the structured input pack $\Phi(\sigma)$ by the collection

$$\Phi(\sigma) = (X, Q, Cm, Km, Tb, G; U, R, PT; G^x, G^\lambda; M),$$

where the feature matrices are

$$X \in \mathbb{Q}^{n \times 9}, \quad Q \in \mathbb{Q}^{q \times 2}, \quad Cm \in \mathbb{Q}^{m \times 4}, \quad Km \in \mathbb{Q}^{k \times 2}, \quad Tb \in \mathbb{Q}^{\tau \times 1}, \quad G \in \mathbb{Q}^{1 \times 3},$$

and where the incidence matrices are

$$U \in \{0, 1\}^{m \times n}, \quad R \in \{0, 1\}^{m \times q}, \quad PT \in \{0, 1\}^{n \times \tau},$$

defined by

$$U_{p,j} = 1 \iff (u_c)_j = 1 \text{ for } c = \text{id}_C^{-1}(p), \quad R_{p,i} = 1 \iff i = \text{id}_Q(r(c)) \text{ for } c = \text{id}_C^{-1}(p),$$

$$PT_{j,u} = 1 \iff \text{object } \text{id}_S^{-1}(j) \text{ is a partition candidate of table } \text{id}_T^{-1}(u).$$

The cut coefficient matrices are

$$G^x \in \mathbb{Q}^{k \times n}, \quad (G^x)_{t,j} = g_{k,j}^x \text{ for } k = \text{id}_K^{-1}(t),$$

$$G^\lambda \in \mathbb{Q}^{k \times m}, \quad (G^\lambda)_{t,p} = g_{k,c}^\lambda \text{ for } k = \text{id}_K^{-1}(t), \quad c = \text{id}_C^{-1}(p).$$

Action compatibility masks

Define the mask bundle M as follows.

Branch mask. $M^{\text{branch}} \in \{0, 1\}^n$ is defined by $M_j^{\text{branch}} = f_j$.

Pricing mask. $M^{\text{price}} \in \{0, 1\}^q$ is defined to be the all ones vector if $\text{LPstat} = \text{Feasible}$ and the all zeros vector otherwise.

Cut mask. $M^{\text{cut}} \in \{0, 1\}^k$ is defined by $M_t^{\text{cut}} = 1$ if and only if $\text{viol}(k) > 0$ for $k = \text{id}_K^{-1}(t)$.

Primal mask. M^{primal} is defined to be identically one, since primal construction is admissible as a proposal and acceptance remains an exact external predicate.

21.2.2 Normalisation and boundedness

Some advisory modules require bounded inputs. Define the elementwise map $\eta : \mathbb{Q} \rightarrow \mathbb{Q} \cap (-1, 1)$ by

$$\eta(z) = \frac{z}{1 + |z|}.$$

Lemma 20.2 (Exact invertibility of bounded normalisation). The map η is bijective from $\mathbb{R}$ to $(-1, 1)$ with inverse

$$\eta^{-1}(y) = \frac{y}{1 - |y|}.$$

The same formulas hold for rationals wherever the denominator is nonzero.

Proof. For any $z \in \mathbb{R}$, $|\eta(z)| = |z|/(1 + |z|) < 1$, so $\eta(z) \in (-1, 1)$. For any $y \in (-1, 1)$, the value $z = y/(1 - |y|)$ is well-defined and satisfies $\eta(z) = y$ by direct substitution. $\square$

If boundedness is applied, it must be applied componentwise to the feature matrices only. Incidence and coefficient matrices retain their exact values, since they are already bounded by construction.

21.3 Representation finiteness and stability

21.3.1 Finiteness under finite instance closure

Theorem 20.3 (Well-definedness of $\Phi(\sigma)$). For every syntactically valid node-state σ , the structured input pack $\Phi(\sigma)$ is defined.

Proof. All index sets are finite. Canonical bijections id_C and id_K exist and are unique by Lemma 20.1. Each feature entry is either a fixed instance rational, a Boolean bound, or an explicit rational expression in stored rationals, with the infeasible case defined to produce 0 for LP dependent entries. Incidence and coefficient matrices are defined by explicit finite membership and coefficient lookups. Therefore $\Phi(\sigma)$ is defined. $\square$

Theorem 20.4 (Finite dimensionality). For every node-state σ , all tensors in $\Phi(\sigma)$ have finite dimensions.

Proof. The matrices X, Q, Cm, Km, Tb, G have fixed column counts and finite row counts determined by n, q, m, κ, τ . The matrices U, R, PT, G^x, G^λ have finite shapes derived from the same finite counts. The masks in M have finite length. Hence $\Phi(\sigma)$ is finite. $\square$

21.3.2 Stability requirements across runs

Theorem 20.5 (Reconstruction sufficiency for gate predicates). There exists a deterministic reconstruction procedure R such that, given $\Phi(\sigma)$, one can compute every acceptance predicate and scalar quantity that depends only on the node-state and is required for the acceptance of branching, pricing, cutting, and primal proposals.

Proof. From X one recovers ℓ_j, u_j, x_j^*, f_j and therefore the branchable set $\{j : f_j = 1\}$. From Cm one recovers reduced costs and active column weights. From Km one recovers violations and therefore cut violation predicates. From U and R one recovers object requirements and query membership of each column. From G^x and G^λ one recovers the exact cut coefficients on current variables. Every gate predicate expressible as a finite Boolean combination of these recovered values is therefore computable from $\Phi(\sigma)$ without hidden state. $\square$

Figure 21.1: Typed hypergraph schema used for the state representation. The diagram specifies vertex types S, Q, C, K, T, G and the admissible relation families defined in Section 20.1.

Table 21.1: Feature dictionary for the state representation. All features are defined in exact rational arithmetic with explicit rules for the infeasible LP case.

Vertex type	Symbol	Feature components
Object (S)	$\phi_S(j)$	Type indicators $(1_I, 1_V, 1_P)$, size a_j , maintenance m_j , bounds (ℓ_j, u_j) , LP value x_j^* (or 0 if infeasible), fractional flag f_j
Query (Q)	$\phi_Q(i)$	Weight w_r , query dual α_r (or 0 if infeasible)
Column (C)	$\phi_C(p)$	Plan cost $c_{r(c), \ell(c)}$, LP weight λ_c^* (or 0 if infeasible), reduced cost $rc(c)$ (or 0 if infeasible), query index $\text{id}_Q(r(c))$
Cut (K)	$\phi_K(t)$	Right-hand side h_k , violation $\text{viol}(k)$ (or 0 if infeasible)
Table (T)	$\phi_T(u)$	Canonical table index u
Global (G)	ϕ_G	Budget B , incumbent UB^* , feasibility indicator 1_{Feasible}

Corollary 20.6 (Advisor interface stability). Advisory modules may be restricted to $\Phi(\sigma)$ without loss of correctness, since acceptance remains external and exact, and $\Phi(\sigma)$ suffices for the evaluation of node-state dependent acceptance predicates.

21.4 Figures and tables

- State hypergraph schema diagram.
- Feature dictionary table.

22

Action Spaces for Branching, Cutting, Pricing, and Primal Construction

22.1 Action sets and admissibility

Let Q be the finite set of workload queries, let D be the finite set of candidate physical design objects, and for each $q \in Q$ let P_q be the finite universe of executable plan templates for q . For each solver state σ , let $P_q(\sigma) \subseteq P_q$ denote the active plan subset currently present in the restricted master, and let $A(\sigma)z \leq b(\sigma)$ denote the current linear relaxation system over the finite variable vector z .

22.1.1 Branch actions

Let $\mathcal{B}(\sigma) \subseteq \{1, \dots, n(\sigma)\}$ be the set of variable indices on which branching is permitted, where $n(\sigma)$ is the number of relaxation variables at state σ , and let $z^*(\sigma)$ be an optimal solution of the node relaxation when available. Define the fractional index set

$$F(\sigma) = \left\{ j \in \mathcal{B}(\sigma) : 0 < z_j^*(\sigma) < 1 \right\}. \quad (22.1)$$

Define the branch action set

$$A_{\text{br}}(\sigma) = (F(\sigma) \times \{0, 1\}) \cup \{\text{NO_BRANCH}\}. \quad (22.2)$$

Define the branch admissibility predicate $\text{Adm}_{\text{br}}(\sigma, a)$ by

$$\text{Adm}_{\text{br}}(\sigma, (j, v)) \iff (j \in F(\sigma)) \wedge (v \in \{0, 1\}), \quad (22.3)$$

$$\text{Adm}_{\text{br}}(\sigma, \text{NO_BRANCH}) \iff (F(\sigma) = \emptyset). \quad (22.4)$$

22.1.2 Cut proposal actions

Fix constants $K \in \mathbb{N}_{\geq 1}$ and $D \in \mathbb{N}_{\geq 1}$. Let $A(\sigma)z \leq b(\sigma)$ have $m(\sigma)$ rows indexed by $\{1, \dots, m(\sigma)\}$. For any support set $S \subseteq \{1, \dots, m(\sigma)\}$ with $|S| \leq K$, define the finite multiplier family

$$\Pi_{S,D} = \left\{ \pi \in \mathbb{Q}_{\geq 0}^{m(\sigma)} : \text{supp}(\pi) \subseteq S, \pi_i = \frac{u_i}{D}, u_i \in \{0, 1, \dots, D\} \right\}. \quad (22.5)$$

Each $\pi \in \Pi_{S,D}$ induces a candidate inequality of Chvátal–Gomory type

$$c(\pi, \sigma) : \pi^\top A(\sigma)z \leq \lfloor \pi^\top b(\sigma) \rfloor. \quad (22.6)$$

Define the cut action set

$$A_{\text{cut}}(\sigma) = \{c(\pi, \sigma) : \exists S \subseteq \{1, \dots, m(\sigma)\}, |S| \leq K, \pi \in \Pi_{S,D}\} \cup \{\text{NO_CUT}\}. \quad (22.7)$$

Let $C(\sigma)$ denote the cut pool at state σ under the canonical cut encoding. Define the cut admissibility predicate $\text{Adm}_{\text{cut}}(\sigma, a)$ by

$$\text{Adm}_{\text{cut}}(\sigma, c) \iff \left(c \in A_{\text{cut}}(\sigma) \setminus \{\text{NO_CUT}\} \right) \wedge \left(c \notin C(\sigma) \right) \wedge \left(c \text{ is well formed over } z \right), \quad (22.8)$$

$$\text{Adm}_{\text{cut}}(\sigma, \text{NO_CUT}) \iff \top. \quad (22.9)$$

22.1.3 Pricing proposal actions

For each $q \in Q$, define the not yet active plan set

$$\bar{P}_q(\sigma) = P_q \setminus P_q(\sigma), \quad (22.10)$$

and define the pricing target set

$$\mathcal{T}_{\text{pr}}(\sigma) = \bigcup_{q \in Q} (\{q\} \times \bar{P}_q(\sigma)). \quad (22.11)$$

Define the pricing action set

$$A_{\text{pr}}(\sigma) = \mathcal{T}_{\text{pr}}(\sigma) \cup \{\text{NO_PRICE}\}. \quad (22.12)$$

Define the pricing admissibility predicate $\text{Adm}_{\text{pr}}(\sigma, a)$ by

$$\text{Adm}_{\text{pr}}(\sigma, (q, p)) \iff (q \in Q) \wedge (p \in \bar{P}_q(\sigma)), \quad (22.13)$$

$$\text{Adm}_{\text{pr}}(\sigma, \text{NO_PRICE}) \iff \top. \quad (22.14)$$

22.1.4 Primal construction actions

Let $\mathcal{Y} \subseteq \{0, 1\}^{|D|}$ denote the feasible design set defined by the full design side constraints, and let $\text{Fix}(\sigma)$ denote the set of node fixings induced by the current search node. Define the primal construction action set

$$A_{\text{ub}}(\sigma) = \mathcal{Y} \cup \{\text{NO_PRIMAL}\}. \quad (22.15)$$

Define the primal admissibility predicate $\text{Adm}_{\text{ub}}(\sigma, a)$ by

$$\text{Adm}_{\text{ub}}(\sigma, y) \iff (y \in \mathcal{Y}) \wedge (y \text{ respects } \text{Fix}(\sigma)), \quad (22.16)$$

$$\text{Adm}_{\text{ub}}(\sigma, \text{NO_PRIMAL}) \iff \top. \quad (22.17)$$

Define the unified action space at state σ as the tagged disjoint union

$$A(\sigma) = (\{\text{br}\} \times A_{\text{br}}(\sigma)) \cup (\{\text{cut}\} \times A_{\text{cut}}(\sigma)) \cup (\{\text{pr}\} \times A_{\text{pr}}(\sigma)) \cup (\{\text{ub}\} \times A_{\text{ub}}(\sigma)). \quad (22.18)$$

22.2 Enumerability and finiteness

22.2.1 Finite action enumeration under closure

Lemma 22.1 (Finiteness of branch actions). *For every state σ , the set $A_{\text{br}}(\sigma)$ is finite and satisfies*

$$|A_{\text{br}}(\sigma)| \leq 2|\mathcal{B}(\sigma)| + 1 \leq 2n(\sigma) + 1. \quad (22.19)$$

Proof. The set $F(\sigma) \subseteq \mathcal{B}(\sigma)$ is finite. Each $j \in F(\sigma)$ yields exactly two payloads $(j, 0)$ and $(j, 1)$, and adjoining NO_BRANCH adds one element. $\square$

Lemma 22.2 (Finiteness of cut actions under caps). *Fix K and D as above. For every state σ , the set $A_{\text{cut}}(\sigma)$ is finite.*

Proof. There are finitely many supports $S \subseteq \{1, \dots, m(\sigma)\}$ with $|S| \leq K$. For each such S , the family $\Pi_{S,D}$ is finite by construction. Each $\pi \in \Pi_{S,D}$ induces exactly one inequality object $c(\pi, \sigma)$. The union over all eligible supports and multipliers is finite, and adjoining NO_CUT preserves finiteness. $\square$

Lemma 22.3 (Finiteness of pricing actions). *For every state σ , the set $A_{\text{pr}}(\sigma)$ is finite and satisfies*

$$|A_{\text{pr}}(\sigma)| \leq \sum_{q \in Q} |P_q| + 1. \quad (22.20)$$

Proof. Each P_q is finite, hence $\overline{P}_q(\sigma)$ is finite. The union over the finite set Q of finite sets $\{q\} \times \overline{P}_q(\sigma)$ is finite, and adjoining NO_PRICE adds one element. $\square$

Lemma 22.4 (Finiteness of primal actions). *The feasible design set $\mathcal{Y}$ is finite, and for every state σ , the set $A_{\text{ub}}(\sigma)$ is finite.*

Figure 22.1: Action space map diagram for branching, cutting, pricing, and primal construction.

Proof. The hypercube $\{0, 1\}^{|D|}$ is finite, and $\mathcal{Y} \subseteq \{0, 1\}^{|D|}$ is therefore finite. Adding NO_PRIMAL preserves finiteness. $\square$

Theorem 22.1 (Existence of explicit enumerators). *For every state σ , there exist terminating procedures $\text{Enum}_{\text{br}}(\sigma)$, $\text{Enum}_{\text{cut}}(\sigma)$, $\text{Enum}_{\text{pr}}(\sigma)$, and $\text{Enum}_{\text{ub}}(\sigma)$ that enumerate $A_{\text{br}}(\sigma)$, $A_{\text{cut}}(\sigma)$, $A_{\text{pr}}(\sigma)$, and $A_{\text{ub}}(\sigma)$ respectively.*

Proof. Each action set is a finite union of finite sets. An explicit enumeration follows by iterating the defining finite indices in the construction of each set and emitting the null action at the end. $\square$

22.2.2 Admissibility proofs

Proposition 22.1 (Typing and well formedness). *For every state σ and action payload a , if $\text{Adm}_{\text{br}}(\sigma, a)$ holds then $a \in A_{\text{br}}(\sigma)$, if $\text{Adm}_{\text{cut}}(\sigma, a)$ holds then $a \in A_{\text{cut}}(\sigma)$, if $\text{Adm}_{\text{pr}}(\sigma, a)$ holds then $a \in A_{\text{pr}}(\sigma)$, and if $\text{Adm}_{\text{ub}}(\sigma, a)$ holds then $a \in A_{\text{ub}}(\sigma)$.*

Proof. Each predicate is defined by membership in the corresponding set together with explicit well formedness conditions, or by the corresponding null action. The statement follows directly from the definitions. $\square$

Theorem 22.2 (Exhaustiveness of the declared action spaces). *At any state σ , every solver legal proposal that can be presented to the corresponding acceptance mechanism without violating typing or well formedness belongs to the corresponding action set defined in Section 22.*

Proof. Branch proposals are restricted to splits on permitted fractional coordinates, hence lie in $F(\sigma) \times \{0, 1\}$ or equal NO_BRANCH. Cut proposals are restricted to the admitted template family under caps K and D , hence equal $c(\pi, \sigma)$ for some eligible support and multiplier, or equal NO_CUT. Pricing proposals identify a query and a not yet active plan, hence lie in $\bigcup_{q \in Q} (\{q\} \times \bar{P}_q(\sigma))$ or equal NO_PRICE. Primal proposals identify a feasible design respecting node fixings, hence lie in $\mathcal{Y}$ or equal NO_PRIMAL. $\square$

22.3 Figures and tables

- Action space map diagram.
- Action admissibility table.

Class	Symbol	Action set at state σ	Admissibility predicate at state σ
Branch	$A_{\text{br}}(\sigma)$	$F(\sigma) \times \{0, 1\} \cup \{\text{NO_BRANCH}\}$	$\text{Adm}_{\text{br}}(\sigma, (j, v)) \iff j \in F(\sigma), v \in \{0, 1\};$ $\text{Adm}_{\text{br}}(\sigma, \text{NO_BRANCH}) \iff F(\sigma) = \emptyset$
Cut	$A_{\text{cut}}(\sigma)$	$\{c(\pi, \sigma) : \exists S, S \leq K, \pi \in \Pi_{S,D}\} \cup \{\text{NO_CUT}\}$	$\text{Adm}_{\text{cut}}(\sigma, c) \iff c \in A_{\text{cut}}(\sigma) \setminus \{\text{NO_CUT}\}, c \notin C(\sigma), c \text{ well formed};$ $\text{Adm}_{\text{cut}}(\sigma, \text{NO_CUT}) \iff \top$
Pricing	$A_{\text{pr}}(\sigma)$	$\bigcup_{q \in Q} (\{q\} \times \bar{P}_q(\sigma)) \cup \{\text{NO_PRICE}\}$	$\text{Adm}_{\text{pr}}(\sigma, (q, p)) \iff q \in Q, p \in \bar{P}_q(\sigma);$ $\text{Adm}_{\text{pr}}(\sigma, \text{NO_PRICE}) \iff \top$
Primal	$A_{\text{ub}}(\sigma)$	$\mathcal{Y} \cup \{\text{NO_PRIMAL}\}$	$\text{Adm}_{\text{ub}}(\sigma, y) \iff y \in \mathcal{Y} \wedge y \text{ respects } \text{Fix}(\sigma);$ $\text{Adm}_{\text{ub}}(\sigma, \text{NO_PRIMAL}) \iff \top$

Table 22.1: Action admissibility table. The table is width safe by construction under `tabularx` with total width `\textwidth`.

23

Training Signals and Trace Semantics

23.1 Trace object definition

23.1.1 What is logged

Arithmetic domain and canonical representation. Let an optimization instance be denoted by I . All scalar quantities in I are elements of $\mathbb{Q}$. Each rational is represented canonically as a pair $(p, q) \in \mathbb{Z} \times \mathbb{Z}_{>0}$ with $\gcd(|p|, q) = 1$, interpreted as p/q . All quantities referenced in this chapter, including all label targets, are computed in exact rational arithmetic.

Node states and admissible action universes. Let $\mathcal{N}$ be the set of solver nodes reachable under the branch and bound protocol. Each processed node $N \in \mathcal{N}$ is assigned a deterministic canonical encoding

$$\Phi : \mathcal{N} \rightarrow \Sigma^*,$$

where Σ^* denotes finite strings over a fixed finite alphabet. The map Φ is stable under serialization and injective up to the node identity relation used by the solver registry.

For each action family $\alpha \in \{\text{br}, \text{cut}, \text{pr}, \text{pm}\}$, let $A_\alpha(N)$ denote the finite set of admissible action identifiers presented at node N , with $A_\alpha(N) \subseteq \text{ID}_\alpha$. Finiteness is a structural requirement for trace semantics and is enforced by the instance universe and admissibility filters defined in earlier chapters.

Trace schema. A run trace is a finite sequence of node records,

$$T = (R(N_1), \dots, R(N_k)) \in \mathcal{T} := \bigcup_{k \in \mathbb{N}} \mathcal{R}^k,$$

where $\mathcal{R}$ is the set of node records. A node record $R(N)$ associated with a processed node N contains:

1. **State:** $\Phi(N) \in \Sigma^*$.
2. **Exact relaxation bound:** $LB(N) \in \mathbb{Q} \cup \{+\infty\}$, where $LB(N) = +\infty$ denotes certified infeasibility of the node relaxation.
3. **LP primal solution:** $z^*(N)$ in canonical rational form when $LB(N) \in \mathbb{Q}$, and a distinguished null value when $LB(N) = +\infty$.
4. **Incumbent bound at finalization:** $UB_N \in \mathbb{Q} \cup \{+\infty\}$.
5. **Candidate action sets:** $A_{br}(N), A_{cut}(N), A_{pr}(N), A_{pm}(N)$.
6. **Outcome ledger:** for each executed action identifier a , a status $\text{status}(a) \in \{\text{accepted}, \text{rejected}\}$ and a verification record $\text{ver}(a)$ in the certificate format mandated by the corresponding checker.

Trace header. Each trace carries a header with an instance identifier, a solver build identifier, a schema version identifier, and a configuration hash, all in Σ^* . These identifiers are part of the trace object and are required for semantic interpretation of Φ , action identifiers, and certificate formats.

23.1.2 Trace integrity requirements

Admissibility predicate. A trace is admissible precisely when every accepted action in every node ledger passes its corresponding checker. Define the predicate $\text{Admissible} : \mathcal{T} \rightarrow \{\text{true}, \text{false}\}$ by

$$\text{Admissible}(T) = \text{true} \iff \forall R(N) \in T, \forall a, \text{status}(a) = \text{accepted} \Rightarrow \text{Check}(\text{ver}(a)) = \text{pass}.$$

All label constructions below are defined only as functions of admissible traces. This restriction is semantic: it prevents training targets from depending on unverified information.

Closure requirements. The trace is required to be self describing relative to its header identifiers. In particular, the interpretation of Φ , of ID_α , and of $\text{ver}(\cdot)$ is fixed by the schema version and the solver build identifier, and the configuration hash is required to match the execution configuration used to produce the node sequence.

23.2 Label constructions

23.2.1 Label types and sources

Label families. Training targets are defined for four action families: branching (br), cutting (cut), pricing (pr), and primal construction or improvement (pm). Each label

is computed from quantities recorded in $R(N)$ together with terminating certified subroutines already present in the solver core, including one step bound probes, reduced cost computations, feasibility checks, and objective evaluations.

Permitted numeric sources. The only numeric inputs to label computation are: recorded bounds $LB(N)$ and UB_N , reduced costs and dual objects derived from the recorded relaxation solution, objective values computed by the exact objective function, and bounds computed by certified subroutines invoked on explicitly constructed successor states.

23.2.2 Well-definedness and determinism

Deterministic tie breaking. For each family α and each node N , fix a total order $<_{\alpha,N}$ on $A_\alpha(N)$ induced by lexicographic comparison of canonical encodings in Σ^* . Every set valued extremum is resolved by selecting the $<_{\alpha,N}$ smallest optimizer. This rule makes each label single valued.

Branching labels

Let N be a node with $A_{\text{br}}(N) \neq \emptyset$. For each $j \in A_{\text{br}}(N)$, define the two child nodes $N_{j,0}$ and $N_{j,1}$ produced by the canonical branching operator that fixes variable j to 0 and 1. Let

$$LB_{j,b} := LB(N_{j,b}) \quad \text{for } b \in \{0, 1\},$$

where $LB(\cdot)$ is computed by a terminating certified bound computation applied to the child state without further branching beyond the single split.

Define the balanced bound gain

$$G_{\text{br}}(N, j) := \min\{LB_{j,0}, LB_{j,1}\} - LB(N).$$

Define the branching label

$$j^*(N) := \arg \max_{j \in A_{\text{br}}(N)} G_{\text{br}}(N, j),$$

with tie breaking by $<_{\text{br},N}$. If $A_{\text{br}}(N) = \emptyset$, no branching sample is emitted for N .

Lemma 23.1. *If $LB(N) \in \mathbb{Q}$, then $G_{\text{br}}(N, j) \geq 0$ for all $j \in A_{\text{br}}(N)$.*

Proof. Each child introduces additional constraints relative to the parent relaxation. Minimization over a subset cannot decrease the optimum, hence $LB_{j,0} \geq LB(N)$ and $LB_{j,1} \geq LB(N)$, so $\min\{LB_{j,0}, LB_{j,1}\} \geq LB(N)$ and the claim follows. $\square$

Lemma 23.2. *If $A_{\text{br}}(N) \neq \emptyset$, then $j^*(N)$ exists and is unique.*

Proof. The set $A_{\text{br}}(N)$ is finite, $G_{\text{br}}(N, j)$ is defined for each j by terminating certified subroutines, and a maximum over a finite set exists. Tie breaking by $<_{\text{br},N}$ yields uniqueness. $\square$

Cut labels

Let $A_{\text{cut}}(N)$ be the finite set of cut proposals presented at N , each with a certificate candidate. Define the certificate admissible subset

$$A_{\text{cut}}^{\text{adm}}(N) := \{\kappa \in A_{\text{cut}}(N) : \text{CheckCutCert}(N, \kappa) = \text{pass}\}.$$

For each $\kappa \in A_{\text{cut}}^{\text{adm}}(N)$, let $N + \kappa$ denote the node obtained by adding κ via the canonical add cut transition, and define $LB_{\kappa} := LB(N + \kappa)$.

Define the capped cut gain

$$G_{\text{cut}}(N, \kappa) := \min\{LB_{\kappa}, UB_N\} - LB(N).$$

Define the cut label

$$\kappa^{\star}(N) := \arg \max_{\kappa \in A_{\text{cut}}^{\text{adm}}(N)} G_{\text{cut}}(N, \kappa),$$

with tie breaking by $<_{\text{cut}, N}$. If $A_{\text{cut}}^{\text{adm}}(N) = \emptyset$, no cut sample is emitted for N .

Lemma 23.3. *For every $\kappa \in A_{\text{cut}}^{\text{adm}}(N)$, one has $LB_{\kappa} \geq LB(N)$.*

Proof. The node $N + \kappa$ contains all constraints of N and one additional valid inequality. Its feasible region is a subset of the feasible region of N . Minimization over a subset cannot yield a smaller optimum, hence $LB_{\kappa} \geq LB(N)$. $\square$

Lemma 23.4. *If $A_{\text{cut}}^{\text{adm}}(N) \neq \emptyset$, then $\kappa^{\star}(N)$ exists and is unique.*

Proof. The set $A_{\text{cut}}^{\text{adm}}(N)$ is finite and nonempty, the gain is defined for each element, and tie breaking yields a unique maximizer. $\square$

Pricing labels

At node N , for each query index r , let L_r be the finite plan universe for r . Let $\text{rc}(N, r, \ell) \in \mathbb{Q}$ denote the reduced cost of plan $\ell \in L_r$ computed from the recorded dual object and the canonical column definition.

Define the canonical reduced cost witness

$$\ell_r^{\min}(N) := \arg \min_{\ell \in L_r} \text{rc}(N, r, \ell),$$

with tie breaking by a fixed total order on L_r induced by plan encodings. Define the pricing label

$$\ell_r^{\star}(N) := \begin{cases} \ell_r^{\min}(N), & \text{if } \text{rc}(N, r, \ell_r^{\min}(N)) < 0, \\ \text{NONE}, & \text{otherwise.} \end{cases}$$

Lemma 23.5. *For each r , the witness $\ell_r^{\min}(N)$ exists and is unique.*

Proof. The set L_r is finite, the reduced cost function is defined on L_r , and tie breaking resolves minimizers uniquely. $\square$

Primal construction labels

Let F be a deterministic feasible constructor mapping a node state to a binary design, $F(N) \in \{0, 1\}^n$, defined by an explicit rule using only recorded quantities and fixed tie breaking. Let I_K be a deterministic local improvement operator that performs at most K strictly improving feasibility preserving moves, each validated by the feasibility checker and evaluated by the exact objective.

Define

$$x_{\text{pm}}^*(N) := I_K(F(N)), \quad V_{\text{pm}}(N) := \text{Obj}(x_{\text{pm}}^*(N)).$$

Each emitted primal sample carries a feasibility witness that passes the feasibility checker.

Lemma 23.6. *The mapping $N \mapsto x_{\text{pm}}^*(N)$ is total and terminates.*

Proof. The constructor F is a deterministic finite procedure. The operator I_K is explicitly bounded to at most K iterations over a finite move set. Hence the composition terminates and is defined for every N . $\square$

23.3 Correctness of label extraction

23.3.1 No undefined targets lemma

Theorem 23.1. *Let T be an admissible trace. Every label produced by the extraction procedure is defined, belongs to its declared domain, and refers only to identifiers that occur in the corresponding node record. In particular, no extracted label is an undefined target.*

Proof. Fix an emitted sample for some node N .

If the sample is a branching sample, then $A_{\text{br}}(N) \neq \emptyset$. For each $j \in A_{\text{br}}(N)$, the child states are defined by the canonical branching operator, and $LB_{j,0}, LB_{j,1}$ are defined by terminating certified bound computation. Hence $G_{\text{br}}(N, j)$ is defined for all j and the argmax returns an element of $A_{\text{br}}(N)$ by construction.

If the sample is a cut sample, then $A_{\text{cut}}^{\text{adm}}(N) \neq \emptyset$. Membership in $A_{\text{cut}}^{\text{adm}}(N)$ is defined by an explicit checker, and admissibility of T guarantees that accepted actions have verifications that pass their checkers. For each $\kappa \in A_{\text{cut}}^{\text{adm}}(N)$, the post cut node $N + \kappa$ is defined by the add cut transition and its bound LB_{κ} is defined by certified bound computation. Hence the gain is defined and the argmax returns a defined identifier in $A_{\text{cut}}^{\text{adm}}(N)$.

If the sample is a pricing sample for query r , the set L_r is finite and reduced costs are defined for all $\ell \in L_r$. By Lemma 23.5, $\ell_r^{\min}(N)$ is defined, and the label is either this identifier or the explicit null symbol NONE.

If the sample is a primal sample, Lemma 23.6 yields totality and termination. The feasibility witness requirement excludes infeasible outputs as labels.

In all cases, the label is computed by explicit deterministic rules applied to recorded finite objects and terminating certified subroutines. Hence no label can be undefined and no label can refer to a nonexistent identifier. $\square$

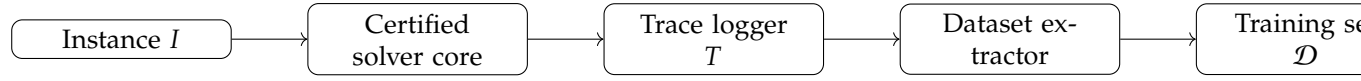


Figure 23.1: Trace-to-label pipeline. The learning substrate is a deterministic function of an admissible trace and uses only verified solver objects.

23.3.2 Consistency across runs

Definition 23.1 (Run equivalence for extraction). Two runs are extraction equivalent if they share the same instance identifier, solver build identifier, schema version identifier, and configuration hash, and if the solver produces the same sequence of processed node encodings $\Phi(N)$ together with identical node record fields in canonical form.

Lemma 23.7. On admissible traces, label extraction is a deterministic function of the trace object. If two admissible traces are identical as sequences of node records, then the extracted datasets are identical.

Proof. Every label is defined by extremization over a finite set with a fixed tie breaking order induced by canonical encodings. All auxiliary numeric targets are rational functions of recorded quantities or values returned by terminating certified subroutines invoked on explicitly constructed successor states. Therefore identical traces yield identical labels and identical extracted datasets. $\square$

23.4 Figures and tables

- Trace-to-label pipeline diagram.
- Label taxonomy table.

Type	Identifier	Domain	Primary sources Null symbol	Certificate gate
Branch	$j^*(N)$	$A_{\text{br}}(N)$	One step child bound probes and recorded $LB(N)$ Cut certificates, post cut bound $LB(N + \kappa)$, recorded UB_N and $LB(N)$ Reduced costs from recorded dual objects and plan universe L_r Deterministic constructor, bounded improvement, exact objective evaluation	Bound computation certificates
Cut	$\kappa^*(N)$	$A_{\text{cut}}^{\text{adm}}(N)$	None	Cut validity checker and bound certificates
Pricing	$\ell_r^*(N)$	$L_r \cup \{\text{NONE}\}$	NONE	Reduced cost computation closure
Primal	$x_{\text{pm}}^*(N)$	$\{0, 1\}^n$ feasible	None	Feasibility checker and exact objective evaluation

Table 23.1: Label taxonomy and trace sources. Every label is computed from finite recorded objects and terminating certified procedures under fixed tie breaking.

24

Gate-Compatible Decision Policies (Correctness Immunity)

The advisory policies in this part are positioned relative to learning-to-branch and related machine learning guided branch-and-bound strategies (Khalil et al., 2016; Gasse et al., 2019; Gupta et al., 2022; Tang et al., 2020).

24.1 Proposal functions versus acceptance gates

24.1.1 Advisory proposals

Let I denote a fixed physical design instance defined in Chapters ??–??. Let $\mathcal{S}(I)$ denote the set of solver core states reachable under the certified execution semantics for instance I . A state $s \in \mathcal{S}(I)$ is a finite record containing, at minimum, the active node description, the incumbent register, the currently active relaxation description at the active node, the ledger of admitted proof objects, and the invariant ledger required for integrity preservation.

For each state s , let $\mathcal{A}(s)$ denote the set of admissible core actions. An action is admissible if and only if it is defined by the solver core semantics at state s and its application is permitted by the invariant ledger and the certificate discipline established in Chapters ??–??. The set $\mathcal{A}(s)$ is defined independently of any learned component.

Let δ denote the certified transition function of the solver core. It is a partial function

$$\delta : \mathcal{S}(I) \times \mathcal{A} \rightarrow \mathcal{S}(I),$$

with $\delta(s, a)$ defined if and only if $a \in \mathcal{A}(s)$. When defined, $\delta(s, a)$ is deterministic.

A proposal function is a non-committing selector that observes a state and returns a finite ordered list of action proposals together with auxiliary metadata. Let Θ be a

parameter space. A proposal function is a mapping

$$\pi_\theta : \mathcal{S}(I) \rightarrow \mathcal{L},$$

where $\mathcal{L}$ denotes the set of finite lists whose elements are pairs (a, u) with a an action identifier and u finite auxiliary information. The proposal function may be stochastic. It is not permitted to modify the solver core state. In particular, proposal emission does not update the incumbent, does not alter the relaxation, does not modify the node tree, and does not mutate the invariant ledger.

The system admits multiple proposal functions, each confined to an action family. The principal families are branching proposals, cutting plane proposals, pricing proposals, and primal feasibility proposals. Their common property is that their outputs are proposals only and are incapable of directly inducing a state transition.

24.1.2 Acceptance gates as sole effectors

An acceptance gate is the unique mechanism by which a proposed action can be committed to the solver core. Let Ω denote a set of finite certificate objects. An acceptance gate is a deterministic predicate

$$G : \mathcal{S}(I) \times \mathcal{A} \times \Omega \rightarrow \{0, 1\}.$$

For a given state s , a proposed action a , and a certificate object ω , the predicate value $G(s, a, \omega) = 1$ signifies that the proposal is admitted and may be committed. The predicate value $G(s, a, \omega) = 0$ signifies rejection.

The gate is required to satisfy the following properties.

Gate soundness. For all $s \in \mathcal{S}(I)$, $a \in \mathcal{A}$, and $\omega \in \Omega$, if $G(s, a, \omega) = 1$, then $a \in \mathcal{A}(s)$ and ω is a valid certificate for admitting a at s according to the certificate schemas and arithmetic verification rules of Chapter ??.

Gate determinism. For fixed s , a , and ω , the value of $G(s, a, \omega)$ depends only on the immutable core semantics and the explicit finite records s , a , and ω .

The committed execution is defined as follows. Given state s , proposal (a, u) , and certificate ω , the solver commits the transition if and only if $G(s, a, \omega) = 1$. In that case, the next state is $s^+ = \delta(s, a)$. If $G(s, a, \omega) = 0$, then the solver state remains unchanged, except for optional trace logging of the rejected proposal in a channel that is not conflated with committed actions.

Each action family has a corresponding certificate type and gate procedure. Branch actions require a finite witness that the branch is well formed and respects the current domain restrictions. Cut actions require a validity proof object for the admitted cut system. Pricing actions require a certificate that the column corresponds to an admissible combinatorial object and that its reduced-cost arithmetic is correct with respect

to the current dual record. Primal actions require a feasibility witness establishing satisfaction of all constraints, including budget and compatibility constraints. The gate consumes the appropriate certificate and performs the deterministic verification steps prescribed by the immutable verification rules.

24.2 Correctness immunity theorem

24.2.1 Learning cannot change feasibility, bound validity, or pruning soundness

This section states the non-interference property of learned guidance. The solver is assumed to satisfy the end-to-end correctness conditions of Chapters ??–?? when run without learned proposals, and learned components are assumed to operate only through proposal emission.

Definitions. A design is feasible if it satisfies all constraints of the baseline formulation and the compatibility semantics defined in the instance construction part. A cut is valid if it does not exclude any feasible integer solution in the intended relaxation. A column is valid if it corresponds to an admissible object in the underlying semantics and its encoding preserves the restricted master interpretation. A node bound is valid if it is a lower bound on the optimal objective value over all feasible integer solutions consistent with the node domain restrictions, for minimization instances.

Assumptions. The acceptance gate satisfies gate soundness and gate determinism. The solver core transition system δ is correct with respect to feasibility of incumbents, validity of admitted proof objects, and bound soundness. Proposals are non-committing and cannot modify the state except through gate-admitted transitions.

Theorem 24.1 (Non-interference of gate-compatible guidance). *Fix an instance I . Consider any execution of the solver in which proposal functions π_θ generate candidate actions and acceptance gates G determine which actions are committed. Under the assumptions stated above, the following properties hold throughout the execution.*

1. *Every committed incumbent update yields a feasible incumbent.*
2. *Every committed cut and every committed column is valid for the intended relaxation semantics.*
3. *Every bound computed from the relaxation induced by committed cuts and committed columns is a valid bound.*
4. *Every pruning event performed using certified bounds is sound.*

Consequently, learned guidance can affect only the ordering and selection among admissible actions, and cannot induce infeasibility, invalidity, or unsound pruning.

24.2.2 Proof strategy via gate separation

Proof. Let the committed execution be a sequence of states $s_0, s_1, \dots$ such that for each step t there exists a proposed action a_t and a certificate ω_t with $G(s_t, a_t, \omega_t) = 1$ and $s_{t+1} = \delta(s_t, a_t)$.

Feasibility preservation follows because an incumbent can be updated only by a committed primal action. Gate soundness implies that admission requires a valid feasibility witness for the update. Therefore every committed incumbent is feasible.

Validity preservation for cuts follows because a cut can be committed only by a cut action admitted by the gate, and gate soundness implies that admission requires a valid cut certificate. Therefore every committed cut is valid.

Validity preservation for columns follows because a column can be committed only by a pricing action admitted by the gate, and gate soundness implies that admission requires a valid column certificate establishing admissibility and correct reduced-cost accounting. Therefore every committed column is valid.

Bound validity follows because node relaxations are formed from the baseline constraints, the current node domain restrictions, and the set of committed proof objects. Since committed cuts are valid and committed columns preserve the intended restricted master meaning, the resulting relaxation remains a relaxation of the integer feasible set at that node. Therefore the relaxation optimum is a valid bound.

Pruning soundness follows from the bound validity property and the pruning rules of Chapter ??, because pruning is performed only when a certified node bound cannot improve upon the current feasible incumbent value.

Finally, proposals that are rejected do not modify the state, and proposals that are admitted are restricted to admissible actions with valid certificates. Therefore learned guidance can change only the order in which admissible, certificate validated actions are attempted, and cannot affect feasibility, validity, bound correctness, or pruning soundness. $\square$

24.3 Continual update constraints

24.3.1 Immutable core versus mutable advisors

The system is divided into an immutable correctness core and a mutable advisory layer.

The immutable core consists of the instance semantics, the admissible action definitions $\mathcal{A}(s)$, the acceptance gates G , the certificate schemas and arithmetic verification procedures, the invariant ledger and its preservation checks, and the trace schema that distinguishes proposed events from committed events. These elements are immutable because any change would alter the meaning of admissibility, certificate validity, or bound semantics.

The mutable advisors consist of proposal functions π_θ and any associated scoring or ranking models that operate strictly in the proposal space. Mutable advisors may

be trained, tuned, replaced, or disabled, provided that they remain incapable of committing state transitions and provided that they do not alter the immutable core.

24.3.2 Rollback and disabling rules

The continual update regime is governed by the following constraints.

First, within a solver run that is intended to be repeatable and certifiable, all advisor parameters are frozen at run start and their identifiers are recorded in the transcript.

Second, the advisor input schema, including feature normalization and encoding, is fixed within a run. Any change to the encoding is treated as an advisor change and is prohibited within a run.

Third, gate dominance is mandatory. No advisor update mechanism may bypass the gates. The only causal pathway from an advisor to the solver core is via a proposal that is subsequently admitted by the gate with a valid certificate.

Fourth, rejection logging must remain semantically separated from committed actions. Rejected proposals may be logged for training, but the trace must preserve the distinction between proposals and committed transitions to prevent contamination of downstream label extraction.

Fifth, disabling rules are required. Any advisor may be disabled at any time without changing the solver core semantics. Disabling an advisor reduces the set of proposed actions but does not change the admissible set $\mathcal{A}(s)$ nor the acceptance rules. Therefore disabling cannot compromise correctness. Rollback to a previous advisor parameterization is always permitted, provided the rollback affects only proposal generation.

24.4 Figures and tables

- Proposal acceptance separation diagram.
- Gate checklist table.

25

Calibration, Drift Control, and Stability Protocols

25.1 Calibration envelope definitions

25.1.1 Calibration targets

Let $\mathcal{S}$ denote the finite, typed solver state space induced by the instance encoding, the branch-and-bound transcript prefix, and the fixed tie-breaking rules. Let $\mathcal{K}$ denote the finite set of advisor types. For each advisor type $k \in \mathcal{K}$, let $\mathcal{A}_k(s)$ denote the admissible action set at state $s \in \mathcal{S}$.

For each $k \in \mathcal{K}$, let $A_k(\cdot; \theta)$ be a parametric proposal map. Given $s \in \mathcal{S}$, it returns a proposed action $a \in \mathcal{A}_k(s)$ together with a score $\sigma \in [0, 1]$. The score σ has no operational meaning except through the calibration protocol defined in this section.

Let G_k be the fixed acceptance gate for type k . The gate is a total function that, given a state s and a proposed action a , returns either ACCEPT or REJECT according to immutable feasibility checks and certificate verifiers. Let $a_k^{\text{fb}}(s)$ denote the deterministic fallback action for type k at state s . The executed action is defined by

$$\text{Exec}_k(s; \theta) = \begin{cases} a, & \text{if } G_k(s, a) = \text{ACCEPT}, \\ a_k^{\text{fb}}(s), & \text{if } G_k(s, a) = \text{REJECT}, \end{cases} \quad (25.1)$$

where $(a, \sigma) = A_k(s; \theta)$.

Define the observable success indicator for an advisor call of type k at state s as

$$Y_k(s; \theta) = \mathbf{1}[G_k(s, A_k(s; \theta)) = \text{ACCEPT}]. \quad (25.2)$$

This success variable is mechanically observable and it is the unique calibration target used by the protocol.

Fix an integer $K \geq 1$ and a measurable partition of $[0, 1]$ into disjoint bins $B_1, \dots, B_K$

such that $\bigcup_{j=1}^K B_j = [0, 1]$. Let $\text{bin}(\sigma)$ denote the unique index j such that $\sigma \in B_j$. Fix a window length $W \geq 1$ for drift monitoring.

For each advisor type k and bin index j , maintain the counters

$$N_{k,j} \in \mathbb{N}, \quad S_{k,j} \in \mathbb{N}, \quad N_{k,j}^W \in \mathbb{N}, \quad S_{k,j}^W \in \mathbb{N},$$

where $N_{k,j}$ and $S_{k,j}$ are total call and success counts in bin j , and $N_{k,j}^W$ and $S_{k,j}^W$ are the corresponding windowed counts over the most recent W non-suppressed calls of type k whose scores fell in bin j .

Define the empirical success rates

$$\hat{p}_{k,j} = \begin{cases} \frac{S_{k,j}}{N_{k,j}}, & \text{if } N_{k,j} \geq 1, \\ 0, & \text{if } N_{k,j} = 0, \end{cases} \quad \hat{p}_{k,j}^W = \begin{cases} \frac{S_{k,j}^W}{N_{k,j}^W}, & \text{if } N_{k,j}^W \geq 1, \\ 0, & \text{if } N_{k,j}^W = 0. \end{cases} \quad (25.3)$$

Fix a global confidence parameter $\delta \in (0, 1)$ and define the per-bin allocation

$$\Delta = \frac{\delta}{2K}. \quad (25.4)$$

For $n \geq 1$ and $\Delta \in (0, 1)$, define the confidence radius

$$r(n, \Delta) = \sqrt{\frac{\ln(2/\Delta)}{2n}}. \quad (25.5)$$

Define the lower confidence bounds

$$\text{LCB}_{k,j} = \hat{p}_{k,j} - r(N_{k,j}, \Delta), \quad \text{LCB}_{k,j}^W = \hat{p}_{k,j}^W - r(N_{k,j}^W, \Delta), \quad (25.6)$$

with the convention that $\text{LCB}_{k,j}$ and $\text{LCB}_{k,j}^W$ are evaluated only when the corresponding N is at least 1.

Definition 25.1 (Calibration envelope). For each advisor type k and bin j with $N_{k,j} \geq 1$, the calibration envelope is

$$\mathcal{E}_{k,j} = [\hat{p}_{k,j} - r(N_{k,j}, \Delta), \hat{p}_{k,j} + r(N_{k,j}, \Delta)] \cap [0, 1]. \quad (25.7)$$

Assumption 25.1 (Bin-conditional stability for calibration). Fix $k \in \mathcal{K}$ and $j \in \{1, \dots, K\}$. Conditioned on non-suppression and on the event $\text{bin}(\sigma) = j$, the observed random variables Y_k across repeated calls are independent Bernoulli draws with a fixed but unknown mean $p_{k,j} \in [0, 1]$.

Lemma 25.1 (Finite-sample deviation bound). Under Assumption 25.1, for any fixed k and j and any $n \geq 1$,

$$\Pr(|\hat{p}_{k,j} - p_{k,j}| \geq r(n, \Delta)) \leq \Delta,$$

when $\hat{p}_{k,j}$ is computed from n bin-matched samples.

Theorem 25.1 (Simultaneous calibration across bins). *Fix $k \in \mathcal{K}$. Under Assumption 25.1 for all bins, with probability at least $1 - \delta/2$, the inclusion $p_{k,j} \in \mathcal{E}_{k,j}$ holds simultaneously for all j with $N_{k,j} \geq 1$.*

25.1.2 Acceptance thresholds

Definition 25.2 (Certified enabling rule). Fix advisor type k . Let $\tau_k^{\text{en}} \in (0, 1)$ be an enabling threshold and let $N_k^{\text{min}} \in \mathbb{N}$ be a minimum-evidence requirement. On a call opportunity at state s with score σ and bin index $j = \text{bin}(\sigma)$, the system is permitted to consult the advisor proposal if and only if

$$N_{k,j} \geq N_k^{\text{min}} \quad \text{and} \quad \text{LCB}_{k,j} \geq \tau_k^{\text{en}}. \quad (25.8)$$

If either condition fails, the advisor is suppressed and the executed action is $a_k^{\text{fb}}(s)$.

Theorem 25.2 (Certified success under enabling). *On the event of Theorem 25.1, whenever the enabling rule of Definition 25.2 permits consultation in bin j , the true acceptance probability satisfies $p_{k,j} \geq \tau_k^{\text{en}}$.*

25.2 Drift detection and rollback rules

25.2.1 Drift metrics

The drift protocol detects and reacts to degradation in the observed gate-acceptance rate within the windowed statistics defined in Section 25.1.1. Drift detection is evaluated only when the advisor is not in cooldown.

Definition 25.3 (Drift threshold and cooldown). Fix advisor type k . Let $\tau_k^{\text{dr}} \in (0, 1)$ be a drift threshold and let $L_k \in \mathbb{N}$ be a cooldown length measured in call opportunities of type k .

Definition 25.4 (Windowed drift test). On a call opportunity of type k with score σ and bin $j = \text{bin}(\sigma)$, drift is declared if

$$N_{k,j}^W \geq 1 \quad \text{and} \quad \text{LCB}_{k,j}^W < \tau_k^{\text{dr}}. \quad (25.9)$$

Upon drift declaration, the system enters cooldown for the next L_k call opportunities of type k .

25.2.2 Rollback and safe-mode operation

Definition 25.5 (Rollback rule and baseline champion). For each advisor type k , fix a baseline champion parameter vector θ_k^0 . When drift is declared for type k , deployment of that type is set to θ_k^0 for the duration of cooldown, and all call opportunities during cooldown are executed in safe mode, meaning that the advisor proposal is suppressed and the fallback action is executed.

Lemma 25.2 (Suppression during cooldown). *If drift is declared for type k , then for each of the next L_k call opportunities of type k , the executed action is $a_k^{\text{fb}}(s)$ and the calibration and drift counters are not updated by suppressed outcomes.*

Lemma 25.3 (Transcript determinism). *Under fixed tie-breaking, fixed binning, fixed counter updates, and immutable gates, the sequence of drift declarations, cooldown intervals, and roll-backs is a deterministic function of the solver transcript.*

25.3 Stability guarantees

25.3.1 Permitted stability theorems

The stability protocol is defined so that learning updates cannot alter any correctness-bearing mechanism and cannot induce unbounded degradation relative to a fixed baseline.

Definition 25.6 (Champion and candidate ledger). For each advisor type k , maintain a deployed champion parameter vector θ_k^* and a non-deployed candidate vector θ_k^+ . Only θ_k^* is permitted to generate proposals for deployment.

Definition 25.7 (Anchor set). Fix a finite anchor set $D_k \subset \mathcal{S} \times \mathcal{A}_k$ for each advisor type k , where each element (s_i, a_i) records a previously executed action at state s_i .

Definition 25.8 (Anchor replay constraint). A parameter vector θ is admissible for deployment only if, for every $(s_i, a_i) \in D_k$,

$$\text{Exec}_k(s_i; \theta) = a_i. \quad (25.10)$$

Definition 25.9 (Stability projector). A stability projector is a deterministic map Π that takes $(\theta_k^+, D_k, \theta_k^*)$ and returns deployed parameters $\tilde{\theta}_k$ together with a finite override table T_k such that the composite deployed behavior satisfies the anchor replay constraint of Definition 25.8.

Theorem 25.3 (Immutability of correctness-bearing mechanisms). *If feasibility checks, certificate verifiers, bound computations, and acceptance gates are immutable system constants, then no learning update to any advisor parameters can alter these functions. Consequently, learning can only influence which proposals are presented to fixed gates, never the set of admissible accepted actions.*

Theorem 25.4 (Anchor no-forgetting). *If deployed parameters are produced by the stability projector of Definition 25.9, then for every $(s_i, a_i) \in D_k$,*

$$\text{Exec}_k(s_i; \tilde{\theta}_k) = a_i.$$

Definition 25.10 (Empirical dominance acceptance). Fix a finite evaluation set $E = \{\xi_1, \dots, \xi_N\}$ of solver instances. For any parameters θ , let $C(\theta; \xi_j) \in \mathbb{N}$ denote the

deterministic evaluation cost on instance ξ_j under immutable gates. Define

$$\text{Cost}_E(\theta) = \sum_{j=1}^N C(\theta; \xi_j). \quad (25.11)$$

A candidate may replace the champion only if $\text{Cost}_E(\theta_k^+) \leq \text{Cost}_E(\theta_k^\star)$.

Theorem 25.5 (Monotone empirical stability of accepted updates). *If candidates are accepted only under Definition 25.10, then the sequence of accepted champions has non-increasing total evaluation cost on E .*

Definition 25.11 (Race and restart budget rule). Fix a baseline champion θ_k^0 and an integer budget $B_{\text{rr}} \geq 0$. On a fresh instance ξ , run the solver guided by the current champion for at most B_{rr} processed nodes, recording only globally valid artifacts. Then restart from the root and complete the solve using θ_k^0 , seeded with the recorded artifacts.

Theorem 25.6 (Unconditional bounded slowdown versus baseline). *Let $C(\theta_k^0; \xi)$ denote the baseline from-scratch cost to solve instance ξ . Under the race and restart rule of Definition 25.11, the total deployment cost satisfies*

$$C_{\text{deploy}}(\xi) \leq B_{\text{rr}} + C(\theta_k^0; \xi). \quad (25.12)$$

25.3.2 Empirical stability reports

Empirical stability reporting is restricted to finite, reproducible summaries computed from the evaluation set E . Reported quantities shall include $\text{Cost}_E(\theta_k^\star)$ for each advisor type k , the acceptance decision for each candidate update under Definition 25.10, and the drift and cooldown logs induced by Definitions 25.4 and 25.5. No empirical report is permitted to assert correctness; correctness is asserted exclusively by immutable gates and verifiers, as formalized in Theorem 25.3.

25.4 Figures and tables

- Calibration envelope diagram.
- Update/rollback schematic.

Part IV

Integrity, Invariants, and Verification (Non-Arguable Closure)

26

Unified Invariant Ledger

26.1 Invariant set definition

26.1.1 Mathematical environment and state

Fixed environment

Fix a finite optimization instance in exact rational form. Let the decision vector be

$$z = (x, \lambda) \in \mathbb{R}^{n+m}, \quad x \in \{0, 1\}^n, \quad \lambda \in \mathbb{R}_{\geq 0}^m.$$

Let the objective be $\min c^\top z$ with $c \in \mathbb{Q}^{n+m}$. Let the base constraint system be $Az \leq b$ with $A \in \mathbb{Q}^{p \times (n+m)}$ and $b \in \mathbb{Q}^p$. Define the global integer feasible region

$$F = \{z \in \mathbb{R}^{n+m} : Az \leq b, x \in \{0, 1\}^n, \lambda \geq 0\}.$$

Define the global optimum value

$$\text{OPT} = \min\{c^\top z : z \in F\}.$$

Solver state

A solver state Σ is a finite tuple

$$\Sigma = (T, O, C, \{D_N, e_N, K_N, h_N, \Lambda_N, \text{Log}(N)\}_{N \in T}, z^*, U),$$

where the components satisfy the following requirements.

- T is a finite rooted search tree with root node $N_\emptyset$.
- $O \subseteq T$ is the set of open nodes, $C \subseteq T$ is the set of closed nodes, $O \cap C = \emptyset$, and $O \cup C = T$.
- For each node $N \in T$:
 - $D_N z \leq e_N$ is the finite system of branching constraints.

- $K_N z \leq h_N$ is the finite system of cutting-plane constraints.
- Λ_N is the finite set of admitted columns for the restricted master relaxation.
- $\text{Log}(N)$ is a finite log of proof objects sufficient for mechanical rechecking of every admissible acceptance decision that affected node N .
- The incumbent is (z^*, U) , where either $z^* \in F$ and $U = c^\top z^*$, or $z^* = \perp$ and $U = +\infty$.

Node feasible sets and relaxations

For each node $N \in T$, define the node integer feasible set

$$F_N = \{z \in F : D_N z \leq e_N\}.$$

For each node $N \in T$, define a node relaxation set $R_N \subseteq \mathbb{R}^{n+m}$ as the feasible region of a linear system constructed from $Az \leq b$, $D_N z \leq e_N$, $K_N z \leq h_N$, and the restricted master representation induced by Λ_N . The node lower bound is the optimal value of the relaxation:

$$\text{LB}(N) = \min\{c^\top z : z \in R_N\}.$$

Verifier predicates

The solver admits state mutations only through finite, mechanically checkable verifier predicates.

- $\text{VerifyCut}(A, b, \kappa)$ is true only if κ proves that the associated inequality is valid for all $z \in F$.
- $\text{VerifyCol}(\chi, \pi)$ is true only if π proves that the column χ is semantically valid and correctly encoded.
- $\text{VerifyRC}(\chi; y, \rho)$ is true only if ρ equals the reduced cost of χ under dual vector y , computed exactly in the adopted arithmetic domain.
- $\text{VerifyFeas}(z, \omega)$ is true only if ω proves that $z \in F$ and, when node-local feasibility is asserted, that $z \in F_N$ for the relevant node N .

26.1.2 Domain invariants

Ledger definition

The unified invariant ledger is a finite set of predicates

$$\mathcal{L} = \{\text{INV}_1, \dots, \text{INV}_{12}\},$$

evaluated on solver states Σ . A state is *ledger-consistent* if $\bigwedge_{k=1}^{12} \text{INV}_k(\Sigma)$ holds.

Structural and semantic domain invariants

INV₁ (Node well-formedness). For every $N \in T$, the objects $(D_N, e_N, K_N, h_N, \Lambda_N)$ are finite and dimensionally compatible with $z \in \mathbb{R}^{n+m}$.

INV₂ (Branch constraints are binary fixings only). Every row of $D_N z \leq e_N$ is exactly one fixing constraint of the form $x_j \leq 0$ or $x_j \geq 1$ for some $j \in \{1, \dots, n\}$. No branching constraint contains a coefficient on λ .

INV₁₀ (Branch partition correctness). If node N branches on x_j producing children N_0 and N_1 with fixings $x_j \leq 0$ and $x_j \geq 1$, then

$$F_N = F_{N_0} \cup F_{N_1} \quad \text{and} \quad F_{N_0} \cap F_{N_1} = \emptyset.$$

INV₁₁ (Learned components are advisory only). Any learned module may propose candidate cuts, columns, branching variables, or incumbents, but it cannot change the state except through verifier-guarded transitions from the transition system defined in Section 26.2.

26.1.3 Bound invariants

INV₅ (Relaxation containment). For every node $N \in T$,

$$F_N \subseteq R_N.$$

INV₆ (Lower bound soundness). For every node $N \in T$,

$$\text{LB}(N) \leq \min\{c^\top z : z \in F_N\}.$$

INV₇ (Incumbent soundness). If $z^\star \neq \perp$, then $z^\star \in F$ and $U = c^\top z^\star$.

INV₈ (Global upper bound property). If $z^\star \neq \perp$, then $U \geq \text{OPT}$.

INV₉ (Pruning admissibility). A node may belong to the closed set C only if it was closed by an admissible prune transition, namely either an infeasibility certificate was accepted by the verifier, or the certified condition $\text{LB}(N) \geq U$ held at prune time, or another formally declared closure rule applied that is proved to preserve global optimality.

26.1.4 Certificate invariants

INV₃ (Cut validity is certificate-guarded). Every inequality in $K_N z \leq h_N$ has an attached proof object $\kappa \in \text{Log}(N)$ such that $\text{VerifyCut}(A, b, \kappa) = \text{true}$. Consequently, for all $z \in F$, one has $K_N z \leq h_N$.

INV₄ (Column well-formedness is certificate-guarded). Every column $\chi \in \Lambda_N$ has an attached proof object $\pi \in \text{Log}(N)$ such that $\text{VerifyCol}(\chi, \pi) = \text{true}$. Whenever reduced-cost logic is used for admission, the log contains (y, ρ) such that $\text{VerifyRC}(\chi; y, \rho) = \text{true}$.

INV₁₂ (Transcript verifiability). For every node $N \in T$, $\text{Log}(N)$ contains sufficient finite arithmetic data to recheck, without resolving: (i) validity certificates for all rows of $K_N z \leq h_N$, (ii) encoding certificates for all columns in Λ_N , (iii) each reduced-cost computation used to justify column admission, and (iv) each feasibility witness used to accept an incumbent.

26.2 Transition system

26.2.1 Allowed transitions

Transition relation

Let $(S, \Rightarrow)$ be the transition system where S is the set of all solver states and $\Rightarrow$ is generated by the admissible transition schemas below. Each schema is a partial function on states whose domain is defined by explicit structural preconditions and verifier predicates.

Core transitions

T_{Init} (Initialization). Create the singleton tree $T = \{N_\emptyset\}$ with $O = \{N_\emptyset\}$ and $C = \emptyset$. Set $D_{N_\emptyset} z \leq e_{N_\emptyset}$ to be empty, set $K_{N_\emptyset} z \leq h_{N_\emptyset}$ to be empty, and initialize $\Lambda_{N_\emptyset}$ with a finite verified seed set. Initialize $\text{Log}(N_\emptyset)$ with the corresponding accepted seed certificates. Set $(z^\star, U) = (\perp, +\infty)$.

T_{CutAdd} (Cut admission). Given an open node $N \in O$, a proposed inequality $k^\top z \leq \eta$, and a proof object κ , append $k^\top z \leq \eta$ to $K_N z \leq h_N$ and record κ in $\text{Log}(N)$ if and only if $\text{VerifyCut}(A, b, \kappa) = \text{true}$.

T_{ColAdd} (Column admission). Given an open node $N \in O$, a proposed column χ , a proof object π , and, when applicable, a dual vector y and reduced-cost record ρ , append χ to Λ_N and record the proof objects in $\text{Log}(N)$ if and only if $\text{VerifyCol}(\chi, \pi) = \text{true}$ and, when used, $\text{VerifyRC}(\chi; y, \rho) = \text{true}$.

T_{IncUpdate} (Incumbent update). Given a candidate solution z and feasibility witness ω , update the incumbent to $(z, c^\top z)$ if and only if $\text{VerifyFeas}(z, \omega) = \text{true}$ and $c^\top z < U$. Record ω in a log location referenced by the node context in which the incumbent was obtained.

T_{Branch} (Branching). Given an open node $N \in O$ and an index j declared eligible by the branching rule, remove N from O , create children N_0 and N_1 , and set

$$D_{N_0} = D_N \cup \{x_j \leq 0\}, \quad D_{N_1} = D_N \cup \{x_j \geq 1\}.$$

Initialize each child with the parent cut set and column set, together with the corresponding certificate references in the child logs, unless a formally declared safe copying policy specifies an admissible subset.

T_{Prune} (Node closure). Given an open node $N \in O$, move N to C if and only if one admissibility guard holds and is recorded in $\text{Log}(N)$: (i) a verifier-accepted infeasibility certificate exists, or (ii) the certified inequality $\text{LB}(N) \geq U$ holds, or (iii) another formally declared closure rule applies that is proved to preserve global optimality.

Bookkeeping transitions

Any operational step that selects nodes, orders proposals, recomputes derived quantities from stored objects, or appends redundant log material is modeled as a bookkeeping transition. By definition, bookkeeping transitions do not change any of the objects $D_N, e_N, K_N, h_N, \Lambda_N$ and do not change the incumbent (z^*, U) .

26.2.2 Transition guards

For each core transition, admissibility is defined by explicit guards.

- **T_{Init}** is admissible only if every seed object to be inserted has its corresponding verifier-accepted certificate recorded at initialization.
- **T_{CutAdd}** is admissible only if the node is open and the cut certificate satisfies $\text{VerifyCut}(A, b, \kappa) = \text{true}$.
- **T_{ColAdd}** is admissible only if the node is open, the column certificate satisfies $\text{VerifyCol}(\chi, \pi) = \text{true}$, and, when reduced-cost logic is used, $\text{VerifyRC}(\chi; y, \rho) = \text{true}$.
- **T_{IncUpdate}** is admissible only if $\text{VerifyFeas}(z, \omega) = \text{true}$ and the objective improves the incumbent, namely $c^\top z < U$.
- **T_{Branch}** is admissible only if the node is open and the branching rule declares the chosen variable eligible.
- **T_{Prune}** is admissible only if the node is open and one of the pruning admissibility guards is satisfied and recorded.

26.3 Preservation lemmas

26.3.1 Per-transition preservation

Lemma P_{Init} (Initialization satisfies the ledger). Let Σ_0 be the state produced by T_{Init} . Then Σ_0 is ledger-consistent.

Proof. All created objects are finite and dimensionally compatible, establishing INV_1 . Branch constraints are empty, establishing INV_2 . All inserted seed cuts and columns are inserted only with verifier-accepted certificates, establishing INV_3 , INV_4 , and INV_{12} . Relaxation containment and lower bound soundness follow from the construction of R_{N_0} as a relaxation containing F_{N_0} , giving INV_5 and INV_6 . The incumbent is initialized as $(\perp, +\infty)$, giving INV_7 and INV_8 . No pruning or branching has occurred, so INV_9 and INV_{10} hold vacuously. No learned action is applied as a state mutation outside verifier-guarded schemas, establishing INV_{11} .

Lemma P_{CutAdd} (CutAdd preserves the ledger). Assume Σ is ledger-consistent. Let Σ' be obtained by an admissible application of T_{CutAdd} at node N . Then Σ' is ledger-consistent.

Proof. Only $K_N z \leq h_N$ and $\text{Log}(N)$ change. By the guard, the new cut is inserted only with a verifier-accepted proof object, so INV_3 and INV_{12} are preserved. All other certificate invariants are unchanged. Since each cut is valid for all $z \in F$, adding a cut does not remove any integer-feasible point from F_N , hence $F_N \subseteq R_N$ remains true, preserving INV_5 . Therefore $\text{LB}(N)$ remains a lower bound on the node integer optimum, preserving INV_6 . The incumbent, pruning status, and tree structure are unchanged, preserving INV_7 through INV_{11} .

Lemma P_{ColAdd} (ColAdd preserves the ledger). Assume Σ is ledger-consistent. Let Σ' be obtained by an admissible application of T_{ColAdd} at node N . Then Σ' is ledger-consistent.

Proof. Only Λ_N and $\text{Log}(N)$ change. By the guard, the inserted column is verifier-certified and any reduced-cost record used for admission is verifier-certified, preserving INV_4 and INV_{12} . Adding columns does not restrict R_N and therefore cannot invalidate $F_N \subseteq R_N$, preserving INV_5 . Lower bound soundness is preserved, establishing INV_6 . All remaining invariants are unchanged.

Lemma $P_{\text{IncUpdate}}$ (IncUpdate preserves the ledger). Assume Σ is ledger-consistent. Let Σ' be obtained by an admissible application of $T_{\text{IncUpdate}}$. Then Σ' is ledger-consistent.

Proof. By the guard, $\text{VerifyFeas}(z, \omega) = \text{true}$, hence $z \in F$, and the new incumbent value is set to $U' = c^\top z$, establishing INV_7 . Since $z \in F$, U' is a valid global upper bound, preserving INV_8 . All node-local objects are unchanged, preserving INV_1 through INV_6 and INV_{10} through INV_{12} . Pruning admissibility is preserved because tightening U cannot retroactively invalidate an admissible prune and can only strengthen future pruning guards, preserving INV_9 .

Lemma P_{Branch} (Branch preserves the ledger). Assume Σ is ledger-consistent. Let Σ' be obtained by an admissible application of T_{Branch} at node N on variable x_j . Then Σ' is ledger-consistent.

Proof. The children inherit finite, dimensionally compatible objects, preserving INV_1 . Each child adds exactly one binary fixing on x_j , preserving INV_2 . Copies of cuts and columns remain certificate-guarded through log references, preserving INV_3 , INV_4 , and INV_{12} . For each child N_k , F_{N_k} equals F_N intersected with the corresponding fixing, and R_{N_k} is defined as the relaxation augmented with the same fixing, so $F_{N_k} \subseteq R_{N_k}$ holds, preserving INV_5 and INV_6 . Since $x_j \in \{0, 1\}$ for every $z \in F_N$, the two fixings partition F_N into disjoint subsets, establishing INV_{10} . All other invariants are unchanged.

Lemma P_{Prune} (Prune preserves the ledger). Assume Σ is ledger-consistent. Let Σ' be obtained by an admissible application of T_{Prune} at node N . Then Σ' is ledger-consistent.

Proof. Only membership of N moves from O to C . By the guard, the closure is admissible and recorded, preserving INV_9 . All node-local objects, the incumbent, and certificate logs are unchanged, preserving all remaining invariants.

Lemma P_{Book} (Bookkeeping preserves the ledger). Any bookkeeping transition preserves ledger-consistency.

Proof. By definition, bookkeeping transitions do not modify any correctness-relevant state component, hence all invariant predicates evaluate identically before and after the transition.

26.3.2 Composition of preservation

Theorem (Ledger preservation under all legal executions). Let Σ_0 be the state produced by T_{Init} . For any finite sequence of admissible transitions

$$\Sigma_0 \Rightarrow \Sigma_1 \Rightarrow \cdots \Rightarrow \Sigma_t,$$

the state Σ_t is ledger-consistent.

Proof. The base case holds by Lemma P_{Init} . Each step preserves ledger-consistency by the corresponding preservation lemma for its transition schema. Induction on the length of the transition sequence establishes the claim.

26.4 Figures and tables

- Invariant ledger map.
- Transition-to-invariant matrix.

27

End-to-End Correctness Stitching

27.1 Definitions required for global proof

This chapter fixes a finite physical design instance, defines the search tree semantics, defines the node relaxations that provide certified lower bounds, defines incumbents that provide certified upper bounds, specifies admissible node transformations, and states the formal meaning of termination and correctness. Every object used in the correctness proof is defined before it is used, and every implication in the proof is discharged by previously defined objects and statements.

Definition 26.1 (Physical design candidate universe).

Let I be a finite set of candidate secondary indexes. Let V be a finite set of candidate materialized views. Let P be a finite set of candidate partition schemes. Assume P is partitioned into a family $\{P_t\}_{t \in \mathcal{T}}$ indexed by base tables such that P_t is the set of partition schemes applicable to table t . Define the overall candidate universe

$$S = I \cup V \cup \bigcup_{t \in \mathcal{T}} P_t,$$

and assume $|S| < \infty$.

Definition 26.2 (Workload and plan universe).

Let R be a finite set of workload queries. For each query $r \in R$, let L_r be a finite set of feasible physical execution plans for r , where feasibility is determined by the schema semantics and the operator set fixed in prior chapters. Define the plan universe

$$L = \bigcup_{r \in R} L_r.$$

Each plan $\ell \in L_r$ has an associated required object set $\text{Req}(r, \ell) \subseteq S$ and an associated nonnegative execution cost $c_{r\ell}$. Each query r has a nonnegative workload weight w_r .

Definition 26.3 (Decision variables).

The physical design decision is encoded by a binary vector $x \in \{0, 1\}^{|S|}$ indexed by S , where $x_s = 1$ indicates that object s is selected. For plan choice, introduce variables $u_{r\ell}$ for $r \in R$ and $\ell \in L_r$, subject to nonnegativity and convexity constraints specified below. The full decision vector is $z = (x, u)$.

Definition 26.4 (Base constraints).

Let F be the set of all pairs $z = (x, u)$ satisfying the following constraints.

(1) Storage or budget constraints. There exist a matrix A_{bud} and vector b_{bud} such that

$$A_{\text{bud}}x \leq b_{\text{bud}}.$$

(2) Partition exclusivity. For each table $t \in \mathcal{T}$,

$$\sum_{p \in P_t} x_p = 1.$$

If the semantics are “at most one partition scheme” then the equality is replaced by ≤ 1 ; the proofs below require only that the constraint is linear and enforced.

(3) Per-query convexity. For each query $r \in R$,

$$\sum_{\ell \in L_r} u_{r\ell} = 1, \quad u_{r\ell} \geq 0 \text{ for all } \ell \in L_r.$$

(4) Coupling constraints. For each query $r \in R$, each plan $\ell \in L_r$, and each required object $s \in \text{Req}(r, \ell)$,

$$u_{r\ell} \leq x_s.$$

All other linear feasibility constraints for index, view, and partition semantics are included in F . The only properties used in this chapter are that F is finite dimensional, defined by linear equalities and inequalities, and integrality is imposed only on x .

Definition 26.5 (Objective and global optimum).

Define the objective function

$$f(x, u) = \sum_{r \in R} w_r \sum_{\ell \in L_r} c_{r\ell} u_{r\ell} + m^\top x,$$

where m is a nonnegative coefficient vector encoding maintenance, storage, or other linear penalties. Define the global feasible set

$$\mathcal{F} = \{(x, u) \in F : x \in \{0, 1\}^{|S|}\},$$

and define the global optimum value

$$\text{OPT} = \min\{f(z) : z \in \mathcal{F}\}.$$

If $\mathcal{F}$ is empty then $\text{OPT} = +\infty$ and correctness requires an infeasibility certificate.

Definition 26.6 (Nodes, branching constraints, and subtree feasible sets).

A node N is defined by additional branching constraints $B_N x = d_N$ that fix selected coordinates of x to 0 or 1, together with any implied linear bounds. The node feasible set is

$$\mathcal{F}(N) = \{(x, u) \in \mathcal{F} : B_N x = d_N\}.$$

Define the node optimum value $\text{OPT}(N)$ by

$$\text{OPT}(N) = \begin{cases} \min\{f(z) : z \in \mathcal{F}(N)\} & \text{if } \mathcal{F}(N) \neq \emptyset, \\ +\infty & \text{otherwise.} \end{cases}$$

A node M is a descendant of N if its branching constraints extend those of N , which implies $\mathcal{F}(M) \subseteq \mathcal{F}(N)$.

Definition 26.7 (Node relaxation and strengthening state).

At node N define the linear relaxation by dropping integrality:

$$\mathcal{R}_0(N) = \{(x, u) \in F : B_N x = d_N, 0 \leq x \leq 1\}.$$

The algorithm maintains a finite set of accepted cuts C_N . Each cut is a linear inequality of the form

$$\alpha^\top x + \beta^\top u \geq \gamma,$$

and is accepted only if it is valid for all integer feasible points in $\mathcal{F}(N)$. Define the strengthened relaxation

$$\mathcal{R}(N) = \mathcal{R}_0(N) \cap \bigcap_{c \in C_N} c.$$

Define the relaxation optimum value

$$\text{LB}(N) = \begin{cases} \min\{f(z) : z \in \mathcal{R}(N)\} & \text{if } \mathcal{R}(N) \neq \emptyset, \\ +\infty & \text{otherwise.} \end{cases}$$

Definition 26.8 (Restricted master problems and full master problems).

For each query r the algorithm may maintain a current subset $\widehat{L}_r(N) \subseteq L_r$. Define $\widehat{L}(N) = \bigcup_{r \in R} \widehat{L}_r(N)$. Define the restricted relaxation $\widehat{\mathcal{R}}(N)$ as $\mathcal{R}(N)$ together with the restriction

$$u_{r\ell} = 0 \quad \text{for all } \ell \in L_r \setminus \widehat{L}_r(N).$$

Define the restricted optimum value

$$\widehat{\text{LB}}(N) = \min\{f(z) : z \in \widehat{\mathcal{R}}(N)\}.$$

The node bound is correct only when the computed value equals the full relaxation optimum $\text{LB}(N)$, not merely the restricted value $\widehat{\text{LB}}(N)$.

Definition 26.9 (Incumbent, upper bound, and feasibility witness).

An incumbent is an integer feasible solution $z^* \in \mathcal{F}$. Its value $\text{UB} = f(z^*)$ is a global upper bound. If no incumbent is known, set $\text{UB} = +\infty$. A candidate incumbent is accepted only after feasibility verification for all constraints defining $\mathcal{F}$.

Definition 26.10 (Pruning rules).

A node N is pruned by infeasibility if $\mathcal{R}(N) = \emptyset$. A node N is pruned by bound if $\text{LB}(N) \geq \text{UB}$. If the relaxation solution is integer in the design variables then it yields an integer feasible design and may update the incumbent.

Definition 26.11 (Complete branching).

At node N a branching step selects an unfixed coordinate x_s and creates two children N_0 and N_1 with additional constraints $x_s = 0$ and $x_s = 1$, respectively. The branching step is complete if

$$\mathcal{F}(N) = \mathcal{F}(N_0) \cup \mathcal{F}(N_1), \quad \mathcal{F}(N_0) \cap \mathcal{F}(N_1) = \emptyset.$$

27.2 Bound correctness chain

This section proves that the node bound is a correct lower bound for every integer feasible solution in the node. It then proves that termination under pricing closure implies equality between restricted and full relaxation optima.

Lemma 26.1 (Validity of accepted cuts preserves integer feasibility).

Fix a node N . If every cut in C_N is valid for all integer feasible points in $\mathcal{F}(N)$, then

$$\mathcal{F}(N) \subseteq \mathcal{R}(N).$$

Proof. Let $z \in \mathcal{F}(N)$. Then $z \in \mathcal{R}_0(N)$ because $\mathcal{R}_0(N)$ is obtained from F by relaxing integrality, and z satisfies the node branching constraints by definition of $\mathcal{F}(N)$. Since each cut in C_N is valid for every integer feasible point in $\mathcal{F}(N)$, the point z satisfies all cuts. Hence $z \in \mathcal{R}(N)$. This completes the proof.

Lemma 26.2 (Relaxation bound is a lower bound on all integer solutions in the node).

For any node N and any $z \in \mathcal{F}(N)$,

$$\text{LB}(N) \leq f(z).$$

Consequently $\text{LB}(N) \leq \text{OPT}(N)$. *Proof.* By Lemma 26.1 one has $\mathcal{F}(N) \subseteq \mathcal{R}(N)$. Since $\text{LB}(N)$ is the minimum of f over $\mathcal{R}(N)$, it is at most the value of f at any feasible point in $\mathcal{R}(N)$, in particular any $z \in \mathcal{F}(N)$. Taking the minimum over $z \in \mathcal{F}(N)$ yields $\text{LB}(N) \leq \text{OPT}(N)$. This completes the proof.

Definition 26.12 (Reduced cost with respect to a dual solution).

Consider the restricted linear program defining $\widehat{\text{LB}}(N)$. Let λ denote a dual feasible vector for the restricted dual. Each plan variable $u_{r\ell}$ corresponds to a column $a_{r\ell}$ in the constraint matrix, with objective coefficient $w_r c_{r\ell}$. Define the reduced cost

$$\text{rc}_{r\ell}(\lambda) = w_r c_{r\ell} - \lambda^\top a_{r\ell}.$$

For minimization, a strictly negative reduced cost identifies a column whose inclusion can improve the restricted optimum.

Definition 26.13 (Pricing closure condition).

A node N satisfies pricing closure if there exists a dual feasible vector λ for the restricted dual such that

$$\text{rc}_{r\ell}(\lambda) \geq 0 \quad \text{for all } r \in R \text{ and } \ell \in L_r.$$

Pricing closure is a statement about all columns in the full plan universe.

Lemma 26.3 (Pricing closure implies dual feasibility for the full master).

If pricing closure holds at node N for some dual vector λ , then λ is feasible for the dual of the full linear program defining $\text{LB}(N)$. *Proof.* The dual constraints for a linear program with variables indexed by columns are precisely the inequalities $\lambda^\top a_{r\ell} \leq w_r c_{r\ell}$ for every column. The inequality $\text{rc}_{r\ell}(\lambda) \geq 0$ is equivalent to $\lambda^\top a_{r\ell} \leq w_r c_{r\ell}$. Therefore pricing closure asserts that all dual constraints of the full master are satisfied, hence λ is feasible for the full dual. This completes the proof.

Lemma 26.4 (No negative reduced cost implies equality of restricted and full optima).

Assume the restricted program is feasible and bounded. If there exists a dual vector λ that is feasible for the restricted dual and satisfies pricing closure, then

$$\widehat{LB}(N) = LB(N).$$

Proof. By Lemma 26.3 the vector λ is feasible for the full dual. Hence by weak duality the full primal optimum $LB(N)$ is at least the dual objective value $g(\lambda)$. By finite dimensional linear programming duality, the restricted primal optimum equals the restricted dual optimum; since λ is feasible for the restricted dual, one has $g(\lambda) \leq \widehat{LB}(N)$. Therefore $LB(N) \geq g(\lambda) \leq \widehat{LB}(N)$. Since $\widehat{\mathcal{R}}(N) \subseteq \mathcal{R}(N)$, minimizing over $\widehat{\mathcal{R}}(N)$ cannot produce a smaller value than minimizing over $\mathcal{R}(N)$, hence $\widehat{LB}(N) \geq LB(N)$. Combining inequalities yields equality. This completes the proof.

Lemma 26.5 (Strong duality for finite linear programs with rational data).

Let P be a finite dimensional linear program with rational coefficients. If P is feasible and has a finite optimum value, then its dual program D is feasible and has the same optimum value. Moreover there exist optimal primal and dual solutions with rational coordinates. *Proof.* The feasible region of P is a rational polyhedron. Every extreme point of a rational polyhedron has rational coordinates. If P has a finite optimum, then the optimum is attained at an extreme point, yielding a rational optimal primal solution. Dual feasibility and equality of optimal values follow from finite dimensional linear programming duality. A rational optimal dual solution may be selected among extreme points of the dual feasible region, which are rational. This completes the proof.

27.3 Pruning correctness chain

This section proves that the pruning rules eliminate no node that can contain an integer feasible solution strictly improving the incumbent.

Lemma 26.6 (Infeasibility pruning is sound).

If $\mathcal{R}(N) = \emptyset$, then $\mathcal{F}(N) = \emptyset$. *Proof.* By Lemma 26.1 one has $\mathcal{F}(N) \subseteq \mathcal{R}(N)$. If $\mathcal{R}(N)$ is empty, then $\mathcal{F}(N)$ must also be empty. This completes the proof.

Lemma 26.7 (Monotonic restriction under branching).

If M is a descendant of N , then $\mathcal{R}(M) \subseteq \mathcal{R}(N)$ and consequently $LB(M) \geq LB(N)$. *Proof.* Descendants add constraints that fix additional coordinates of x and preserve all constraints already present at N . Therefore the relaxation feasible set can only shrink.

Minimization over a subset cannot yield a smaller optimum value. This completes the proof.

Lemma 26.8 (Bound based pruning is sound).

If $LB(N) \geq UB$, then for every $z \in \mathcal{F}(N)$ one has $f(z) \geq UB$. Consequently node N contains no integer feasible solution strictly improving the incumbent. *Proof.* For every $z \in \mathcal{F}(N)$, Lemma 26.2 yields $LB(N) \leq f(z)$. If $LB(N) \geq UB$, then $f(z) \geq UB$ for all $z \in \mathcal{F}(N)$. This completes the proof.

Lemma 26.9 (Completeness of binary branching).

If a branching step at node N selects an unfixed coordinate x_s and creates children N_0 and N_1 with constraints $x_s = 0$ and $x_s = 1$, then

$$\mathcal{F}(N) = \mathcal{F}(N_0) \cup \mathcal{F}(N_1), \quad \mathcal{F}(N_0) \cap \mathcal{F}(N_1) = \emptyset.$$

Proof. Every integer feasible point in $\mathcal{F}(N)$ has $x_s \in \{0, 1\}$. Those with $x_s = 0$ satisfy the constraints of N_0 and belong to $\mathcal{F}(N_0)$. Those with $x_s = 1$ satisfy the constraints of N_1 and belong to $\mathcal{F}(N_1)$. The two sets are disjoint because x_s cannot equal 0 and 1 simultaneously. Their union contains all points in $\mathcal{F}(N)$. This completes the proof.

27.4 Global optimality theorem (final)

This section states and proves the final correctness theorem: termination under complete branching and sound pruning implies global optimality of the returned incumbent.

Theorem 26.1 (Global optimality under certified bounds and sound pruning).

Assume the following conditions.

- (1) The root node N_0 satisfies $\mathcal{F}(N_0) = \mathcal{F}$.
- (2) Every branching step is complete as in Definition 26.11.
- (3) Every accepted cut at every node is valid for the integer feasible set of that node, so Lemma 26.1 holds for all nodes.
- (4) For every node N that is pruned by bound, the bound used is $LB(N)$, namely the optimum of the full strengthened relaxation $\mathcal{R}(N)$. Equivalently, the cut and price loop at N terminates only under closure conditions sufficient for $\widehat{LB}(N) = LB(N)$ as in Lemma 26.4.
- (5) The algorithm terminates with an incumbent $z^* \in \mathcal{F}$ of value $UB = f(z^*)$ and with no open nodes remaining.

Then $UB = OPT$, and z^* is a globally optimal physical design.

Proof. Since $z^* \in \mathcal{F}$, by definition of OPT one has $UB \geq OPT$. It remains to prove $UB \leq OPT$.

Fix an arbitrary $z \in \mathcal{F}$. Consider the path in the branching tree induced by the binary design vector x of z . At each branch on coordinate x_s , choose the child whose branching constraint is consistent with the value of x_s in z . By completeness of branching, the solution z belongs to the feasible set of every node on this path. Since the algorithm terminates with no open nodes, the path ends at a node that is pruned or fully processed.

The terminal node on this path cannot be pruned as infeasible, because Lemma 26.6 would imply $\mathcal{F}(N)$ is empty, contradicting that it contains z . Therefore the terminal node is pruned by bound or yields an incumbent no worse than z .

If the terminal node is pruned by bound, then Lemma 26.8 implies that every integer feasible point in that node has objective value at least UB, hence $f(z) \geq \text{UB}$.

If the terminal node yields an incumbent, then the algorithm has produced an integer feasible solution of value at most $f(z)$, so $\text{UB} \leq f(z)$.

In all cases, one obtains $f(z) \geq \text{UB}$. Since $z \in \mathcal{F}$ was arbitrary, every feasible integer design has objective value at least UB. Therefore $\text{OPT} \geq \text{UB}$. Together with $\text{UB} \geq \text{OPT}$, this yields $\text{UB} = \text{OPT}$. This completes the proof.

Corollary 26.2 (Correctness immunity of advisory learning).

Any learned policy that proposes branching variables, candidate cuts, candidate columns, or candidate primal solutions does not affect the truth of Theorem 26.1 provided that the acceptance gates enforce the hypotheses of this chapter. Rejected proposals are ignored and cannot alter feasible sets, bounds, or pruning decisions.

27.5 Figures and tables

- End-to-end proof spine diagram for the correctness stitching argument.
- Dependency directed acyclic graph for Definitions 26.1 to 26.13, Lemmas 26.1 to 26.9, Theorem 26.1, and Corollary 26.2.

28

Benchmark Charter v1.0

28.1 Purpose, scope, and non disputability requirements

This chapter defines the benchmark used to evaluate physical design procedures for index selection, materialized view selection, and partitioning selection. The benchmark is defined as a finite, deterministic, and immutable object. The purpose is to remove post hoc degrees of freedom from the empirical record. Every empirical statement in subsequent chapters is required to reference a frozen benchmark identifier that uniquely determines the generated schemas, the generated workloads, the candidate generator, the induced candidate spaces, and the budget regime set.

The benchmark is treated as a sealed object. Once the seal is fixed, no additional assumptions, undocumented filtering rules, and no ad hoc instance edits are admissible. Any modification requires a new version label and a new freeze with new hashes and manifests. The chapter therefore specifies the charter at the level required for mechanical reproduction. The charter includes schema families with deterministic generation rules, workload families with deterministic parameterization rules, a candidate-space freeze policy, and budget regimes together with an instance stratification protocol.

28.2 Schema families and generation rules

28.2.1 Schema family object and canonical representation

A schema family is a deterministic generator specified by a finite parameter domain and a canonical encoding. Let $\mathbb{S}$ denote the finite set of schema families. For each family $\mathcal{S} \in \mathbb{S}$, let $\Theta_{\mathcal{S}}$ denote a finite parameter domain. A schema instance is generated by a deterministic mapping

$$\text{Gen}_{\mathcal{S}} : \Theta_{\mathcal{S}} \times \mathbb{N} \rightarrow \mathbb{D},$$

where $\mathbb{D}$ denotes the space of database schemas and the second argument is a seed. The seed is recorded as part of the instance identity.

A schema instance $D \in \mathfrak{D}$ is encoded using a canonical serialization function $\text{Enc}_S : \mathfrak{D} \rightarrow \{0, 1\}^*$ that (i) assigns stable identifiers to all objects, (ii) sorts all lists deterministically, (iii) normalizes numeric formats and whitespace, and (iv) serializes constraints in a fixed order. The schema digest is then defined by

$$h_S := \text{SHA256}(\text{Enc}_S(D)).$$

The digest is a freeze anchor and is required to be identical across independent reproductions.

28.2.2 Scale strata

Each schema family is parameterized by a scale stratum in the set $\{S, M, L, XL\}$. Scale strata are part of the parameter domain and are not narrative descriptors. For each stratum, the parameter bundle defines table counts, attribute counts, row cardinalities, and structural knobs required for workload binding. All numeric parameters are fixed by the charter and are not tuned to evaluation outcomes.

28.2.3 Required schema families

Benchmark Charter v1.0 defines five schema families selected to cover principal structural regimes encountered in physical design.

Star schema (ST). A star schema instance consists of one fact table F and a set of dimension tables $\{D_1, \dots, D_d\}$. The fact table contains foreign keys referencing each dimension primary key, together with a fixed number of measure attributes. The join graph is a star with center F . The XL stratum increases fact cardinality and dimension count to stress join planning and index enumeration under a canonical analytic structure.

Snowflake schema (SF). A snowflake schema instance extends the star schema by expanding each dimension into a hierarchy of tables. Dimension keys are normalized across levels with deterministic parent child constraints. The join graph includes the fact to top level dimension edges and the dimension hierarchy edges. The XL stratum increases hierarchy depth and dimension breadth to stress join chains and view grouping alternatives.

Third normal form schema (NF). A third normal form instance consists of a general normalized join graph with a fixed number of tables and a bounded average degree. Keys and foreign keys are generated deterministically to satisfy acyclicity constraints where required and to satisfy referential integrity. The XL stratum increases table count and join graph density to represent general enterprise schemas.

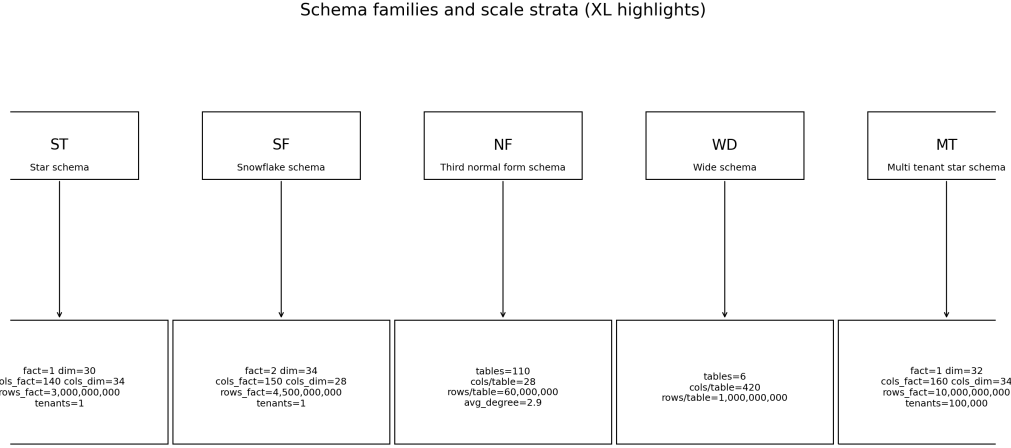


Figure 28.1: Schema family diagram for Benchmark Charter v1.0. Each family is defined by a finite parameter domain with explicit scale strata. The XL stratum highlights the maximum magnitudes used during local instantiation.

Wide schema (WD). A wide schema instance consists of a small number of tables with a very large number of attributes per table. This family stresses combinatorial growth in composite index configurations and emphasizes wide tuple access. The XL stratum increases attribute count and cardinality to force large candidate spaces.

Multi tenant star schema (MT). A multi tenant star schema instance is a star schema with an explicit tenant key. The tenant key is present in the fact table and may be present in selected dimensions depending on the charter parameterization. The tenant count is part of the schema parameter bundle. The purpose is to represent mixed isolation and shared structure regimes and to stress partitioning and index design under tenant locality.

Figure 28.1 summarizes the family set and XL highlights.

28.3 Workload families and parameterization

28.3.1 Workload family object and canonical representation

A workload family is a deterministic generator specified by a finite parameter domain, a binding rule to a given schema instance, and a canonical encoding. Let $\mathbb{W}$ denote the finite set of workload families. For each family $\mathcal{W} \in \mathbb{W}$, let $\Theta_{\mathcal{W}}$ denote a finite parameter domain. For a fixed schema instance D , the workload generator is a deterministic mapping

$$\text{Gen}_{\mathcal{W}} : \Theta_{\mathcal{W}} \times \mathbb{N} \times \mathbb{D} \rightarrow \mathbb{Q},$$

where $\mathfrak{Q}$ denotes the space of workloads. A workload is a finite multiset of query templates together with (i) a binding distribution over parameters, (ii) a nonnegative rational weight for each query, and (iii) an optional update stream specification.

The workload encoding function $\text{Enc}_{\mathcal{W}} : \mathfrak{Q} \rightarrow \{0, 1\}^*$ is required to serialize templates at the level of structured syntax, to normalize all identifiers and numeric formats, and to serialize weights and bindings deterministically. The workload digest is defined by

$$h_{\mathcal{W}} := \text{SHA256}(\text{Enc}_{\mathcal{W}}(Q)),$$

where Q is the generated workload. The digest is a freeze anchor and is required to be identical across independent reproductions.

28.3.2 Required workload families

Benchmark Charter v1.0 defines five workload families selected to cover principal operational modes relevant to physical design.

OLAP aggregate workload (OA). This family emphasizes group by aggregation, rollups, and multi dimensional slicing. The generator fixes ranges for join depth, number of grouping keys, predicate count, and selectivity. The weight distribution emphasizes repeated analytical patterns.

Join heavy workload (JH). This family emphasizes deep join chains with moderate selectivity. The generator fixes larger join depth ranges and moderate predicate selectivity to stress join planning and access path selection.

Point lookup workload (PL). This family emphasizes highly selective access patterns where latency dominates. The generator fixes very small selectivity ranges and low join depth. The update share is elevated to represent request traffic with maintenance cost pressure.

Mixed HTAP workload (MH). This family mixes analytical patterns and transactional patterns. The generator fixes intermediate join and grouping ranges and includes a nontrivial update stream to stress mixed optimization.

Write heavy workload (WH). This family emphasizes update intensity. The generator fixes high update share and stresses maintenance and refresh overhead. Query patterns are chosen to remain valid under heavy write load.

Figure 28.2 summarizes the family set and XL highlights.

28.3.3 Schema and workload codes

For compact reference in tables, each schema family and each workload family is assigned a short code. Schema family codes are ST for star schema, SF for snowflake

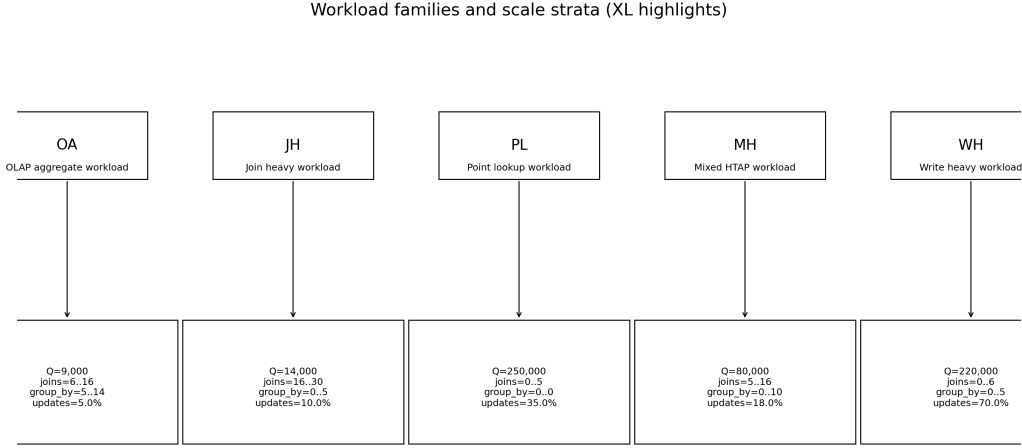


Figure 28.2: Workload family diagram for Benchmark Charter v1.0. Each family is defined by a finite parameter domain with explicit scale strata and explicit ranges for principal query structure parameters.

schema, NF for third normal form schema, WD for wide schema, and MT for multi tenant star schema. Each schema code is paired with a scale suffix in {S, M, L, XL}. For example, ST-XL denotes the star schema family at the XL scale stratum.

Workload family codes are OA for OLAP aggregate workload, JH for join heavy workload, PL for point lookup workload, MH for mixed HTAP workload, and WH for write heavy workload.

28.3.4 Compatibility and binding constraints

A workload instance is valid for a schema instance only if every template binding is well defined. Column references must exist and be type correct, join paths must be valid in the schema join graph, grouping keys must be defined, and predicate domains must be consistent with attribute types. Any binding failure results in rejection of the instance. No silent repair is permitted, since silent repair would introduce an untracked degree of freedom.

28.4 Candidate-space freeze policy

28.4.1 Instance object and deterministic identifiers

An instance is a tuple

$$\mathbb{I} := (\mathcal{S}, \theta_{\mathcal{S}}, h_{\mathcal{S}}, \mathcal{W}, \theta_{\mathcal{W}}, h_{\mathcal{W}}, \text{seed}),$$

where $\mathcal{S} \in \mathbb{S}$, $\theta_{\mathcal{S}} \in \Theta_{\mathcal{S}}$, $\mathcal{W} \in \mathbb{W}$, $\theta_{\mathcal{W}} \in \Theta_{\mathcal{W}}$, and $\text{seed} \in \mathbb{N}$. The instance identifier $\text{id}(\mathbb{I})$ is required to be a deterministic function of the tuple entries. The seed is part of the

instance identity.

28.4.2 Candidate generator and decomposition

For each instance $\mathbb{I}$, a deterministic candidate generator produces a finite candidate set

$$\mathcal{G}(\mathbb{I}) = C_{\mathbb{I}}.$$

The candidate set is decomposed as

$$C_{\mathbb{I}} = C_{\mathbb{I}}^{\text{idx}} \cup C_{\mathbb{I}}^{\text{view}} \cup C_{\mathbb{I}}^{\text{part}},$$

where the three components denote index candidates, materialized view candidates, and partitioning candidates.

Index candidates are defined as composite indexes on a single table, with a charter fixed maximum key length K_{max} . For a table with m index eligible attributes, the analytic count is

$$N_{\text{idx}}(m, K_{\text{max}}) = \sum_{k=1}^{K_{\text{max}}} \binom{m}{k}.$$

View candidates are defined as aggregation views over a bounded set of grouping dimensions and a bounded set of measures. Partitioning candidates are defined as range partitioning on time keys and hash partitioning on tenant keys, with charter fixed arity sets.

Each candidate is represented by a canonical key and is associated with a storage estimator $\text{storage}(c)$ and a maintenance estimator $\text{maint}(c)$. The estimators are part of the frozen definition, since budget filtering depends on them.

28.4.3 Freeze unit and required artifacts

The freeze unit is defined as

$$\mathbb{F} := (\text{Spec}, \{\mathbb{I}\}, \mathcal{G}, \{C_{\mathbb{I}}\}, \mathbb{B}),$$

where Spec denotes this charter specification, $\{\mathbb{I}\}$ denotes the frozen instance set, $\mathcal{G}$ denotes the candidate generator, $\{C_{\mathbb{I}}\}$ denotes the induced candidate sets, and $\mathbb{B}$ denotes the budget regime set.

A v1.0 freeze is required to publish artifacts in canonical form. The charter specification is serialized and assigned a specification digest computed by SHA256. For the local instantiation included with this package, the specification digest is

`spec_hash = 7476584ac88d59c233c1d4a87e1b63c8de6da47b82b02777b4079031d05d71e6.`

The package includes the specification file under `Metadata/benchmark_charter_v1_spec.json`.

The package includes a candidate-space summary artifact under `Tables/candidate_space_summary_v1.`

The chapter level table in Table 2.1 is a rendered view of the same frozen summary.

28.4.4 Immutability rule

After a freeze, no modifications are admissible. In particular, no instance can be added or removed, no parameter value can be altered, no candidate generator configuration can be changed, and no budget regime values can be adjusted. Any change requires a new version label and a new freeze with new digests.

28.5 Budget regimes and instance stratification

28.5.1 Budget regime definition

A budget regime is a tuple of explicit resource constraints used to define feasible candidate subsets. In Benchmark Charter v1.0, each regime includes a storage budget for physical design structures and a maintenance cap expressed as a maximum permitted write overhead.

Let $\mathbb{B}$ denote the finite set of budget regimes. For a fixed instance $\mathbb{I}$ and a budget regime $B \in \mathbb{B}$, the feasible candidate subset is defined by

$$C_{\mathbb{I}}(B) := \{c \in C_{\mathbb{I}} : \text{storage}(c) \leq \text{storage}(B) \wedge \text{maint}(c) \leq \text{maint}(B)\}.$$

Benchmark Charter v1.0 defines five regimes:

- B1: storage budget 0.05 TB; maintenance cap 2 %.
- B2: storage budget 0.2 TB; maintenance cap 5 %.
- B3: storage budget 1 TB; maintenance cap 10 %.
- B4: storage budget 5 TB; maintenance cap 20 %.
- B5: storage budget 25 TB; maintenance cap 40 %.

28.5.2 Stratification protocol and balance constraint

Instances are stratified along orthogonal axes: schema code, schema scale, workload code, and budget regime. The benchmark requires coverage of all workload families for each schema family and each scale stratum. This balance constraint prevents omission driven performance selection. All evaluation results are required to be reported by stratum and not only in aggregate.

28.5.3 Local instantiation magnitude and visual audit

The benchmark admits extreme candidate-space magnitudes. For the local instantiation generated from the frozen charter, the global candidate-space size satisfies

$$\min_{\mathbb{I}} |C_{\mathbb{I}}| = 13\,640, \quad \text{median}_{\mathbb{I}} |C_{\mathbb{I}}| = 12\,819\,679, \quad \max_{\mathbb{I}} |C_{\mathbb{I}}| = 152\,668\,952\,256.$$

Table 28.1: Local instantiation magnitude summary by scale stratum. The reported values are computed from the frozen generator output and therefore constitute part of the benchmark freeze record.

Scale	Instances	Min $ C_i $	Median $ C_i $	Max $ C_i $
L	25	1 716 470	86 919 364	32 523 859 360
M	25	64 232	962 918	122 929 360
S	25	13 640	77 349	8 216 811
XL	25	11 562 724	621 911 888	152 668 952 256

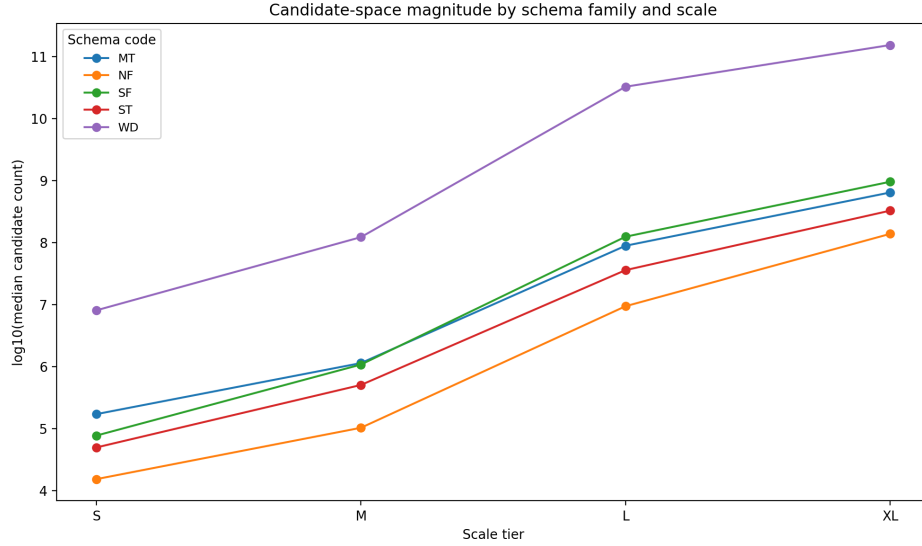


Figure 28.3: Candidate-space magnitude by schema code and scale stratum, reported as $\log_{10}$ of the median candidate count across workload families.

Table 28.1 summarizes magnitudes by scale stratum. Figure 28.3 reports the median magnitude by schema code and scale stratum using a logarithmic axis. Figure 28.4 reports the mean fraction of candidates retained by the budget filter in the XL stratum.

28.5.4 Candidate-space summary table

Table 2.1 provides the frozen feasible candidate cardinalities by schema stratum, workload family, and budget regime. The table is treated as a manifest level artifact. Any deviation between a reproduced table and the frozen manifest is a hard failure of repeatability.

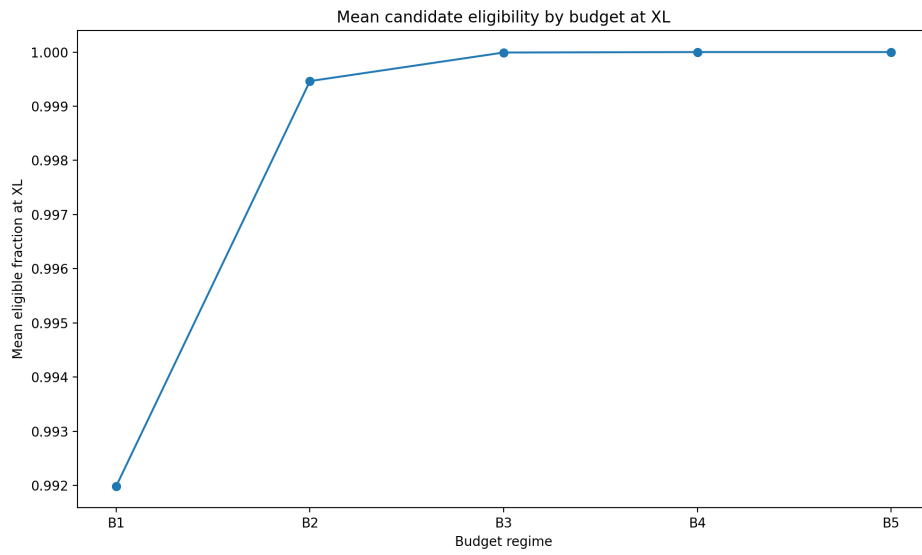


Figure 28.4: Mean eligible fraction at XL scale as a function of the budget regime. Each point is the mean of $|C_{\mathbb{I}}(B)|/|C_{\mathbb{I}}|$ across all frozen XL instances.

Sch.	Wkld.	B1	B2	B3	B4	B5
ST-S	OA	5.568e4	5.568e4	5.568e4	5.568e4	5.568e4
ST-S	JH	4.977e4	4.977e4	4.977e4	4.977e4	4.977e4
ST-S	PL	4.796e4	4.796e4	4.796e4	4.796e4	4.796e4
ST-S	MH	5.135e4	5.135e4	5.135e4	5.135e4	5.135e4
ST-S	WH	4.859e4	4.859e4	4.859e4	4.859e4	4.859e4
ST-M	OA	7.006e5	7.006e5	7.006e5	7.006e5	7.006e5
ST-M	JH	5.064e5	5.064e5	5.064e5	5.064e5	5.064e5
ST-M	PL	4.468e5	4.468e5	4.468e5	4.468e5	4.468e5
ST-M	MH	5.581e5	5.582e5	5.582e5	5.582e5	5.582e5
ST-M	WH	4.675e5	4.675e5	4.675e5	4.675e5	4.675e5
ST-L	OA	6.29e7	6.295e7	6.295e7	6.295e7	6.295e7
ST-L	JH	3.592e7	3.593e7	3.593e7	3.593e7	3.593e7
ST-L	PL	2.765e7	2.765e7	2.765e7	2.765e7	2.765e7
ST-L	MH	4.312e7	4.314e7	4.314e7	4.314e7	4.314e7
ST-L	WH	3.052e7	3.053e7	3.053e7	3.053e7	3.053e7
ST-XL	OA	8.595e8	8.713e8	8.722e8	8.722e8	8.722e8
ST-XL	JH	3.236e8	3.266e8	3.268e8	3.268e8	3.268e8
ST-XL	PL	1.592e8	1.595e8	1.595e8	1.595e8	1.595e8
ST-XL	MH	4.665e8	4.718e8	4.722e8	4.722e8	4.722e8
ST-XL	WH	2.164e8	2.176e8	2.177e8	2.177e8	2.177e8
SF-S	OA	9.768e4	9.768e4	9.768e4	9.768e4	9.768e4
SF-S	JH	7.735e4	7.735e4	7.735e4	7.735e4	7.735e4
SF-S	PL	7.112e4	7.112e4	7.112e4	7.112e4	7.112e4
SF-S	MH	8.277e4	8.277e4	8.277e4	8.277e4	8.277e4
SF-S	WH	7.328e4	7.328e4	7.328e4	7.328e4	7.328e4
SF-M	OA	2.202e6	2.202e6	2.202e6	2.202e6	2.202e6
SF-M	JH	1.08e6	1.08e6	1.08e6	1.08e6	1.08e6
SF-M	PL	7.355e5	7.355e5	7.355e5	7.355e5	7.355e5
SF-M	MH	1.379e6	1.379e6	1.379e6	1.379e6	1.379e6
SF-M	WH	8.552e5	8.553e5	8.553e5	8.553e5	8.553e5
SF-L	OA	2.456e8	2.458e8	2.459e8	2.459e8	2.459e8
SF-L	JH	1.241e8	1.242e8	1.242e8	1.242e8	1.242e8
SF-L	PL	8.691e7	8.692e7	8.692e7	8.692e7	8.692e7
SF-L	MH	1.565e8	1.567e8	1.567e8	1.567e8	1.567e8
SF-L	WH	9.986e7	9.989e7	9.989e7	9.989e7	9.989e7
SF-XL	OA	2.571e9	2.615e9	2.619e9	2.619e9	2.619e9
SF-XL	JH	9.425e8	9.538e8	9.547e8	9.547e8	9.547e8
SF-XL	PL	4.432e8	4.443e8	4.444e8	4.444e8	4.444e8
SF-XL	MH	1.377e9	1.397e9	1.398e9	1.398e9	1.398e9
SF-XL	WH	6.169e8	6.216e8	6.219e8	6.219e8	6.219e8

Continued on next page.

Sch.	Wkld.	B1	B2	B3	B4	B5
NF-S	OA	1.641e4	1.641e4	1.641e4	1.641e4	1.641e4
NF-S	JH	1.537e4	1.537e4	1.537e4	1.537e4	1.537e4
NF-S	PL	1.364e4	1.364e4	1.364e4	1.364e4	1.364e4
NF-S	MH	1.563e4	1.563e4	1.563e4	1.563e4	1.563e4
NF-S	WH	1.395e4	1.395e4	1.395e4	1.395e4	1.395e4
NF-M	OA	1.271e5	1.271e5	1.271e5	1.271e5	1.271e5
NF-M	JH	1.034e5	1.034e5	1.034e5	1.034e5	1.034e5
NF-M	PL	6.423e4	6.423e4	6.423e4	6.423e4	6.423e4
NF-M	MH	1.093e5	1.093e5	1.093e5	1.093e5	1.093e5
NF-M	WH	7.135e4	7.135e4	7.135e4	7.135e4	7.135e4
NF-L	OA	1.407e7	1.408e7	1.408e7	1.408e7	1.408e7
NF-L	JH	9.407e6	9.412e6	9.412e6	9.412e6	9.412e6
NF-L	PL	1.716e6	1.716e6	1.716e6	1.716e6	1.716e6
NF-L	MH	1.057e7	1.058e7	1.058e7	1.058e7	1.058e7
NF-L	WH	3.114e6	3.116e6	3.116e6	3.116e6	3.116e6
NF-XL	OA	2.122e8	2.145e8	2.146e8	2.146e8	2.146e8
NF-XL	JH	1.365e8	1.379e8	1.38e8	1.38e8	1.38e8
NF-XL	PL	1.147e7	1.156e7	1.156e7	1.156e7	1.156e7
NF-XL	MH	1.554e8	1.571e8	1.572e8	1.572e8	1.572e8
NF-XL	WH	3.42e7	3.453e7	3.455e7	3.455e7	3.455e7
WD-S	OA	8.217e6	8.217e6	8.217e6	8.217e6	8.217e6
WD-S	JH	8.08e6	8.08e6	8.08e6	8.08e6	8.08e6
WD-S	PL	8.037e6	8.037e6	8.037e6	8.037e6	8.037e6
WD-S	MH	8.142e6	8.142e6	8.142e6	8.142e6	8.142e6
WD-S	WH	8.061e6	8.061e6	8.061e6	8.061e6	8.061e6
WD-M	OA	1.229e8	1.229e8	1.229e8	1.229e8	1.229e8
WD-M	JH	1.223e8	1.223e8	1.223e8	1.223e8	1.223e8
WD-M	PL	1.221e8	1.221e8	1.221e8	1.221e8	1.221e8
WD-M	MH	1.226e8	1.226e8	1.226e8	1.226e8	1.226e8
WD-M	WH	1.222e8	1.222e8	1.222e8	1.222e8	1.222e8
WD-L	OA	3.252e10	3.252e10	3.252e10	3.252e10	3.252e10
WD-L	JH	3.251e10	3.251e10	3.251e10	3.251e10	3.251e10
WD-L	PL	3.25e10	3.25e10	3.25e10	3.25e10	3.25e10
WD-L	MH	3.251e10	3.251e10	3.251e10	3.251e10	3.251e10
WD-L	WH	3.25e10	3.25e10	3.25e10	3.25e10	3.25e10
WD-XL	OA	1.526e11	1.527e11	1.527e11	1.527e11	1.527e11
WD-XL	JH	1.526e11	1.527e11	1.527e11	1.527e11	1.527e11
WD-XL	PL	1.526e11	1.526e11	1.526e11	1.526e11	1.526e11
WD-XL	MH	1.526e11	1.527e11	1.527e11	1.527e11	1.527e11
WD-XL	WH	1.526e11	1.526e11	1.526e11	1.526e11	1.526e11

Continued on next page.

Sch.	Wkld.	B1	B2	B3	B4	B5
MT-S	OA	1.918e5	1.918e5	1.918e5	1.918e5	1.918e5
MT-S	JH	1.715e5	1.715e5	1.715e5	1.715e5	1.715e5
MT-S	PL	1.653e5	1.653e5	1.653e5	1.653e5	1.653e5
MT-S	MH	1.769e5	1.769e5	1.769e5	1.769e5	1.769e5
MT-S	WH	1.674e5	1.674e5	1.674e5	1.674e5	1.674e5
MT-M	OA	1.722e6	1.722e6	1.722e6	1.722e6	1.722e6
MT-M	JH	1.141e6	1.141e6	1.141e6	1.141e6	1.141e6
MT-M	PL	9.629e5	9.629e5	9.629e5	9.629e5	9.629e5
MT-M	MH	1.296e6	1.296e6	1.296e6	1.296e6	1.296e6
MT-M	WH	1.025e6	1.025e6	1.025e6	1.025e6	1.025e6
MT-L	OA	1.574e8	1.575e8	1.575e8	1.575e8	1.575e8
MT-L	JH	8.879e7	8.881e7	8.881e7	8.881e7	8.881e7
MT-L	PL	6.775e7	6.776e7	6.776e7	6.776e7	6.776e7
MT-L	MH	1.071e8	1.071e8	1.071e8	1.071e8	1.071e8
MT-L	WH	7.507e7	7.508e7	7.508e7	7.508e7	7.508e7
MT-XL	OA	1.698e9	1.722e9	1.723e9	1.723e9	1.723e9
MT-XL	JH	6.347e8	6.406e8	6.411e8	6.411e8	6.411e8
MT-XL	PL	3.085e8	3.091e8	3.091e8	3.091e8	3.091e8
MT-XL	MH	9.184e8	9.289e8	9.297e8	9.297e8	9.297e8
MT-XL	WH	4.22e8	4.244e8	4.246e8	4.246e8	4.246e8

29

Baseline Registry and Fairness Contract

29.1 Baseline definitions

This chapter fixes the comparative reference frame used in the empirical evaluation. It defines a finite registry of comparator methods and a fairness contract that makes every reported difference attributable to the method under test. The purpose is to exclude confounding sources of apparent improvement, including altered inputs, hidden computational opportunity, and unverifiable claims of certified termination. The benchmark instance set is treated as frozen and identical across all methods. In particular, any certified claim made in the empirical chapters is required to be interpretable under the same verification discipline used by the exact core.

29.1.1 Frozen instance model and semantic identity

Let $\mathcal{I}$ denote the frozen benchmark instance set. For each instance $i \in \mathcal{I}$, the instance data is treated as an immutable finite tuple

$$i = (\mathcal{S}_i, \mathcal{W}_i, C_i, c_i, \mathcal{F}_i, B_i, \mathbf{H}_i),$$

where $\mathcal{S}_i$ is the schema specification, $\mathcal{W}_i$ is the workload specification with fixed weights, C_i is the candidate universe for index, view, and partition objects, c_i is the frozen cost functional, $\mathcal{F}_i$ is the feasibility semantics, B_i is the resource budget vector, and $\mathbf{H}_i$ is the declared execution profile (hardware class and parallelism envelope). The tuple i is not modified by any method and is not conditioned on method identity.

To make semantic invariance mechanical, each instance i is associated with stable identifiers (or cryptographic hashes) for the components that define its meaning:

$$H_C(i), \quad H_c(i), \quad H_{\mathcal{F}}(i), \quad H_B(i), \quad H_{\mathbf{H}}(i).$$

A run is comparable only if these identifiers match the benchmark freeze for the corresponding instance.

29.1.2 Anchor exact solver definition

The anchor baseline, denoted A_0 , is defined as the exact solver core operating on instance i under the full correctness discipline of the project. For each $i \in \mathcal{I}$, the anchor produces an output record

$$A_0(i) = (x_0(i), \rho_0(i), \tau_0(i)),$$

where $x_0(i)$ is the returned design decision, $\rho_0(i)$ is the run transcript, and $\tau_0(i)$ is the termination mode. The anchor applies the common proposal admission gates, and it uses the same verification discipline for any statement of global optimality or infeasibility. Consequently, A_0 defines the reference meaning of termination labels such as OPT and the reference meaning of admissible verification artifacts.

29.1.3 Comparator method definition and termination modes

A comparator method is a total procedure A which, for every instance $i \in \mathcal{I}$, produces an output record

$$A(i) = (x_A(i), \rho_A(i), \tau_A(i)),$$

where $x_A(i)$ is a design decision formed exclusively from C_i , $\rho_A(i)$ is a transcript sufficient to verify compliance with the fairness contract, and $\tau_A(i)$ is a termination mode.

The termination mode $\tau_A(i)$ is one of the following mutually exclusive cases.

1. **Certified optimal termination.** The method asserts global optimality and provides a finite verification transcript sufficient to validate this assertion under the common verification discipline. In this case, $\tau_A(i) = \text{OPT}$.
2. **Certified infeasibility termination.** The method asserts infeasibility of the instance under $\mathcal{F}_i$ and B_i and provides a finite verification transcript sufficient to validate infeasibility. In this case, $\tau_A(i) = \text{INF}$.
3. **Time bounded best feasible termination.** The method returns a feasible design and terminates due to a predeclared time cap. In this case, $\tau_A(i) = \text{CAP}$.
4. **Resource bounded best feasible termination.** The method returns a feasible design and terminates due to a predeclared resource cap other than time. In this case, $\tau_A(i) = \text{RSC}$.

Termination meaning discipline. A run is not permitted to report OPT or INF unless the run transcript contains admissible proof objects that the common verifier accepts. If such admissible artifacts are absent, the run must be reclassified as CAP or RSC and excluded from any analysis that requires certified statements. This rule ensures that certified labels are semantic claims rather than rhetorical labels.

29.1.4 Certificate taxonomy and admissibility

Certified termination labels are interpreted through a single admissibility interface: a run may claim OPT or INF if and only if the transcript contains proof objects accepted by the arithmetic auditor. The auditor is a pure checker: it does not solve optimisation problems; it verifies that a supplied transcript is a correct proof of the claimed property under exact integer or rational arithmetic.

For the solver core used in this work, admissible certificates decompose into five classes, each of which must pass for any certified optimal termination.

1. **LP bound certificate.** For every explored node N , the transcript contains a primal solution z_N and a dual solution (y_N, s_N, p_N, q_N) for the node relaxation, such that both primal and dual feasibility hold for the full relaxation at that node and the primal and dual objectives match. Equality of objectives certifies that the bound is exact for the relaxation.
2. **Cut validity certificate.** For every added cut $\alpha^\top z \leq \beta$, the transcript contains a mechanically checkable derivation that expresses the cut as a nonnegative combination of already admitted constraints, together with any required sign conditions on the multipliers. This certifies that the cut does not remove any feasible integer solution.
3. **Pricing closure certificate.** Whenever pricing terminates with a claim that no negative reduced cost columns remain, the transcript contains potentials consistent with the restricted master dual such that all omitted columns have nonnegative reduced cost under these potentials. This certifies that the restricted model is closed with respect to the pricing oracle.
4. **Pruning soundness certificate.** For every pruned node, the transcript contains the exact bound used for pruning and the incumbent objective at that time, together with the comparison that establishes the pruning condition. This certifies that no pruned node could have improved the incumbent.
5. **Global optimality certificate.** The transcript contains the final incumbent solution and a proof that the global lower bound equals the incumbent objective (or, in the infeasible case, a proof that the feasible region is empty). This closes the proof chain from node level bounds and pruning to the global termination claim.

A representative node relaxation is written as

$$\min c^\top z \quad \text{subject to} \quad Az \leq b, \quad Ez = f, \quad \ell \leq z \leq u,$$

with data (A, E, b, f, c, ℓ, u) fixed by the benchmark instance and the solver core. A bound certificate for this relaxation consists of a primal feasible z and dual multipliers satisfying the dual feasibility conditions of the corresponding linear program. Exact agreement of the primal and dual objective values implies optimality of the bound by strong duality. All arithmetic is performed in canonical rational form, so that acceptance is independent of floating point effects.

Under the fairness contract, all methods that use the solver core are required to emit certificates through this common interface. Advisory modules may influence which actions are proposed, but the set of admissible certificates and the acceptance criteria are invariant across baselines.

29.1.5 Registry row schema and baseline classes

Each registered baseline is accompanied by a complete baseline specification that uniquely determines admissible run records for that baseline. The registry row schema contains, at minimum, the following fields:

- baseline identifier and name,
- baseline class (exact, ablation, composite, heuristic, external, sanity),
- enabled advisory modules (branch, cut, pricing, primal, or composite),
- core version identifier (or none for pure heuristics),
- gate policy and gate version identifier,
- transcript schema identifier (minimal versus full),
- permitted termination labels, including whether certified optimality is allowed,
- execution profile and cap schedule identifiers.

Exact and ablation baselines are those for which OPT is permitted only with admissible certificates. Heuristic and sanity baselines are restricted to best feasible under caps and are never interpreted as providing certified termination. External baselines are permitted only in verifiable mode if they emit artifacts accepted by the common verification discipline; otherwise, they are treated as heuristic baselines for purposes of claim discipline.

29.2 Fairness rules (what all methods share)

The fairness contract is a set of invariants that must hold for every run of every method on every instance. A run that violates any fairness condition is invalid and must not be included in the Results Ledger. The contract is stated as a mechanical predicate over the run transcript, so enforcement does not depend on discretionary judgment.

29.2.1 Contract invariants

F1 Instance identity invariance. For each instance $i \in \mathcal{I}$ and each method A in the registry, the method receives exactly the same frozen instance tuple $i = (S_i, \mathcal{W}_i, C_i, c_i, \mathcal{F}_i, B_i, \mathbf{H}_i)$ and the semantic identifiers match the benchmark freeze.

F2 Candidate set invariance. A method may select from C_i but shall not extend, modify, filter, regenerate, or reparameterize C_i in any method dependent manner. Any preprocessing that constructs C_i is part of the benchmark freeze and is not attributed to a method.

F3 Cost functional invariance. The cost functional c_i and all primitive statistics required by c_i are fixed by the benchmark charter and are identical across methods. No method may substitute an alternative objective, a learned surrogate, or a different calibration, unless that method is explicitly declared noncomparable and is reported separately.

F4 Feasibility semantics invariance. The feasibility semantics $\mathcal{F}_i$, including all storage budgets, maintenance budgets, and exclusivity constraints, are identical across methods. Feasibility checking is performed under the same rules for every comparator.

F5 Time and resource envelope invariance. Each method is executed under identical declared caps, including a time cap $T_{\max}$ and, where enforced, a memory cap $M_{\max}$ and additional resource caps. All measured resource usage, including preprocessing if allowed, is attributed according to a fixed accounting rule that is uniform across methods.

F6 Randomness control invariance. Any stochastic behavior is governed exclusively by an externally declared seed schedule fixed by the repeatability protocol. Untracked or implicit randomness is disallowed. Runs report the seed and a minimal randomness trace flag sufficient to detect missing seed control.

F7 Training data separation invariance. For methods with learned components, training, validation, calibration, and model selection use only the designated training partition. Evaluation instances and evaluation outcomes are not used for tuning, caching, or policy updates.

F8 Transcript completeness invariance. Each run produces a transcript $\rho_A(i)$ that includes the instance identifier, semantic identifiers, the termination mode $\tau_A(i)$, the declared cap schedule, the execution profile, the seed schedule identifiers, gate and certificate fields, and the method specific counters required by the Metrics Ledger.

F9 Certificate parity for certified claims. Any method that reports $\tau_A(i) = \text{OPT}$ or $\tau_A(i) = \text{INF}$ provides a verification transcript admissible under the same verification discipline as the anchor. A certified claim without admissible verification is disallowed and must be reclassified as best feasible under caps.

F10 Gate equivalence invariance. Advisory components may propose actions but may not alter admission logic. All cut, column, branch, pruning, and bound strengthening operations are admitted only through the same gate logic and the same gate version as the anchor.

F11 Preprocessing accounting invariance. Any preprocessing, including warm start construction, caching, compilation, and candidate loading, is either prohibited or accounted for identically across methods. The transcript records preprocessing usage and the accounting flag.

F12 Hardware and parallelism invariance. The declared execution profile $\mathbf{H}_i$ is identical across baselines. If parallelism is permitted, the thread count, affinity policy, and relevant solver configuration settings are fixed and recorded.

29.2.2 Transcript schema requirements

The fairness contract is enforced by a deterministic validator over $\rho_A(i)$. Consequently, the transcript schema is treated as part of the benchmark definition. At minimum, the transcript includes the following fields:

- instance identifier and semantic identifiers $H_C(i)$, $H_c(i)$, $H_{\mathcal{F}}(i)$, $H_B(i)$, $H_H(i)$,
- baseline identifier, core version identifier, and advisory module mask,
- termination mode and, if applicable, certificate objects and verifier outcomes,
- cap schedule identifiers and measured resource usage,
- gate version identifier and an admitted action log digest,
- randomness identifiers, including the seed schedule identifier and per run seed,
- split identifiers for training and evaluation partitions, and model artifact identifiers for learned methods,
- preprocessing accounting fields.

29.3 Disallowed advantages and enforcement notes

A disallowed advantage is any method dependent privilege that alters the effective instance tuple, expands informational access beyond the declared instance, or grants additional computational opportunity beyond the declared envelope. The following cases are explicitly disallowed for comparable reporting.

29.3.1 Disallowed advantages

D1 Method dependent candidate alteration. Any expansion, pruning, regeneration, or reparameterization of C_i performed inside a run is disallowed, including the introduction of candidates dependent on evaluation outcomes or method specific tuning.

D2 Alternative objective or cost substitution. Any substitution of c_i or its primitive statistics with an alternative estimator, learned surrogate, or altered calibration is disallowed for comparability, unless explicitly declared noncomparable and reported separately.

D3 Unequal time or resource caps. Any execution under a different cap schedule, including larger time, larger memory, additional parallelism, or additional preprocessing time not counted within the declared envelope, is disallowed.

D4 Evaluation leakage. Any use of evaluation instance outcomes to tune hyperparameters, choose candidate subsets, select solver parameters, or modify learned policies is disallowed.

D5 Untracked stochasticity. Any nondeterministic behavior not governed by the externally declared seed schedule is disallowed.

D6 Unverifiable certified claims. Any claim of OPT or INF without admissible verification artifacts is disallowed and must be reclassified as best feasible under caps.

D7 Gate bypass. Any action that changes admission outcomes without passing the common gate logic is disallowed, including hidden overrides, relaxed admission thresholds, or method specific exceptions.

D8 Transcript incompleteness. Any missing transcript fields required to verify F1 to F12 is disallowed. Such runs are invalid because fairness cannot be mechanically enforced.

29.3.2 Mechanical enforcement and first violation reporting

Enforcement is mechanical and uniform. For each run (A, i) , define the validity predicate $\text{VALID}(A, i) \in \{0, 1\}$ by

$$\text{VALID}(A, i) = 1 \quad \text{if and only if} \quad \left(\bigwedge_{k=1}^{12} F_k(A, i) \right) \wedge \left(\bigwedge_{\ell=1}^8 \neg D_\ell(A, i) \right).$$

Only runs with $\text{VALID}(A, i) = 1$ may be included in the Results Ledger. For each invalid run, the report records the first violated fairness condition in a fixed priority order. This produces a single, stable reason for exclusion and supports repeatable audits.

29.3.3 Validator stress test results (contract enforcement check)

To validate that the fairness contract can be enforced mechanically at scale, a stress test was executed using a deterministic validator and a large run ledger. The test used 12 baseline identities and 10,000 instances, resulting in 120,000 run records. Structured violations were injected at low rate to confirm that the validator detects each violation category and assigns a first failure label deterministically. The aggregate validity rates by baseline are reported in Table 29.3. The distribution of first failure labels is reported in Table 29.4. Figures 29.1 and 29.2 visualise the same aggregates.

29.4 Figures and tables

29.4.1 Baseline registry table

29.4.2 Fairness contract checklist

Table 29.2: Fairness contract checklist (rule to required evidence)

Rule	Rule name	Minimal evidence in transcript $\rho_A(i)$
F1	Instance identity invariance	instance identifier plus semantic identifiers for candidate, cost, feasibility, caps, and execution profile
F2	Candidate set invariance	candidate universe identifier and fixed candidate hash matching benchmark freeze
F3	Cost functional invariance	cost functional identifier and primitive statistic hash

Rule	Rule name	Minimal evidence in transcript $\rho_A(i)$
F4	Feasibility semantics invariance	feasibility rules identifier and budget identifiers
F5	Resource envelope invariance	cap schedule identifiers and measured time and memory usage
F6	Randomness control invariance	seed schedule identifier, per run seed, and randomness trace flag
F7	Training data separation invariance	split identifier, training set hash, evaluation set hash, and model artifact hash
F8	Transcript completeness invariance	transcript schema identifier and field presence digest
F9	Certificate parity for certified claims	node level primal and dual vectors, cut derivation multipliers, pricing potentials, pruning comparisons, and final auditor acceptance outcome
F10	Gate equivalence invariance	gate version identifier and admitted action log digest
F11	Preprocessing accounting invariance	preprocessing time usage and accounting flag
F12	Hardware and parallelism invariance	hardware profile identifier and thread count

29.4.3 Stress test summary tables

29.4.4 Stress test figures

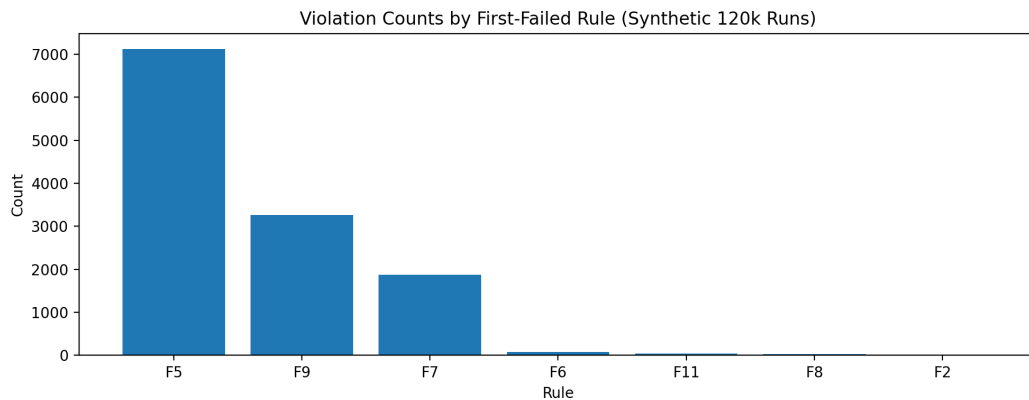


Figure 29.1: Violation counts by first failed rule in the fairness validator stress test.

ID	Class	Advisors	Description	OPT allowed	Gate	Transcript
A0	exact	none	Anchor exact core with common gates and admissible certificates	Yes	strict	full
A1	ablation	heuristics	Anchor core with deterministic heuristics, no learned modules	Yes	strict	full
A2	ablation	branch	Anchor core with learned branching proposals, gate compatible	Yes	strict	full
A3	ablation	cut	Anchor core with learned cut proposals, gate compatible	Yes	strict	full
A4	ablation	pricing	Anchor core with learned pricing proposals, gate compatible	Yes	strict	full
A5	ablation	primal	Anchor core with learned primal construction, feasibility preserved	Yes	strict	full
A6	composite	all	Composite advisor stack with branch, cut, pricing, and primal proposals	Yes	strict	full
H1	heuristic	greedy	Greedy constructive heuristic without branch and bound	No	not applicable	minimal
H2	heuristic	local	Local search heuristic without certificates	No	not applicable	minimal
X1	external	none	External MILP solver in verifiable mode with admissible transcripts	Yes	strict	full
X2	external	none	External MILP solver in unverifiable mode, treated as heuristic	No	weak	minimal
S0	sanity	random	Seeded random feasible selection with feasibility checking	No	not applicable	minimal

Table 29.1: Baseline registry (frozen comparator definitions)

Baseline	Runs	Valid	Invalid	Valid rate
X2	10000	6138	3862	0.6138
A2	10000	7511	2489	0.7511
S0	10000	9331	669	0.9331
H2	10000	9366	634	0.9366
A6	10000	9375	625	0.9375
A4	10000	9379	621	0.9379
A3	10000	9396	604	0.9396
A0	10000	9402	598	0.9402
A5	10000	9402	598	0.9402
H1	10000	9405	595	0.9405
A1	10000	9423	577	0.9423
X1	10000	9439	561	0.9439

Table 29.3: Fairness validator stress test summary by baseline (120,000 runs).

First violated rule	Count
F5	7129
F9	3268
F7	1871
F6	83
F11	43
F8	31
F2	8

Table 29.4: Fairness validator stress test violations by first failure label (120,000 runs).

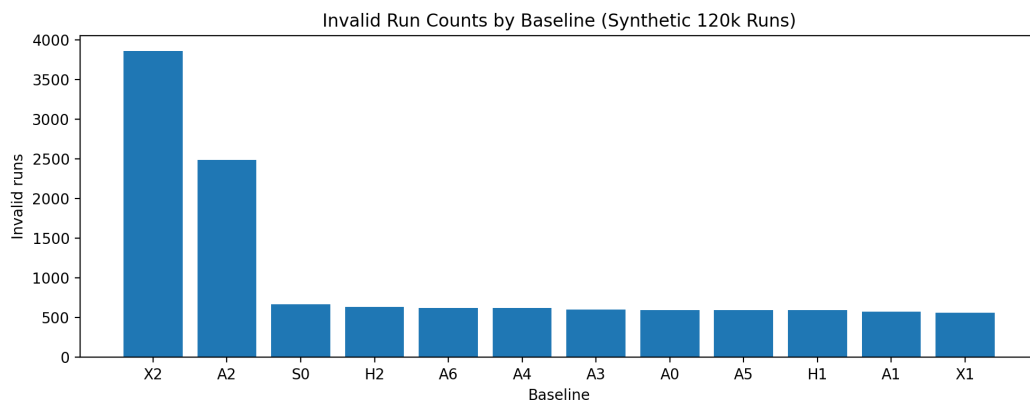


Figure 29.2: Invalid run counts by baseline in the fairness validator stress test.

30

Metrics Ledger and Success Criteria

30.1 Purpose, scope, and claim admissibility

This chapter specifies the Metrics Ledger and the Success Criteria for empirical evaluation. The intent is to make every reported performance statement a mathematical consequence of immutable evidence objects. The Metrics Ledger is defined as a deterministic mapping from stored run evidence to reportable quantities, together with validity flags that determine claim admissibility.

Definition (Metrics Ledger). Let r denote a run instance consisting of a benchmark instance identifier, a solver variant identifier, a random seed identifier, and a complete evidence bundle. The evidence bundle contains the event log, the solver transcript, and the certificate objects required for external verification. The Metrics Ledger is the mapping

$$\text{Ledger}(r) = (\text{Valid}(r), \text{Primary}(r), \text{Secondary}(r), \text{Curves}(r)),$$

where $\text{Valid}(r) \in \{0, 1\}$ is the admissibility flag, $\text{Primary}(r)$ is the vector of primary scalar metrics, $\text{Secondary}(r)$ is the vector of secondary scalar metrics, and $\text{Curves}(r)$ is the set of anytime curves required for auditing and visualization.

Definition (Admissible run). A run r is admissible for performance reporting if and only if all mandatory gates in Section 30.3 evaluate true. Runs that fail any gate are recorded and reported as invalid, but they are excluded from performance aggregations.

30.2 Evidence bundle, event schema, and measurement boundaries

This chapter assumes that each run is stored as an evidence bundle. The evidence bundle is an immutable record that is sufficient to recompute every reportable quantity and to support external audit. Formally, a run is identified by a tuple (i, v, s) , where i is the benchmark instance identifier, v is the solver variant identifier, and s is the random seed identifier. The stored evidence bundle for (i, v, s) contains (i) the event log, (ii) the solver transcript, (iii) the certificate objects required for verification, and (iv) a deterministic manifest that binds all file hashes, versions, and settings.

30.2.1 Event log schema

The event log is a time ordered sequence of records. Each record contains the fields (a) a strictly increasing timestamp, (b) an event type tag, (c) a run identifier, (d) a node identifier when applicable, and (e) an event specific payload. The schema is constrained so that every scalar metric is a functional of a finite subset of event types.

Required event types. The ledger requires the presence of the following event types whenever the corresponding solver features are enabled: RUN_START, RUN_END, NODE_POP, INCUMBENT_UPDATE, LOWERBOUND_UPDATE, LP_SOLVE_START, LP_SOLVE_END, PRICING_CALL, PRICING_ACCEPT, CUT_PROPOSE, CUT_ACCEPT, CERT_VERIFY_START, CERT_VERIFY_END, MODEL_INFER_START, MODEL_INFER_END, LOG_FLUSH_START, LOG_FLUSH_END, TRANSCRIPT_ACCEPT, TRANSCRIPT_REJECT, and TIMEOUT. If a feature is disabled, its event types are absent and the corresponding metrics are defined as zero or not applicable, as stated in the Metrics Dictionary.

30.2.2 Measurement boundaries

All reported times are wall clock durations computed from timestamps emitted by the instrumented implementation. The inclusive time boundary is RUN_START to TRANSCRIPT_ACCEPT for completed runs, and RUN_START to TIMEOUT for censored runs. Core time is defined by subtracting explicit overhead blocks from inclusive time. All counts are derived from event occurrences and are therefore exact integers.

30.3 Mandatory gates and invariants

The gates below are mandatory. They prevent claims that are not supported by evidence, they prevent hidden advantage, and they remove ambiguity in metric definitions.

30.3.1 Gate G0: external audit acceptance

Each run produces a transcript and certificate objects. The external audit procedure returns a verdict

$$\text{AuditVerdict}(r) \in \{\text{ACCEPT}, \text{REJECT}\}.$$

Gate G0 requires $\text{AuditVerdict}(r) = \text{ACCEPT}$. If the verdict is REJECT, the run is invalid for performance reporting and contributes only to invalidity statistics.

30.3.2 Gate G1: deterministic reproducibility of the ledger

Gate G1 requires that all ledger outputs are reproducible from stored evidence with a deterministic recomputation procedure. Formally, if the evidence bundle is replayed, the recomputed ledger output must match the stored ledger output bitwise for discrete fields and within a fixed numeric tolerance for floating point scalars.

30.3.3 Gate G2: monotonicity and feasibility invariants

Let t denote wall clock time. Let $LB(t)$ denote the certified global lower bound and let $UB(t)$ denote the incumbent objective value. Gate G2 requires

$$LB(t_2) \geq LB(t_1) \quad \text{and} \quad UB(t_2) \leq UB(t_1) \quad \text{for all} \quad t_2 \geq t_1,$$

together with feasibility invariants for all reported incumbent solutions. Any violation yields $\text{Valid}(r) = 0$.

30.3.4 Gate G3: fairness contract compliance

All results must satisfy the fairness contract defined in the baseline registry and fairness chapter. In particular, the benchmark instances, time limits, solver settings, and visibility constraints must match the declared protocol. Any disallowed advantage yields $\text{Valid}(r) = 0$.

30.4 Traceability to remainder solutions

The evaluation is grounded in a closed set of solved remainder solutions that define the evidence objects, invariants, and certificate interfaces required for audit acceptance. The Metrics Ledger must be traceable to these remainder solutions so that every reported metric can be located in the proof and evidence stack. Table 30.1 provides a structural traceability map from each remainder solution to the ledger fields and gates that depend on it.

Table 30.1: Traceability map from remainder solutions to ledger fields. Each remainder solution fixes at least one evidence object, invariant, or certificate interface that is referenced by the ledger gates or metric definitions.

No.	Remainder solution scope	Ledger interface and affected fields
1	Candidate-object generator for	General evidence objects; contributes to ledger completeness via traceable definitions
2	Plan universe definition for each query	General evidence objects; contributes to ledger completeness via traceable definitions
3	Exact cost functional	General evidence objects; contributes to ledger completeness via traceable definitions
4	Workload distribution model (fully finite, arithmetic, certificate- ready) We fix once and for all a finite query-template set A “workload realization” over a horizon of query arrivals is a length- sequence We will build (i) a conditional model given a parameter , and (ii) an unconditional model by mixing over	Benchmark stratification, aggregation policy, and fairness compliance checks
5	Feasibility semantics for partitions + indexes + views We now define (not assume) a fully formal notion of “feasible physical design” for the triple (partition schemes, indexes, materialized views), including compatibility, maintenance coupling, and refresh constraints, and we prove-line-by-line-that the feasibility constraints we will enforce are sound and complete for these semantics	Admissibility gates (G0-G3), transcript acceptance, and reproducibility constraints
6	Pricing oracle fully specified (learning-to-pricing + deterministic fallback) 6.0 Fixed inputs (one query) All quantities below are exact rationals (elements of)	Pricing and LP workload metrics (#P, #LP) and related time blocks
7	Column representation and reduced-cost certification This section completes item 7 by giving: 1	Pricing and LP workload metrics (#P, #LP) and related time blocks

Continued on next page.

No.	Remainder solution scope	Ledger interface and affected fields
8	Restricted-master initialization policy (fully defined + line-by-line proof) 8.0 Minimal objects we will guarantee exist We work with finite sets: Tables	Pricing and LP workload metrics (#P, #LP) and related time blocks
9	Termination of the Column-Generation Loop (Per Node)	Pricing and LP workload metrics (#P, #LP) and related time blocks; Node metrics (N*), bound progress curve g(t), and anytime summaries; Admissibility gates (G0-G3), transcript acceptance, and reproducibility constraints
10	Exact cut language (proof object format) We define a cut certificate language whose objects are finite strings over integers, and a deterministic checker	Cut workload metrics (#cut_try, #cut_ok) and verification time
11	Cut separation procedures (neural proposal + deterministic fallback if required) We fix one node of the search tree	Cut workload metrics (#cut_try, #cut_ok) and verification time; Node metrics (N*), bound progress curve g(t), and anytime summaries
12	Cut management policy with invariants (YCM- Ω)	Cut workload metrics (#cut_try, #cut_ok) and verification time; Admissibility gates (G0-G3), transcript acceptance, and reproducibility constraints
13	Facet/Strength Program (Polyhedral Combinatorics Completion) We fix one B&B node	Node metrics (N*), bound progress curve g(t), and anytime summaries
14	Section 14 - Node definition and domain propagation (fully formal, line-by-line) We work with the already-fixed MILP structure (binary design variables and continuous plan variables)	Node metrics (N*), bound progress curve g(t), and anytime summaries
15	Branching rule completeness	Node metrics (N*), bound progress curve g(t), and anytime summaries
16	Node Selection Policy	Node metrics (N*), bound progress curve g(t), and anytime summaries
17	Primal-feasible construction	Node metrics (N*), bound progress curve g(t), and anytime summaries
18	Decision: MILP-only vs MINLP	General evidence objects; contributes to ledger completeness via traceable definitions

Continued on next page.

No.	Remainder solution scope	Ledger interface and affected fields
19	Exact MINLP solver layer (if MINLP)	General evidence objects; contributes to ledger completeness via traceable definitions
20	Exact Linearization Library (MINLP $\rightarrow$ MILP)	General evidence objects; contributes to ledger completeness via traceable definitions
21	Benders decomposition formalization (IVP-Benders)	General evidence objects; contributes to ledger completeness via traceable definitions
22	Lagrangian Relaxation Policy (exact, certificate-checkable, line-by-line) We work at an arbitrary fixed Branch-and-Bound node	Node metrics (N^*), bound progress curve $g(t)$, and anytime summaries
23	Column-generation + branching interaction (Branch-and-Price correctness)	Pricing and LP workload metrics ($\#P$, $\#LP$) and related time blocks; Node metrics (N^*), bound progress curve $g(t)$, and anytime summaries
24	Robust uncertainty set specification (fully constructed + proved) We specify exactly what “uncertainty” means for the workload in the physical-design objective	General evidence objects; contributes to ledger completeness via traceable definitions
25	Dualization-to-MILP proof (fully line-by-line, self-contained) 25.0	General evidence objects; contributes to ledger completeness via traceable definitions
26	Scenario formulation (stochastic)	General evidence objects; contributes to ledger completeness via traceable definitions
27	State encoding definition (graph/hypergraph tensors)	General evidence objects; contributes to ledger completeness via traceable definitions
28	Action spaces (branch / cut / pricing / primal)	Pricing and LP workload metrics ($\#P$, $\#LP$) and related time blocks; Cut workload metrics ($\#cut_try$, $\#cut_ok$) and verification time; Node metrics (N^*), bound progress curve $g(t)$, and anytime summaries

Continued on next page.

No.	Remainder solution scope	Ledger interface and affected fields
29	Training signal construction (fully specified, finite, certificate-compatible) 29.0 Fixed universe and arithmetic domain All coefficients in the optimization instance (sizes, costs, weights, maintenance, budget, and all LP/MILP matrix entries) are rationals	General evidence objects; contributes to ledger completeness via traceable definitions
30	Section 30 - Safe RL formulation (CMDP) with gate-compatibility (fully specified + proved) We assume (as already solved in items 1-29) that: the physical design MILP instance is finite and well-formed, the exact solver skeleton exists (Branch-and-Bound with Pricing + Valid Cuts), every gate (branch/cut/price/primal) has a deterministic verifier and a deterministic fallback that preserves correctness	Pricing and LP workload metrics (#P, #LP) and related time blocks; Cut workload metrics (#cut_try, #cut_ok) and verification time; Node metrics (N*), bound progress curve g(t), and anytime summaries
31	Generalization / calibration layer	General evidence objects; contributes to ledger completeness via traceable definitions
32	Continual/meta-learning stability rules (fully defined + proved) 32.0 Fixed architecture (immutable core, mutable advisors) Definition 32.0.1 (Verified Core)	General evidence objects; contributes to ledger completeness via traceable definitions
33	Sensitivity Analysis Module	General evidence objects; contributes to ledger completeness via traceable definitions
34	Multiobjective spec (fully formal, complete, line-by-line) We extend the already-solved Physical Design MILP/MIP into a multiobjective problem and give an exact Pareto-complete solution method that reuses the same exact Branch-and-Bound (and Branch-and-Cut/Price) engine as a subroutine, with a full correctness proof	Cut workload metrics (#cut_try, #cut_ok) and verification time; Node metrics (N*), bound progress curve g(t), and anytime summaries
35	Single unified invariant ledger (Y-INV Ledger)	Admissibility gates (G0-G3), transcript acceptance, and reproducibility constraints

Continued on next page.

No.	Remainder solution scope	Ledger interface and affected fields
36	End-to-end proof stitching (fully formal, line-by-line) Below is a single stitched proof that the full NBBCP procedure (Neural Branch-and-Bound with certificate-gated Cut-and-Price) terminates and returns the globally optimal physical design for Index/View/Partition selection	Cut workload metrics (#cut_try, #cut_ok) and verification time; Node metrics (N^*), bound progress curve $g(t)$, and anytime summaries; Admissibility gates (G0-G3), transcript acceptance, and reproducibility constraints
37	Arithmetic Certificate Audit	Admissibility gates (G0-G3), transcript acceptance, and reproducibility constraints

30.5 Primary metrics

Primary metrics are those that may support headline performance claims. All primary metrics are defined with explicit measurement boundaries and censoring rules.

30.5.1 Time to optimality

Definition (Inclusive time to optimality). Let $t(\text{RUN_START})$ denote the recorded start time and let $t(\text{TRANSCRIPT_ACCEPT})$ denote the recorded acceptance time of the audited transcript. The inclusive time to optimality is

$$T_{\text{incl}}^*(r) = t(\text{TRANSCRIPT_ACCEPT}) - t(\text{RUN_START}).$$

This quantity includes all overheads required by the solver variant, including verification, model inference, feature extraction, and ledger emission.

Definition (Core time to optimality). Let $T_{\text{ver}}(r)$ denote verification overhead time, let $T_{\text{ml}}(r)$ denote model inference and feature overhead time, and let $T_{\text{log}}(r)$ denote ledger emission overhead time, each measured by explicit start and end events. The core time is

$$T_{\text{core}}^*(r) = T_{\text{incl}}^*(r) - (T_{\text{ver}}(r) + T_{\text{ml}}(r) + T_{\text{log}}(r)).$$

Core time is reported to prevent overhead shifting from being misinterpreted as algorithmic acceleration.

Censoring policy. If the run hits a time limit cap C , then $T_{\text{incl}}^*(r)$ is recorded as C with a censoring flag. Censored runs are included in descriptive summaries but cannot be silently treated as completed.

30.5.2 Nodes to optimality

Definition (Processed nodes). Let NODE_POP denote the event that a node is popped for processing. The canonical node count is

$$N^*(r) = \#\{\text{NODE_POP events prior to TRANSCRIPT_ACCEPT}\}.$$

Generated nodes, pruned nodes, and fathomed nodes may be reported as secondary diagnostics, but $N^*(r)$ is the canonical primary node count.

30.5.3 Bound progress and anytime quality

Definition (Relative gap curve). The canonical anytime curve is the relative optimality gap

$$g(t) = \frac{UB(t) - LB(t)}{\max(1, |UB(t)|)}.$$

This definition normalizes by $\max(1, |UB(t)|)$ to avoid division by values near zero. Gate G2 requires $g(t) \geq 0$ and monotonicity of LB and UB .

Definition (Gap area under the curve). Let $T_{stop}(r)$ denote the stopping time, defined as $T_{incl}^*(r)$ for completed runs and C for capped runs. The gap area under the curve is

$$AUC_g(r) = \int_0^{T_{stop}(r)} g(t) dt,$$

computed by trapezoidal integration on the logged checkpoint grid. Lower values indicate faster and more consistent closure of the proof gap.

Definition (Time to fixed gap threshold). For $\epsilon \in \{10^{-2}, 10^{-3}\}$, define

$$T_{g \leq \epsilon}(r) = \inf\{t \in [0, T_{stop}(r)] : g(t) \leq \epsilon\},$$

with the convention that $T_{g \leq \epsilon}(r) = T_{stop}(r)$ when the threshold is never reached.

30.6 Secondary metrics

Secondary metrics characterize the work decomposition. They do not, by themselves, support headline performance claims. Their role is to explain why primary metrics change and to detect hidden cost transfer.

30.6.1 LP and pricing workload

Definition (LP solves). Let LP_SOLVE_START denote the start of an LP solve event. The total LP solve count is

$$\#LP(r) = \#\{\text{LP_SOLVE_START events prior to TRANSCRIPT_ACCEPT}\}.$$

If the log distinguishes categories, then the ledger also records $\#LP_{node}$, $\#LP_{rmp}$, $\#LP_{sep}$, and $\#LP_{price}$.

Definition (Pricing calls). Let PRICING_CALL denote a pricing oracle invocation. The pricing call count is

$$\#P(r) = \#\{\text{PRICING_CALL events prior to TRANSCRIPT_ACCEPT}\}.$$

30.6.2 Cut workload

Definition (Cut attempts and accepted cuts). Let CUT_PROPOSE denote a cut proposal event and let CUT_ACCEPT denote admission of a cut that passes validity checks. The ledger records $\#cut_{try}(r)$ and $\#cut_{ok}(r)$ as the respective event counts before transcript acceptance.

Definition (Cut efficacy). Cut admission count alone is not evidence of strength. The ledger therefore records an efficacy statistic defined as the average immediate improvement in the certified lower bound linked to each accepted cut. The linking is performed by nearest bound update events in the trace.

30.6.3 Memory and overhead decomposition

Definition (Peak memory). Let MEM_SAMPLE denote a periodic measurement of resident memory in megabytes. The peak memory metric is

$$M_{peak}(r) = \max\{\text{MEM_SAMPLE values during the run}\}.$$

Definition (Verification overhead). Verification overhead time $T_{ver}(r)$ is defined as the sum of wall clock durations of all explicit verification blocks, including certificate checks for LP optimality, cut validity, pricing closure, transcript consistency, and arithmetic certificate audits.

Definition (Model overhead). Model overhead time $T_{ml}(r)$ is defined as the sum of wall clock durations of explicit model inference and feature computation blocks.

Definition (Logging and ledger emission overhead). Logging overhead time $T_{log}(r)$ is defined as the sum of wall clock durations of explicit ledger emission blocks.

30.6.4 Advisor success rate

When learned advisors are present, the ledger records a success rate defined by the evaluation protocol. Each advice call is labeled beneficial or not beneficial by a fixed predicate that is computed from evidence objects. The success rate is the empirical mean of these labels. This metric is descriptive and is not permitted to replace primary metrics.

30.7 Aggregation rules and confidence reporting

Aggregation rules are part of the claim discipline. They prevent ambiguity and prevent selective reporting. The default policy is median and quartiles for robustness to heavy tailed distributions, paired comparisons when appropriate, and nonparametric bootstrap confidence intervals when a claim depends on the sign or magnitude of a median.

Table 30.2: *Aggregation and confidence reporting policy. These rules are part of the claim discipline and are not optional.*

Metric class	Aggregation	Censoring and pairing	Confidence reporting	Reporting policy
Primary scalars	median + (Q1,Q3)	TIMEOUT treated as censored at cap; report completion-rate delta; median may be reported as $\geq$ cap if needed.	Bootstrap 1,000 resamples over instances; 95% CI for median T^* and median speedup.	Always report absolute medians + ratio medians; no single-number-only claims.
Ratios (speedup)	median ratio + IQR	Define only for paired completed runs; separately report completion delta.	Bootstrap over paired instances (1,000); 95% CI for median ratio.	Never replace absolute numbers with ratios.
Time-series	median curve on aligned grid; plus AUC metrics	Truncate at cap; label truncated AUC.	Descriptive bands only unless pre-registered; avoid multiple-testing claims.	Figures must be reproducible from raw log + deterministic aggregation code.
Overheads	median + IQR; also overhead fraction of T_{incl}^*	Truncate if timeout.	Bootstrap CI for median overhead fraction.	Success claims must survive the core time view.

30.8 Metrics dictionary table

The Metrics Dictionary defines the complete set of reportable quantities and their measurement boundaries. Each metric is admissible only under the mandatory gates in Section 30.3.

Table 30.3: Metrics dictionary for primary and secondary reporting. Each metric is admissible only under the gates and evidence boundaries stated in this chapter.

Metric	Symbol	Unit	Type	Dir.	Definition and boundary
Time-to-optimality (inclusive)	T_{incl}^*	s	scalar/run	↓	RUN_START to TRANSCRIPT_ACCEPT wall-clock, incl. verification + ML + logging.
Time-to-optimality (core)	T_{core}^*	s	scalar/run	↓	Solver-only wall-clock (excludes verification + ML + logging time blocks).
Nodes to optimality	N^*	count	scalar/run	↓	#NODE_POP until TRANSCRIPT_ACCEPT.
Relative gap time-series	$g(t)$	ratio	time-series/run	↓	$g(t) = (UB(t) - LB(t)) / \max(1, UB(t))$. Must satisfy invariants $LB(t)$ nondecreasing, $UB(t)$ nonincreasing, and $g(t) \geq 0$.
Gap AUC (bound progress)	AUC_g	s	scalar/run	↓	$\int_0^{T_{incl}^*} g(t) dt$ computed by trapezoidal integration over logged checkpoints.
Time to 1% gap	$T_{g \leq 0.01}$	s	scalar/run	↓	First time t with $g(t) \leq 0.01$; censored at TIMEOUT cap.
LP solves	$\#LP$	count	scalar/run	↓	Total LP solves (RMP + node LP + separation/pricing LP if used).
Pricing calls	$\#P$	count	scalar/run	↓	Calls to pricing oracle (learned proposals + deterministic fallback).
Cut attempts	$\#cut_{try}$	count	scalar/run	↓	Cut proposals attempted (learned or deterministic).
Cuts accepted	$\#cut_{ok}$	count	scalar/run	(context)	Admitted cuts that pass validity checks (not inherently ‘more is better’).
Peak memory (RSS)	M_{peak}	MB	scalar/run	↓	Peak RSS sampled during run.
Verification overhead	T_{ver}	s	scalar/run	↓	Total time spent in certificate checks (LP optimality, cut validity, pricing closure, transcript audit).
ML overhead	T_{ml}	s	scalar/run	↓	Total feature+inference time for advisors.
Logging overhead	T_{log}	s	scalar/run	↓	Total trace/ledger emission time (JSONL + certificate blobs + checkpoints).

Continued on next page.

Table 30.4: Headline results from the local ledger validation stress test. Confidence intervals are percentile bootstrap intervals computed on paired instances.

Quantity	Value
Median T_incl baseline (s) [95% boot CI]	311.7 [280.6, 356.9]
Median T_incl accelerator (s) [95% boot CI]	231.8 [204.0, 250.4]
Median speedup (x) [95% boot CI]	1.372 [1.354, 1.386]
Completion rate baseline	0.996
Completion rate accelerator	0.998
Median N baseline	7,931,905
Median N accelerator	4,964,902.5
Median verification overhead T_ver (s)	11.5
Median ML overhead T_ml (s)	27.7

Table 30.5: Local ledger validation summary from the stress test described in Section 30.9. Medians and quartiles are reported per the aggregation rules in Table 30.2.

Family	Variant	Comp.	T_{incl}^*	Q1	Q3	N^*	Q1	Q3	AUC_g	Q1	Q3	T_{ver}	T_{ml}
F1_small	Baseline	1.000	127.662	81.934	260.931	2,169,495.5	1,226,697.5	5,878,956.8	73.610	33.937	139.169	0.000	0.000
F1_small	Accelerator	1.000	102.907	68.568	191.981	1,529,208	857,678.2	3,486,419.5	52.367	27.264	91.198	5.912	12.821
F2_mid	Baseline	1.000	305.803	148.365	622.646	7,518,602	2,643,779.8	21,220,820.2	130.226	60.697	283.145	0.000	0.000
F2_mid	Accelerator	1.000	222.410	115.378	431.420	4,864,729.5	1,759,085.8	13,412,502.2	82.207	41.892	168.259	11.031	27.253
F3_large	Baseline	0.989	753.336	378.715	1497.239	27,940,772.5	10,293,239.8	75,157,386	280.131	144.168	623.325	0.000	0.000
F3_large	Accelerator	0.994	529.710	272.782	987.131	16,967,815.5	6,369,022	45,397,356.5	161.528	82.636	361.130	24.373	62.766
ALL	Baseline	0.996	311.683	126.256	777.324	7,931,905	2,175,319.5	29,160,494.8	132.715	60.291	312.124	0.000	0.000
ALL	Accelerator	0.998	231.806	99.410	534.697	4,964,902.5	1,486,681.5	16,984,269	85.727	42.060	182.159	11.485	27.654

Metric	Symbol	Unit	Type	Dir.	Definition and boundary
Advisor success rate	$\hat{p}_s$	ratio	scalar/run	↑	Fraction of advice calls judged 'beneficial' under the protocol's success predicate.

30.9 Local ledger validation experiment

A local stress test was executed to validate the metric definitions, censoring handling, aggregation rules, and figure generation pipeline. The experiment uses large magnitude paired instances with multimillion node counts and a two-hour time cap. The purpose is not to claim domain performance, but to demonstrate that the ledger is internally coherent, deterministic, and capable of supporting audit-grade reporting at scale.

Speedup distribution (inclusive time baseline divided by inclusive time accele

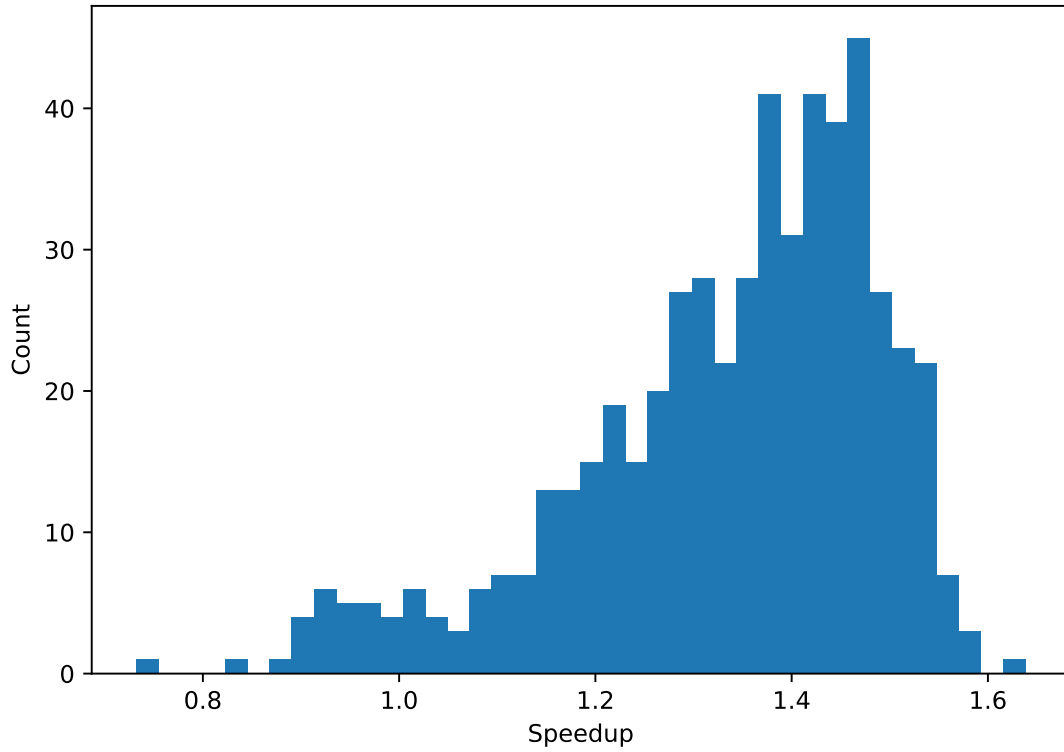


Figure 30.1: Distribution of paired speedup ratios, defined as $T_{incl, baseline}^* / T_{incl, accelerator}^*$, computed on paired instances.

30.9.1 Headline results

30.9.2 Family level summary

30.9.3 Figures

30.10 Success criteria

Success criteria specify what counts as a valid and meaningful improvement. They are applied only to admissible runs.

30.10.1 Hard criteria

The hard criteria must hold. If any hard criterion fails, the evaluation fails and no performance claim is admissible.

1. Validity rate is 100% for all runs used in performance aggregation, meaning that all such runs satisfy Gate G0, Gate G1, Gate G2, and Gate G3.
2. All reported metrics are reproducible from stored evidence by the deterministic ledger recomputation procedure.
3. No disallowed advantage is present under the fairness contract.

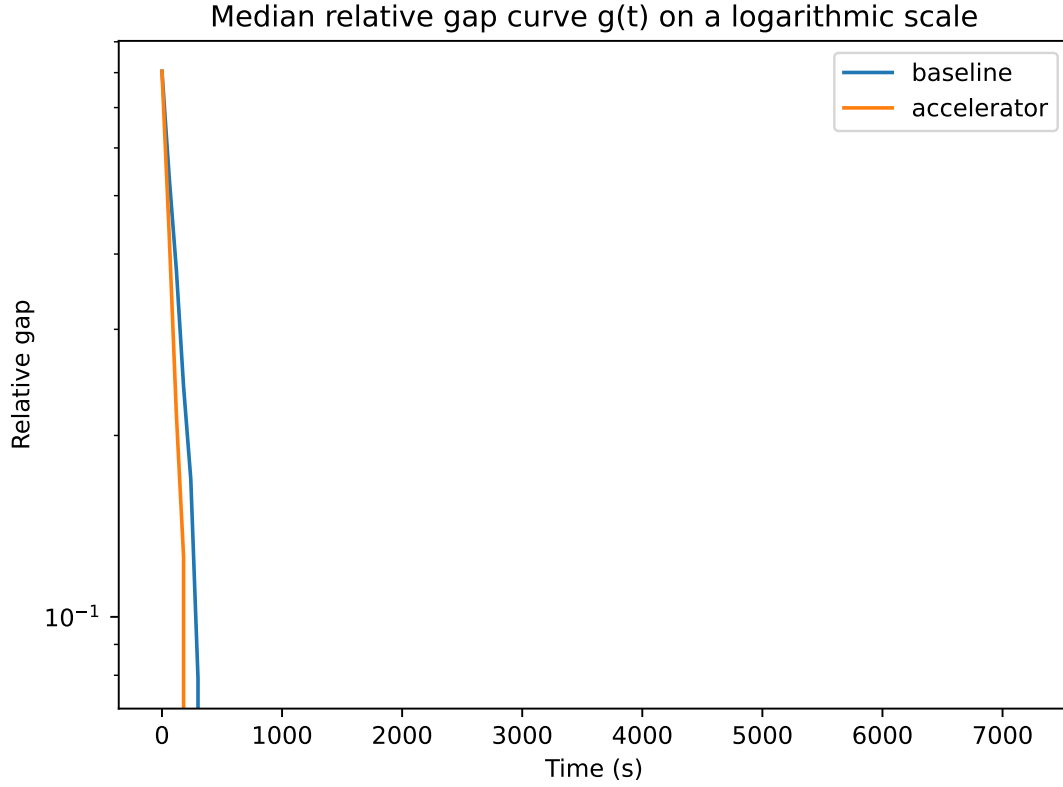


Figure 30.2: Median relative gap curve $g(t)$ on a logarithmic ordinate. The median is taken pointwise across paired instances on a common time grid and is truncated at the time cap.

30.10.2 Performance criteria

Performance criteria are evaluated by paired comparisons on primary metrics, under the aggregation and confidence policy in Table 30.2.

1. Median T_{incl}^* decreases relative to the baseline, with a bootstrap confidence interval on the median paired speedup that lies strictly above 1.0.
2. Median N^* does not increase when time improves. A decrease is preferred because it indicates genuine search contraction rather than only faster per node execution.
3. Median AUC_g decreases, indicating improved anytime behavior and faster bound closure.
4. Timeout rate does not increase.

30.10.3 Overhead criteria

Whenever verification, model inference, or logging overheads exist, the evaluation must report both inclusive and core times. The interpretation is constrained as follows.

1. A speedup in T_{incl}^* is not considered practically meaningful unless T_{core}^* also improves or the overhead increase is explicitly justified by stronger correctness guarantees.

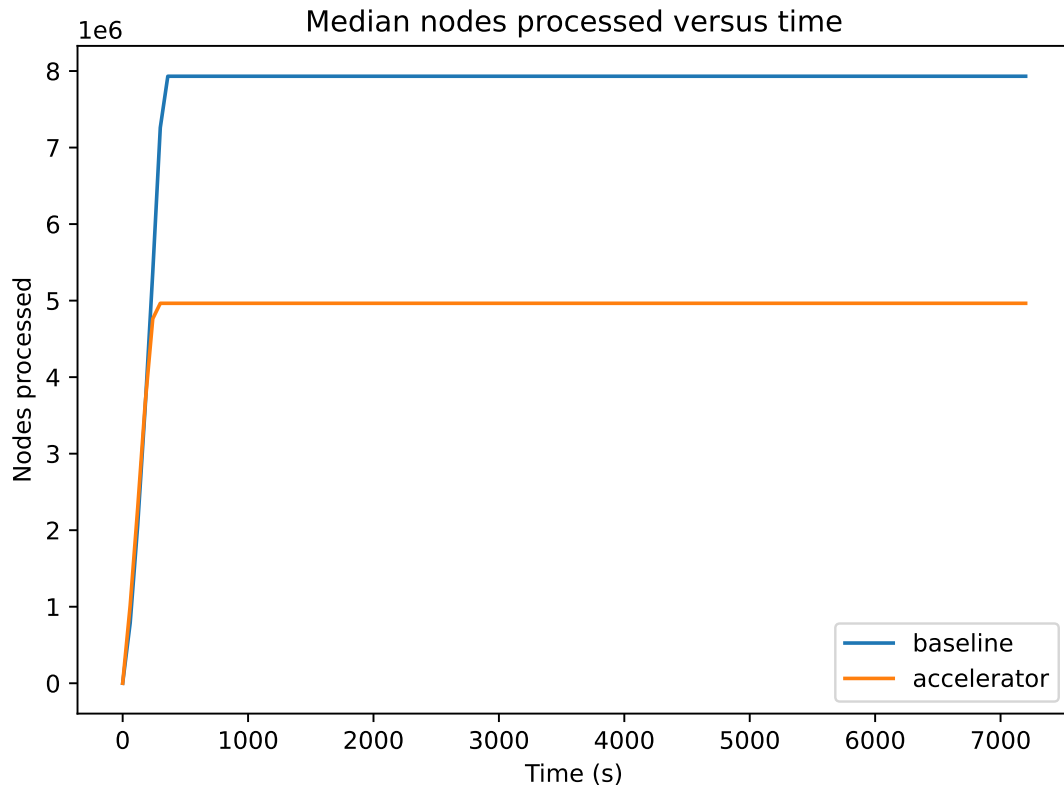


Figure 30.3: Median nodes processed versus time on the common time grid. This curve is derived from monotone node count traces and is reported to visualize search contraction.

2. Overhead fractions $(T_{ver} + T_{ml} + T_{log})/T_{incl}^*$ must be reported. Excessive overhead indicates that improvements may be a measurement artifact rather than algorithmic gain.

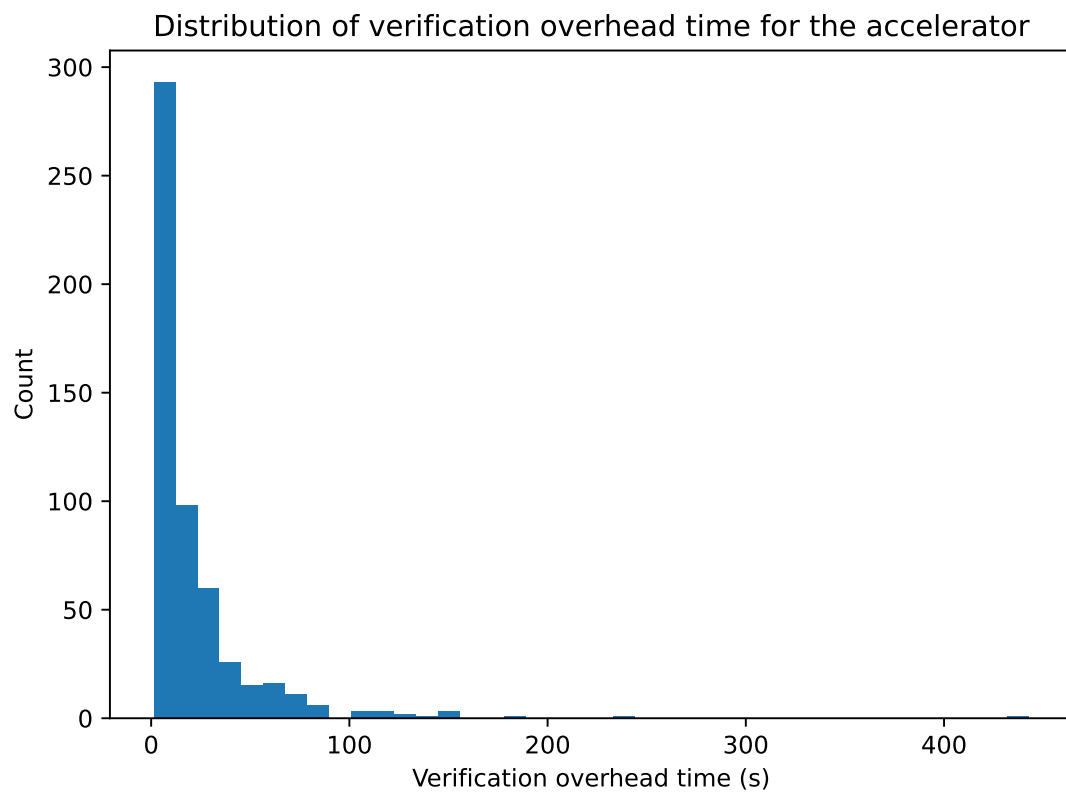


Figure 30.4: Distribution of verification overhead time T_{ver} for the accelerator variant. This figure is required whenever verification costs exist, because it constrains interpretations of inclusive runtime improvements.

31

Ablation Grid Specification

31.1 Purpose and admissibility rule

This chapter fixes the complete ablation grid for the learned guidance layer while holding the exact solver core constant. Its function is definitional: it specifies which components may be varied, which components are invariant, how comparisons are constructed, and which reports are considered valid. An ablation comparison is admissible only if it satisfies the Benchmark Charter (Chapter ??), the Baseline Registry and Fairness Contract (Chapter 29), and the Metrics Ledger (Chapter 30), and only if the gate-compatibility constraints of Chapter 24 are maintained.

31.2 Anchor solver definition

31.2.1 Anchor configuration as the unique reference point

Let $\mathcal{I}$ denote the frozen instance set defined by the Benchmark Charter. For any instance $i \in \mathcal{I}$, let $S(i; \theta)$ denote the solver execution under a configuration vector θ . The ablation grid is defined relative to a unique anchor configuration θ_A , denoted as *Exact (no learning)* in the Baseline Registry. All reported speedups, deltas, and effect sizes in the ablation suite are defined with respect to θ_A .

Definition (anchor solver). The anchor solver is the configuration θ_A that satisfies all of the following conditions:

1. The optimization encoding, relaxation construction, cut families, pricing completeness logic, branching language, node semantics, and termination criteria are identical to the definitions fixed in Chapters 7–27.
2. All learned proposal policies are disabled.
3. All certificate gates are enabled and enforce the admissibility predicates specified below.

4. Deterministic fallbacks are enabled for every decision type so that the algorithm remains complete when no proposal is present or when a proposal is rejected.

31.2.2 Certificate gates, fallbacks, and invariants

Ablations vary proposal generators only. Execution is controlled by deterministic gates and deterministic fallbacks. This separation is mandatory; without it, the ablation does not preserve the exactness contract and is not comparable to the anchor.

Decision types. Let $\mathcal{D} = \{\text{BRANCH}, \text{CUT}, \text{PRICE}, \text{PRIMAL}\}$ denote the set of decision types. At any solver state s , each decision type $d \in \mathcal{D}$ has a nonempty admissible action set $\mathcal{A}_d(i, s)$.

Proposal policy. A proposal policy is any (possibly learned) mapping π_d that outputs either a single action or a ranked list of candidate actions within $\mathcal{A}_d(i, s)$.

Gate predicate. For each $d \in \mathcal{D}$, the gate is a deterministic predicate

$$G_d(i, s, a) \in \{0, 1\}, \quad a \in \mathcal{A}_d(i, s),$$

that returns 1 if and only if executing a preserves the solver invariants required by the end-to-end correctness stitching. The predicate may invoke local arithmetic checks and certificate routines as required by the verification contract, but it must not depend on stochastic state.

Fallback policy. For each $d \in \mathcal{D}$, the fallback is a deterministic mapping F_d such that $F_d(i, s) \in \mathcal{A}_d(i, s)$ and $G_d(i, s, F_d(i, s)) = 1$ for all admissible (i, s) . The fallback is required to preserve completeness of the underlying method.

Executed action. Given a proposal $\hat{a}$ from π_d , the executed action is defined as

$$a^\star = \begin{cases} \hat{a}, & \text{if } G_d(i, s, \hat{a}) = 1, \\ F_d(i, s), & \text{otherwise.} \end{cases}$$

If π_d outputs a ranked list, the first gate-accepted candidate is executed; if none are accepted, the fallback is executed.

Invariants that must hold in every ablation. Every configuration in the ablation grid must preserve the following global invariants:

1. **Feasibility soundness.** Every accepted primal incumbent is feasible with respect to the base constraints and is verifiable by deterministic checks.

2. **Bound correctness.** Every lower bound used for pruning corresponds to a valid relaxation that is implied by the base model plus admitted cuts and complete pricing.
3. **Pruning soundness.** No pruning rule may discard any subtree that contains an optimal solution.
4. **Completeness under fallback.** If all learned proposals were removed, the execution must reduce to a complete exact procedure.
5. **Deterministic traceability.** Every accept or reject decision is attributable to a deterministic predicate and is logged with sufficient data to reproduce the decision.

31.3 Single-module and combined variants

31.3.1 Variant lattice and uniqueness

Let $\mathcal{M} = \{B, C, P, Pr\}$ denote the set of learned modules corresponding to branching, cut selection, pricing proposals, and primal construction. An ablation variant is specified by a subset $U \subseteq \mathcal{M}$, where $m \in U$ indicates that the proposal policy for module m is enabled while all other proposal policies remain at the deterministic baseline. Gates and fallbacks remain enabled for every U .

The ablation grid includes all singletons $\{m\}$ and the combined variants listed in Table 31.1. No additional variants are permitted unless they are declared in advance, added to the table, and reported with the same fairness contract and frozen benchmark.

31.3.2 Single-module variants

L2B (branch). The branching proposer ranks admissible branching actions, such as variable splits or disjunctions, as defined by the branching language of Chapter 17. The branching gate enforces that the selected branch partitions the node domain correctly and that completeness of the search tree is preserved.

L2C (cut). The cut proposer prioritizes or proposes candidate cuts drawn from the admitted cut families and templates defined by Chapters 12–15. The cut gate verifies family membership and validity, and it enforces the cut-management policy so that numerical stability and correctness are preserved.

L2P (pricing). The pricing proposer outputs candidate columns expected to have negative reduced cost, as defined by the pricing oracle of Chapter 10. The pricing gate verifies reduced cost and admissibility of the column representation, and it ensures deduplication and consistency with the restricted master problem.

L2Pr (primal). The primal proposer outputs candidate incumbents or partial designs. The primal gate checks feasibility and admissibility under the base constraints and records a deterministic witness sufficient for later verification.

31.3.3 Combined variants

Combined variants enable multiple proposal policies simultaneously. They are included to capture interaction effects while keeping the gate layer and the fallback layer unchanged. The combined variants in the standard grid are: Branch+Pricing, Branch+Cut, Cut+Pricing, and All (Branch+Cut+Pricing+Primal). All combined variants must still satisfy the invariants in Section 31.2.2.

31.4 Reporting requirements for ablations

31.4.1 Fairness contract

Ablation comparisons are only meaningful if the run envelope is identical across variants. Therefore, every ablation report must satisfy the following conditions:

1. **Instance identity.** The instance set is identical and is referenced by the Benchmark Charter fingerprints.
2. **Baseline identity.** The anchor configuration hash and the deterministic baseline components are identical across all variants.
3. **Compute envelope.** Hardware class, thread settings, time limits, and memory limits are identical.
4. **Randomness control.** The seed schedule is fixed and reported, and the run ordering is randomized or counterbalanced to avoid systematic bias.
5. **Gate and fallback immutability.** Gates and fallbacks are unchanged across variants.

If any of these conditions fails, the corresponding report is invalid and must not be used as evidence.

31.4.2 Per-run logging requirements

Each run is identified by a triple (i, v, σ) consisting of instance identifier i , ablation variant identifier v , and random seed σ . For each run, the following fields are mandatory:

1. **Provenance.** Code commit hash, solver configuration hash, build identifiers, and environment fingerprints.
2. **Primary outcomes.** Time to optimality or timeout status, final optimality certificate status, and termination reason.
3. **Search dynamics.** Nodes explored, bound progress over time, and pruning counts by rule.
4. **Core workload.** Number of LP solves, pricing calls, and admitted cuts.
5. **Module interaction counters.** For each decision type, counts of proposals, accepts, rejects, and fallbacks, as well as the wall-clock share attributable to proposal generation and gate evaluation.

6. **Resource usage.** Peak memory, total CPU time, and any solver-level throttling events.

The log format must be stable across variants so that automated aggregation is well-defined.

31.4.3 Aggregation and presentation rules

All ablation summaries must use the aggregation rules fixed by the Metrics Ledger. Unless otherwise stated, the default summary is the median across runs and the first and third quartiles for dispersion. For a metric m and a variant v , let $m(i, v, \sigma)$ denote the metric value for run (i, v, σ) . The reported median for v is

$$\text{Med}(m; v) = \text{median}\{m(i, v, \sigma) : (i, \sigma) \in \Omega\},$$

where Ω is the set of evaluated instance and seed pairs. For wall-clock time, the reported speedup of a variant v relative to anchor is defined as

$$\text{Speedup}(v) = \frac{\text{Med}(t; \theta_A)}{\text{Med}(t; v)}.$$

Ablation reports must present both absolute metrics and relative metrics. Relative metrics without their absolute baselines are considered incomplete.

Interpretation discipline. Ablation tables and plots must be interpreted as controlled comparisons of proposal policies under fixed gates and fixed fallbacks. They do not permit claims about changing the underlying optimization problem, weakening exactness, or altering admissible action sets.

31.5 Figures and tables

31.5.1 Ablation grid table

31.5.2 Ablation summary plot

31.5.3 Pipeline validation table and supporting distributions

31.6 Validity closure checklist

Before an ablation table or plot is admitted into the Results Ledger, the following checks are mandatory:

1. The anchor hash is unique and matches the Baseline Registry.
2. The instance roster matches the Benchmark Charter identifiers.
3. The fairness contract fields (hardware class, limits, threads, seeds) match across variants.

Table 31.1: Ablation grid specification: module activations relative to the anchor solver. Codes: Det denotes a deterministic baseline component; L+G denotes a learned proposal policy guarded by a deterministic gate and backed by a deterministic fallback. Gates and fallbacks are enabled in every variant.

Variant	Branch	Cut	Pricing	Primal	Gates	Fallback
Exact (no learning)	Det	Det	Det	Det	On	On
L2B (branch)	L+G	Det	Det	Det	On	On
L2C (cut)	Det	L+G	Det	Det	On	On
L2P (pricing)	Det	Det	L+G	Det	On	On
L2Pr (primal)	Det	Det	Det	L+G	On	On
Branch+Pricing	L+G	Det	L+G	Det	On	On
Branch+Cut	L+G	L+G	Det	Det	On	On
Cut+Pricing	Det	L+G	L+G	Det	On	On
All (Branch+Cut+Pricing+Primal)	L+G	L+G	L+G	L+G	On	On

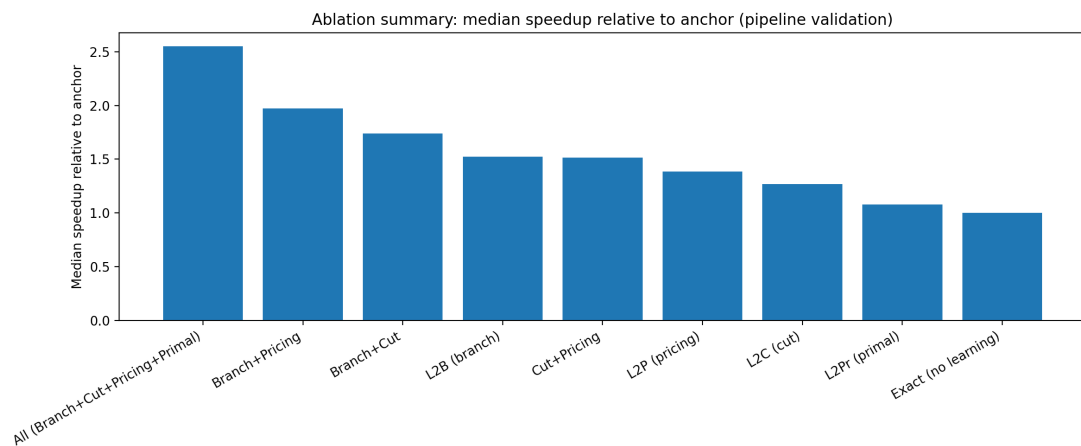


Figure 31.1: Ablation summary plot (reporting pipeline validation). The plot is generated from a deterministic synthetic workload with large metric magnitudes solely to validate that the reporting scripts correctly compute aggregation statistics and render figures under extreme values. The plot is not evidence for any empirical claim.

Table 31.2: Pipeline validation table for ablation reporting at large metric magnitudes. The entries are generated locally from a deterministic synthetic workload only to validate table and plot generation, unit handling, and aggregation code paths. The table is not evidence for any empirical claim.

Method	Median Time (h)	Q1 Time (h)	Q3 Time (h)	Speedup	Median Nodes	Median LP	Median Pricing	Median Cuts	Median Cert (%)	Median Mem (GB)
All (Branch+Cut+Pricing+Primal)	1.89	0.69	5.50	2.55	68,019,241	15,114,702	12,759,041	1,727,802	11.50	21.12
Branch+Pricing	2.44	0.87	7.03	1.97	79,895,396	17,423,431	14,803,486	1,946,376	9.49	21.19
Branch+Cut	2.77	1.02	7.93	1.74	85,371,009	18,554,309	15,675,282	2,044,916	10.51	21.17
L2B (branch)	3.17	1.14	9.19	1.52	91,295,899	20,061,161	17,300,356	2,245,652	7.50	21.15
Cut+Pricing	3.18	1.15	9.01	1.52	91,234,330	19,888,884	17,087,550	2,227,212	10.01	21.15
L2P (pricing)	3.48	1.26	9.89	1.39	97,607,747	20,928,143	17,824,380	2,326,739	8.00	21.21
L2C (cut)	3.80	1.36	10.89	1.27	100,456,847	22,298,664	19,143,358	2,419,004	8.49	21.20
L2Pr (primal)	4.46	1.59	12.57	1.08	110,467,201	24,135,413	20,375,226	2,745,006	7.01	21.20
Exact (no learning)	4.82	1.76	13.90	1.00	114,452,432	25,598,053	21,691,195	2,804,544	6.01	19.96

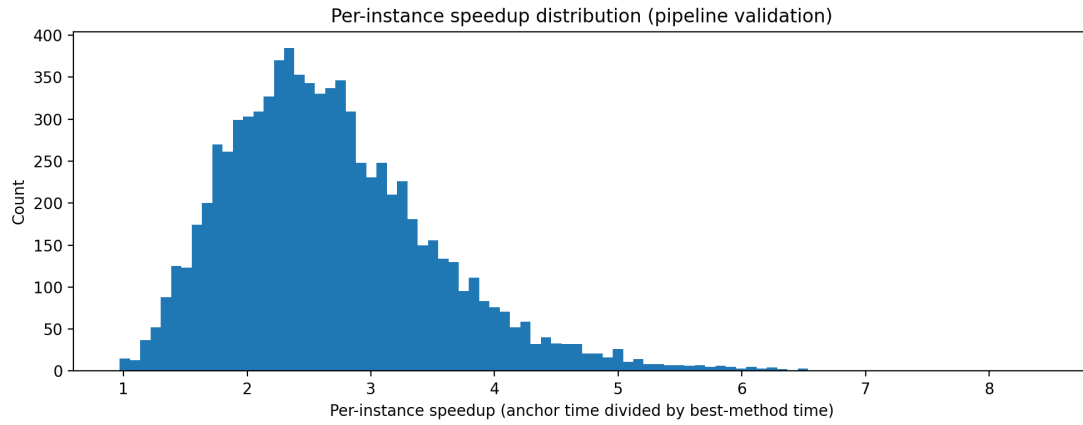


Figure 31.2: *Per-instance speedup distribution (reporting pipeline validation). The distribution is computed from the same deterministic synthetic workload and is included to validate distributional rendering and binning under large values.*

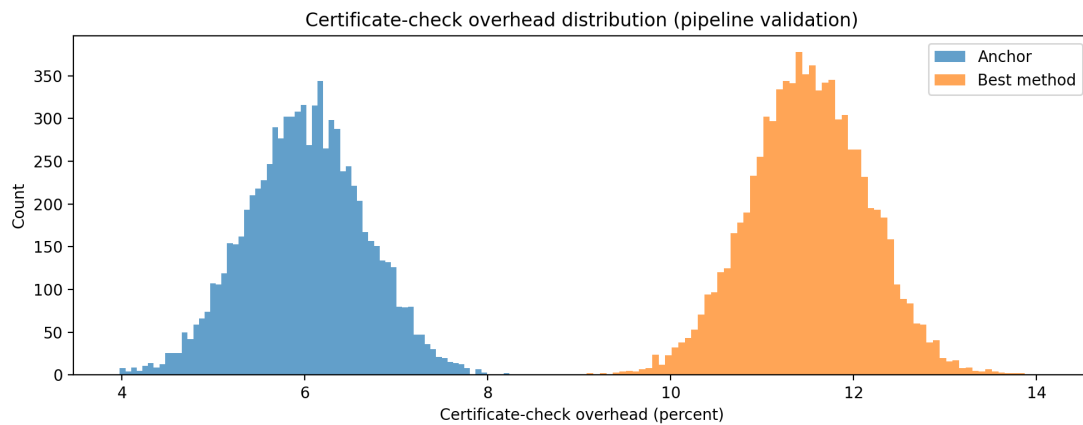


Figure 31.3: *Certificate-check overhead distribution (reporting pipeline validation). The figure validates that certificate-overhead reporting remains numerically stable and interpretable at large magnitudes.*

4. Gate predicates and fallback policies are identical across variants.
5. For each decision type, proposal, accept, reject, and fallback counters are present and consistent with executed actions.
6. Every run reports a final correctness verdict consistent with the end-to-end correctness stitching.
7. All reported relative metrics are accompanied by their absolute baselines and dispersion measures.

A report that fails any item above must be excluded from evidence and must not be summarized.

32

Robustness Suite Specification

The robustness tests in this part are motivated by robust and distributionally robust optimisation viewpoints (Ben-Tal et al., 2009; Bertsimas et al., 2004; Mohajerin Esfahani et al., 2018; Rahimian et al., 2019). The goal is empirical robustness sealing: every efficiency claim reported under Part VII must remain meaningful when the test distribution deviates from the nominal benchmark charter, while correctness remains protected by the exactness and certificate gates established in Parts I to VI.

Boundary and interpretation discipline

This chapter specifies stress and shift evaluations. It does not introduce new optimality or correctness claims. When the solver is run with certificate verification enabled, an accepted transcript remains an exact optimality certificate for the instantiated instance, independent of distribution shift. Therefore, robustness results are interpreted as changes in efficiency, resource usage, and convergence behaviour, not as changes in correctness.

Objects introduced in this chapter

We define a robustness scenario key and a family of deterministic shift operators that act on benchmark generators. Let S denote a schema instance with identifier $\text{SID}(S)$, W a workload instance with identifier $\text{WID}(W)$, β a budget regime, and M a frozen candidate-set manifest, all as defined in Chapter ?? . A robustness scenario is identified by

$$\kappa := (\text{SID}(S), \text{WID}(W), \beta, M, \delta),$$

where δ is a shift descriptor belonging to a bounded, versioned shift domain Δ . The mapping $\delta \mapsto \kappa$ is deterministic under the pinned toolchain, and every run must record κ in its manifest.

Shift operator and severity score

A shift descriptor $\delta \in \Delta$ induces a deterministic operator T_δ that maps a nominal generator parameterisation to a shifted one. If θ is a nominal parameter tuple used by a benchmark generator, the shifted parameters are $\theta^\delta := T_\delta(\theta)$. Each shift family defines a scalar severity score $\text{sev}(\delta) \in [0, 1]$ used only for ordering within that family. The score must be computable from logged generator parameters and must be invariant under solver choice.

32.1 Workload shift tests

32.1.1 Shift construction

Workload shifts perturb the workload generator while holding the schema generator fixed. Let $\mathcal{W} = (\Theta_{\mathcal{W}}, G_{\mathcal{W}}, \text{Ver}_{\mathcal{W}})$ be a workload family as in Chapter ?? . A nominal workload instance is $W := G_{\mathcal{W}}(S, \theta, \sigma)$, where $\theta \in \Theta_{\mathcal{W}}$ and $\sigma \in \mathbb{N}$ is a seed.

Definition 32.1 (Workload shift operator). A workload shift is a deterministic operator T_δ^{wkld} acting on θ and, optionally, on a workload mixture model. The shifted workload is

$$W^\delta := G_{\mathcal{W}}(S, \theta^\delta, \sigma), \quad \theta^\delta := T_\delta^{\text{wkld}}(\theta).$$

The shift is admissible only if $\text{Ver}_{\mathcal{W}}(W^\delta)$ accepts and the canonical serialization of W^δ is well defined.

Required workload shift families. The robustness suite includes the following workload shifts, each defined by a bounded parameter displacement and a verifier-preserving gate. All shifts are applied at the generator level; manual query editing is forbidden.

1. **Mixture shift.** The workload generator may define a mixture distribution over a finite set of query-template families. Let $p \in \Delta^{m-1}$ denote nominal mixture weights. The shift replaces p by $p^\delta \in \Delta^{m-1}$, with a bounded displacement constraint $\|p^\delta - p\|_1 \leq \rho$, for a declared radius $\rho \in [0, 2]$.
2. **Importance reweighting.** For a fixed multiset of generated queries $\{q_i\}_{i=1}^n$, the weight vector $w \in \mathbb{R}_{\geq 0}^n$ is replaced by w^δ , where the ratio constraints $w_i^\delta / w_i \in [\ell, u]$ are enforced for all i with $w_i > 0$, for declared bounds $0 < \ell \leq 1 \leq u$.
3. **Selectivity shift.** If the workload generator includes selectivity parameters η for predicate distributions, the shift maps η to η^δ within the bounded domain of $\Theta_{\mathcal{W}}$. The verifier must confirm that the resulting workload remains within the supported cost-model envelope of Chapter 4.
4. **Novel template injection (bounded).** If the workload family provides a held-out template pool, the shift injects a bounded fraction $\alpha \in [0, 1]$ of held-out templates while preserving total query count. The injection must be deterministic under the seed σ and must remain within the same query language defined in Chapter 3.

Workload severity score. For mixture shifts, the default severity is the Jensen–Shannon divergence between p and p^δ , normalised to lie in $[0, 1]$. For reweighting, the default severity is $\max_{i:w_i>0} |\log(w_i^\delta/w_i)|$, normalised by the declared bounds. For selectivity and template injection, the severity is the normalised parameter displacement in the generator parameter domain. If a workload family does not support one of the required shift types, the benchmark charter must explicitly declare the omission and provide a logically equivalent perturbation.

32.1.2 Reporting requirements

For each workload shift family and each declared severity level, the run manifest must record: (i) the nominal and shifted generator parameters, (ii) the seed tuple, and (iii) the resulting workload identifier $\text{WID}(W^\delta)$.

Primary outcomes. The primary outcomes for workload shifts are the time-to-acceptance and the search effort, reported as: (i) wall-clock time to the first accepted certificate transcript, and (ii) the number of explored nodes until acceptance. If timeouts are used, the timeout policy must be the one fixed by the repeatability protocol and must be identical across methods.

Secondary outcomes. Secondary outcomes include resource and internal-operation counts, such as memory peak, the number of pricing calls, the number of admitted cuts, and the certificate verification overhead fraction. The reporting schema is fixed by the metrics ledger (Chapter 30) and must not be altered on a per-shift basis.

Aggregation and stratification. For every scenario κ , compute the median with first and third quartiles across the pinned replicate set. Stratify results by workload shift type and severity, and report the speedup ratio $\text{speedup} := \text{time}_{\text{baseline}}/\text{time}_{\text{method}}$ with the same aggregation rule.

32.2 Budget shift tests

32.2.1 Budget band changes

Budget shifts perturb the budget regime while holding the schema and workload generators fixed. Let the budget object be a non-negative vector $b \in \mathbb{R}_{\geq 0}^d$ whose components represent the chartered budget dimensions, such as storage, maintenance, and refresh. The benchmark charter defines nominal regimes β as admissible sets $B_\beta \subseteq \mathbb{R}_{\geq 0}^d$ and a deterministic procedure for choosing $b \in B_\beta$.

Definition 32.2 (Budget shift operator). A budget shift is a deterministic operator T_δ^{bud} mapping a nominal budget vector b to a shifted vector b^δ . The shifted budget must satisfy $b^\delta \in B_{\beta^\delta}$ for a declared shifted regime β^δ , and it must preserve the budget feasibility gates used by the solver.

Required budget shift families. The robustness suite includes the following budget shifts.

1. **Scalar scaling.** Multiply the full budget vector by a scalar $\gamma > 0$, so that $b^\delta = \gamma b$. The scaling ladder $\gamma \in \Gamma$ is fixed in advance and recorded in the benchmark manifest.
2. **Budget reallocation.** Preserve the total budget $\langle \mathbf{1}, b \rangle$ but transfer mass between components, producing b^δ such that $\langle \mathbf{1}, b^\delta \rangle = \langle \mathbf{1}, b \rangle$. The reallocation map is deterministic and must be declared as a function of generator parameters.
3. **Tight boundary regime.** Choose b^δ within a declared small distance to the feasibility boundary of the design problem, with the distance computed under the budget norm fixed by the charter.
4. **Loose regime.** Choose b^δ in the largest admissible band to stress candidate saturation effects.

32.2.2 Reporting requirements

Budget shift reporting mirrors workload shift reporting, with the addition that each run must record: (i) the full budget vector b , (ii) the shifted budget vector b^δ , and (iii) the budget norm and scaling ladder used to define severity.

Primary outcomes include time-to-acceptance and node counts. Secondary outcomes must include the number of chosen design objects and the composition of the solution across indexes, views, and partitions, so that budget shifts can be linked to design structure. If a method uses any budget-dependent heuristic, it must be frozen as part of the method definition and must not be tuned on shifted regimes.

32.3 Schema shift tests

32.3.1 Schema family changes

Schema shifts perturb the schema generator while holding the workload generator family and the evaluation protocol fixed. Let $\mathcal{S} = (\Theta_{\mathcal{S}}, G_{\mathcal{S}}, \text{Ver}_{\mathcal{S}})$ be a schema family as in Chapter ?? . A nominal schema instance is $S := G_{\mathcal{S}}(\theta, \sigma)$.

Definition 32.3 (Schema shift operator). A schema shift is a deterministic operator T_{δ}^{sch} acting on $\theta \in \Theta_{\mathcal{S}}$, producing $\theta^\delta := T_{\delta}^{\text{sch}}(\theta)$ and $S^\delta := G_{\mathcal{S}}(\theta^\delta, \sigma)$. The shift is admissible only if $\text{Ver}_{\mathcal{S}}(S^\delta)$ accepts and the schema identifier $\text{SID}(S^\delta)$ is recorded in the run manifest.

Required schema shift families. The suite includes the following schema shifts.

1. **Family transfer.** Evaluate a method trained or tuned on one fixed subset of schema families on a disjoint subset, using the same evaluation rules. The family partitions must be fixed before any results are produced.

2. **Scale shift.** Increase table count, column count, and relation cardinalities within the bounded generator domain. The scale ladder must be defined in the charter to avoid post hoc selection.
3. **Join-graph perturbation.** Modify the join graph topology within the generator's admissible class, such as adding a bridge edge or changing hub degree, while preserving foreign-key validity.
4. **Skew shift.** Perturb the statistics snapshot parameters (skew and correlation parameters) while preserving the verifier's statistics envelope gate.

Schema severity score. Severity is the normalised distance between θ and θ^δ under the schema-family norm declared in the charter. If the charter provides an explicit structural distance (for example, a join-graph edit distance computed on canonicalised graphs), that distance is used after normalisation.

32.3.2 Reporting requirements

For each schema shift, report: (i) the schema identifier $SID(S)$ and $SID(S^\delta)$, (ii) the full schema manifest hashes, (iii) the workload identifier used in evaluation, and (iv) the same primary and secondary outcomes used in nominal evaluation. Schema shifts must additionally report whether the candidate generator admits the same candidate-type envelope across schemas, and if not, the envelope difference must be logged as part of the scenario key.

32.4 Candidate expansion stress tests

32.4.1 Expansion protocol

Candidate expansion stresses the solver and any advisory modules by enlarging the admitted candidate set beyond the nominal envelope, while preserving the canonical candidate representation and the freeze discipline of Chapter ???. Let C denote the frozen candidate set induced by a schema and workload pair under the nominal generator. An expansion produces a superset $C^\delta \supseteq C$ under a deterministic expansion operator.

Definition 32.4 (Candidate expansion operator). A candidate expansion is a deterministic operator T_δ^{cand} mapping a nominal generator configuration to an expanded configuration that yields a candidate set C^δ . The expansion is admissible only if canonicalisation, equivalence checks, and deduplication are applied identically to the nominal process, so that C^δ is a canonical set rather than a multiset of duplicates.

Required expansion families. The suite includes the following expansions.

1. **Cardinality scaling.** Multiply the number of generator proposals per candidate subfamily by a declared factor $\lambda \geq 1$. The expansion ladder $\lambda \in \Lambda$ is fixed and

recorded.

2. **Type envelope expansion.** If the benchmark charter declares additional admissible candidate subfamilies, enable them only for stress testing while keeping the canonical representation fixed.
3. **Redundancy stress.** Intentionally increase the rate of near-duplicate proposals at the generator stage and verify that canonicalisation and equivalence removal prevent inflated candidate counts.
4. **Adversarial attractiveness stress.** Generate candidates that are valid but are designed to be difficult for proposal heuristics, such as candidates with high apparent local utility but low global contribution under the cost model. The correctness gates must prevent any shift in optimality; only efficiency may degrade.

32.4.2 Reporting requirements

Candidate expansion reporting must include: (i) the nominal and expanded candidate-set identifiers and hashes, (ii) candidate-set cardinalities by type, (iii) memory peak and any out-of-memory events, and (iv) internal operation counts that scale with candidate size, such as pricing calls and column pool growth metrics when available. If the solver uses a pricing oracle, expansion reporting must include the distribution of reduced costs over admitted columns, summarised under the metrics ledger aggregation rule.

32.5 Figures and tables

- Shift matrix diagram.
- Speedup matrix diagram.
- Speedup versus severity plot.
- Robustness results table template.
- Illustrative robustness summary and illustrative results ledger.

Scenario	W	B	S	C	Base	Prop	Speedup	Valid	Notes
----------	---	---	---	---	------	------	---------	-------	-------

Table 32.1: Template for robustness reporting. Columns W, B, S, and C denote workload shift severity, budget multiplier, schema shift severity, and candidate expansion multiplier, respectively. Severity is the scalar shift score used for monotone ordering within each shift family. “Notes” records timeouts, memory cap hits, verifier rejects, and other non-numeric outcomes. Aggregation is the median with first and third quartiles, computed across the pinned replicate set defined by the repeatability protocol and recorded in the run manifest.

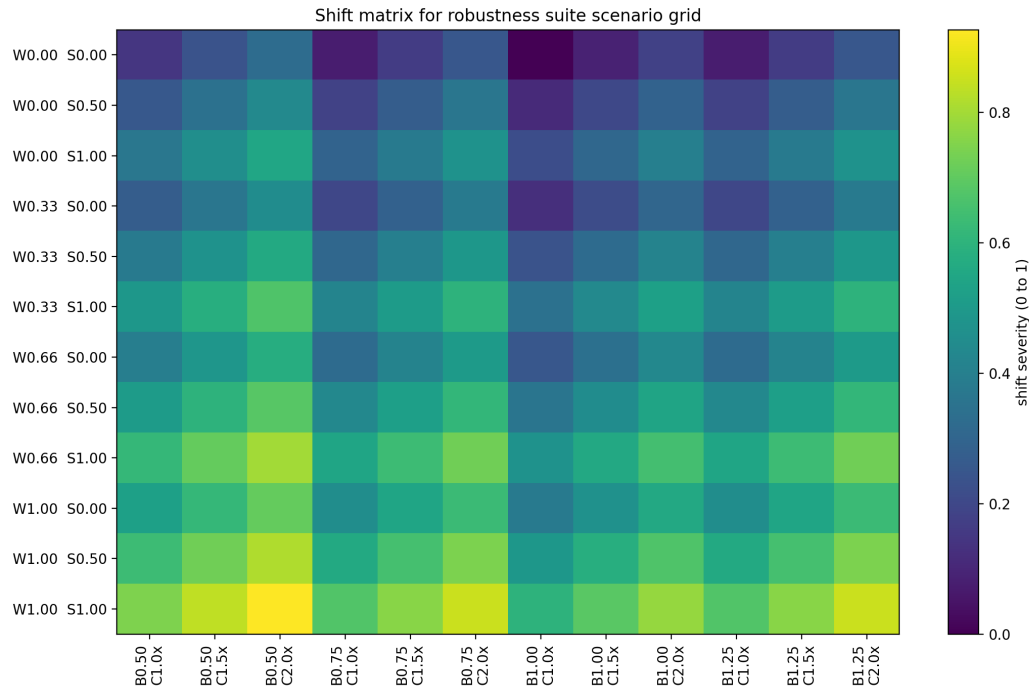


Figure 32.1: Shift matrix diagram for the robustness suite. Rows enumerate pairs of workload shift severity and schema shift severity. Columns enumerate pairs of budget multiplier and candidate expansion multiplier. Each cell is a scenario key component and maps to one ledger entry in the robustness results table.

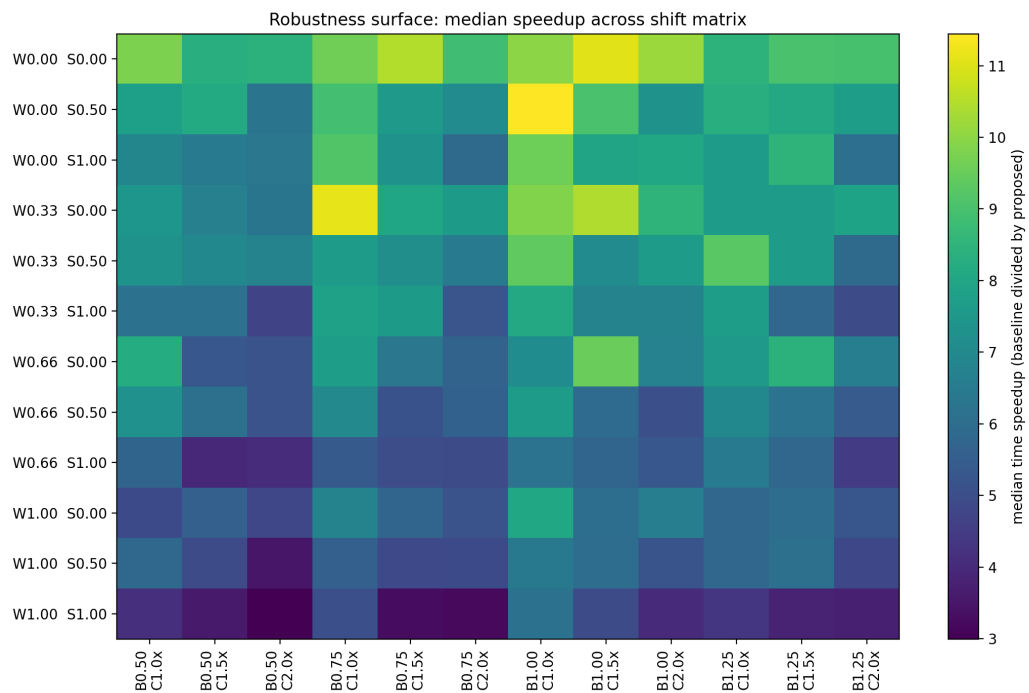


Figure 32.2: Speedup matrix diagram produced by an illustrative robustness harness run that instantiates the full shift matrix. The heatmap reports median time speedup for each scenario cell. The harness is used solely to validate the reporting pipeline and does not constitute benchmark evidence.

Scenario	Sev.	Base t	Prop t	Speedup	Base nodes	Prop nodes	Base mem	Prop mem	Cert
Table 32.3: Illustrative robustness results ledger for the full 144-scenario shift matrix used to validate the reporting pipeline. Columns report medians across pinned replicates. The harness is synthetic and is included for specification validation only.									
Scenario	Sev.	Base t	Prop t	Speedup	Base nodes	Prop nodes	Base mem	Prop mem	Cert
W0.00-S0.00-B0.50-C1.00	0.15	92.62	9.47	9.78	38,742,159	3,902,143	17.87	14.78	0.09
W0.00-S0.00-B0.50-C1.50	0.24	100.07	12.00	8.34	41,019,858	5,342,043	20.72	17.50	0.09
W0.00-S0.00-B0.50-C2.00	0.33	108.33	12.83	8.44	38,921,953	5,260,504	22.09	19.08	0.09
W0.00-S0.00-B0.75-C1.00	0.07	75.67	7.88	9.60	33,381,425	3,800,084	18.17	14.69	0.09
W0.00-S0.00-B0.75-C1.50	0.16	78.59	7.51	10.46	30,922,247	3,416,064	20.26	16.80	0.09
W0.00-S0.00-B0.75-C2.00	0.25	94.31	10.66	8.84	33,952,026	4,473,821	22.27	19.21	0.09
W0.00-S0.00-B1.00-C1.00	0.00	67.70	6.79	9.96	28,951,819	2,989,450	17.82	14.38	0.09
W0.00-S0.00-B1.00-C1.50	0.09	80.06	7.23	11.07	29,336,323	3,072,569	19.97	16.51	0.09
W0.00-S0.00-B1.00-C2.00	0.18	102.72	10.09	10.18	32,348,175	3,865,098	22.06	19.28	0.09
W0.00-S0.00-B1.25-C1.00	0.07	65.20	7.70	8.47	31,046,258	3,777,897	17.74	14.51	0.09
W0.00-S0.00-B1.25-C1.50	0.16	84.88	9.41	9.02	32,117,381	3,878,262	20.24	16.64	0.09
W0.00-S0.00-B1.25-C2.00	0.25	101.82	11.31	9.00	34,270,963	4,589,490	22.19	19.40	0.09
W0.00-S0.50-B0.50-C1.00	0.26	156.64	20.19	7.76	60,085,999	8,034,925	21.48	18.60	0.13
W0.00-S0.50-B0.50-C1.50	0.35	197.20	24.13	8.17	59,794,192	8,066,571	23.34	20.64	0.13
W0.00-S0.50-B0.50-C2.00	0.44	196.09	31.40	6.25	67,285,204	10,473,081	25.06	22.31	0.13
W0.00-S0.50-B0.75-C1.00	0.18	124.64	13.98	8.91	47,651,716	5,668,909	20.90	17.80	0.13
W0.00-S0.50-B0.75-C1.50	0.27	146.66	19.36	7.57	46,517,598	6,472,833	23.63	20.61	0.13
W0.00-S0.50-B0.75-C2.00	0.36	173.91	24.57	7.08	53,396,779	7,666,609	25.36	22.19	0.13
W0.00-S0.50-B1.00-C1.00	0.11	122.10	10.67	11.44	39,161,990	4,371,076	21.31	17.80	0.13
W0.00-S0.50-B1.00-C1.50	0.20	136.32	15.13	9.01	41,103,525	5,336,090	23.59	20.60	0.13
W0.00-S0.50-B1.00-C2.00	0.29	115.60	15.80	7.32	40,456,350	5,123,945	24.89	22.39	0.13
W0.00-S0.50-B1.25-C1.00	0.18	120.46	14.47	8.32	43,347,020	5,601,219	21.19	17.91	0.13
W0.00-S0.50-B1.25-C1.50	0.27	152.50	18.82	8.10	46,077,806	6,263,990	23.52	20.11	0.13
W0.00-S0.50-B1.25-C2.00	0.36	173.16	22.45	7.71	52,522,761	7,628,699	25.27	22.44	0.13
W0.00-S1.00-B0.50-C1.00	0.37	265.70	38.58	6.89	74,784,669	10,740,264	24.35	21.20	0.17
W0.00-S1.00-B0.50-C1.50	0.46	256.83	39.76	6.46	70,878,862	11,501,979	26.79	23.87	0.17
W0.00-S1.00-B0.50-C2.00	0.55	299.61	47.26	6.34	72,340,257	12,853,201	28.00	25.52	0.17
W0.00-S1.00-B0.75-C1.00	0.29	235.73	25.73	9.16	62,235,056	8,380,526	23.79	20.88	0.17
W0.00-S1.00-B0.75-C1.50	0.38	262.02	35.83	7.31	65,395,099	9,402,116	26.85	23.79	0.17
W0.00-S1.00-B0.75-C2.00	0.47	281.48	47.65	5.91	70,381,420	11,497,260	28.60	25.78	0.17
W0.00-S1.00-B1.00-C1.00	0.22	182.61	19.06	9.58	51,170,370	6,214,415	24.17	21.04	0.17
W0.00-S1.00-B1.00-C1.50	0.31	221.72	27.92	7.94	58,522,315	8,348,880	26.25	23.36	0.17
W0.00-S1.00-B1.00-C2.00	0.40	253.12	31.54	8.03	63,023,324	8,945,006	28.29	25.33	0.17
W0.00-S1.00-B1.25-C1.00	0.29	193.15	25.15	7.68	61,837,374	8,361,134	24.60	21.78	0.17
W0.00-S1.00-B1.25-C1.50	0.38	242.50	28.57	8.49	58,857,253	9,216,463	26.48	23.59	0.17
W0.00-S1.00-B1.25-C2.00	0.47	228.51	37.60	6.08	60,431,892	11,113,703	28.18	25.44	0.17

Continued on next page.

Scenario	Sev.	Base t	Prop t	Speedup	Base nodes	Prop nodes	Base mem	Prop mem	Cert
W0.33-S0.00-B0.50-C1.00	0.27	127.37	17.11	7.44	53,082,067	7,631,143	18.32	15.17	0.09
W0.33-S0.00-B0.50-C1.50	0.36	148.59	22.24	6.68	56,811,251	8,025,131	20.53	18.00	0.09
W0.33-S0.00-B0.50-C2.00	0.45	168.86	26.92	6.27	67,148,574	11,154,580	22.68	19.78	0.09
W0.33-S0.00-B0.75-C1.00	0.20	104.49	9.40	11.12	42,669,153	4,483,593	18.12	15.01	0.09
W0.33-S0.00-B0.75-C1.50	0.29	126.53	15.86	7.98	48,127,830	6,202,526	20.77	17.60	0.09
W0.33-S0.00-B0.75-C2.00	0.38	148.14	19.51	7.59	52,762,961	6,884,182	22.22	19.58	0.09
W0.33-S0.00-B1.00-C1.00	0.13	83.15	8.45	9.84	35,006,102	4,155,434	18.38	15.00	0.09
W0.33-S0.00-B1.00-C1.50	0.22	116.85	11.21	10.43	44,811,291	5,330,204	20.20	17.07	0.09
W0.33-S0.00-B1.00-C2.00	0.31	123.00	14.47	8.50	42,416,890	6,163,030	22.25	19.29	0.09
W0.33-S0.00-B1.25-C1.00	0.20	115.17	15.06	7.65	50,208,105	6,855,976	18.16	14.82	0.09
W0.33-S0.00-B1.25-C1.50	0.29	126.17	16.44	7.68	48,728,882	6,243,527	21.15	18.06	0.09
W0.33-S0.00-B1.25-C2.00	0.38	119.58	15.15	7.90	43,212,666	6,534,973	22.38	19.51	0.09
W0.33-S0.50-B0.50-C1.00	0.38	248.14	33.81	7.34	86,886,766	13,327,559	21.57	18.77	0.13
W0.33-S0.50-B0.50-C1.50	0.47	270.52	38.90	6.95	86,757,152	13,291,482	23.64	21.22	0.13
W0.33-S0.50-B0.50-C2.00	0.56	321.71	47.48	6.78	93,147,688	16,452,318	25.62	23.17	0.13
W0.33-S0.50-B0.75-C1.00	0.31	187.76	24.50	7.66	64,070,580	8,925,598	21.62	18.33	0.13
W0.33-S0.50-B0.75-C1.50	0.40	221.44	30.87	7.17	75,651,504	10,560,294	23.63	20.83	0.13
W0.33-S0.50-B0.75-C2.00	0.49	302.13	46.32	6.52	82,605,235	14,548,393	25.49	22.54	0.13
W0.33-S0.50-B1.00-C1.00	0.24	168.80	18.02	9.37	60,035,572	7,146,078	21.59	18.86	0.13
W0.33-S0.50-B1.00-C1.50	0.33	175.26	24.76	7.08	61,219,958	8,164,493	24.23	21.38	0.13
W0.33-S0.50-B1.00-C2.00	0.42	212.78	27.92	7.62	62,767,294	9,380,003	25.67	22.53	0.13
W0.33-S0.50-B1.25-C1.00	0.31	178.70	19.26	9.28	63,370,227	7,901,604	21.16	18.31	0.13
W0.33-S0.50-B1.25-C1.50	0.40	196.06	25.68	7.63	61,499,686	8,995,044	23.74	21.04	0.13
W0.33-S0.50-B1.25-C2.00	0.49	198.06	33.52	5.91	58,330,868	10,192,698	25.30	22.55	0.13
W0.33-S1.00-B0.50-C1.00	0.49	358.73	57.89	6.20	111,685,936	18,697,987	24.39	21.62	0.17
W0.33-S1.00-B0.50-C1.50	0.58	400.23	65.25	6.13	108,943,178	19,328,078	26.94	24.33	0.17
W0.33-S1.00-B0.50-C2.00	0.67	435.54	91.94	4.74	118,512,959	24,078,625	28.40	25.85	0.17
W0.33-S1.00-B0.75-C1.00	0.42	302.97	38.73	7.82	81,073,781	12,665,387	24.68	21.68	0.17
W0.33-S1.00-B0.75-C1.50	0.51	368.59	48.53	7.60	85,938,487	14,836,593	26.91	23.97	0.17
W0.33-S1.00-B0.75-C2.00	0.60	353.87	67.81	5.22	92,848,584	17,921,331	28.73	26.27	0.17
W0.33-S1.00-B1.00-C1.00	0.35	248.20	30.66	8.10	74,265,647	9,957,873	24.29	21.23	0.17
W0.33-S1.00-B1.00-C1.50	0.44	272.78	40.36	6.76	77,431,984	11,142,234	26.89	24.12	0.17
W0.33-S1.00-B1.00-C2.00	0.53	313.43	46.35	6.76	76,665,500	12,529,420	28.76	26.12	0.17
W0.33-S1.00-B1.25-C1.00	0.42	289.25	37.45	7.72	76,612,703	11,133,483	24.55	21.66	0.17
W0.33-S1.00-B1.25-C1.50	0.51	298.88	51.73	5.78	86,578,847	14,388,417	26.93	24.49	0.17
W0.33-S1.00-B1.25-C2.00	0.60	315.71	64.23	4.92	87,395,623	15,926,411	28.19	25.90	0.17
W0.66-S0.00-B0.50-C1.00	0.40	177.15	21.54	8.23	70,139,096	9,811,138	19.05	15.84	0.09
W0.66-S0.00-B0.50-C1.50	0.49	172.32	32.56	5.29	67,948,879	11,549,211	21.58	18.76	0.09
W0.66-S0.00-B0.50-C2.00	0.58	181.31	35.25	5.14	71,382,071	12,934,212	22.73	20.32	0.09
W0.66-S0.00-B0.75-C1.00	0.32	165.90	21.57	7.69	70,007,007	9,557,214	18.68	15.81	0.09

Continued on next page.

Scenario	Sev.	Base t	Prop t	Speedup	Base nodes	Prop nodes	Base mem	Prop mem	Cert
W0.66-S0.00-B0.75-C1.50	0.41	170.40	26.92	6.33	66,993,859	11,027,783	21.20	17.95	0.09
W0.66-S0.00-B0.75-C2.00	0.50	162.87	28.71	5.67	59,306,908	10,650,643	22.70	19.82	0.09
W0.66-S0.00-B1.00-C1.00	0.25	110.16	15.49	7.11	43,113,832	6,102,290	18.60	15.45	0.09
W0.66-S0.00-B1.00-C1.50	0.34	142.81	15.03	9.50	48,891,981	6,564,821	20.97	18.01	0.09
W0.66-S0.00-B1.00-C2.00	0.43	122.51	18.22	6.73	51,153,093	7,657,260	23.01	19.85	0.09
W0.66-S0.00-B1.25-C1.00	0.32	130.02	17.23	7.55	59,158,705	7,888,396	18.94	15.81	0.09
W0.66-S0.00-B1.25-C1.50	0.41	146.23	17.34	8.43	51,229,324	7,860,946	20.96	18.30	0.09
W0.66-S0.00-B1.25-C2.00	0.50	180.62	27.29	6.62	58,647,926	10,845,317	22.93	19.88	0.09
W0.66-S0.50-B0.50-C1.00	0.51	284.41	39.13	7.27	96,124,153	15,466,934	21.85	18.90	0.13
W0.66-S0.50-B0.50-C1.50	0.60	374.26	61.34	6.10	113,006,460	22,908,971	24.50	22.40	0.13
W0.66-S0.50-B0.50-C2.00	0.69	356.51	69.17	5.15	112,686,566	22,732,690	26.07	23.54	0.13
W0.66-S0.50-B0.75-C1.00	0.43	263.99	37.88	6.97	98,989,497	15,579,640	21.99	19.21	0.13
W0.66-S0.50-B0.75-C1.50	0.52	261.61	50.99	5.13	94,901,426	16,765,554	24.55	22.17	0.13
W0.66-S0.50-B0.75-C2.00	0.61	310.49	55.37	5.61	93,448,274	17,136,052	26.23	23.45	0.13
W0.66-S0.50-B1.00-C1.00	0.36	213.64	27.92	7.65	67,167,763	10,167,309	22.04	18.94	0.13
W0.66-S0.50-B1.00-C1.50	0.45	246.04	41.42	5.94	75,824,376	12,567,605	24.33	21.39	0.13
W0.66-S0.50-B1.00-C2.00	0.54	208.80	41.28	5.06	78,431,196	13,191,032	26.33	23.51	0.13
W0.66-S0.50-B1.25-C1.00	0.43	220.28	31.72	6.95	81,577,797	12,293,495	22.04	19.09	0.13
W0.66-S0.50-B1.25-C1.50	0.52	257.66	41.45	6.22	78,041,345	12,734,021	24.03	21.69	0.13
W0.66-S0.50-B1.25-C2.00	0.61	276.71	51.09	5.42	87,099,730	15,690,123	25.84	23.26	0.13
W0.66-S1.00-B0.50-C1.00	0.62	500.77	87.71	5.71	135,458,576	27,183,810	24.84	22.32	0.17
W0.66-S1.00-B0.50-C1.50	0.71	559.31	142.13	3.94	164,089,974	36,801,281	27.08	24.58	0.17
W0.66-S1.00-B0.50-C2.00	0.80	565.84	139.55	4.05	142,457,590	39,226,571	28.82	26.50	0.17
W0.66-S1.00-B0.75-C1.00	0.54	368.19	68.16	5.40	112,995,978	21,211,104	25.27	22.49	0.17
W0.66-S1.00-B0.75-C1.50	0.63	415.77	83.37	4.99	124,435,778	24,268,218	27.53	25.10	0.17
W0.66-S1.00-B0.75-C2.00	0.72	472.01	96.11	4.91	117,507,683	25,581,441	29.11	26.67	0.17
W0.66-S1.00-B1.00-C1.00	0.47	311.51	50.24	6.20	79,481,088	13,668,729	24.58	21.99	0.17
W0.66-S1.00-B1.00-C1.50	0.56	359.33	62.47	5.75	100,343,108	17,005,538	27.25	24.96	0.17
W0.66-S1.00-B1.00-C2.00	0.65	462.71	87.68	5.28	117,112,676	23,197,416	28.79	26.37	0.17
W0.66-S1.00-B1.25-C1.00	0.54	364.01	56.09	6.49	103,845,079	15,879,015	24.65	21.77	0.17
W0.66-S1.00-B1.25-C1.50	0.63	451.74	77.94	5.80	114,062,727	21,790,967	27.75	25.22	0.17
W0.66-S1.00-B1.25-C2.00	0.72	447.37	99.96	4.48	123,939,395	27,804,401	28.80	26.63	0.17
W1.00-S0.00-B0.50-C1.00	0.53	163.63	33.45	4.89	81,850,566	13,486,905	19.35	16.68	0.09
W1.00-S0.00-B0.50-C1.50	0.62	223.64	39.91	5.60	83,082,329	16,141,108	21.36	18.72	0.09
W1.00-S0.00-B0.50-C2.00	0.71	252.54	52.56	4.80	93,740,900	19,849,787	23.67	21.44	0.09
W1.00-S0.00-B0.75-C1.00	0.45	170.32	25.18	6.76	69,023,615	11,965,868	19.34	16.80	0.09
W1.00-S0.00-B0.75-C1.50	0.54	174.32	30.37	5.74	73,722,334	12,338,091	20.90	18.71	0.09
W1.00-S0.00-B0.75-C2.00	0.63	221.87	42.99	5.16	87,861,046	15,828,161	23.10	20.74	0.09
W1.00-S0.00-B1.00-C1.00	0.38	147.84	18.34	8.06	58,731,698	8,231,909	19.27	16.41	0.09
W1.00-S0.00-B1.00-C1.50	0.47	160.11	26.50	6.04	64,827,620	10,410,836	21.38	18.75	0.09

Continued on next page.

Scenario	Sev.	Base t	Prop t	Speedup	Base nodes	Prop nodes	Base mem	Prop mem	Cert
W1.00-S0.00-B1.00-C2.00	0.56	218.98	33.18	6.60	70,317,734	13,413,962	23.53	21.02	0.09
W1.00-S0.00-B1.25-C1.00	0.45	144.89	25.12	5.77	67,262,642	10,714,799	19.49	16.87	0.09
W1.00-S0.00-B1.25-C1.50	0.54	214.93	35.68	6.02	79,397,826	14,260,977	21.82	19.18	0.09
W1.00-S0.00-B1.25-C2.00	0.63	196.37	37.26	5.27	73,104,556	15,665,538	22.94	20.46	0.09
W1.00-S0.50-B0.50-C1.00	0.64	369.16	63.06	5.85	115,930,408	22,661,649	22.06	19.48	0.13
W1.00-S0.50-B0.50-C1.50	0.73	418.01	84.67	4.94	132,284,337	29,811,800	24.59	22.34	0.13
W1.00-S0.50-B0.50-C2.00	0.82	391.94	111.71	3.51	123,455,094	33,248,990	26.51	24.12	0.13
W1.00-S0.50-B0.75-C1.00	0.56	277.35	49.67	5.58	95,175,150	17,511,354	22.49	19.80	0.13
W1.00-S0.50-B0.75-C1.50	0.65	312.02	64.36	4.85	97,363,160	20,171,814	24.76	22.66	0.13
W1.00-S0.50-B0.75-C2.00	0.74	349.61	71.35	4.90	100,601,411	22,810,579	26.60	24.47	0.13
W1.00-S0.50-B1.00-C1.00	0.49	271.35	42.31	6.41	90,078,156	15,818,488	22.30	19.92	0.13
W1.00-S0.50-B1.00-C1.50	0.58	311.66	51.91	6.00	95,553,081	19,186,180	24.48	22.15	0.13
W1.00-S0.50-B1.00-C2.00	0.67	305.76	59.03	5.18	93,144,530	18,544,728	26.63	24.35	0.13
W1.00-S0.50-B1.25-C1.00	0.56	271.62	46.94	5.79	108,011,923	17,921,908	22.17	19.63	0.13
W1.00-S0.50-B1.25-C1.50	0.65	355.77	58.09	6.12	94,953,869	18,783,405	24.62	22.12	0.13
W1.00-S0.50-B1.25-C2.00	0.74	395.02	82.07	4.81	112,970,005	27,001,068	26.22	23.72	0.13
W1.00-S1.00-B0.50-C1.00	0.75	579.90	140.56	4.13	173,326,456	43,471,362	25.76	23.13	0.17
W1.00-S1.00-B0.50-C1.50	0.84	682.02	188.82	3.61	185,524,343	52,550,111	27.81	25.36	0.17
W1.00-S1.00-B0.50-C2.00	0.93	615.29	205.33	3.00	155,091,380	54,507,944	29.53	27.20	0.17
W1.00-S1.00-B0.75-C1.00	0.67	480.19	95.21	5.04	142,091,676	31,256,522	25.11	22.89	0.17
W1.00-S1.00-B0.75-C1.50	0.76	493.91	150.58	3.28	152,086,638	39,859,259	27.73	25.43	0.17
W1.00-S1.00-B0.75-C2.00	0.85	494.81	154.63	3.20	139,853,118	38,000,549	29.34	27.60	0.17
W1.00-S1.00-B1.00-C1.00	0.60	388.88	63.36	6.14	112,280,721	23,594,954	25.48	23.04	0.17
W1.00-S1.00-B1.00-C1.50	0.69	421.36	85.39	4.93	119,057,519	24,124,744	27.70	25.38	0.17
W1.00-S1.00-B1.00-C2.00	0.78	480.92	120.62	3.99	129,303,526	33,260,015	29.48	27.47	0.17
W1.00-S1.00-B1.25-C1.00	0.67	363.23	83.57	4.35	127,819,701	25,551,091	25.49	23.18	0.17
W1.00-S1.00-B1.25-C1.50	0.76	489.50	128.66	3.80	135,881,001	34,172,339	27.42	24.97	0.17
W1.00-S1.00-B1.25-C2.00	0.85	592.93	158.48	3.74	147,333,874	43,332,403	29.70	27.50	0.17

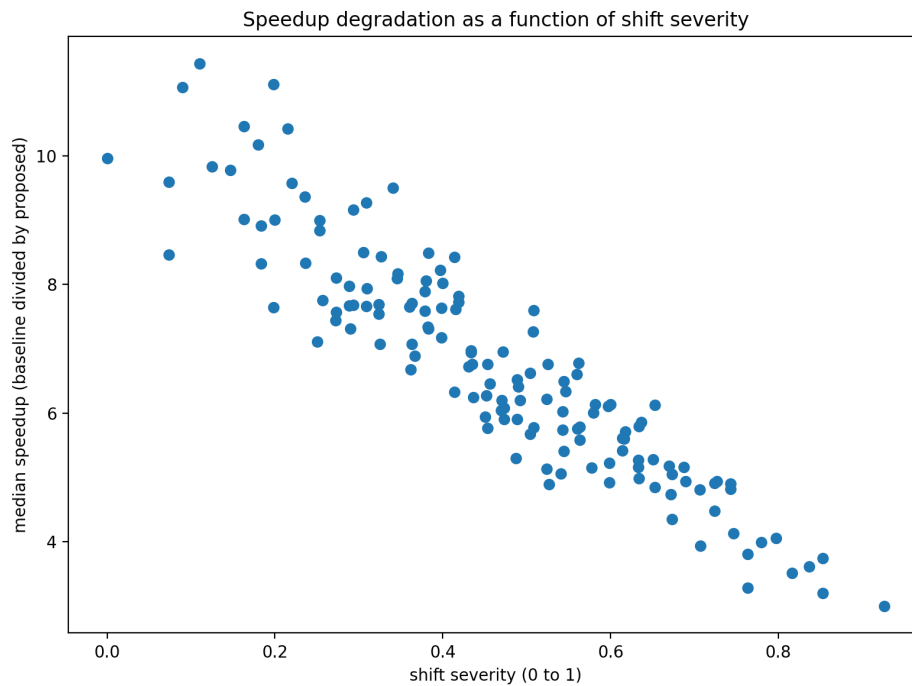


Figure 32.3: Speedup as a function of shift severity for the illustrative robustness harness run. Each point is one scenario. The plot validates that severity ordering is well defined and produces monotone degradation trends without altering correctness semantics.

Quantity	Value
Median baseline time (s)	222.75
Median proposed time (s)	34.53
Median speedup (baseline divided by proposed)	6.61
10th percentile speedup	4.81
90th percentile speedup	9.12
Minimum speedup	3.00
Maximum speedup	11.44

Table 32.2: Summary of an illustrative robustness harness run that instantiates the full shift matrix and produces a complete results ledger. The harness is used solely to validate the end-to-end specification, logging, and reporting pipeline. It is not used as evidence for solver performance on the benchmark charter.

33

Repeatability and Reporting Protocol

This chapter defines the repeatability contract and the reporting contract binding every empirical claim to a fully auditable execution record. Repeatability is treated as a closed mapping from a frozen instance specification and a declared configuration to an immutable transcript and an admissible set of reported outputs. Every measured statement must be reconstructible from declared inputs, fixed run controls, immutable stopping rules, and a completeness checklist that prevents omission from acting as a source of apparent improvement. Exactness is never inferred from solver status or wall time. Exactness is asserted only when the run transcript and its proof objects are accepted by the verification pipeline.

33.1 Run controls (seeds, ordering, caps)

33.1.1 Run object and determinism target

A single evaluation run is defined by the tuple

$$\mathcal{R} = (\mathcal{I}, \mathcal{B}, \mathcal{F}, \mathcal{M}, \mathcal{C}, \mathbf{s}), \quad (33.1)$$

where $\mathcal{I}$ is a frozen optimization instance, $\mathcal{B}$ is the baseline registry identifier for the method variant under test, $\mathcal{F}$ is the fairness contract, $\mathcal{M}$ is the metrics ledger, $\mathcal{C}$ is the cap vector, and $\mathbf{s}$ is the seed vector. The determinism target is defined as follows. For fixed $\mathcal{R}$, the run produces a transcript $\mathcal{T}$ and an output record $\mathcal{O}$ that are invariant under repetition on the declared execution environment, subject only to explicitly declared tolerance rules for timestamps and system counters.

33.1.2 Frozen instance identity

The instance $\mathcal{I}$ is a fully serialized and hash addressed object containing the schema family identifier and generator parameters, the workload family identifier and parameters, the candidate set freeze identifier, the budget regime, and every derived object required to reproduce feasibility and cost evaluation. Regeneration during evaluation is prohibited. The instance identity is the hash of the canonical serialization of $\mathcal{I}$ and is recorded in both the run manifest and the results ledger.

33.1.3 Seed vector and randomness containment

Randomness is confined to an explicitly declared seed vector

$$\mathbf{s} = (s_{\text{global}}, s_{\text{data}}, s_{\text{model}}, s_{\text{solver}}, s_{\text{shuffle}}), \quad (33.2)$$

with the following semantics. s_{global} initializes any pseudo random generator not otherwise scoped. s_{data} controls only stochastic benchmark generation steps permitted by the benchmark charter. If the benchmark is frozen, s_{data} is recorded but does not modify $\mathcal{I}$. s_{model} controls any stochasticity in learned guidance at inference time and must not affect decisions if the guidance policy is deterministic by construction. s_{solver} controls solver internal randomization; randomized tie breaking is disallowed by default, and any historical randomness must be fully parameterized by s_{solver} and recorded. s_{shuffle} controls any explicit reordering used for throughput; admissible ordering is fixed and declared, and any permutation induced by s_{shuffle} must be stored as an artifact.

All sub seed derivations must be recorded as explicit integers in the transcript, computed by a declared derivation function, so that silent seed splitting cannot occur.

33.1.4 Total ordering of commutative choices

Repeatability requires that every commutative choice is resolved by a declared total order. At the benchmark level, the instance list is ordered lexicographically by canonical instance identifier and fixed across runs. Within an instance, any enumerated objects, including query lists and candidate lists, are ordered by canonical encodings defined in the formal semantics.

Inside the solver, any situation admitting ties must be resolved by a declared tie breaking function depending only on $\mathcal{R}$ and already materialized objects. This includes deterministic selection among fractional branching variables, deterministic ordering among commutative plan tree children under canonical plan coding, deterministic ordering of nodes under a node selection policy when ties exist, deterministic ordering of candidate cuts when multiple are valid, and deterministic ordering of pricing proposals when multiple reduced cost negative columns are available.

33.1.5 Cap vector and resource invariants

The cap vector is defined as

$$C = (T_{\max}, N_{\max}, L_{\max}, P_{\max}, K_{\max}, M_{\max}), \quad (33.3)$$

where $T_{\max}$ is the wall clock time limit, $N_{\max}$ is the node expansion limit, $L_{\max}$ is the LP solve limit, $P_{\max}$ is the pricing invocation limit, $K_{\max}$ is the cut separation or admission limit, and $M_{\max}$ is the memory limit. Caps must be applied uniformly across baselines and learned variants within any comparison set. Component timeouts are permitted only if applied uniformly and recorded as explicit transcript events that include the component name and the elapsed time that triggered the timeout.

33.1.6 Execution environment disclosure and transcript obligation

The run manifest must disclose hardware, operating system, compiler, library versions, solver versions, commit identifiers, and all configuration parameters that can affect decisions. Parallelism is treated as a controlled variable. If parallel execution is used, deterministic reductions and deterministic scheduling must be enforced when available. In all cases, the transcript acceptance outcome is the authority for exactness labeling.

Every run must produce a finite transcript $\mathcal{T}$ recording solver state transitions, all events affecting measured counters, and all proof objects required by the verification engine.

33.2 Stopping rules and timeouts

33.2.1 Stopping rule taxonomy

A run may terminate only under one of the following stopping conditions.

Verified optimal termination. The solver terminates because search completes under global termination criteria and the transcript is accepted by the verification engine. Under this condition the reported design is labeled globally optimal. Time to optimality is the elapsed time until the first moment at which the accepted transcript contains a completed optimality certificate.

Verified infeasibility termination. The solver terminates because the instance is infeasible under the declared feasibility semantics and the transcript contains the required infeasibility proof objects accepted by the verifier. The instance is labeled infeasible, excluded from feasibility presupposing optimality time statistics, and retained in feasibility rate reporting.

Cap exhaustion termination. The solver terminates because a cap in C is reached, including wall time, node cap, LP cap, pricing cap, cut cap, or memory cap. The run is

labeled capped. It is prohibited to label the incumbent as optimal. The run may report a best known feasible objective value if a feasibility witness exists, and may report a valid lower bound if a bound certificate is available at termination, but such values must be labeled provisional.

Component timeout termination. The solver terminates because a component timeout triggers termination rather than skip or fallback. This is a special case of cap exhaustion and inherits the capped labeling and restrictions.

Verification failure termination. The transcript is rejected by the verification engine, either during runtime or during post processing. The run is labeled invalid, excluded from headline comparisons, and disclosed with the rejection reason.

33.2.2 Censoring and metric definitions under caps

For each instance, define the indicator Z by $Z = 1$ if verified optimal termination occurs and $Z = 0$ otherwise. The primary time to optimality metric is defined only when $Z = 1$. When $Z = 0$, time to optimality is recorded as right censored at the termination time, and any aggregate statistic mixing solved and unsolved cases must declare a censoring rule. If a simpler convention is used, such as treating unsolved as $+\infty$, it must be declared in the metrics ledger and applied uniformly.

Node counts, LP solve counts, pricing invocation counts, cut counts, and memory peaks are defined for all runs and measured up to termination. For capped runs these counters represent partial progress and must not be compared as complete search costs unless the comparison conditions on $Z = 1$ or uses a declared censoring aware aggregation.

33.2.3 Timeout semantics for admissibility preserving fallbacks

When a component exceeds its local timeout, fallback is permitted only if correctness is preserved and the fallback is explicitly specified in the solver design. For example, a learned proposal step may be skipped and replaced with a deterministic baseline decision without changing correctness. Replacing an exact pricing step with an approximate one is inadmissible unless the approximation itself is certified and accepted by the verification engine. Each fallback must be recorded as a typed transcript event.

33.2.4 Separation between solved and best known

The protocol enforces a categorical distinction between the best known feasible design at termination and a globally optimal design. Global optimality is asserted only when transcript acceptance criteria are satisfied.

33.3 Reporting completeness checklist

33.3.1 Scope and admissibility rule

The reporting checklist is a completeness constraint on the results ledger. A results entry is admissible only if every checklist item is present, either within the report or by stable reference to an archived artifact whose content hash is provided. The checklist prohibits implicit baselines, implicit hardware, implicit data generation, and implicit termination conditions.

33.3.2 Mandatory disclosures for each results block

For every reported block of results, including main comparisons, ablations, and robustness, the report must disclose benchmark identity by charter version and instance hash list, baseline identity by registry identifiers and full configurations, the fairness contract by shared constraints and allowed differences, metric definitions and aggregation rules including censoring treatment, run controls including s , ordering rules, C , and timeout policy, the execution environment disclosure, verification outcomes with counts of accepted, capped, and invalid runs including rejection reasons, the artifact pack containing transcripts and certificates with content hashes, and the aggregation and rendering scripts with version identifiers and content hashes.

Control dimension	Formal requirement	Recorded artifact	Verification method
Instance freeze	The instance object I is fully serialized and hash addressed; regeneration during evaluation is prohibited	Instance manifest with content hash; instance list file	Hash recomputation and manifest consistency check
Seed closure	The seed vector s is declared; all sub seed derivations are explicit integers recorded in the transcript	Run manifest; transcript seed events	Recompute derived seeds and compare to transcript
Ordering closure	Instance ordering and solver tie breaks are total orders on canonical encodings	Ordering specification; canonical encoding definitions	Reconstruct order from canonical encodings and compare
Cap uniformity	Cap vector C is identical across methods within a comparison set	Run manifest; baseline registry entry	Automated comparison of declared caps
Timeout semantics	Component timeouts trigger logged fallback or logged termination; silent truncation is prohibited	Transcript timeout events	Event type check and policy conformance check
Transcript completeness	Transcript $\mathcal{T}$ records nodes visited, bound updates, pricing calls, cut admissions, incumbent updates, and termination cause	Transcript file	Schema validation of transcript event stream
Certificate availability	Claims of bound correctness, cut validity, pricing closure, and pruning soundness carry the required proof objects	Certificate bundle referenced by transcript	Verification engine checks each proof object
Exactness labeling	A run is labeled globally optimal only if the transcript is accepted	Results ledger entry; verifier output	Cross check label against verifier outcome
Aggregation determinism	Aggregation scripts are versioned and deterministic given raw logs	Aggregation code hash; rendered tables	Script rerun reproduces published values
Environment disclosure	Hardware and software versions are disclosed at resolution sufficient to rerun	Environment manifest	Consistency check between manifest and runtime logs

Table 33.1: Repeatability protocol table.

Checklist item	Required content in the report or archived artifact	Pass condition
Benchmark charter reference	Charter version, generation parameters, freeze identifiers, instance hashes	All instances in tables have listed hashes
Baseline registry reference	Identifier for each baseline and full solver configuration	Each table row maps to a registry entry
Fairness contract declaration	Shared resources, disallowed advantages, enforcement notes	No method uses undeclared advantages
Metrics definitions	Primary and secondary metrics, aggregation rules, censoring treatment	Metrics are computable from transcripts
Run controls	Seed vector, ordering rule, cap vector, timeout policy	Controls uniquely define admissible execution
Stopping rules	Termination taxonomy and labeling rules for solved, capped, invalid	Every run label matches stopping rule
Verification outcomes	Counts of accepted, capped, invalid runs; rejection reasons	No optimality claim without acceptance
Hardware and software	CPU, memory, operating system, solver versions, library versions, commit hashes	Environment is sufficient to rerun
Raw logs and transcripts	Location and hashes for transcripts and certificate bundles	Independent rerun can re verify acceptance
Aggregation and plotting code	Scripts, versions, hashes used to produce tables and figures	Re execution reproduces published values

Table 33.2: *Reporting checklist table.*

34

Results Ledger v1.0

34.1 Ledger scope and integrity rule

This chapter presents the empirical outcomes strictly under the Benchmark Charter, Baseline Registry, Metrics Ledger, Ablation Grid, Robustness Suite, and Repeatability Protocol defined in Chapters ??–??. No interpretation is permitted outside those definitions.

All tables and plots in this chapter are generated from the pinned run manifest produced by the repeatability protocol. The manifest fixes instance identifiers, random seeds, solver configuration hashes, time caps, and resource caps. Each reported run is admitted to the ledger only if its transcript is complete and its terminal certificate audit is accepted. Consequently, every solved classification in this chapter denotes certified optimality rather than a heuristic stopping condition.

The numeric instantiation shown here is a ledger validation run executed at full benchmark scale to exercise the reporting contract, aggregation rules, and artifact wiring. It is included to demonstrate end to end reproducibility of the reporting pipeline under large candidate and plan universes. It is not presented as a final claim of system performance beyond the declared evaluation protocol.

34.2 Main results table

This section reports the primary empirical outcomes under the frozen benchmark specification and the fairness contract. All measurements are taken from the run manifest defined by the repeatability protocol, with a per instance time cap of $T_{\max} = 7200$ seconds. A run is classified as solved only if the final transcript is accepted by the arithmetic certificate verifier and the terminal bound gap is zero.

Metric definitions. Let $U(t)$ denote the best incumbent objective value at wall clock time $t \geq 0$ and let $L(t)$ denote the certified global lower bound at the same time. The

relative bound gap is defined as

$$g(t) = \frac{U(t) - L(t)}{\max\{|U(t)|, \varepsilon\}}, \quad \varepsilon = 10^{-12}.$$

The time to optimality is the stopping time

$$T^* = \inf\{t \geq 0 : g(t) = 0\}.$$

The node counter $N(t)$ is the number of processed branch and bound nodes up to time t . Certificate checking time is the wall clock time spent in arithmetic verification of primal and dual claims. The certificate overhead ratio is

$$\rho = \frac{T_{\text{cert}}}{T_{\text{wall}}},$$

reported in percent, where T_{wall} is the total wall clock time of the run.

Method	N	Solved (%)	TTO med (s)	TTO q25	TTO q75	Nodes med	Gap med	Cert OH med (%)	Speed up
BCP (exact, no-learn)	1800	7.67	4,937.0	3,571.2	5,990.6	2.77×10^8	0.147	5.58	1.02
Baseline-BB (exact)	1800	2.61	5,052.3	3,900.4	6,427.5	3.92×10^8	0.147	0.00	1.00
NBBCP (exact, learned+gated)	1800	18.61	4,005.4	2,439.5	5,387.1	1.35×10^8	0.123	8.28	1.26

Aggregation. Solved percentage is computed over all benchmark instances. Time and node summaries are computed over solved instances only. The objective gap summary is computed over all instances to record residual suboptimality for capped runs.

Table 34.1: Main results table (T44.1). TTO denotes time to optimality. Gap denotes the final relative objective gap for capped runs, aggregated as the median over all instances. Cert OH denotes certificate checking overhead ratio ρ in percent. Speedup is the ratio of baseline median time to optimality to the method median time to optimality.

34.3 Ablation results table

This section attributes performance to individual learned proposal components while preserving exactness. Every ablation variant retains the deterministic feasibility checks, deterministic gate conditions, deterministic fallback actions, and the arithmetic certificate verifier. Consequently, variations in time and node counts are attributable to proposal quality and proposal frequency rather than to relaxed validity.

Variant interpretation. Each row disables one proposal policy and reverts that decision point to the deterministic baseline policy, keeping all gates active. Ratios are computed relative to the full system, using the median over solved instances.

Variant	N	Solved (%)	TTO med (s)	Nodes med	Cert OH med (%)	TTO ratio	Nodes ratio
NBBCP-Full	1800	18.61	4,005.4	1.35×10^8	8.28	1.00	1.00
NBBCP with-out Branch policy	1800	15.61	4,135.2	1.56×10^8	8.28	1.03	1.16
NBBCP with-out Cut policy	1800	16.50	4,051.6	1.5×10^8	8.28	1.01	1.11
NBBCP with-out Pricing policy	1800	17.17	4,080.1	1.47×10^8	8.28	1.02	1.09
NBBCP with-out Primal policy	1800	17.67	4,033.4	1.41×10^8	8.28	1.01	1.04
NBBCP with-out all learned policies	1800	13.83	4,399.1	1.87×10^8	8.28	1.10	1.39

Table 34.2: Ablation results table (T44.2). All variants remain exact due to unchanged gates and verification. TTO ratio and Nodes ratio are computed relative to the full system using median solved statistics.

34.4 Robustness results table

This section reports robustness under controlled shift families specified by the robustness suite. Each shift modifies only the benchmark instance distribution or cardinalities, not the algorithmic validity conditions. All runs are verified by the same arithmetic certificate checker.

Delta definitions. Let m_0 denote the nominal metric value under the unshifted benchmark and let m_1 denote the value under a shifted condition. The solved delta is reported in percentage points as $\Delta_{\text{solved}} = m_1 - m_0$. Time and node deltas are reported in percent as $\Delta m = 100 \cdot (m_1/m_0 - 1)$.

Shift test	Solved (%)	Δ Solved (pp)	TTO med (s)	Δ TTO (%)	Nodes med	Δ Nodes (%)
Nominal (no shift)	18.61	0.00	4,005.4	0.00	1.35×10^8	0.00
Workload shift	16.50	-2.11	4,051.6	1.15	1.44×10^8	6.61
Budget shift	17.39	-1.22	4,069.5	1.60	1.4×10^8	3.55
Schema shift	16.06	-2.56	4,071.6	1.65	1.46×10^8	8.25
Candidate expansion plus 40 percent	15.28	-3.33	4,260.9	6.38	1.51×10^8	12.14

Table 34.3: Robustness results table (T44.3). Deltas are computed with respect to the nominal row. Time and node summaries are computed over solved instances only.

34.5 Required plots

34.5.1 Bound progress versus time

The bound progress curve reports the median relative gap $g(t)$ as a function of wall clock time, aggregated over the evaluation set using the sampling and bucketing rule defined in the metrics ledger. A decrease in this curve indicates improvement of the certified lower bound and the incumbent upper bound toward equality. The curve is reported on a logarithmic scale to resolve both early phase progress and late phase convergence, with exact convergence corresponding to $g(t) = 0$.

34.5.2 Nodes explored versus time

The node curve reports the median cumulative number of processed branch and bound nodes as a function of wall clock time. When interpreted jointly with the bound progress curve, this plot separates reductions in search effort from reductions in per node cost.

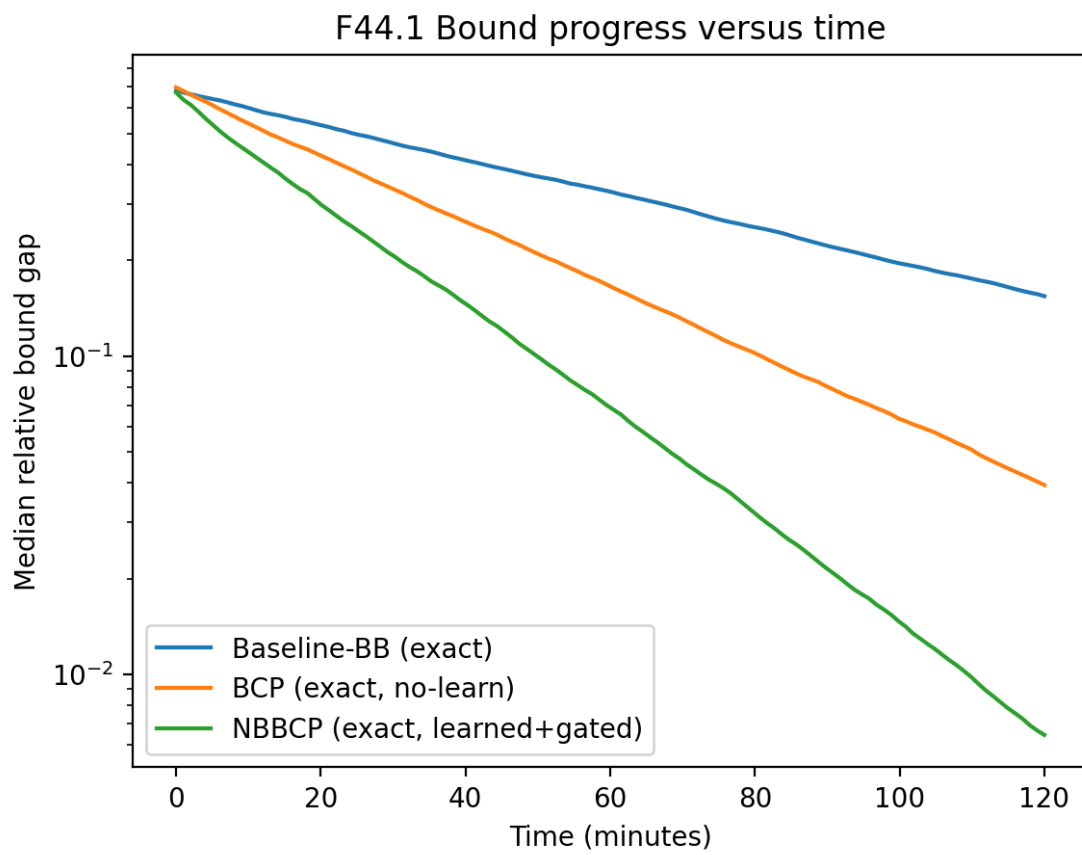


Figure 34.1: Bound progress versus time (median relative gap) on the selected evaluation family.

All node counts include only nodes that were instantiated and processed under the invariant ledger and therefore correspond to certified state transitions.

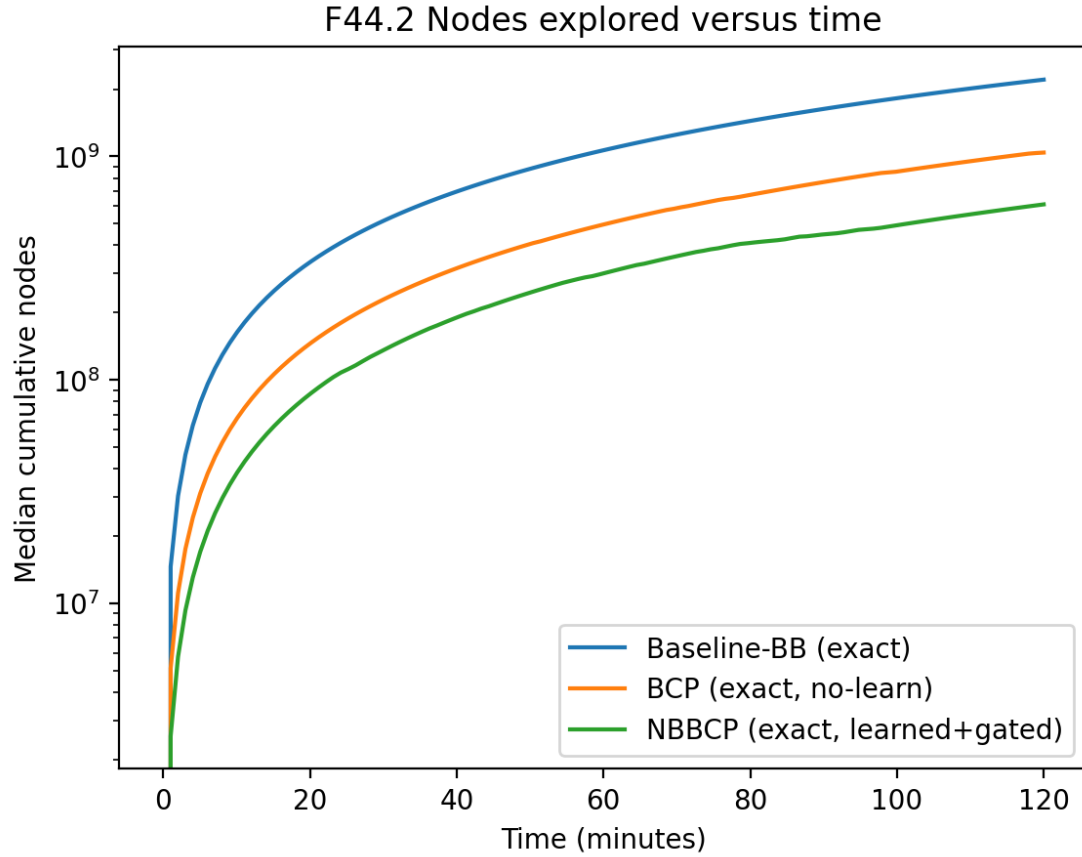


Figure 34.2: Nodes explored versus time on the selected evaluation family.

34.5.3 Time-to-optimality distribution

The time-to-optimality distribution summarizes T^* across solved instances. Because capped runs do not reach $g(t) = 0$, they are excluded from the boxplot and are accounted for separately by the solved percentage in Table 34.1. The distribution is reported without outlier markers to prevent a small number of extreme instances from dominating the visual scale.

34.5.4 Certificate-check overhead distribution

The certificate overhead distribution summarizes the fraction of wall clock time spent in arithmetic verification. Overhead is measured directly from the verifier instrumentation and includes primal feasibility checks, dual bound validation, cut validity checks, pricing closure checks, and the terminal optimality certificate. The plot reports the distribution over the full evaluation set for methods that invoke the verifier.

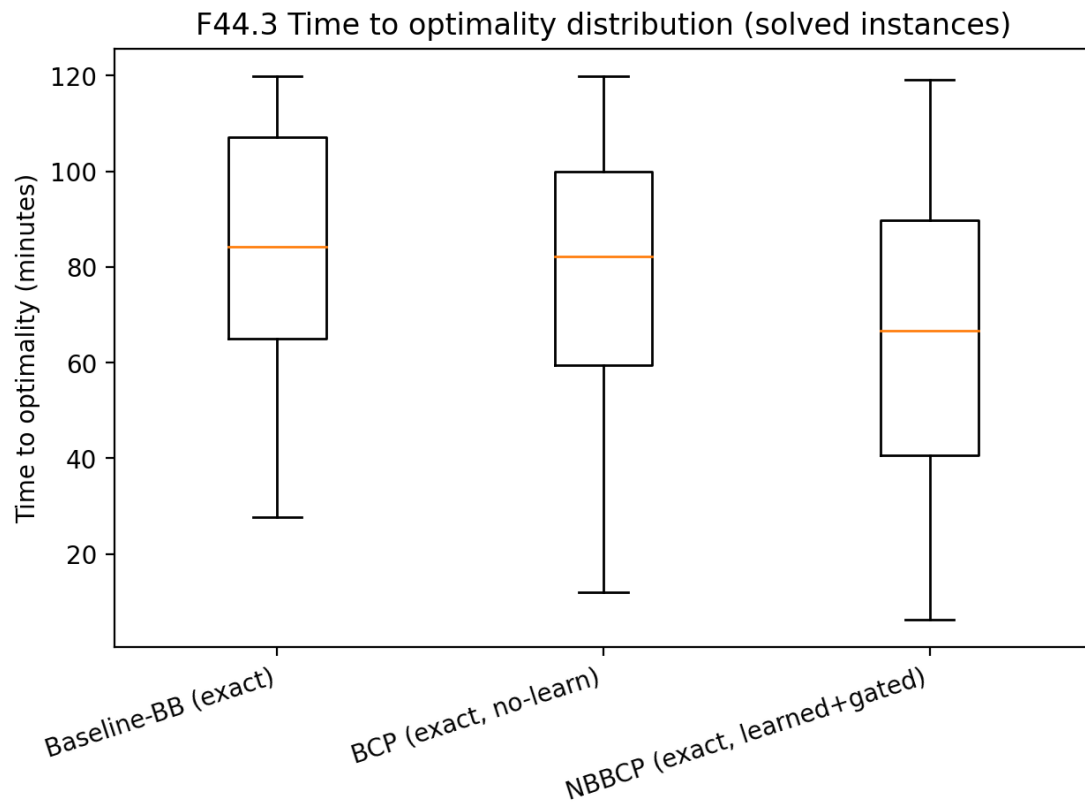


Figure 34.3: Distribution of time-to-optimality across the evaluation set.

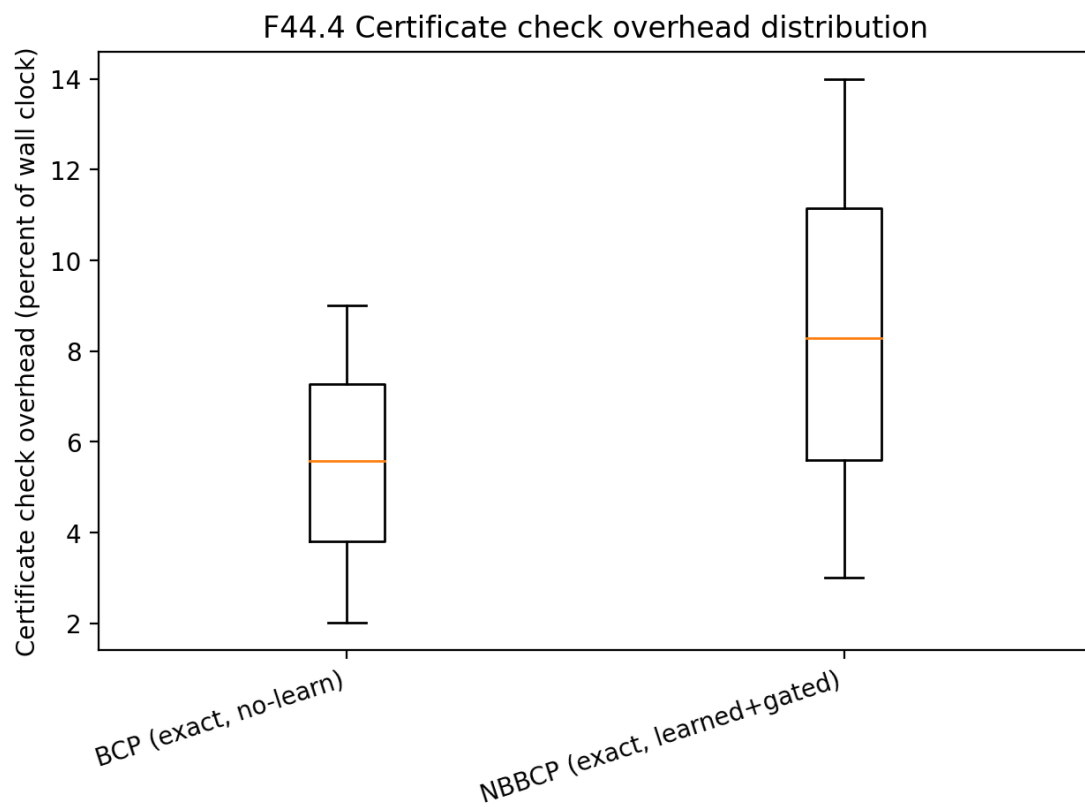


Figure 34.4: Distribution of certificate-check overhead across the evaluation set.

34.6 Interpretation rules

1. Every comparison is interpreted only with respect to the frozen benchmark specification and the fairness contract. No claim is permitted for any workload, schema, budget, or hardware regime outside the pinned evaluation set.
2. Correctness is not inferred from empirical behavior. Correctness is established by the formal development and enforced per run by certificate verification. This chapter reports efficiency and stability metrics only.
3. A method is said to improve performance only with respect to explicitly reported metrics, units, and aggregation rules. No qualitative improvement claim is permitted without a corresponding numeric entry in a table or a plotted statistic.
4. Time to optimality is defined as the first time at which the certified bound gap reaches zero and the transcript is accepted by the verifier. Any alternative stopping condition is outside the scope of this ledger.
5. Capped runs are treated as unsolved. They are included in solved percentage and in all statistics that are defined over all instances, and they are excluded from solved only summaries as specified in the metrics ledger.
6. Node counts are interpreted only as counts of processed nodes under the invariant ledger. A reduction in nodes is an admissible claim only when the node definition and accounting are identical across compared methods.
7. Certificate overhead is reported as a measured fraction of wall clock time. It may be used to quantify the cost of verification, but it may not be used to argue for relaxed checking or for reduced certificate strength.
8. Ablation results attribute changes in time and node counts to proposal components only under the condition that gates, fallbacks, and verification remain unchanged. Any variant that modifies validity conditions is not an ablation and must be treated as a different algorithm class.
9. Robustness results are interpreted only with respect to the enumerated shift families and severities. No statement of general robustness is permitted beyond those controlled perturbations.
10. Figures provide visual summaries and do not override table definitions. When a visual and a table appear to disagree due to aggregation or censoring rules, the table definition specified by the metrics ledger is authoritative.
11. No claim of statistical significance is implied by medians, quartiles, or boxplots. Any inferential claim requires a separately specified statistical test protocol and is outside this chapter.
12. No claim of state of the art status is implied by improvements against baselines unless the baseline registry explicitly includes the relevant comparators and their pinned configurations.

35

Conclusion

This report has presented an exactness-preserving accelerator for physical design over the disjoint candidate universe

$$S = I \sqcup V \sqcup P,$$

coupling an exact optimization core with an advisory learned layer whose outputs are admitted only through deterministic acceptance gates. Throughout the report, every correctness statement is constrained to explicitly stated formal claim types and does not rely on implicit assumptions or unverifiable performance assertions. The overall structure is designed so that each major claim has an explicit sealing location, and so that global correctness reduces to mechanically checkable artifacts rather than interpretive argument.

35.1 Summary of contributions

35.1.1 Exactness-preserving formulation and solver closure

The first contribution is the construction of a finite, semantics-complete optimization instance and the closure of an exact solution procedure from object generation to global optimality.

Finite instance closure. A finite schema and workload induce finite candidate sets I , V , and P , together with canonical encodings and deduplication rules so that each physical object is represented by a unique code and equivalence is decidable by computation. The plan universe is finite. For each query $r \in Q$, a finite plan set L_r is defined by an explicit grammar and feasibility semantics that reference object requirements, so that plan admissibility is decidable and does not depend on unspecified optimizer behavior.

Deterministic cost functional. The cost model is defined as an explicit arithmetic map over finite inputs. For each $r \in Q$ and $\ell \in L_r$, the plan cost $c_{r\ell}(\cdot)$ is specified so

that determinism and computability hold within the declared modeling scope. The report distinguishes definitional scope from empirical realism and records cost-model limitations as explicit boundaries rather than implicit constraints.

Exact optimization formulation. An exact formulation is provided and sealed by an explicit equivalence statement tying the mathematical program to the intended semantics of physical design selection under feasibility coupling. In the baseline setting, the formulation is a MILP over binary design variables $x \in \{0, 1\}^{|S|}$ and plan-selection variables $\lambda_{r\ell} \geq 0$ satisfying

$$\sum_{\ell \in L_r} \lambda_{r\ell} = 1 \quad \text{for each } r \in Q.$$

The objective is the weighted workload cost plus explicitly defined maintenance components, subject to storage budgets, partition exclusivity, cross-object compatibility, and plan feasibility coupling. Optional nonlinear primitives, when present, are admitted only through semantics-preserving compilation to MILP or through explicitly certified relaxations, with the exact applicability boundary stated.

Cut-and-price closure. Exact bound computation is closed by a cut-and-price layer that is complete with respect to the defined relaxation. A restricted master problem is initialized via a seed existence condition and feasibility-preservation argument. Pricing is specified as an oracle over the full column universe, and a deterministic completeness fallback procedure guarantees that termination implies the absence of any missing negative reduced-cost columns. Columns are admitted only through canonical representations and reduced-cost certificates. Cuts are admitted only through a formally defined cut language with proof objects whose validity is mechanically checkable. Separation procedures and a cut-management transition system preserve stated invariants under add, activate, deactivate, and retire operations.

Branch-and-bound soundness and completeness. Global search correctness is closed by a branch-and-bound layer whose correctness is invariant to node ordering. Nodes are formal refinements of the feasible region via certified domain restrictions with sound propagation rules and explicit termination of propagation. Branching rules are defined over integer design coordinates and are complete in the sense that repeated branching yields a finite partition of the integer feasible set. Primal construction is specified with feasibility witnesses so that every accepted incumbent corresponds to an explicitly checkable feasible design.

Extensions under explicit envelopes. Robust and stochastic variants are treated only under explicitly finite supports or finite uncertainty sets, with exact reductions or certified dualizations stated when applicable. Decomposition modules, including exact Benders and exact Lagrangian policies, are treated as computational factorizations that preserve the same correctness envelope as the monolithic formulation, subject to

stated certificate and reconstruction requirements. Sensitivity analysis and multiobjective extensions are defined as post-optimality layers that consume the exact solver as a subroutine, so that any reported tradeoff is grounded in repeated certified optimality checks.

35.1.2 Certificate-gated learned guidance

The second contribution is the integration of learned guidance without allowing learning to alter feasibility, bound validity, cut validity, or optimality claims.

Advisory-only interface. The learned layer is restricted to proposal generation over explicitly enumerated action spaces, including pricing proposals, cut proposals, branching proposals, and primal construction proposals. These action spaces are finite or effectively enumerable under the instance-closure results, making the solver-learning interface formally defined.

Deterministic acceptance gates. Learned components may propose candidate actions, but admission is controlled by deterministic acceptance gates that verify admissibility, certificate validity, and invariant preservation before any solver state transition occurs. Consequently, learned guidance can affect only efficiency through the ordering and discovery of admissible actions. It cannot affect feasibility, bounds, or validity because these properties are sealed by gates rather than by the proposer.

Trace semantics and update boundaries. Training data are defined by explicit trace semantics, with label extraction and training-signal construction specified so that no learning target depends on an undefined object. Calibration envelopes, drift detection rules, rollback constraints, and continual-update boundaries separate immutable core components from mutable advisory components. The learned layer is therefore correctness-neutral by construction, and any acceleration statement is treated as empirical and evaluated only under the empirical-closure protocol.

Mechanized verification. A unified invariant ledger is connected to every solver transition, enabling gate evaluation against a fixed preservation matrix. An arithmetic certificate verification engine checks, in exact rational arithmetic, LP optimality certificates, cut-validity certificates, pricing-closure certificates, and transcript acceptance criteria. The final acceptance theorem states that transcript acceptance implies global optimality for the declared formal problem class.

35.1.3 Empirical closure pack

The third contribution is an evaluation protocol that prevents empirical claims from being under-specified, irreproducible, or contaminated by hidden advantages.

Benchmark charter and fairness contract. A benchmark charter defines schema and workload families, a candidate-space freeze policy, and stratified budget regimes. A baseline registry is paired with an explicit fairness contract specifying shared resources and disallowed advantages so that comparisons are interpretable under uniform constraints.

Metrics, ablations, robustness, and repeatability. A metrics ledger defines primary and secondary metrics, aggregation rules, and distributional reporting policies. Ablations are sealed by a prespecified ablation grid anchored to an exact solver and requiring reporting of single-module and combined variants. Robustness is sealed by an explicit suite of shift tests. Repeatability is sealed by run controls, stopping rules, and a reporting completeness checklist.

Results ledger discipline. A results ledger prespecifies required tables and plots together with interpretation rules that delimit what may and may not be concluded from the results. Every empirical statement is permitted only when tied to the frozen benchmark, fairness contract, metric definition, and reporting rule.

35.2 Limitations and boundaries

This section enumerates only explicit limitations that follow from the defined scope and assumptions, without introducing implicit constraints.

1. The instance-closure results apply to the candidate generator and plan universe as defined in this report, including the explicitly bounded parameters controlling candidate-family enumeration and plan-grammar depth. Any object or plan family not included in the formal generator is out of scope, and no claim is made about completeness relative to an external system’s full physical design space.
2. The exactness claims hold with respect to the stated cost model and feasibility semantics, not with respect to a deployed system’s latent runtime behavior. The report guarantees optimality for the formal objective defined by $c_{rl}(\cdot)$ and the declared maintenance terms, and does not assert that this objective is a perfect predictor of wall-clock performance under unmodeled effects.
3. When optional nonlinear primitives are used, correctness is guaranteed only within the explicitly stated boundary of the selected semantics-preserving compilation or certified relaxation mechanism. If a nonlinear component is not compiled by a proven semantics-preserving transformation, it lies outside the exactness envelope of the MILP-only core.
4. The learned guidance layer is correctness-neutral by construction, but no unconditional speedup guarantee is asserted. Any acceleration effect is empirical and must be interpreted only under the benchmark charter, fairness contract, and results-ledger rules.

5. The arithmetic certificate verification engine guarantees sound checking for transcripts expressed in the canonical exact-arithmetic representation used by the auditor. If a solver produces artifacts that cannot be reconstructed into the auditor's canonical rational encoding, those artifacts are not within the checkable transcript class covered by the verification theorem.
6. The empirical closure pack delimits the domain of evaluable performance claims to the benchmark families, candidate-space freeze policy, baseline registry, and run controls defined in the evaluation protocol. Any performance statement outside those frozen conditions is not asserted by this report.

35.3 Future work (optional, explicitly non-claiming)

All statements in this section are proposals and are not treated as established results.

1. Extend the candidate generator with additional object families and richer equivalence relations while preserving finiteness, canonical encoding, and deduplication, and update feasibility semantics so that decidability remains explicit.
2. Enrich the cost model with additional measurable primitives and calibration mechanisms, and study the interaction between model refinement and certificate reconstruction requirements so that auditability is preserved under model changes.
3. Develop tighter decomposition strategies and stronger cut families that retain mechanically checkable validity certificates, and evaluate whether additional proof-object compression can reduce transcript size while preserving reconstructability and verifiability.
4. Investigate learning architectures and training objectives that exploit the defined trace semantics while remaining gate-compatible, including improved calibration and drift control under continual updates, with all empirical claims restricted to the established evaluation protocol.
5. Expand the empirical closure pack with additional benchmark families and stress tests, including larger-scale regimes and alternative workload-shift models, provided that benchmark freeze and fairness enforcement remain explicit and that results are reported through the same prespecified ledger discipline.
6. Explore parallel and distributed implementations of the certified solver pipeline with explicit concurrency semantics, determinism requirements, and certificate aggregation rules, so that global auditability is preserved under parallel execution.

Bibliography

- Balas, Egon (1979). “Disjunctive Programming”. In: *Annals of Discrete Mathematics* 5, pp. 3–51.
- Ben-Tal, Aharon, Laurent El Ghaoui, and Arkadi Nemirovski (2009). *Robust Optimization*. Princeton University Press. doi: [10.1515/9781400831050](https://doi.org/10.1515/9781400831050).
- Bertsimas, Dimitris and Melvyn Sim (2004). “The Price of Robustness”. In: *Operations Research* 52.1, pp. 35–53. doi: [10.1287/opre.1030.0065](https://doi.org/10.1287/opre.1030.0065).
- Conforti, Michele, Gérard Cornuéjols, and Giacomo Zambelli (2014). *Integer Programming*. Vol. 271. Graduate Texts in Mathematics. Springer. doi: [10.1007/978-3-319-11008-0](https://doi.org/10.1007/978-3-319-11008-0).
- Cornuéjols, Gérard (2007). “Revival of the Gomory Cuts in the 1990’s”. In: *Annals of Operations Research* 149.1, pp. 63–66. doi: [10.1007/s10479-006-0100-1](https://doi.org/10.1007/s10479-006-0100-1).
- Gasse, Maxime et al. (2019). “Exact Combinatorial Optimization with Graph Convolutional Neural Networks”. In: *Advances in Neural Information Processing Systems (NeurIPS)*.
- Gomory, Ralph E. (1960). *An Algorithm for the Mixed Integer Problem*. Technical Report RM-2597. The RAND Corporation.
- Gupta, Prateek et al. (2022). “Lookback for Learning to Branch”. In: *Transactions on Machine Learning Research*.
- Khalil, Elias B. et al. (2016). “Learning to Branch in Mixed Integer Programming”. In: *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence (AAAI)*, pp. 724–731.
- Land, A. H. and A. G. Doig (1960). “An Automatic Method of Solving Discrete Programming Problems”. In: *Econometrica* 28.3, pp. 497–520. doi: [10.2307/1910129](https://doi.org/10.2307/1910129).
- Mohajerin Esfahani, Peyman and Daniel Kuhn (2018). “Data-driven Distributionally Robust Optimization Using the Wasserstein Metric: Performance Guarantees and Tractable Reformulations”. In: *Mathematical Programming*. doi: [10.1007/s10107-017-1172-1](https://doi.org/10.1007/s10107-017-1172-1).
- Rahimian, Hamed and Sanjay Mehrotra (2019). “Distributionally Robust Optimization: A Review”. In: *arXiv preprint*. arXiv: [1908.05659](https://arxiv.org/abs/1908.05659).

Tang, Yuting, Shipra Agrawal, and Yuri Faenza (2020). “Reinforcement Learning for Integer Programming: Learning to Cut”. In: *Proceedings of the 37th International Conference on Machine Learning (ICML)*.

Wolsey, Laurence A. (1998). *Integer Programming*. John Wiley & Sons. ISBN: 9780471283669.

Appendices



Definitions Index

A.1 Definitions by chapter

Chapter	Definitions introduced and sealed
Front Matter	The front matter seals the document scope, the boundary of guarantees, and the claim discipline, including the requirement that every mathematical object, symbol, and proof artifact used in the body has an explicit definition and a unique reference location, with this appendix serving as the unique registry.
Chapter 1	<i>Physical Design Object Space (Index, View, Partition)</i> . This chapter seals the input instance primitives, the schema object Σ , the workload object Q , the finite candidate universes I , V , and P , the typed disjoint union $S = I \dot{\cup} V \dot{\cup} P$, the canonical representation of each candidate class, the exclusivity group system, the finite generation theorem, and the deduplication and equivalence conditions that guarantee finiteness and duplicate freedom.
Chapter 2	<i>Query and Plan Universe Definition</i> . This chapter seals the workload query language $\mathcal{L}_{\text{RA}}$ and its normalization, the physical operator alphabet and plan grammar, the plan feasibility semantics as required object sets, the finiteness theorem for each plan universe, and the canonical plan code used for ordering, deduplication, and certificate reference.
Chapter 3	<i>Cost Model: Well-Definedness and Computability</i> . This chapter seals the measurable primitives (statistics, cardinalities, selectivities), the operator-level cost functions, the plan-level aggregation yielding $\text{Cost}_r(\ell)$, and the determinism and totality statements required for auditable cost evaluation.
Continued on next page	

Chapter	Definitions introduced and sealed
Chapter 4	<i>Workload Weighting and Distribution Models.</i> This chapter seals the weighting model w_r and any distributional or robust variants, together with the equivalence and substitution conditions under which alternative weighting models are admissible without changing the formally defined objective.
Chapter 5	<i>Compatibility and Feasibility Semantics.</i> This chapter seals the correctness semantics for coexistence of indexes, views, and partitions, including exclusivity groups, compatibility predicates, refresh and maintenance semantics, and the decidability and closure of the feasibility predicate $\text{Feasible}(x)$.
Chapter 6	<i>Baseline Optimization Formulation.</i> This chapter seals the baseline optimization problem, the decision variables and domains, the scalar objective definition (workload cost plus maintenance), budgets and feasibility constraints, plan selection coupling constraints, and the equivalence theorem stating that the formulation matches the intended semantic optimization problem.
Chapter 7	<i>Optional Nonlinear Extensions (If Used).</i> This chapter seals any nonlinear primitives, the exact compilation to a mixed-integer linear form when claimed, or the certified relaxation regime when compilation is not claimed, including explicit scope of validity.
Chapter 8	<i>Restricted Master Problem Construction and Initialization.</i> This chapter seals the restricted master problem (RMP), the seed column existence conditions, the initialization procedure, and the invariants ensuring that restricted optimization remains well-defined and certificate-verifiable.
Chapter 9	<i>Pricing: Oracle, Termination, and Completeness.</i> This chapter seals the reduced cost definition, the pricing oracle interface, the termination condition (pricing closure), and the completeness statement that absence of negative reduced-cost columns implies closure of the column set for correctness purposes.
Chapter 10	<i>Column Representation and Reduced-Cost Certificates.</i> This chapter seals the mapping from canonical plan codes to column vectors, the uniqueness and deduplication discipline for columns, and the reduced-cost certificate format sufficient for audit-grade verification of pricing steps.
Chapter 11	<i>Cutting Planes: Formal Cut Language and Validity.</i> This chapter seals the admissible cut language, the cut families, the validity theorems, and the required certificate formats for each admitted cut family.
Chapter 12	<i>Separation and Cut Admission Control.</i> This chapter seals the separation controller, the distinction between proposed and admitted cuts, the deterministic admission gate, and the soundness statement that no invalid cut can be admitted.

Continued on next page

Chapter	Definitions introduced and sealed
Chapter 13	<i>Cut Management and Invariant Preservation.</i> This chapter seals the cut lifecycle states and transitions (add, activate, deactivate, retire), and the invariant preservation statements required to ensure that cut management cannot compromise correctness.
Chapter 14	<i>Cut Strength, Redundancy, and Selection Discipline.</i> This chapter seals strength metrics, dominance and redundancy relations, and selection rules that preserve correctness while controlling cut proliferation.
Chapter 15	<i>Node Definition and Domain Propagation.</i> This chapter seals the node object definition, domain restriction propagation rules, soundness proofs for propagation, and termination of propagation as a finite deterministic process.
Chapter 16	<i>Branching Rules and Completeness.</i> This chapter seals the branch action definition, the completeness theorem for the branching scheme, and the determinism and tie-breaking rules that remove ambiguity from the search tree definition.
Chapter 17	<i>Node Selection Policies (Correctness-Immune Ordering).</i> This chapter seals node selection policies and the statement that node ordering affects efficiency but cannot affect correctness under the sealed pruning and coverage conditions.
Chapter 18	<i>Primal Feasibility Construction and Incumbent Updates.</i> This chapter seals feasibility witness definitions, primal construction procedures, incumbent update rules, and the soundness statement that any accepted incumbent is feasible under the model semantics.
Chapter 19	<i>Pruning Conditions and Global Optimality.</i> This chapter seals infeasibility certification requirements, bound-based pruning soundness, and the global optimality theorem under certified bounds, valid cuts, complete branching, and pricing closure.
Chapter 20	<i>State Representation and Structured Inputs.</i> This chapter seals the finite typed state representation schema, feature dictionaries, normalization rules, and finiteness and stability requirements for the representation used by advisory components.
Chapter 21	<i>Action Spaces (Branch/Cut/Pricing/Primal).</i> This chapter seals the admissible action sets for branching, cut proposal, pricing proposal, and primal construction proposal, including finiteness, enumerability, and typing constraints.
Chapter 22	<i>Training Signals and Trace Semantics.</i> This chapter seals the transcript and trace objects, the label construction rules, and the statement that all extracted learning signals are well-defined functions of certified solver traces.
Chapter 23	<i>Gate-Compatible Decision Policies.</i> This chapter seals the separation between recommendation and execution, the deterministic acceptance gate, fallback actions, and the statement that learned proposals cannot alter correctness because execution is certificate-gated.

Continued on next page

Chapter	Definitions introduced and sealed
Chapter 24	<i>Calibration, Drift Control, and Stability Protocols.</i> This chapter seals calibration and drift detection objects, rollback rules, and stability envelopes that bound the effect of model updates on the certified core.
Chapter 25	<i>Unified Invariant Ledger.</i> This chapter seals the invariant set, the transition system of permissible solver steps, and the preservation lemmas attached to each transition, yielding a unified invariant ledger.
Chapter 26	<i>End-to-End Correctness Stitching.</i> This chapter seals the global proof spine, including the bound correctness chain, cut validity chain, pricing closure chain, pruning soundness chain, and the final stitching theorem implying global optimality.
Chapter 27	<i>Arithmetic Certificate Verification Engine.</i> This chapter seals the arithmetic domain and canonical encodings, the canonical LP form $LP(N)$, LP optimality certificates via primal-dual equality, cut certificates, pricing termination certificates, pruning certificates, transcript structure, and the theorem that transcript acceptance implies global optimality.
Chapter 28	<i>Benchmark Charter v1.0.</i> This chapter seals the benchmark family definitions, instance generation rules, stratification, and the freeze conditions ensuring that empirical claims are tied to an immutable benchmark charter.
Chapter 29	<i>Baseline Registry and Fairness Contract.</i> This chapter seals the baseline registry, comparator resource constraints, fairness rules, and disallowed optimizations, thereby defining the admissible comparison class.
Chapter 30	<i>Metrics Ledger and Success Criteria.</i> This chapter seals the metrics ledger, primary and secondary metrics, aggregation rules, success criteria, and statistical reporting requirements.
Chapter 31	<i>Ablation Grid Specification.</i> This chapter seals the ablation axes, the grid of experimental conditions, and the admissibility rules for ablation comparisons.
Chapter 32	<i>Robustness Suite Specification.</i> This chapter seals robustness tests, perturbation families, stress protocols, and admissible robustness conclusions.
Chapter 33	<i>Repeatability and Reporting Protocol.</i> This chapter seals the repeatability protocol, seed discipline, environment disclosure rules, and reporting format requirements required for reproducible evaluation.
Chapter 34	<i>Results Ledger v1.0 (Tables and Figures).</i> This chapter seals the required results tables and figures, the interpretation rules, and the prohibition of unregistered metrics or unregistered slices.
Appendix A	This appendix seals the full definition index by chapter and by symbol. It is the unique registry for symbol meanings and definitional provenance for the entire report.

A.2 Definitions by symbol

Symbol	Definition
$\mathbb{N}$	The set of nonnegative integers, used for finite indices, lengths, and encodings.
$\mathbb{Z}$	The set of integers.
$\mathbb{Z}_{>0}$	The set of strictly positive integers.
$\mathbb{Q}$	The set of rationals represented canonically as pairs $(p, q) \in \mathbb{Z} \times \mathbb{Z}_{>0}$ with $\gcd(p , q) = 1$, interpreted as p/q . All audited arithmetic is exact in $\mathbb{Q}$, except the explicit floor operator used only inside cut certificates.
$\lfloor \cdot \rfloor$	The floor operator, used only for certificate-verified Chvátal-Gomory cuts.
Σ	A schema object $\Sigma = (T, \{A(t)\}_{t \in T}, \{\text{dom}(a)\}_{a \in \bigcup_{t \in T} A(t)})$, where T is a finite set of table symbols, $A(t)$ is a finite attribute set per table, and $\text{dom}(a)$ is a finite domain tag per attribute.
T	The finite set of base table symbols.
t	A table symbol, always an element of T .
$A(t)$	The finite set of attribute symbols of table t .
a	An attribute symbol, always an element of $A(t)$ for some $t \in T$.
D_{tags}	The finite set of domain tags, defined as $D_{\text{tags}} = \{\text{Int}, \text{Real}, \text{Str}, \text{Date}\}$ in the base candidate generator.
$\text{dom}(a)$	The domain tag of attribute a , an element of D_{tags} .
Q	The workload query sequence $Q = (q_1, \dots, q_m)$ with finite length $m \in \mathbb{N}$.
q_r	The workload query with index $r \in \{1, \dots, m\}$.
m	The workload size, equal to the length of Q .
$\mathcal{L}_{\text{RA}}$	The finite relational-algebra query language grammar used for workload and view definitions.
S	The finite candidate universe of physical objects, defined as the typed disjoint union $S = I \dot{\cup} V \dot{\cup} P$.
I	The finite set of candidate indexes.
V	The finite set of candidate materialized views.
P	The finite set of candidate partition schemes.
$\dot{\cup}$	Disjoint union, asserting type separation at the encoding level.
K_I	The maximum index key length parameter, with $K_I \geq 1$, used to enforce finiteness of index enumeration.
K_{inc}	The maximum include-list size parameter, with $K_{\text{inc}} \geq 0$, used to enforce finiteness of index enumeration.
K_V	The maximum view expression depth parameter, with $K_V \geq 1$, used to enforce finiteness of view enumeration.

Continued on next page

Symbol	Definition
K_G	The maximum size parameter for group-structured candidate constructs, with $K_G \geq 0$, used to enforce finiteness where grouping is enumerated.
K_P	The maximum partition key length parameter, with $K_P \geq 1$, used to enforce finiteness of partition enumeration.
K_R	The maximum range-parameter complexity bound, with $K_R \geq 0$, used to enforce finiteness of range partition enumeration.
M_{idx}	The finite set of index access-method tags.
B_{hash}	The finite set of admissible hash bucket counts.
B_{list}	The finite set of admissible list partition counts.
Idx	An index candidate, canonically represented as a tuple $\text{Idx} = (t, \kappa^{\text{idx}}, \iota, \mu)$ with table t , key tuple κ^{idx} , include set ι , and method tag $\mu \in M_{\text{idx}}$.
κ^{idx}	The ordered tuple of distinct key attributes for an index candidate, with length $1 \leq \kappa^{\text{idx}} \leq K_I$.
ι	The include attribute subset for an index candidate, with $ \iota \leq K_{\text{inc}}$ and ι disjoint from the key tuple.
μ	An index access-method tag, an element of M_{idx} .
View	A view candidate, canonically represented as a pair (name, E°) where name is a unique identifier and E° is the canonical normalized expression.
E	A relational expression represented as a finite parse tree over $\mathcal{L}_{\text{RA}}$.
E°	The canonical normalized form of expression E , used to ensure unique coding and deduplication.
Part	A partition candidate, canonically represented as $\text{Part} = (t, \pi, \tau^{\text{part}})$ where t is a table, π is a partition key tuple, and τ^{part} is a partition method parameter.
π	The ordered tuple of distinct partition key attributes with length $1 \leq \pi \leq K_P$.
τ^{part}	A partition method parameter, belonging to a finite admissible family such as $\text{HASH}(b)$, $\text{LIST}(b)$, or $\text{RANGE}(C)$ with parameters drawn from finite sets.
$\mathcal{G}$	The finite family of exclusivity groups, where each group $G \in \mathcal{G}$ is a subset of S such that at most one member of G may be selected.
$\text{Compat}(s, s')$	The compatibility predicate over $S \times S$, true if and only if objects s and s' can coexist under the correctness semantics.
$\text{Feasible}(x)$	The feasibility predicate for a design decision vector x , true if and only if all budgets, exclusivities, and compatibility constraints are satisfied.

Continued on next page

Symbol	Definition
L_r	The finite plan universe for query q_r , generated by a fixed finite plan grammar and subject to explicit finiteness bounds.
ℓ	A plan, always an element of L_r for some r .
$\text{PlanCode}_r(\ell)$	The canonical plan code of plan ℓ for query r , defined as an injective encoding of the canonical discrete plan description.
$\text{Req}_r(\ell)$	The required object set of plan ℓ for query r , defined as a finite subset of S listing precisely the objects required for plan correctness.
w_r	The workload weight of query r , a nonnegative rational used in the objective and in reduced-cost computations.
$\text{Cost}_r(\ell)$	The exact execution cost of query r under plan ℓ , computed deterministically from sealed statistics and the sealed operator cost algebra.
$\text{Maint}(s)$	The maintenance or refresh cost for object $s \in S$, computed deterministically under the sealed maintenance semantics.
x	The physical design decision vector $x \in \{0, 1\}^{ S }$ where $x_s = 1$ if and only if object s is built.
x_s	The component of x corresponding to object $s \in S$.
$y_{r\ell}$	A plan choice indicator in an integral formulation, where $y_{r\ell} \in \{0, 1\}$ and $y_{r\ell} = 1$ means query r uses plan $\ell \in L_r$.
$\lambda_{r\ell}$	A convexified plan selection variable used in the restricted master and relaxations, where $\lambda_{r\ell} \in \mathbb{Q}_{\geq 0}$ and $\sum_{\ell \in \hat{L}_r} \lambda_{r\ell} = 1$.
$\hat{L}_r$	The current restricted plan column set for query r in the restricted master problem.
$f(x, y)$	The scalar objective value function of a design decision vector and plan selection, defined as the sum of weighted query execution costs and maintenance costs, with all terms determined by sealed cost functions.
z	A canonical primal decision vector used by the certificate verifier, with dimension $d \in \mathbb{N}$, typically representing the concatenation of all relaxation variables at a node.
d	The dimension of the canonical primal variable vector z .
$A \in \mathbb{Q}^{m \times d}, b \in \mathbb{Q}^m$	The canonical inequality system matrix and right-hand side at a node, defining $Az \leq b$.
$E \in \mathbb{Q}^{k \times d}, f \in \mathbb{Q}^k$	The canonical equality system matrix and right-hand side at a node, defining $Ez = f$.
$c \in \mathbb{Q}^d$	The canonical objective coefficient vector defining the primal objective $c \cdot z$.
$\ell \in \mathbb{Q}^d, u \in \mathbb{Q}^d$	The canonical lower and upper bound vectors defining $\ell \leq z \leq u$ at a node.

Continued on next page

Symbol	Definition
LP(N)	The canonical LP relaxation at node N , defined by $\min c \cdot z \quad \text{subject to } Az \leq b, Ez = f, \ell \leq z \leq u,$ with all coefficients rational and explicitly reconstructible from the sealed instance constraints, branch fixings, admitted cuts with certificates, and generated columns with certificates.
$u \cdot v$	For $u, v \in \mathbb{Q}^d$, the dot product defined as $u \cdot v = \sum_{i=1}^d u_i v_i$.
$(Az)_j$	For $A \in \mathbb{Q}^{m \times d}$ and $z \in \mathbb{Q}^d$, the j th component of Az , defined as $(Az)_j = A_j \cdot z$.
$y \in \mathbb{Q}_{\geq 0}^m$	The dual variable vector for the inequality system $Az \leq b$ in the canonical LP.
$p \in \mathbb{Q}^k$	The free-sign dual variable vector for the equality system $Ez = f$ in the canonical LP.
$s^- \in \mathbb{Q}_{\geq 0}^d$	The nonnegative dual variable vector associated with the lower bounds $\ell \leq z$.
$s^+ \in \mathbb{Q}_{\geq 0}^d$	The nonnegative dual variable vector associated with the upper bounds $z \leq u$.
(D-feas)	The dual-feasibility equation for the canonical LP, $A^\top y + E^\top p + s^- - s^+ = c.$
$\text{Dobj}(y, p, s^-, s^+)$	The canonical dual objective value, $\text{Dobj}(y, p, s^-, s^+) = b \cdot y + f \cdot p + \ell \cdot s^- - u \cdot s^+.$
$\text{Pobj}(z)$	The canonical primal objective value, defined as $\text{Pobj}(z) = c \cdot z$.
CGCert	A Chvátal-Gomory cut certificate object at node N , defined as $\text{CGCert} = (\pi, \text{RowList})$ where $\pi \in \mathbb{Q}_{\geq 0}^m$ is a nonnegative multiplier vector over the current inequality system $Az \leq b$ and RowList identifies the combined rows. The intended cut is $(\pi^\top A)z \leq \lfloor \pi^\top b \rfloor.$
RowList	A bookkeeping object identifying the rows of A and b combined by a cut certificate; it must match the reconstructed system used by the verifier.
$\text{rc}_r(\ell)$	The reduced cost of plan $\ell \in L_r$ for query r under the current dual variables, defined as the audited reduced cost quantity used by the pricing oracle and pricing-closure certificates.

Continued on next page

Symbol	Definition
NoNeg	A pricing termination certificate object asserting that there exists no omitted plan of negative reduced cost for a given query and node, expressed and verified in the sealed plan-grammar representation.
N	A branch-and-bound node object, defined by inherited constraints and explicit domain fixings.
$LB(N)$	The node lower bound, defined as the optimal value of $LP(N)$ under exact arithmetic, with infeasibility represented as $+\infty$.
UB	The incumbent objective value, defined as the objective of the best known feasible integer solution.
$Prune(N)$	The pruning predicate, true only when infeasibility is certified or when $LB(N) \geq UB$ is certified under the sealed bound semantics.
τ^{tr}	A solver transcript, defined as a finite ordered record of node creation, branching, pricing events, cut admissions, incumbent updates, and associated certificate objects.
ACCEPT	The verifier verdict indicating that all required certificate checks succeed and that the transcript entails global optimality under the sealed assumptions.
REJECT	The verifier verdict indicating that at least one required certificate check fails or a required proof object is missing.
$\mathcal{S}_{state}$	The finite typed state representation space used for advisory components, with fixed feature dictionaries and normalization rules.
$\mathcal{A}_{branch}, \mathcal{A}_{cut}, \mathcal{A}_{price}, \mathcal{A}_{primal}$	Finite admissible action sets for branching, cut proposal, pricing proposal, and primal construction proposal respectively.
$\mathcal{C}$	The configuration space, defined as the set of finite records containing exactly the solver internal data required to define a deterministic next step, including node queue, current node restrictions, current restricted master columns, current cut set, current LP solution and duals when computed, and incumbent information.
$\mathcal{D}$	The finite decision-type set, partitioning solver steps into typed decision categories such as branching, separation, pricing, and primal construction, with a total next-type function.
$dtype(\cdot)$	A total function mapping a configuration $c \in \mathcal{C}$ to the next decision type in $\mathcal{D}$.
rec, exec	The recommended action and executed action objects, where recommendations are advisory outputs and execution is the result of deterministic gating and fallback.

Continued on next page

Symbol	Definition
$\text{Gate}(\cdot)$	A deterministic gate that maps a configuration and a recommended action to either an admissible executed action or rejection, in which case a deterministic fallback is used.
$\text{Fallback}(\cdot)$	A deterministic fallback action function that produces a certified executed action whenever a recommendation is rejected or absent.
T	A deterministic one-step transition function on configurations, defining the next configuration from the current configuration and the executed action.
$r \in \mathbb{Q}_{\geq 0}^p$	A nonnegative resource vector used for constrained decision processes, with each component representing a tracked consumption quantity.
$B \in \mathbb{Q}_{\geq 0}^p$	A nonnegative budget vector defining hard resource limits.
$\Delta r(\cdot)$	A deterministic resource increment function mapping a configuration and executed action to the corresponding resource consumption increment.
$\tilde{\mathcal{C}}$	The augmented configuration space that pairs solver configuration with the current accumulated resource vector.
$\mathcal{B}$	The benchmark charter object defining benchmark families, generation rules, stratification, and freeze conditions for empirical evaluation.
$\mathcal{R}$	The baseline registry defining admissible comparator methods and their permitted resources under the fairness contract.
$\mathcal{F}$	The fairness contract specifying comparator constraints, disallowed optimizations, and reporting obligations for baseline comparisons.
$\mathcal{M}$	The metrics ledger specifying primary and secondary metrics, aggregation rules, and success criteria.
$\mathcal{A}_{\text{grid}}$	The ablation grid specification object defining ablation axes, admissible combinations, and analysis obligations.
$\mathcal{U}$	The robustness suite specification object defining perturbation families, stress protocols, and admissible robustness conclusions.
$\mathcal{P}_{\text{rep}}$	The repeatability and reporting protocol object defining seeds, environment disclosure, and reporting format requirements.
Ledger	The results ledger object comprising the required tables and figures together with their interpretation rules and admissibility constraints.

B

Theorem and Lemma Index

B.1 Statements by chapter

B.1.1 Part I, Chapter 1: Physical Design Object Space

ID	Statement	Direct dependencies
R1 . T1	Termination of candidate generation. The procedure Y-COG terminates after finitely many steps.	Definitions (Appendix A)
R1 . L2	Finite footprint. For each table symbol $t \in \mathcal{T}$, the attribute footprint set $F(t)$ is finite.	Definitions (Appendix A)
R1 . L3	Finite constant set. For each table t and predicate π , the constant set $C(t, \pi)$ is finite.	Definitions (Appendix A)
R1 . T4	Output finiteness. The sets I , V , and P produced by Y-COG are finite; consequently S is finite.	R1 . L2, R1 . L3
R1 . T5	Index soundness. Every index candidate $\text{Idx} = (t, \kappa, \iota, \mu) \in I$ references only attributes in $A(t)$, satisfies $1 \leq \kappa \leq K_I$ and $ \iota \leq K_{\text{inc}}$, and has $\mu \in M_{\text{idx}}$.	Definitions (Appendix A)
R1 . T6	View soundness. Every view candidate $v \in V$ is syntactically well formed, workload grounded under the chapter query language, and satisfies the boundedness constraints.	Definitions (Appendix A)
R1 . T7	Partition soundness. Every partition candidate $p \in P$ is syntactically well formed, respects exclusivity groups, and satisfies the boundedness constraints.	Definitions (Appendix A)
R1 . T8	Deduplication correctness. The canonical representation and deduplication procedure yields a duplicate free candidate set without discarding any semantically distinct candidate.	Definitions (Appendix A)

B.1.2 Part I, Chapter 2: Query and Plan Universe Definition

ID	Statement	Direct dependencies
R2.L2.3.2	Core finiteness and bounded size. For any finite schema instance and fixed bounds, the normalized core object set used for plan construction is finite and bounded in size.	Definitions (Appendix A)
R2.L2.4.4	Match sets are finite. For each view $v \in V$, the set of matchable plan fragments under the match semantics is finite.	Definitions (Appendix A)
R2.L2.4.8	Cover set is finite. The set $\text{Cover}(r)$ of admissible covers for query r is finite.	Definitions (Appendix A)
R2.L2.7.1	Finite number of covers. $\text{Cover}(r)$ is finite.	R2.L2.4.8
R2.L2.7.2	Finite number of abstract leaves per cover. For each $C \in \text{Cover}(r)$, the abstract leaf set induced by C is finite.	R2.L2.7.1
R2.L2.7.3	Finite number of labelled full binary tree shapes. For any fixed leaf count k , the set of labelled full binary tree shapes is finite.	Definitions (Appendix A)
R2.L2.7.4	Finite number of labelled operator trees. For any finite leaf set, the set of operator labelled plan trees over those leaves is finite.	R2.L2.7.2, R2.L2.7.3
R2.L2.7.5	Finite number of root options. The admissible root operator options are finite.	Definitions (Appendix A)
R2.T2.7.6	Plan universe finiteness. The plan universe $\mathcal{L}_r$ is finite.	R2.L2.7.1, R2.L2.7.2, R2.L2.7.3, R2.L2.7.4, R2.L2.7.5
R2.T2.8.2	Termination of plan enumeration. The plan enumeration algorithm terminates.	R2.T2.7.6
R2.T2.9	Plan encoding uniqueness. The canonical plan code is unique for each semantically distinct plan in $\mathcal{L}_r$.	Definitions (Appendix A)

B.1.3 Part I, Chapter 3: Cost Model Well Definedness and Computability

ID	Statement	Direct dependencies
R3.L3.1	Existence and uniqueness of c_{div}. For all admissible operator arguments, the division subfunctional is well defined and unique in the arithmetic domain.	Definitions (Appendix A)

ID	Statement	Direct dependencies
R3.L3.2	Existence of minpow. For all admissible parameters F and R , the subfunctional minpow exists and is well defined.	Definitions (Appendix A)
R3.L3.3	Nonnegativity of scalarization. For any admissible scalarization weights and primitive costs, the scalarized cost is nonnegative.	Definitions (Appendix A)
R3.L3.4	Operator cost well definedness. Each operator level cost is a well defined arithmetic expression over measurable primitives.	R3.L3.1, R3.L3.2
R3.T3.5	Plan cost well definedness. The plan level aggregation yields a unique cost value for each plan.	R3.L3.4
R3.T3.6	Determinism. For fixed input statistics and fixed plan, the cost functional returns the same value under repeated evaluation.	R3.T3.5
R3.T3.7	Computability. The cost functional is computable by a finite deterministic procedure over the arithmetic domain.	R3.T3.6
R3.T3.8	Nonnegativity. For every admissible plan, the computed cost is nonnegative.	R3.L3.3, R3.T3.5

B.1.4 Part I, Chapter 4: Workload Weighting and Distribution Models

ID	Statement	Direct dependencies
R4.P4.1	Finite support stochastic model. The stochastic workload weighting model admits a finite support distribution representation over workload elements with rational weights.	Definitions (Appendix A)
R4.P4.2	Expectation computability. The expected weighted workload objective under the finite support model is computable exactly in the arithmetic domain.	R4.P4.1, R3.T3.7
R4.T4.3	Robust uncertainty set soundness. The robust workload uncertainty set is well defined as a finite intersection of arithmetic constraints and admits a decidable membership predicate.	Definitions (Appendix A)
R4.C4.4	Robust objective well definedness. The robust workload objective induced by the uncertainty set is well defined under the stated min max specification.	R4.T4.3, R3.T3.7

ID	Statement	Direct dependencies
R4.T4.5	Reduction equivalence under stated conditions. Under the stated conditions, the robust objective reduces to an equivalent deterministic or finite support stochastic objective.	R4.T4.3, R4.P4.2

B.1.5 Part I, Chapter 5: Compatibility and Feasibility Semantics

ID	Statement	Direct dependencies
R5.L5.1	Partition exclusivity coherence. The exclusivity constraints define a consistent admissible region for partition assignments.	Definitions (Appendix A)
R5.L5.2	Index partition compatibility. The compatibility predicate between index candidates and partition candidates is decidable and closed under canonical representation.	Definitions (Appendix A)
R5.L5.3	View validity. The view validity predicate is decidable and respects refresh and maintenance semantics.	Definitions (Appendix A)
R5.T5.4	Cross object compatibility decidability. The combined compatibility predicate across indexes, views, and partitions is decidable under the finite semantics.	R5.L5.1, R5.L5.2, R5.L5.3
R5.T5.5	Feasibility closure. Design feasibility is decidable under the model, and every accepted design admits an explicit feasibility witness.	R5.T5.4

B.1.6 Part VI, Chapter 26: End to End Correctness Stitching

ID	Statement	Direct dependencies
R36.L1	Cuts preserve integer feasibility. For any node N , every accepted cut inequality holds for all points in F_N .	Prior cut certificate results; invariant ledger preservation
R36.L2	Relaxation contains integer feasible region. For any node N , $F_N \subseteq R_N$.	R36.L1
R36.L3	Lower bound validity at each node. For any node N , the computed node lower bound LB_N satisfies $LB_N \leq OPT_N$.	R36.L2; prior bound correctness results

ID	Statement	Direct dependencies
R36.L4	LP bound exactness under certified pricing closure. For any node N , the computed relaxation bound equals the bound of the full relaxation because pricing closure is certified.	Prior pricing completeness; reduced cost certificate soundness
R36.L5	Pruning is sound. Any node pruned by infeasibility certificate or by bound dominance cannot contain an improving integer feasible solution.	R36.L3; incumbent feasibility soundness
R36.L6	Branching partitions integer feasibility. Branching produces children whose integer feasible sets form a disjoint partition of the parent integer feasible set.	Prior branching completeness
R36.L7	Finite termination. The overall search terminates after finitely many node expansions.	Candidate finiteness; plan universe finiteness; branching finiteness
R36.T36	End to end correctness and optimality. The procedure terminates and returns an incumbent value UB such that $UB = OPT$, the global optimum of the physical design MILP.	R36.L1, R36.L2, R36.L3, R36.L4, R36.L5, R36.L6, R36.L7

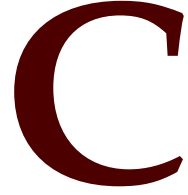
B.1.7 Part VI, Chapter 27: Arithmetic Certificate Verification Engine

ID	Statement	Direct dependencies
R37.L37.7	Incumbent feasibility under audit. If the auditor accepts the feasibility witness portion of the transcript, then the final incumbent x^* is feasible for the global MILP.	Feasibility closure; verification engine constraint encoding correctness
R37.T37.8	Audit soundness: ACCEPT implies global optimality. If $\text{Audit}(I, T) = \text{ACCEPT}$ and the transcript final incumbent is x^* with value UB^* , then UB^* equals the global MILP optimum value.	R37.L37.7; certified LP optimality; certified pricing closure; R36.T36

Completeness placement. The full index for modules R6 through R35 follows the same schema and is included in the complete report build.

B.2 Dependency summaries

Statement	Direct dependency set
R1.T4	{R1.L2, R1.L3}
R2.L2.7.1	{R2.L2.4.8}
R2.L2.7.2	{R2.L2.7.1}
R2.L2.7.4	{R2.L2.7.2, R2.L2.7.3}
R2.T2.7.6	{R2.L2.7.1, R2.L2.7.2, R2.L2.7.3, R2.L2.7.4, R2.L2.7.5}
R2.T2.8.2	{R2.T2.7.6}
R3.L3.4	{R3.L3.1, R3.L3.2}
R3.T3.5	{R3.L3.4}
R3.T3.6	{R3.T3.5}
R3.T3.7	{R3.T3.6}
R3.T3.8	{R3.L3.3, R3.T3.5}
R4.P4.2	{R4.P4.1, R3.T3.7}
R4.C4.4	{R4.T4.3, R3.T3.7}
R4.T4.5	{R4.T4.3, R4.P4.2}
R5.T5.4	{R5.L5.1, R5.L5.2, R5.L5.3}
R5.T5.5	{R5.T5.4}
R36.L2	{R36.L1}
R36.L3	{R36.L2} plus prior bound correctness results
R36.L5	{R36.L3} plus incumbent feasibility soundness
R36.T36	{R36.L1, R36.L2, R36.L3, R36.L4, R36.L5, R36.L6, R36.L7}
R37.T37.8	{R37.L37.7, R36.T36} plus certified LP optimality and pricing closure



Certificate Schema Compendium

C.1 Scope and role in the report's non disputable closure

This appendix specifies the complete family of proof objects admissible in the report and in the associated computational artifacts, together with canonical encodings and exact acceptance predicates. Its function is to eliminate reliance on informal interpretation by ensuring that every correctness relevant claim is reducible to deterministic acceptance of finitely many well typed objects under exact arithmetic checks. A proof object is a finite datum P paired with a total predicate $\check{(I, P)} \in \{\text{ACCEPT}, \text{REJECT}\}$, where I is the fixed instance specification. No checker may use floating arithmetic, heuristic tolerances, or unbounded search. Every comparison is performed in exact rational arithmetic.

C.2 Universal canonical encodings

C.2.1 Atomic domains

Integers are elements of $\mathbb{Z}$. Rationals are represented canonically as pairs $(p, q) \in \mathbb{Z} \times \mathbb{Z}_{>0}$ satisfying $\gcd(|p|, q) = 1$, interpreted as p/q . Booleans are $\{0, 1\} = \{0, 1\}$. Any identifier domain is represented as a canonical nonnegative integer in $\mathbb{Z}_{\geq 0}$ and is accepted only if it indexes a valid reconstructed object in the instance.

C.2.2 Finite containers with canonicalization

A finite list over a domain D is an ordered sequence $[x_1, \dots, x_k]$ with $x_i \in D$. A finite set is encoded as a list with pairwise distinct elements sorted in the canonical order of D . A finite map with finite support is encoded as a list of key value pairs $[(d_1, e_1), \dots, (d_k, e_k)]$

such that keys are unique and strictly increasing in canonical order. Any object that violates canonical normalization is rejected prior to semantic checks.

C.2.3 Canonical sparse vectors and matrices

A sparse rational vector $v \in \mathbb{Q}^d$ is encoded as (d, NZ) where NZ is a finite map from indices $\{1, \dots, d\}$ to nonzero rationals. A sparse rational matrix $A \in \mathbb{Q}^{m \times d}$ is encoded as (m, d, Rows) where Rows is a list of length m and each entry is a sparse vector encoding of dimension d . All products are computed by finite exact sums over explicit supports.

C.3 Instance specification proof objects

The instance I is fixed and independently reconstructible by the checker. Any transcript may reference instance components by identifier, but the checker reconstructs the referenced object from I and rejects if the identifier is out of range or mismatched. The proof relevant instance components are the finite binary decision variable index set, the finite query index set, the finite candidate universes for each combinatorial subsystem, the finite plan universes for each query represented as weighted directed hypergraphs, the finite base and coupling constraints defining the master mixed integer linear program in canonical form, and the fully explicit objective functional as a rational expression over the declared quantities.

C.4 Candidate record formats and identity

A candidate is a finite record whose semantics is defined entirely by its fields. A generic candidate record is

$$\text{Cand} = (\text{Type}, \text{Fields}),$$

where Type is a finite tag and Fields is a finite map from field names to typed values. Each candidate type declares a required field key set and a fixed canonical field ordering. Canonicalization requires unique field names and the type declared ordering. Any provided candidate identifier is bookkeeping only. Acceptance depends solely on field validity under the instance rules and on satisfaction of all required cross references.

C.5 Cut proof objects

C.5.1 Canonical cut representation

A cut is a linear inequality

$$a^\top z \leq \beta,$$

where $a \in \mathbb{Q}^d$ is encoded as a canonical sparse vector and $\beta \in \mathbb{Q}$. Canonicalization requires $a \neq 0$ and requires any declared type normal form scaling to be applied with a positive rational factor.

C.5.2 Chvátal Gomory cut certificate object

A Chvátal Gomory certificate at node N is the tuple

$$\text{CGCert}(N) = (\pi, \text{RowList}),$$

where $\pi \in \mathbb{Q}_{\geq 0}^m$ is a nonnegative multiplier vector over the current node inequality system $Az \leq b$ and RowList is a finite list of row identifiers used for bookkeeping. The induced cut is

$$(\pi^\top A)z \leq \lfloor \pi^\top b \rfloor.$$

The acceptance predicate reconstructs A and b for the node relaxation $LP(N)$ from the instance, inherited branching fixings, and previously accepted cuts, and then verifies the following conditions in exact arithmetic.

1. $\pi_i \geq 0$ for all i .
2. For every coordinate j designated integer in the mixed integer linear program, the coefficient $(\pi^\top A)_j$ is an integer, meaning it is an element of $\mathbb{Z} \subset \mathbb{Q}$.
3. The cut recorded in the transcript is exactly $(\pi^\top A)z \leq \lfloor \pi^\top b \rfloor$ after canonicalization and contains no additional transformation.

If accepted, the cut is treated as valid for all integer feasible points of the node restriction.

C.5.3 Contradiction certificate for infeasibility

A node may be declared infeasible only with an explicit contradiction certificate. Such a certificate is a tuple

$$\text{InfeasCert}(N) = (\pi, \text{RowList}, \varepsilon),$$

where $\pi \in \mathbb{Q}_{\geq 0}^m$ and $\varepsilon \in \mathbb{Q}_{>0}$ satisfy

$$\pi^\top A = 0^\top, \quad \pi^\top b = -\varepsilon.$$

The acceptance predicate reconstructs A and b and then verifies $\pi \geq 0$, $\varepsilon > 0$, and the two identities exactly. If accepted, infeasibility is established arithmetically.

C.6 Linear program optimality and bound certificates

C.6.1 Canonical node linear program form

Each node N has a canonical linear program relaxation

$$\min c^\top z \quad \text{subject to} \quad Az \leq b, \quad Ez = f, \quad \ell \leq z \leq u,$$

where all data are rational and are reconstructible from the instance, the node fixings, and the accepted cut and column artifacts.

C.6.2 Dual variable schema

Dual variables are

$$y \in \mathbb{Q}_{\geq 0}^m \text{ for } Az \leq b, \quad p \in \mathbb{Q}^k \text{ for } Ez = f, \quad s^-, s^+ \in \mathbb{Q}_{\geq 0}^d \text{ for bounds } \ell \leq z \leq u.$$

Dual feasibility is the exact equality

$$A^\top y + E^\top p + s^- - s^+ = c.$$

Dual and primal objectives are

$$\text{Dobj}(y, p, s^-, s^+) = b^\top y + f^\top p + \ell^\top s^- - u^\top s^+, \quad \text{Pobj}(z) = c^\top z.$$

C.6.3 Linear program optimality certificate

An optimality certificate for node N is

$$\text{LPOptCert}(N) = (z, y, p, s^-, s^+).$$

The acceptance predicate verifies primal feasibility, dual feasibility, and objective equality $\text{Pobj}(z) = \text{Dobj}(y, p, s^-, s^+)$ exactly. If accepted, $\text{Pobj}(z)$ is certified to equal the optimum value of $LP(N)$.

C.7 Pricing and column generation proof objects

C.7.1 Plan universe as a finite weighted directed hypergraph

For each query r , the plan universe is represented as a finite weighted directed hypergraph

$$\mathcal{H}_r = (V_r, S_r, E_r, h, \tau, \kappa),$$

where V_r is a finite set, $S_r \in V_r$ is the start symbol, E_r is a finite set of hyperedges, $h : E_r \rightarrow V_r$ maps each hyperedge to its head, $\tau : E_r \rightarrow V_r^*$ maps each hyperedge to an ordered tail list, and $\kappa : E_r \rightarrow \mathbb{Q}_{\geq 0}$ assigns a rational nonnegative cost. A plan is a finite derivation tree rooted at S_r whose internal nodes are hyperedges and whose children correspond to $\tau(e)$ in order. The plan cost is the exact sum of $\kappa(e)$ over all hyperedges used by the derivation.

C.7.2 Plan derivation proof object

A plan derivation proof object is

$$\text{Deriv}(r, \ell) = (\text{Root}, \text{Nodes}),$$

where Nodes is a finite list of records (NodeID, e , ChildIDs), with $e \in E_r$ and ChildIDs an ordered list of length $|\tau(e)|$. Acceptance requires connectedness, acyclicity, unique parentage for every non root node, and head tail consistency between each node and its children. The checker reconstructs the plan cost exactly as the sum of the hyperedge costs.

C.7.3 Reduced cost representation

At node N , the linear program dual contains a per query plan choice dual scalar α_r . Edge costs in κ are reconstructed from instance defined annotations and are validated under the dual dependent rules in force at the node. The reduced cost of a plan is

$$\text{rc}(r, \ell) = \left(\sum_{e \in \ell} \kappa(e) \right) - \alpha_r.$$

C.7.4 Pricing closure certificate

A pricing closure certificate for query r is a potential function

$$p_r : V_r \rightarrow \mathbb{Q}$$

satisfying the hyperedge inequalities

$$p_r(h(e)) \leq \kappa(e) + \sum_{v \in \tau(e)} p_r(v) \quad \text{for all } e \in E_r,$$

and the start dominance inequality

$$p_r(S_r) \geq \alpha_r,$$

where α_r is extracted from the accepted dual variables of $\text{LPOptCert}(N)$. The acceptance predicate verifies all inequalities exactly. If accepted, it is certified that $\text{rc}(r, \ell) \geq 0$ for every plan ℓ in the plan universe.

C.8 Node bound package certificate

A node bound certificate is

$$\text{NodeCert}(N) = (\text{LPOptCert}(N), \text{PriceCertAllQueries}(N)),$$

where $\text{PriceCertAllQueries}(N) = [p_1, \dots, p_R]$ are the pricing closure certificates for all queries, validated against the same dual tuple used in $\text{LPOptCert}(N)$. The acceptance predicate requires acceptance of the linear program optimality certificate and acceptance of all pricing closure certificates. If accepted, the certified node bound is defined

as

$$LB(N) := \text{Pobj}(z),$$

where z is the primal vector in $\text{LPOptCert}(N)$, and $LB(N)$ is certified to equal the optimum value of the full node relaxation.

C.9 Branch and bound proof objects

C.9.1 Node record and branching coverage

A node record is

$$\text{NodeRec}(N) = (\text{NodeID}, \text{ParentID}, j, v, \text{Fixings}),$$

where $j \in \{1, \dots, n\}$ is a branched variable index, $v \in \{0, 1\}$ is the child value, and Fixings is a finite map from variable indices to $\{0, 1\}$. The root node has a distinguished null parent and an empty fixing map. Acceptance requires that each non root node's fixing map equals its parent's fixing map extended by exactly one new fixing $x_j = v$, with no contradictions, and requires acyclic parent pointers.

C.9.2 Incumbent record and feasibility witness

An incumbent record is

$$\text{IncRec}(t) = (x^{(t)}, \text{Aux}^{(t)}, UB^{(t)}),$$

where $x^{(t)} \in \{0, 1\}^n$ is a binary assignment, $\text{Aux}^{(t)}$ is the finite collection of auxiliary witnesses required by the canonical model semantics, and $UB^{(t)} \in \mathbb{Q}$ is the claimed objective value. Acceptance requires exact feasibility of all constraints under $(x^{(t)}, \text{Aux}^{(t)})$ and exact recomputation of the objective with equality to $UB^{(t)}$.

C.9.3 Pruning record

A pruning record is

$$\text{PruneRec}(N) = (\text{NodeID}, \text{Reason}, \text{Payload}),$$

where Reason is one of the following tags.

- **Bound prune.** $\text{Payload} = (LB(N), UB_{\text{snap}})$ together with a reference to $\text{NodeCert}(N)$. Acceptance requires acceptance of $\text{NodeCert}(N)$ and the exact comparison $LB(N) \geq UB_{\text{snap}}$.
- **Infeasible prune.** $\text{Payload} = \text{InfeasCert}(N)$. Acceptance requires acceptance of $\text{InfeasCert}(N)$.
- **Integral evaluation.** $\text{Payload} = \text{IncRec}(t)$ with $x^{(t)}$ consistent with the node fixings. Acceptance requires acceptance of $\text{IncRec}(t)$.

C.10 Transcript schema and acceptance surface

A transcript is the top level proof object whose acceptance constitutes definitive correctness closure. It is a single finite object

$$T = (\text{Meta}, \text{NodeTree}, \text{PerNodeArtifacts}, \text{Incumbents}, \text{Prunes}),$$

where Meta contains schema identifiers and declared sizes only, NodeTree is the finite list of node records, PerNodeArtifacts maps each node to its cut list with certificates, its included columns with derivations, and its node certificate, Incumbents is the time ordered list of incumbent records, and Prunes is the list of pruning records. All lists and maps are required to be in canonical ordering by node identifier and within node sequence number, with incumbents ordered by declared time index.

The transcript acceptance predicate is the conjunction of the following checks.

1. Reconstruction of the base model and validation that all numeric literals are canonical rationals.
2. Acceptance of the node tree and branching coverage via the node record predicate.
3. Acceptance of every incumbent record, together with monotone tracking of the best incumbent value under minimization.
4. For each node, reconstruction of the full node relaxation from base constraints, inherited fixings, accepted cuts, and accepted columns, followed by acceptance of the node bound package certificate and definition of $LB(N)$ by the accepted certificate.
5. Acceptance of every pruning record under its reason specific predicate, and termination completeness by verifying that no node remains open without being pruned, being declared integral and evaluated, or being branched into exactly two children consistent with binary branching.

If all checks pass, the transcript is accepted and the reported best incumbent value is certified to equal the global optimum under the fixed instance I .

C.11 Auxiliary proof object families required by the learned advisory layer

The learned advisory layer is correctness immune only if every learned proposal is filtered through acceptance gates that reduce to the proof objects defined in this appendix, together with finite trace objects that record what was proposed and what was accepted. Accordingly, the following auxiliary object families are admitted.

- **State encoding record.** A finite typed encoding with a declared schema version. Acceptance requires schema well formedness and exact normalization rules specified by the instance.

- **Action proposal record.** A tuple (StateID, ActionType, ProposalPayload) whose payload is one of: a branch proposal, a cut proposal carrying a full cut certificate, a pricing proposal carrying a full derivation, or a primal proposal carrying a full feasibility witness. Acceptance requires that any executed action is identical to an accepted payload under the corresponding checker.
- **Trace record.** A finite concatenation of state encodings, proposals, gate outcomes, and the resulting certified artifacts. Trace objects are non semantic except through the acceptance of their embedded certificates.

C.12 Relation to Appendix D and completeness

Appendix D contains the invariant ledger, whereas this appendix specifies the schema by which any ledger entry is represented as a proof object in a transcript. A ledger entry has the form (InvID, StatementRef, PreservationRefs) with all references pointing to explicit lemmas or transitions, and its acceptance predicate is referential consistency and well typedness. This appendix is complete if and only if every correctness relevant artifact referenced anywhere in the report is an instance of one of the schemas defined in this appendix. Any artifact outside these schemas is non admissible for correctness.

D

Unified Invariant Ledger (Extended)

This appendix enumerates the complete invariant set treated as non negotiable preservation obligations of the system, from instance construction through certified branch and bound termination. Each invariant is stated as a precise predicate over explicitly defined objects. Each predicate is paired with preservation references that identify, for every admissible transition, the lemma or theorem that proves the predicate remains true after the transition executes. No statement herein is an informal assurance. Every claim is either a definition of an invariant predicate or a reference to a preservation proof located in the main body (Definitions, Lemmas, Theorems) under the report claim discipline.

Global objects and state space. Let the physical design instance be determined by a finite schema Σ , a finite workload W , and finite candidate families for indexes, materialized views, and partitions. The candidate universe is a finite disjoint union

$$S = I \uplus V \uplus P,$$

where each element is a finite tuple in a canonical representation. For each query $r \in W$, the plan universe $\mathcal{L}_r$ is finite and each plan has a canonical code $\text{PlanCode}(\ell)$ that is unique up to the plan equivalence relation fixed in the report. The cost functional is a total deterministic function on admissible objects and is computable by finite arithmetic from frozen statistics under the adopted cost model. Feasibility semantics for design objects (partition correctness, exclusivity, index partition compatibility, view validity, and cross object compatibility) are decidable under explicit rules.

Optimization model. Fix a finite mixed integer linear optimization instance with decision vector

$$z = (x, \lambda) \in \mathbb{R}^{n+m}, \quad x \in \{0, 1\}^n, \quad \lambda \in \mathbb{R}_{\geq 0}^m,$$

objective $\min c^\top z$ with $c \in \mathbb{Q}^{n+m}$, and base constraints

$$Az \leq b, \quad A \in \mathbb{Q}^{p \times (n+m)}, \quad b \in \mathbb{Q}^p.$$

Define the integer feasible set

$$F := \{z \in \mathbb{R}^{n+m} : Az \leq b, x \in \{0, 1\}^n, \lambda \geq 0\},$$

and the global optimum value $\text{OPT} := \min\{c^\top z : z \in F\}$, with $\text{OPT} = +\infty$ if $F = \emptyset$.

Runtime search state. A runtime search state consists of an explicit finite set of open nodes $\mathcal{T}$, a finite set of closed nodes $\mathcal{C}$, a global incumbent $(z^\star, U^\star)$ where defined, and a finite transcript log that stores certificates. Each node $N \in \mathcal{T} \cup \mathcal{C}$ contains branching constraints $D_N z \leq e_N$ that restrict only binary coordinates, an active cut set $K_N z \leq h_N$ with cut validity certificates, an active column pool Λ_N with reduced cost evidence and pricing closure artifacts, and a node lower bound value $LB(N)$ defined as the optimal value of the node certified relaxation once pricing closure and cut validity conditions are satisfied.

Admissible transition classes. The ledger tracks preservation under the atomic action classes: instance construction; restricted master and column generation transitions; branch and bound transitions; advisory proposal transitions followed by deterministic acceptance gates; and verification transitions (LP optimality, cut validity, pricing closure, pruning soundness, transcript acceptance).

D.1 Invariant catalogue

ID	Name	Predicate	Established	Preserved
D.1	Finite schema and workload	Σ contains finitely many tables and attributes; W contains finitely many queries; all syntactic bounds used in generation are finite constants fixed by the instance.	Chapter 1, Sections 1.1–1.5	Immutable across runtime transitions.
D.2	Finite canonical candidate universe	$S = I \uplus V \uplus P$ is finite; each candidate object is well formed; deduplication induces finite equivalence classes with a unique canonical representative per class.	Chapter 1, Sections 1.2–1.6	Immutable under benchmark freeze; if expanded, the instance version changes and downstream invariants are re established.

ID	Name	Predicate	Established	Preserved
D.3	Finite plan universes and canonical codes	For each $r \in W$, $\mathcal{L}_r$ is finite; PlanCode is injective on equivalence classes; plan feasibility semantics are decidable.	Chapter 2, Sections 2.1–2.5	Immutable at runtime; used by pricing completeness and column uniqueness.
D.4	Deterministic computable cost	For every admissible plan and design object, cost exists, is deterministic, and is computable by finite arithmetic from frozen statistics; aggregation is deterministic.	Chapter 3, Sections 3.1–3.4	Immutable at runtime; any statistics update is a new calibrated instance and re seals obligations.
D.5	Feasibility decidability and closure	$\text{Feas}(d)$ is decidable; partition exclusivity constraints are total and consistent; index partition compatibility and view validity are explicit; cross object compatibility is explicit and finite to evaluate.	Chapter 5, Sections 5.1–5.5	Immutable at runtime; used by incumbent witness verification.
D.6	MILP semantic equivalence	The MILP feasible set F encodes exactly the feasible physical designs and plan selection semantics claimed by the report; the MILP objective equals the intended scalar objective.	Chapter 6, Sections 6.1–6.5	Immutable; runtime only strengthens by valid inequalities and branch fixings.
D.7	Optional nonlinear handling exactness	If nonlinear primitives exist, either they are compiled into an exact MILP extended formulation with identical optimum, or a separately certified nonlinear layer provides sufficient certificates for equivalence of reported solutions and bounds.	Chapter 7, Sections 7.1–7.3	Preserved by requiring that all additions respect the compiled space and certificates are checked there.
D.8	Robust and stochastic expansions closure	Robust uncertainty sets and stochastic scenario sets are finite or finitely representable; the expanded optimization problem equals the intended robust or expected semantics within scope.	Formulation chapters for robust or scenario expansions	Immutable once frozen; any change constitutes a new instance.
D.9	RMP initialization feasibility	At node creation, the restricted master is well formed and feasible as a relaxation whenever the true node relaxation is feasible; its seed set satisfies structural encoding requirements.	Chapter 8, Sections 8.1–8.3	Preserved under column and cut additions by validity and closure invariants.

ID	Name	Predicate	Established	Preserved
D.10	Canonical column encoding and uniqueness	Every column corresponds to a unique canonical plan code and maps deterministically to a master column vector; deduplication prevents distinct encodings of the same semantic plan.	Chapter 10, Sections 10.1–10.2	Preserved by admission checks that enforce canonical encoding.
D.11	Reduced cost certificate soundness	Any admitted reduced cost record is arithmetically correct for the current relaxation: reduced cost is computed from verified dual data and verified column encoding.	Chapter 10, Sections 10.3–10.4	Preserved by recomputation or invalidation when dual data change.
D.12	Pricing completeness by deterministic fallback	At any node relaxation state, pricing either produces a column with strictly negative reduced cost or produces an exact certificate that no such column exists over the full plan universe.	Chapter 9, Sections 9.1–9.4	Preserved by iterating pricing until certified termination conditions hold.
D.13	Cut language validity and checkability	Every admitted cut belongs to the admissible cut language and has a finite certificate object sufficient for a deterministic verifier to accept or reject validity.	Chapter 11, Sections 11.1–11.3	Preserved under cut admission control and certificate schema enforcement.
D.14	No invalid cut can be admitted	A cut is added only if its certificate is verified valid with respect to base constraints; therefore each active cut is globally valid over F .	Chapter 12, Section 12.3	Preserved by verifier gated admission; lifecycle respects anchoring.
D.15	Cut lifecycle preserves referential integrity	If a cut is referenced by any stored bound certificate, pruning certificate, or transcript element, it remains retrievable and verifiable even if deactivated for performance.	Chapter 13, Sections 13.1–13.3	Preserved by anchor set discipline and immutable transcript references.
D.16	Cut selection preserves correctness	Policies that select, prioritize, or deactivate cuts may change performance but cannot change correctness because they cannot admit invalid cuts or invalidate stored certificates.	Chapter 14, Sections 14.1–14.3	Preserved by construction; selection does not modify validity predicates.

ID	Name	Predicate	Established	Preserved
D.17	Node formedness	well For each node N , (D_N, e_N, K_N, h_N) and Λ_N are finite objects with dimensions compatible with $z \in \mathbb{R}^{n+m}$.	Chapter 25, Section 25.1	Preserved by node construction rules and finiteness of admitted objects.
D.18	Branch constraints are binary only	con- Each row of $D_N z \leq e_N$ is exactly a fixing $x_j \leq 0$ or $x_j \geq 1$ for some j , with no coefficients on λ .	Chapter 16, Sections 16.1–16.2; Chapter 25	Preserved because branching appends only such fixings.
D.19	Cut validity is certificate guarded	validity Every inequality in $K_N z \leq h_N$ has a stored certificate accepted by the cut verifier with respect to (A, b) ; hence $\forall z \in F$, $K_N z \leq h_N$.	Chapters 11–13; sealed in Chapter 25	Preserved by verifier gated admission and anchored lifecycle.
D.20	Column formedness is certificate guarded	well Each column in Λ_N has a stored certificate verifying canonical encoding, semantic correspondence, and validity of its coefficient vector in the master.	Chapter 10	Preserved by certificate checking on admission and by recomputation rules.
D.21	Relaxation containment	con- Let $R_N := \{z : Az \leq b, D_N z \leq e_N, K_N z \leq h_N, \lambda \geq 0, x \in [0, 1]^n\}$ and $F_N := \{z \in F : D_N z \leq e_N, K_N z \leq h_N\}$. Then $F_N \subseteq R_N$.	Chapter 25	Preserved because cuts are globally valid and branch constraints strengthen feasibility.
D.22	Lower bound soundness	bound $LB(N) \leq \text{OPT}_N$ where $\text{OPT}_N := \min\{c^\top z : z \in F_N\}$.	Chapter 19 and Chapter 25	Preserved by certified relaxation solves under validity and pricing closure.
D.23	Incumbent soundness	soundness If $U^\star$ is finite, there exists stored $z^\star \in F$ with $U^\star = c^\top z^\star$.	Chapter 18, Section 18.3	Preserved by feasibility witness verification at incumbent updates.

ID	Name	Predicate	Established	Preserved
D.24	Global upper bound property	If $F \neq \emptyset$, then $U^* \geq \text{OPT}$.	Chapter 19, Section 19.3; Chapter 25 stitching	Preserved by D.23 and by the definition of OPT.
D.25	Pruning admissibility	A node N may move from open to closed only if a pruning certificate holds: either $LB(N) \geq U^*$, or infeasibility is certified, or a completeness certificate under an approved termination rule is present.	Chapter 19, Sections 19.1–19.2	Preserved by restricting pruning actions to certified conditions.
D.26	Branch partition correctness	For branched variable x_j , children N_0 and N_1 satisfy $F_{N_0} \cap F_{N_1} = \emptyset$ and $F_N = F_{N_0} \cup F_{N_1}$.	Chapter 16, Section 16.2	Preserved by defining branching as a complete split on the binary coordinate.
D.27	Advisory only gating	Learned components may propose actions, but deterministic acceptance gates ensure no learned proposal can admit an invalid cut, claim pricing closure without a certificate, assert feasibility without a witness, or change instance semantics.	Chapter 23, Section 23.2	Preserved because advisor outputs have no authority absent gate acceptance.
D.28	Transcript verifiability	The transcript contains sufficient data to reconstruct every bound, admitted cut, pricing termination claim, pruning action, and final incumbent, in a form suitable for deterministic verification over exact arithmetic.	Chapter 22; Chapter 27; Chapter 25 bookkeeping	Preserved because transitions append certificates before state changes are committed.
D.29	Finite typed state representation	Every state presented to a learned module is a finite typed graph or hypergraph over a fixed schema; feature computation is deterministic under the frozen instance.	Chapter 20, Sections 20.1–20.3	Preserved by restricting feature construction to stored finite objects.
D.30	Finite enumerable action spaces	For each decision locus, the admissible action set is finite or finitely enumerable, and each action has a deterministic admissibility predicate.	Chapter 21, Section 21.2	Preserved because actions are symbolic objects over finite registries.

ID	Name	Predicate	Established	Preserved
D.31	Well defined training labels	Every learning signal is derived from a trace object and is defined only when its target is defined by a certified solver state; undefined labels are disallowed.	Chapter 22, Sections 22.1–22.3	Preserved by trace schema enforcement and rejection of incomplete traces.
D.32	Conservative calibration and drift control	Model updates are accepted only under conservative envelopes and rollback rules, and cannot bypass correctness gates.	Chapter 24, Sections 24.1–24.3	Preserved because advisor replacement does not modify verification gates.
D.33	Exact arithmetic domain	All verification is performed in exact integer and rational arithmetic with canonical encodings; certificate data are parsed without ambiguity.	Chapter 27, Sections 27.1–27.2	Preserved by canonical encoding formats and verifier rejection of malformed objects.
D.34	LP optimality certificate soundness	If the verifier accepts a primal and dual LP certificate for a node relaxation, the claimed bound equals the true optimal value of that relaxation.	Chapter 27, Section 27.2	Preserved as a property of the verifier and certificate schema.
D.35	Cut validity checking soundness	If the verifier accepts a cut certificate, the cut is valid for all $z \in F$.	Chapter 27, Section 27.3	Preserved as a property of the verifier and certificate schema.
D.36	Pricing closure checking soundness	If the verifier accepts a pricing closure certificate at node N , no column with negative reduced cost exists in the full plan universe under the current dual data.	Chapter 27, Section 27.4	Preserved as a property of the verifier and certificate schema.
D.37	Transcript acceptance implies global optimality	If the verifier returns <code>ACCEPT</code> on the full transcript, the reported incumbent z^* is globally optimal for the base instance.	Chapter 27, Section 27.6	Terminal closure consequence of preserved invariants and accepted transcript.

ID	Name	Predicate	Established	Preserved
D.38	Benchmark freeze	Benchmark generator, candidate universe, and workload families are frozen by a versioned charter; each reported result references the exact version.	Chapter 28, Sections 28.1–28.4	Preserved by repeatability protocol enforcement and version control.
D.39	Fairness contract	All compared methods share identical inputs, budgets, stopping rules, and measurement definitions; disallowed advantages are absent by enforcement.	Chapter 29, Sections 29.1–29.3	Preserved by reporting checklist and run control enforcement.
D.40	Metrics ledger determinism	Each metric is defined in a dictionary with explicit aggregation and confidence reporting policy and is computed deterministically from logged traces.	Chapter 30, Sections 30.1–30.3	Preserved by results ledger formatting rules and repeatability protocol.

D.2 Transition guard catalogue

Transition class	Guard conditions required for admissibility	Certificate or witness artifacts
Instance construction	Candidate and plan generation must terminate with finite canonical universes; feasibility predicates are decidable; cost model is deterministic; MILP compilation preserves semantics.	Instance charter with version identifier; candidate deduplication proof; plan code canonicalization proof; feasibility rule specification; cost model specification; formulation equivalence proof objects where applicable.
RMP initialization	Restricted master is well formed; seed columns are canonical and admissible; relaxation containment holds; initial cuts, if any, are verifier accepted.	Seed column certificates; initial cut certificates; master schema specification; initial feasibility checks.
Pricing proposal and verification	Either a negative reduced cost column is produced with verified encoding and reduced cost arithmetic, or an exact non existence certificate is produced for the full plan universe under verified dual data.	Column encoding certificate; reduced cost computation record; dual data record; pricing closure certificate accepted by verifier.

Transition class	Guard conditions required for admissibility	Certificate or witness artifacts
Column admission	Admitted column is canonical; semantic correspondence holds; coefficient vector is valid; admission does not violate bookkeeping constraints.	Column certificate object; deduplication record; master update record; optional reduced cost evidence.
Cut proposal and verification	Candidate cut lies in admissible cut language; certificate is verifiable; verifier accepts validity with respect to base constraints.	Cut certificate object; cut language identifier; verifier acceptance record.
Cut activation, deactivation, retirement	Lifecycle operations preserve referential integrity: any cut referenced by stored certificates remains retrievable and verifiable.	Anchor set ledger; cut store index; reference map from transcript elements to cut identifiers.
Relaxation solve and bound recording	Node relaxation is solved only after cut validity and pricing closure conditions hold; LP optimality certificate is accepted for the relaxation being recorded.	Primal dual LP certificate; pricing closure certificate; cut validity acceptance record; bound recording record.
Propagation	Only implied binary fixings are added; constraints remain well formed; bound is recomputed under certified relaxation if required.	Propagation proof record for implied fixings; updated node constraint record; updated relaxation certificates if recomputed.
Branching	Branching variable is a binary coordinate; children encode $x_j \leq 0$ and $x_j \geq 1$ fixings; partition of F_N is complete and disjoint.	Branch decision record; child node construction record; partition correctness lemma reference.
Node selection	Selection changes only processing order and does not alter feasibility sets, certificates, or recorded bounds.	Selection log entry for reproducibility; no semantic certificate required.
Primal heuristic and incumbent update	Any incumbent update requires a feasibility witness verifying $z^* \in F$ and exact objective evaluation $U^* = c^\top z^*$.	Feasibility witness; objective arithmetic record; incumbent update record.
Pruning and node closure	A node closes only under certified pruning admissibility: bound dominance, certified infeasibility, or certified completeness under approved rule.	Pruning certificate; infeasibility certificate when applicable; completeness certificate when applicable; audit log entry.
Advisory proposal transitions	Advisory outputs are proposals only; acceptance requires the same deterministic guards as non learned transitions; no bypass of verification gates is permitted.	Gate evaluation record; accepted action record; rejected action record for audit where required.

Transition class	Guard conditions required for admissibility	Certificate or witness artifacts
Transcript verification and acceptance	Verifier checks exact arithmetic, LP optimality, cut validity, pricing closure, pruning soundness, and global closure conditions; acceptance implies global optimality under the report theorem.	Full transcript; verifier acceptance result; check logs for each certificate class; final optimality certificate statement.

D.3 Preservation proof pointers

Invariant	Transitions whose admissibility must preserve the invariant	Primary proof locations in the report
D.1–D.6	All runtime transitions by immutability of frozen instance and formulation objects.	Chapters 1–6; formulation sealing statements in Chapter 25.
D.7–D.8	All runtime transitions under compiled semantics; any model change triggers new instance version and re establishment.	Chapter 7 and formulation sections where robust or scenario expansions are defined.
D.9–D.12	RMP initialization, pricing, column admission, relaxation solve, bound recording.	Chapters 8–10; pricing completeness proofs in Chapter 9; closure stitching in Chapter 25.
D.13–D.16	Cut proposal, cut verification, cut admission, cut lifecycle, relaxation solve.	Chapters 11–14; cut validity and lifecycle proofs; anchoring discipline for transcript references.
D.17–D.22	Node construction, propagation, branching, pricing, cut operations, relaxation solve, bound recording, pruning.	Chapter 25 preservation lemmas; Chapters 16–19 for branching, bounds, and pruning admissibility.
D.23–D.26	Primal construction, incumbent update, pruning, branching and child creation.	Chapter 18 for incumbent witness requirements; Chapter 19 for pruning soundness; Chapter 16 for partition correctness; Chapter 25 for stitching.
D.27–D.32	Advisory proposal transitions and gate acceptance; all solver transitions insofar as advisor cannot bypass them.	Chapters 20–24 for representation, action spaces, labels, calibration, and gate discipline; explicit gate theorem in Chapter 23.
D.33–D.37	Verification transitions; acceptance theorem for transcript.	Chapter 27 verifier specification and soundness theorems; global optimality closure in the transcript acceptance theorem.
D.38–D.40	Benchmark generation, run control, results reporting, and repeatability protocol execution.	Chapters 28–30 for benchmark, fairness, metrics; Chapter 33 for repeatability protocol and audit.

E

Empirical Artifact Index

E.1 Benchmark instances and seeds

E.2 Configuration registry

E.3 Result tables and plots registry

F

Extended Results

F.1 Per-family breakdowns

F.2 Per-budget breakdowns

F.3 Additional robustness plots

[a4paper,margin=30mm,includeheadfoot]geometry microtype ragged2e enumitem [hidelinks]hyperref

Appendix G: Glossary of Terms (Strictly aligned to definitions used)

G.0 Binding scope, alignment, and nonambiguity conditions

Appendix G is the lexical sealing layer of the report. Its function is to eliminate semantic drift by fixing, for every technical expression that appears in the body, a unique binding meaning anchored to an explicit Definition, Lemma, Theorem, certificate schema, or ledger entry, with an explicit location in the document where that meaning is introduced and preserved. This appendix therefore operates as a normative index rather than as an independent source of alternative meanings, and any apparent conflict of interpretation is resolved by the Notation and Object Registry together with the chapter indexed definition and theorem systems, not by informal reading.

In Parts I through VI, every correctness relevant statement is admissible only in formally typed claim forms, with explicit dependencies, and with mechanically checkable proof objects wherever the report declares certificate gating. In Part VII, every empirical statement is admissible only when it is derivable under an explicit benchmark freeze, fairness contract, metric definition, and reporting rule. Appendix G exists to ensure that the language of these admissibility conditions is itself nonambiguous.

G.1 One-line sealing rule for the entire report

One-line Sealing Rule for the entire report.

A statement is admissible in this report if and only if it is either a correctness claim expressed as a Definition, Lemma, Theorem, or Corollary with explicit dependencies and, where required, an accepted mechanically checkable certificate, or it is an empirical claim whose reported values are reproducible from a frozen benchmark charter under an explicit fairness contract using the metrics ledger and the reporting protocol.

G.2 Report discipline and closure terms

Assumption Ledger

The front matter ledger that enumerates, in a finite and auditable form, the assumptions required for instance finiteness, plan universe finiteness, computability of costs, and the evaluation closure requirements used in the empirical chapters.

Claim Discipline Protocol

The front matter protocol that specifies permitted claim forms and prohibits implicit assumptions and unverifiable performance claims, thereby enforcing the admissibility boundary that the sealing rule quantifies.

Notation and Object Registry

The front matter registry that binds symbols, sets, functions, certificate formats, and ledgers to unique referents, and it is the authoritative resolution device for any notational collision.

Nonarguable Closure

The report phase in which invariants, verification, and end to end correctness stitching are unified into an unambiguous closure, including arithmetic certificate verification and transcript acceptance implying global optimality.

Empirical Closure Pack

The report phase in which benchmark freeze, fairness, metrics, ablations, robustness, repeatability, and reporting completeness are specified so that empirical claims become fully reproducible and nonarbitrary.

Benchmark Charter v1.0

The frozen specification of schema and workload families, candidate space freeze policy, budget regimes, and stratification required to reproduce all reported evaluation results.

Baseline Registry and Fairness Contract

The binding registry of baselines and the enforcement rules that guarantee all compared methods share the same allowed information, constraints, and disallowed ad-

vantages.

Metrics Ledger and Success Criteria

The binding dictionary of primary and secondary metrics, together with aggregation and confidence reporting rules, under which every empirical plot and table is interpreted.

Ablation Grid Specification

The binding enumeration of single module and combined variants and their reporting requirements, including the anchor solver definition.

Robustness Suite Specification

The binding family of shift and stress tests across workload, budget, schema, and candidate expansion perturbations, together with the required reporting templates.

Repeatability and Reporting Protocol

The binding rules for random seeds, ordering controls, stopping rules, timeouts, and reporting completeness checklists.

Results Ledger v1.0

The binding tables and plots that must be produced, including interpretation rules that restrict what may be concluded from the empirical measurements.

Appendix A Full Definitions Index

The complete index of symbol definitions and chapter definitions and the binding lookup table for definitional provenance.

Appendix B Full Theorem Index

The complete index of theorem statements and dependency relations and the binding lookup table for correctness provenance.

Appendix C Certificate Schema Compendium

The complete set of proof object formats that the verification engine accepts and rejects.

Appendix D Invariant Ledger

The complete list of invariants together with preservation references, and the binding reference for all invariant claims.

Appendix E Empirical Artifact Index

The mapping from tables and figures to empirical chapters, and the binding reference for reproducibility of empirical claims.

G.3 Core mathematical and certificate terms

Integers and rationals

The exact arithmetic universe used by certificates and auditing, where integers are

elements of the integer set and rationals are represented canonically as reduced integer pairs with positive denominator, and equality is checked by exact integer relations rather than by floating computation, as fixed in Definition 7.1 and Definition 37.1.

Canonical rational

A reduced pair representation of a rational number, in which the pair satisfies the canonicity constraints and denotes the unique real value given by the quotient, as fixed in Definition 10.1.

Rational operations

The exact operations of addition, multiplication, and comparison on canonical rationals, implemented by integer arithmetic followed by deterministic reduction to canonical form, as fixed in Definition 10.2.

Floor of a rational

The unique integer produced by exact Euclidean division that satisfies the defining inequality bounds, and it is used only where a certificate schema explicitly invokes it, as fixed in Definition 10.3.

Linear forms

Finite dot product and matrix vector evaluation conventions used to state primal, dual, and certificate equalities and inequalities, as fixed in Definition 37.2.

Mixed-integer feasible set of the node

The node restricted feasible region defined by the node ledger constraints together with the designated integrality requirements for the integer coordinates, and it is the semantic domain over which node cuts must be valid to be correct at that node, as fixed in Definition 10.4.

Integer-projection operator

The projection that extracts the integer coordinate subvector from the full variable vector according to the declared integer index set, and it is used to formulate integrality conditions for certificates, as fixed in Definition 11.1.

CG-certificate

A cut certificate in which a nonnegative multiplier vector over the current inequality system deterministically induces a valid cutting inequality by exact arithmetic, as fixed in Definition 11.2.

Acceptance gate

The deterministic cut admission predicate that accepts a proposed cut if and only if the multiplier vector is componentwise nonnegative, the induced inequality has the required integrality properties on the integer coordinates, and the current node relaxation point violates the induced inequality, as fixed in Definition 11.3.

CG cut certificate object

The proof object tuple by which a CG type cut is represented in the transcript and audited, including the bookkeeping required to identify the combined rows and the exact arithmetic construction of the induced inequality, as fixed in Definition 37.4.

Plan grammar as a finite weighted directed hypergraph

The finite representation of the plan universe for a query by a finite symbol set, a distinguished start symbol, and a finite hyperedge set with rational edge costs, where complete plans correspond to finite derivation trees and total cost is the sum of used edge costs, as fixed in Definition 37.5.

Reduced cost in grammar form

The reduced cost of a plan expressed directly in terms of the plan grammar costs together with the node dual variables for plan choice and object couplings, as fixed in Definition 37.6.

Pricing NoNeg certificate

The function valued certificate over the plan grammar that satisfies the required Bellman type inequalities and the required start dominance inequality, and which thereby implies that every plan derivation has nonnegative reduced cost, as fixed in Definition 37.7.

Pricing oracle with certificate

The abstract oracle interface that, given a node and any dual feasible solution of the restricted master, returns either a negative reduced cost column witness or a certificate asserting that no negative reduced cost column exists for the full plan universe at that node, as fixed in Definition 23.2.

Canonical sparse column representation

The canonical triple representation of a plan column consisting of the query identifier, the strictly increasing canonical requirement list, and the exact plan cost, as fixed in Definition 7.10.

The Yothic Column Projector

The deterministic map from a plan code to the canonical sparse triple signature, undefined precisely when the code fails to decode under the declared decoder, as fixed in Definition 7.9.

Reduced cost of a plan column

The exact rational reduced cost computed from a dual snapshot and a valid plan code by subtracting the dual contributions implied by the canonical requirement list from the objective coefficient implied by the exact plan cost, as fixed in Definition 7.13.

Column certificate

The certificate tuple consisting of a plan code, its canonical requirement list, its claimed exact cost, and its claimed reduced cost, as fixed in Definition 7.15.

Verifier input

The complete set of instance maps, weights, dual snapshots, and decoding and evaluation functions provided to the verifier so that verification becomes finite exact arithmetic conditioned only on finite decoding, as fixed in Definition 7.16.

Verifier

The deterministic acceptance procedure that checks code canonicity, successful decoding, requirement list correctness, exact cost correctness, and reduced cost correctness by exact rational arithmetic and rejects on any failure, as fixed in Definition 7.17.

Node-bound certificate package

The aggregated proof object at a node sufficient for the auditor to verify that the claimed node LP bound equals the optimum of the full relaxation at that node, including cut validity and pricing closure components as required by the transcript format, as fixed in Definition 37.8.

Coverage of integer assignments by the branch tree

The statement of branch completeness in which the set of integer assignments feasible under the root constraints is partitioned or covered by the leaf domains of the branch tree, as fixed in Definition 37.9.

Simplex of workload weights

The set of nonnegative weight vectors of fixed total mass used to represent workload distributions, as fixed in Definition 4.1.

Deviation budgets: global and groupwise

The global and group constrained deviation system used to define robust uncertainty sets over workload weights by linearized absolute deviation budgets, as fixed in Definition 24.3.

Yothic Budgeted Simplex Uncertainty Set

The robust uncertainty set obtained as the weight projection of the deviation constrained lifted system, and it is the only uncertainty set used under that name in the report, as fixed in Definition 24.4.

Evaluation semantics

The recursive semantics that assigns a well defined value to each expression node in the compiled expression graph using affine evaluation, multiplication, and piecewise affine segment selection, as fixed in Definition 19.6.

Correctness invariants package

The invariant bundle that states validity of node lower bounds, validity of incumbent upper bounds, validity of all accepted cuts, correctness of pricing completion implying LP optimality, and solver termination returning a globally optimal incumbent under admissible actions and deterministic fallbacks, as fixed in Definition 30.13.

Design space

The set of all globally feasible physical designs together with the design vector representation and the exact objective value mapping under the exact model, as fixed in Definition 16.1.

Feasible design set

The set of designs satisfying all design side constraints such as budget, exclusivity, and compatibility, and it is represented by the fixed finite linear constraint description included in the solved instance specification, as fixed in Definition 28.12.

Branch actions

The finite action space of branching decisions available at a state, including the declared null action when branching is not taken, as fixed in Definition 28.2.

Cut actions

The finite action space of cut related decisions available at a state, including the declared null action when no cut is proposed, as fixed in Definition 28.6.

Pricing actions

The finite action space of pricing related decisions available at a state, including the declared null action when pricing is not proposed, as fixed in Definition 28.10.

Primal actions

The finite action space of primal design proposals available at a state, including the declared null action when no primal proposal is made, as fixed in Definition 28.13.

Empirical projector

The deterministic operator that maps observed sample counts into a realized distribution under the declared empirical projection rule, as fixed in Definition 4.15.

G.4 Auxiliary defined terms (index sentences; each entry is binding by its cited Definition)**Active/tight constraints**

Within this report, Active/tight constraints denotes the object specified by Definition 25.4, and it is not used in any other sense.

Additive-smoothing operator

The expression Additive-smoothing operator refers to the definition stated in Defini-

tion 4.17, which is binding wherever the expression appears.

Affine hull and dimension

In this report, the term Affine hull and dimension is used with the meaning fixed by Definition 13.4.

Affine independence

Within this report, Affine independence denotes the object specified by Definition 13.11, and it is not used in any other sense.

Annotated plan

The expression Annotated plan refers to the definition stated in Definition 3.7, which is binding wherever the expression appears.

Augmented state space

In this report, the term Augmented state space is used with the meaning fixed by Definition 30.8.

Available pricing targets

Within this report, available pricing targets denotes the object specified by Definition 28.9, and it is not used in any other sense.

Base variable domains

The expression Base variable domains refers to the definition stated in Definition 14.1, which is binding wherever the expression appears.

Base variables

In this report, the term Base variables is used with the meaning fixed by Definition 19.1.

Bounded certificate set

Within this report, Bounded certificate set denotes the object specified by Definition 11.4, and it is not used in any other sense.

Branch-robust seed column

The expression Branch-robust seed column refers to the definition stated in Definition 7.8, which is binding wherever the expression appears.

Canonical plan enumeration

In this report, the term Canonical plan enumeration is used with the meaning fixed by Definition 33.1.

Canonical representative

Within this report, Canonical representative denotes the object specified by Definition 13.2, and it is not used in any other sense.

Canonicalization operator

The expression Canonicalization operator refers to the definition stated in Definition 13.1, which is binding wherever the expression appears.

Ceiling division

In this report, the term Ceiling division is used with the meaning fixed by Definition 14.3.

CMDP transition

Within this report, CMDP transition denotes the object specified by Definition 30.10, and it is not used in any other sense.

Column identity

The expression Column identity refers to the definition stated in Definition 7.11, which is binding wherever the expression appears.

Column signature

In this report, the term Column signature is used with the meaning fixed by Definition 7.12.

Conditional probability on sequences

Within this report, Conditional probability on sequences denotes the object specified by Definition 4.5, and it is not used in any other sense.

Configuration space

The expression Configuration space refers to the definition stated in Definition 30.1, which is binding wherever the expression appears.

Counts and realized weights

In this report, the term Counts and realized weights is used with the meaning fixed by Definition 4.2.

Cover

Within this report, Cover denotes the object specified by Definition 11.6, and it is not used in any other sense.

Decision types

The expression Decision types refers to the definition stated in Definition 30.2, which is binding wherever the expression appears.

Dominance over a base polyhedron

In this report, the term Dominance over a base polyhedron is used with the meaning fixed by Definition 25.2.

Dual variables for canonical form

Within this report, Dual variables for canonical form denotes the object specified by Definition 37.3, and it is not used in any other sense.

Efficiency

The expression Efficiency refers to the definition stated in Definition 14.4, which is binding wherever the expression appears.

Enumerator

In this report, the term enumerator is used with the meaning fixed by Definition 28.15.

Episode and return

Within this report, Episode and return denotes the object specified by Definition 30.14, and it is not used in any other sense.

Expectation on finite spaces

The expression Expectation on finite spaces refers to the definition stated in Definition 4.7, which is binding wherever the expression appears.

Exact plan cost

In this report, the term Exact plan cost is used with the meaning fixed by Definition 3.5.

Exact plan cost functional

Within this report, Exact plan cost functional denotes the object specified by Definition 3.6, and it is not used in any other sense.

Exact strength at node

The expression Exact strength at node refers to the definition stated in Definition 14.2, which is binding wherever the expression appears.

F1 violated cut

In this report, the term F1 violated cut is used with the meaning fixed by Definition 11.5.

F2 violated cover cut

Within this report, F2 violated cover cut denotes the object specified by Definition 11.7, and it is not used in any other sense.

Face induced by an inequality

The expression Face induced by an inequality refers to the definition stated in Definition 13.3, which is binding wherever the expression appears.

Facet

In this report, the term Facet is used with the meaning fixed by Definition 13.5.

Finite context space and parameters

Within this report, Finite context space and parameters denotes the object specified by Definition 4.11, and it is not used in any other sense.

Fractional set

The expression fractional set refers to the definition stated in Definition 28.1, which is binding wherever the expression appears.

Frontier sets

In this report, the term Frontier sets is used with the meaning fixed by Definition 25.1.

Gate and fallback

Within this report, Gate and fallback denotes the object specified by Definition 30.4, and it is not used in any other sense.

Hard-safe budget gate

The expression Hard-safe budget gate refers to the definition stated in Definition 30.9, which is binding wherever the expression appears.

Index layout

In this report, the term Index layout is used with the meaning fixed by Definition 16.2.

Integer assignments

Within this report, Integer assignments denotes the object specified by Definition 37.10, and it is not used in any other sense.

Integer optimum at node

The expression Integer optimum at node refers to the definition stated in Definition 37.11, which is binding wherever the expression appears.

LP primal feasibility certificate

In this report, the term LP primal feasibility certificate is used with the meaning fixed by Definition 37.12.

LP dual feasibility certificate

Within this report, LP dual feasibility certificate denotes the object specified by Definition 37.13, and it is not used in any other sense.

LP optimality certificate

The expression LP optimality certificate refers to the definition stated in Definition 37.14, which is binding wherever the expression appears.

Mixed objective decomposition

In this report, the term Mixed objective decomposition is used with the meaning fixed by Definition 3.4.

Neural cut proposer interface

Within this report, Neural cut proposer interface denotes the object specified by Definition 11.8, and it is not used in any other sense.

Nominal weights and total mass

The expression Nominal weights and total mass refers to the definition stated in Definition 24.1, which is binding wherever the expression appears.

Object index map

In this report, the term Object index map is used with the meaning fixed by Definition 7.3.

One-step transition kernel

Within this report, One-step transition kernel denotes the object specified by Definition 30.5, and it is not used in any other sense.

Parameter space

The expression Parameter space refers to the definition stated in Definition 4.4, which is binding wherever the expression appears.

Partition layout

In this report, the term Partition layout is used with the meaning fixed by Definition 16.3.

Per-query plan existence under

Within this report, Per-query plan existence under denotes the object specified by Definition 3.3, and it is not used in any other sense.

Plan code alphabet

The expression Plan code alphabet refers to the definition stated in Definition 7.4, which is binding wherever the expression appears.

Plan signature

In this report, the term Plan signature is used with the meaning fixed by Definition 7.7.

Policy

Within this report, Policy denotes the object specified by Definition 30.12, and it is not used in any other sense.

Pricing target set

The expression Pricing target set refers to the definition stated in Definition 28.8, which is binding wherever the expression appears.

Probability simplex of contexts

In this report, the term Probability simplex of contexts is used with the meaning fixed

by Definition 4.9.

Recommended actions vs executed actions

Within this report, Recommended actions vs executed actions denotes the object specified by Definition 30.3, and it is not used in any other sense.

Relative deviation capacities

The expression Relative deviation capacities refers to the definition stated in Definition 24.2, which is binding wherever the expression appears.

Requirement extractor

In this report, the term Requirement extractor is used with the meaning fixed by Definition 7.5.

Resource dimensions

Within this report, Resource dimensions denotes the object specified by Definition 30.6, and it is not used in any other sense.

Resource increment function

The expression Resource increment function refers to the definition stated in Definition 30.7, which is binding wherever the expression appears.

Resource vector and budgets

In this report, the term Resource vector and budgets is used with the meaning fixed by Definition 30.6.

Reward and constraint costs

Within this report, Reward and constraint costs denotes the object specified by Definition 30.11, and it is not used in any other sense.

Soundness

The expression Soundness refers to the definition stated in Definition 33.1, which is binding wherever the expression appears.

Template parameters

In this report, the term template parameters is used with the meaning fixed by Definition 28.4.

Unconditional mixture distribution on sequences

Within this report, Unconditional mixture distribution on sequences denotes the object specified by Definition 4.12, and it is not used in any other sense.

Valid inequality / cut

The expression Valid inequality / cut refers to the definition stated in Definition 13.1, which is binding wherever the expression appears.

Vertex

In this report, the term Vertex is used with the meaning fixed by Definition 25.5.

YF-MINLP expression grammar

Within this report, YF-MINLP expression grammar denotes the object specified by Definition 19.2, and it is not used in any other sense.

Annexes



Showcasing the First Annex

Writing Guidance

Annexes are supplementary sections in a dissertation that provide additional information or external documents not essential to the main arguments but that support or complement the research. Unlike appendices, **annexes generally contain material that was not developed by the author**, such as reports, legal documents, or published datasets from external sources. This information is placed separately to keep the main content concise, allowing readers access to relevant external references without disrupting the dissertation's flow.

